

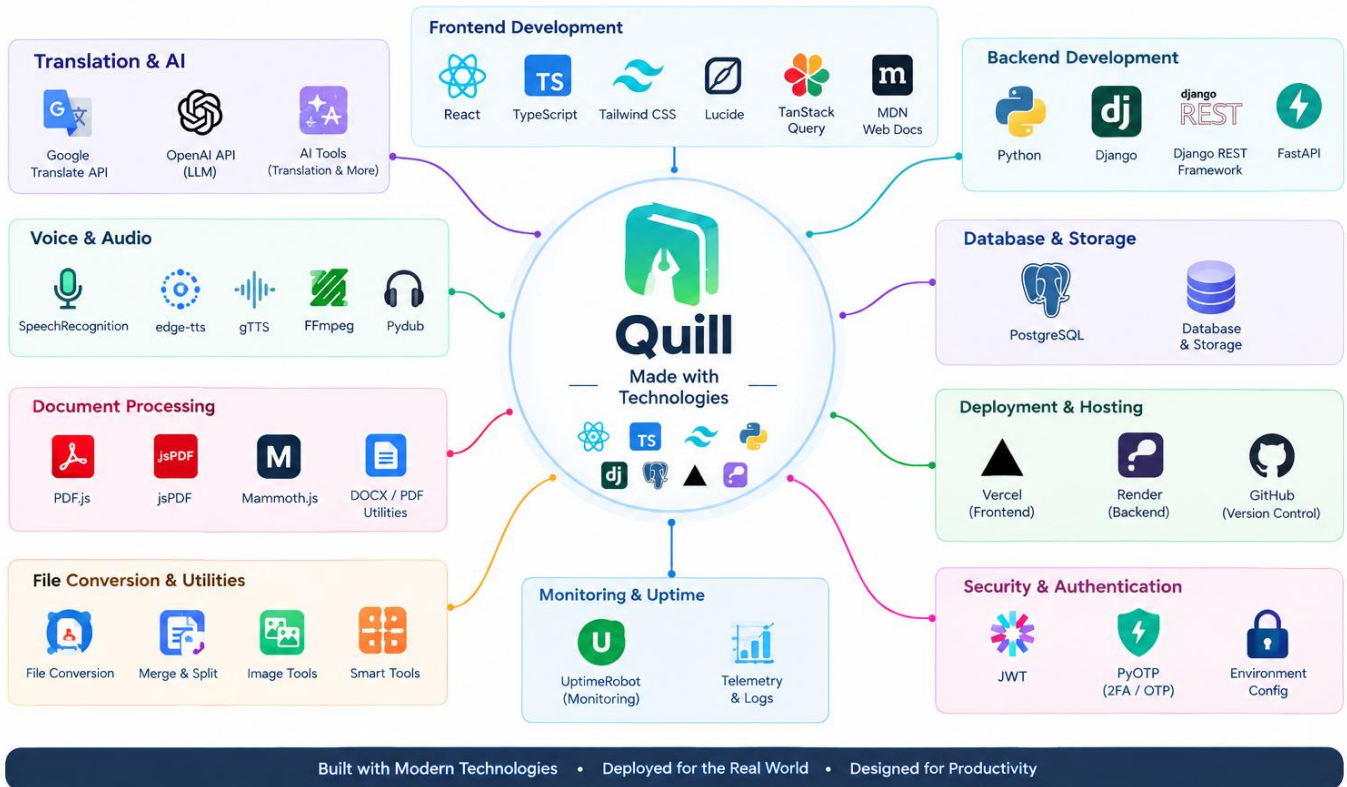
INDEX / TABLE OF CONTENTS

S. No.	Section
1	Introduction to Quill
2	Problem Statement & Motivation
3	Objectives of the Project
4	Why Quill? — Idea & Vision
5	Overview of the System
6	Key Features of Quill
7	Translation Module
8	Document Translation Module
9	Combine & Convert Module
10	Smart Tools & Utility Module
11	User Authentication & Security
12	History, Activity & User Management
13	System Architecture
14	Frontend Architecture
15	Backend Architecture
16	Database Design
17	API Architecture & Communication
18	Important Algorithms & Processing Logic
19	File Processing Workflow
20	Translation Workflow
21	PDF Merge & File Reordering Algorithm
22	Document Processing & Conversion Logic
23	Error Handling & Validation
24	Performance & Optimization
25	Technology Stack
26	Project Folder Structure
27	Development Process
28	Testing & Debugging
29	Deployment — Vercel & Render

30	Uptime Monitoring with UptimeRobot
31	Security & Environment Configuration
32	Challenges Faced & Solutions
33	Future Improvements
34	Conclusion
35	References & Resources

Quill – Full Stack Productivity Workspace

Translation • Voice • Documents • Conversion • Smart Tools • History • Authentication



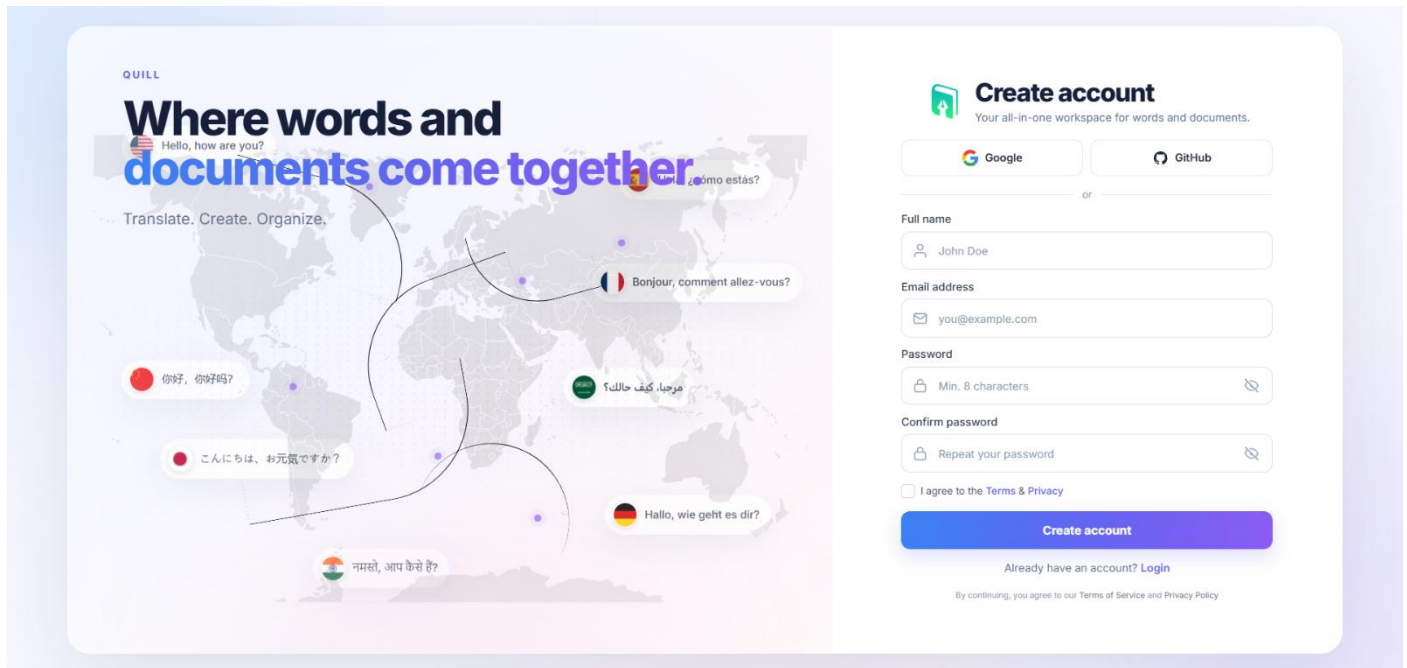
Built by : Aarush Khandelwal

“To create a single, simple workspace that brings translation, document processing, file conversion, and smart tools together instead of using multiple separate platforms.”

1. Introduction to Quill

Quill is a web-based productivity platform developed to bring different day-to-day document and language-related tasks into one place. The main idea behind the project is to make common activities such as **translation, document processing, file conversion, PDF management, and utility calculations** easier to access from a single workspace.

While working with documents, users often need to move between different tools for different tasks. For example, one website may be used for translation, another for merging PDFs, another for converting Word files, and another for simple calculations or unit conversion. Quill was developed to reduce this dependency on multiple platforms by providing these functions through a common interface.



The platform provides separate workspaces for **Translation, Documents, Combine & Convert, Smart Tools, and History**. The translation workspace supports text and voice-based workflows, while the document workspace allows users to upload and process complete documents. The Combine & Convert section provides tools for merging, converting, compressing, rotating, organizing, and modifying files. Quill also includes utility tools for areas such as numbers, currency, units, data, electrical calculations, and other specific conversions.

Another important part of Quill is its user-based workflow. Users can create an account, securely access the platform, perform different operations, and keep track of their previous activities through the History section. This makes the application more useful for repeated work instead of treating every operation as a one-time task.

From the technical side, Quill is built as a full-stack application. The frontend is organized into reusable React components and pages, while the backend is developed using Django and Django REST Framework. The two layers communicate through APIs, allowing file operations, translation requests, authentication, and history management to be handled through the backend. The project is structured into modules such as users, translation, documents, combine, tools, and history.

Quill is deployed as a cloud-based application, with the frontend hosted on **Vercel** and the backend hosted on **Render**. The source code is maintained on GitHub, making the project easier to manage and update during development.

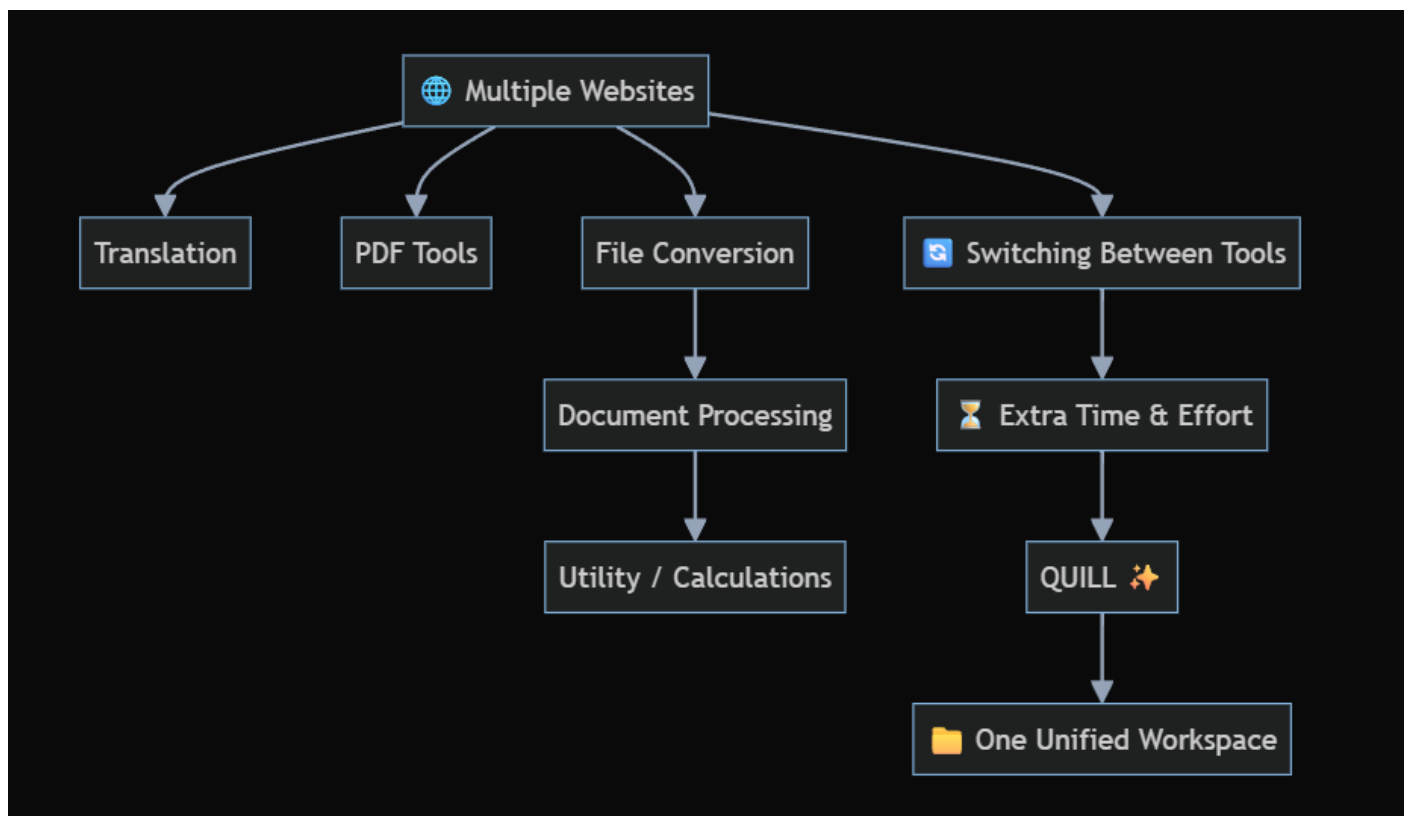
Overall, Quill was developed with one clear purpose: **to provide a simple and unified workspace for working with language, documents, files, and everyday productivity tasks.**

2. Problem Statement & Motivation

Working with digital documents has become a regular part of both academic and professional work. A single task may involve several different operations such as translating a document, converting its format, merging files, compressing a PDF, organizing pages, or performing a quick unit or currency conversion. In practice, these tasks are often handled using different websites or applications, which makes the overall workflow slower and less organized.

The main problem that led to the development of Quill was this fragmentation. Instead of having one place for related tasks, users may need to upload the same file to different services multiple times, learn different interfaces, and keep track of results from separate platforms. This becomes especially inconvenient when several operations are part of the same workflow.

Quill was designed to address this by bringing these operations into a single workspace. The application combines translation, document processing, file conversion, PDF operations, and utility tools under a common interface. The project documentation describes dedicated workspaces for **translation, documents, combine and convert operations, smart tools, and activity history**.



Another motivation was to make document-related workflows easier to follow. For example, a document translation workflow in Quill follows a clear sequence: **upload the document → select or detect the language → translate → review the result → download the output**. Similarly, file operations follow a sequence of selecting a tool, uploading files, arranging or converting them, processing them, and downloading the result.

The project was also motivated by the need for a more consistent user experience. Instead of designing every tool as a completely separate application, Quill uses shared navigation, cards, controls, validation messages, processing states, and history features. This gives different tools a similar interaction pattern, making the platform easier to understand as the number of available features grows.

From the development perspective, Quill was also an opportunity to solve real full-stack problems rather than building only a static interface. The project involves **frontend state management, REST API communication, authentication, file uploads, document processing, error handling, history management, and cloud deployment**.

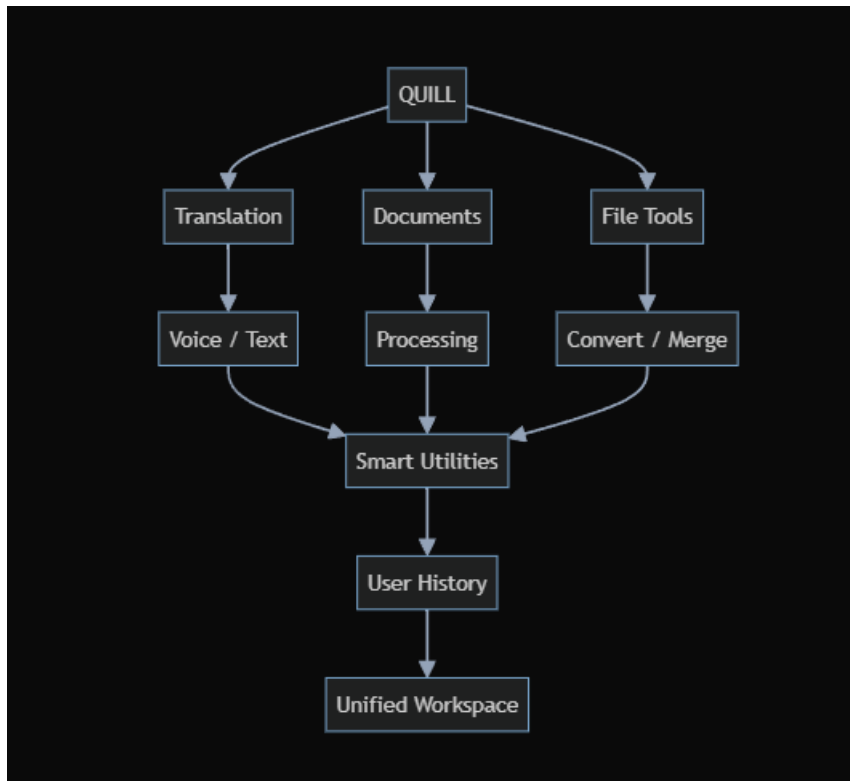
The backend is divided into modules such as **translation, documents, combine, tools, and history**, reflecting the different responsibilities of the application.

The motivation behind Quill can therefore be summarized in one sentence:

Create one practical workspace where users can handle language, documents, files, and everyday utility tasks without constantly switching between different tools.

3. Objectives of the Project

The main objective of **Quill** is to provide a single web-based workspace where users can perform different language, document, file-processing, and utility-related tasks without depending on multiple separate applications.



The project was developed with the following objectives:

3.1 Create a Unified Workspace

Bring translation, document processing, file conversion, PDF utilities, and smart tools together in one application.

3.2 Simplify Document Workflows

Make common operations such as merging, splitting, compressing, rotating, organizing, converting, and watermarking documents easier to perform through a consistent interface.

3.3 Support Multiple Translation Workflows

Provide different ways of working with language, including text and voice-based translation workflows and document translation.

3.4 Make Document Translation Practical

Allow users to upload a document, select or detect languages, process the document, review the translated content, and download the result in supported formats.

3.5 Provide Useful Everyday Tools

Include utilities for areas such as **number, currency, units, data, electrical, health, and specific calculations or conversions.**

3.6 Maintain User Activity History

Keep users' previous translation, document, tool, and combine activities organized in one history workspace, with options for filtering, searching, previewing, downloading, and deleting records.

3.7 Build a Scalable Full-Stack System

Separate the frontend, backend, and processing responsibilities so that individual modules can be maintained and improved independently. The current project structure separates areas such as users, translation, documents, combine, tools, and history.

3.8 Provide a Simple and Consistent User Experience

Use common layouts, navigation, controls, feedback messages, and workflows across the application so that users can move between different tools without learning a completely new interface each time.

3.9 Deploy the Application for Real Use

Make the project available as a cloud-hosted application with the frontend deployed on **Vercel** and the backend on **Render**.

3.10 Learn and Apply Full-Stack Development

The project also serves as a practical implementation of frontend development, backend APIs, authentication, file handling, document processing, database interaction, debugging, deployment, and monitoring.

4. Why Quill? — Idea & Vision

The idea behind Quill came from a simple problem: many common tasks related to **translation, documents, PDFs, file conversion, and utility tools** are usually handled through different websites or applications. A user may need one website for translation, another for PDF operations, another for file conversion, and additional tools for smaller tasks. This makes the overall workflow slower and less convenient.

Quill was designed to bring these commonly used functions together in **one workspace**. Instead of building the project around only a single feature, the idea was to create a platform where users can move from one task to another without leaving the application.

The main vision of Quill is to provide a **simple, organized, and practical workspace** for everyday digital tasks. The platform combines translation, document processing, PDF operations, file conversion, smart utilities, and activity history under a common interface.

Another important part of the idea was to keep the experience consistent. Different modules should follow a similar workflow: **select a tool, provide the required input, process it, review the result, and download or save the output**. This makes the application easier to understand even when the user moves between different types of tools.

From a development perspective, Quill was also created as a practical full-stack project. It brings together frontend development, backend APIs, file processing, database operations, authentication, deployment, and monitoring in a single application.

4.1 Vision of Quill

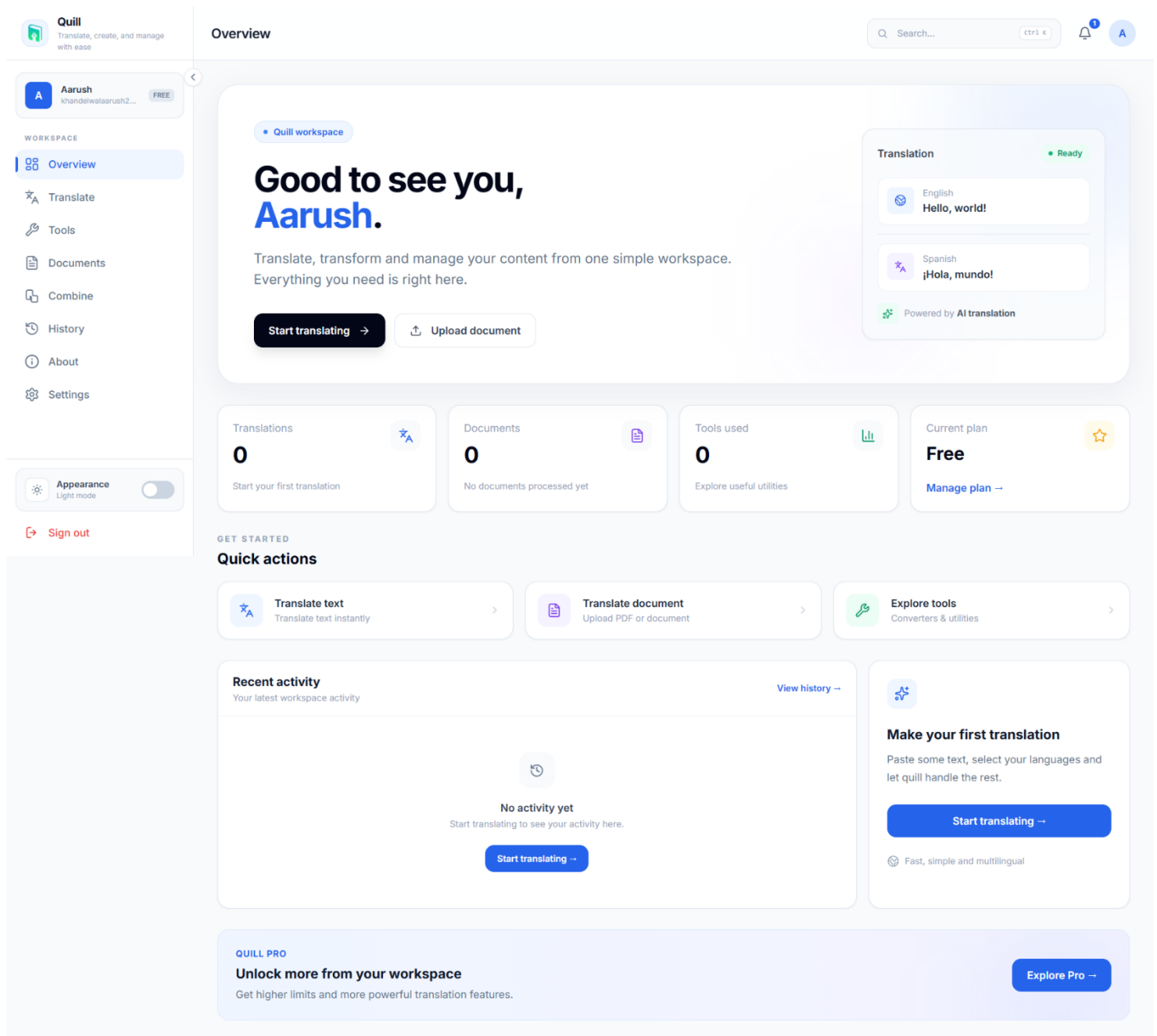
The vision can be represented through four main ideas:

- **One Workspace** — multiple useful functions in one platform.
- **Simple Workflow** — clear steps from input to final output.
- **Practical Tools** — features focused on real and common user requirements.
- **Connected System** — translation, documents, tools, history, and account features working together.

The goal is not just to provide individual tools, but to create a connected environment where these tools can be used as part of one complete workflow.

5. Overview of the System

Quill is designed as a **full-stack web application** that brings translation, document processing, file conversion, PDF utilities, and other practical tools into a single workspace. The system is divided into different functional modules, but they are connected through a common frontend, backend API layer, database, and file-processing workflow. The overall design allows a user to select a task, provide the required input, process it through the backend, and receive the final result in the browser. Quill's published project description identifies React/TypeScript/Tailwind CSS on the frontend, Python/Django/Django REST Framework on the backend, and document/file-processing libraries such as PDF.js, jsPDF, DOCX and image-processing tools.



5.1 Main Components of the System

The major parts of Quill can be divided into the following modules:

1. User Interface

The frontend provides the main workspace where users can access translation, document, conversion, combine, and utility features. It also manages user interactions such as file selection, options, progress, results, and downloads.

2. Translation Module

This module handles text and voice-based translation workflows. Users can enter text or provide speech, select source and target languages, and review the translated output.

3. Document Processing Module

Users can upload documents for processing and translation. The system handles the uploaded file, performs the required processing, and prepares the translated or converted output.

4. Combine & Convert Module

This section contains tools for PDF, Word, and image operations. It includes operations such as merging, splitting, compression, rotation, watermarking, image conversion, PDF conversion, and other file-related tasks.

5. Smart Tools Module

Quill also provides utility tools for categories such as numbers, currency, units, data, electrical calculations, health-related calculations, and other specific utilities.

6. History and User Management

Completed activities can be connected with the user's account and history, allowing previous translations and tool operations to be reviewed and managed.

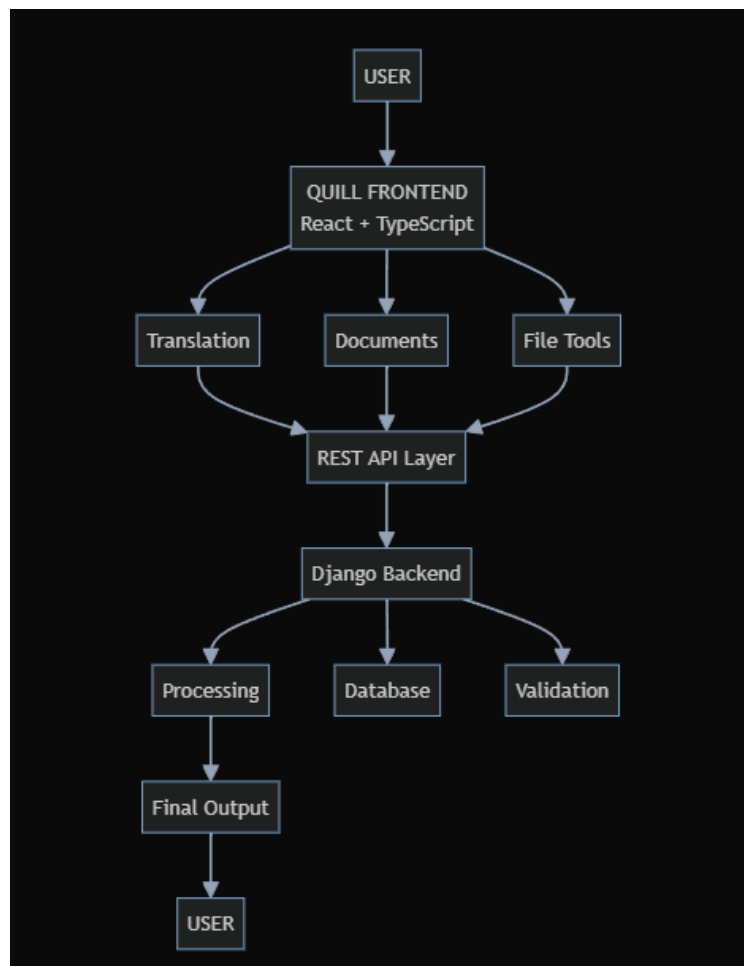
7. Backend and Database

The frontend communicates with a Django-based REST API. The backend handles validation, business logic, file processing, authentication-related operations, and communication with the database and processing services.

5.2 Basic Working Flow

The overall working of Quill can be understood through a simple flow:

High-level working flow of the Quill system



The important point in this architecture is that the user does not need to directly interact with the backend or processing libraries. The frontend provides the interface, while the backend receives requests, performs the required operation, and sends the result back to the user.

5.3 Overall System Workflow

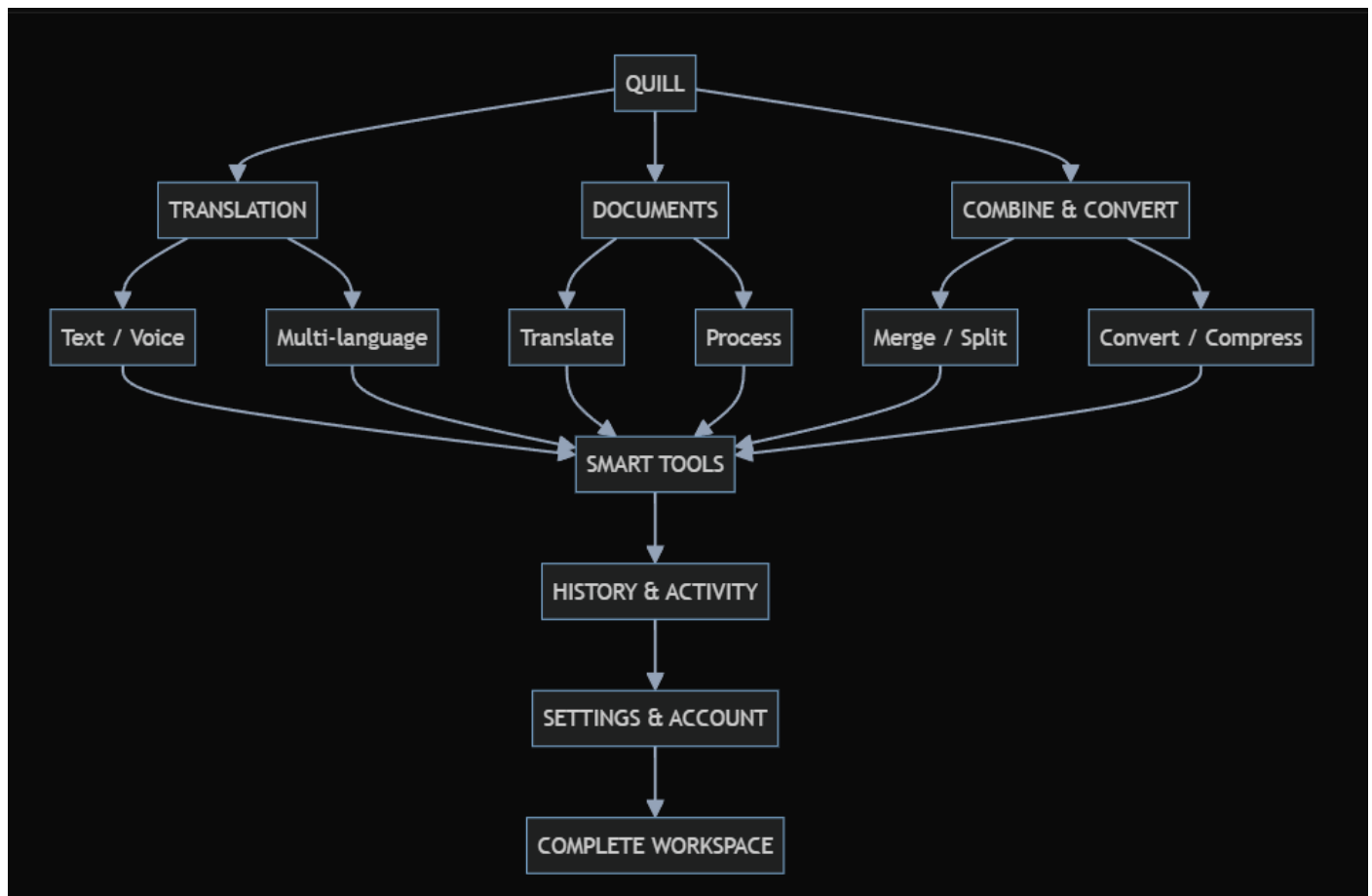
A typical operation follows these steps:

Input → Tool Selection → User Options → API Request → Backend Processing → Result Generation → Preview/Download → History

This common workflow is used across many parts of Quill, although the exact processing steps differ between translation, document processing, and file operations.

6. Key Features of Quill

Quill is designed as a multi-purpose workspace where users can perform different tasks related to **translation, documents, PDF files, file conversion, and utility calculations** without moving between separate applications. The system combines these functions through a common interface while keeping each feature organized into its own workflow.



The major features of Quill are described below.

6.1 Translation

Quill supports multiple translation workflows for different types of input. Users can work with text or voice, select source and target languages, and review the translated result. The translation interface also provides options related to language selection, translation style, formality, and output speed.

6.2 Voice Translation

The platform supports speech-based workflows in which users can record or provide audio, convert speech into text, translate the content, and generate translated speech where required. This makes the translation system useful for situations where typing is not convenient.

6.3 Document Translation

Users can upload documents and process them through a dedicated document workflow. The system supports language selection or detection, document translation, review of the processed content, and downloading the resulting file.

6.4 PDF and File Operations

Quill provides a collection of file-processing tools for common document tasks. These include PDF merging, splitting, compression, rotation, watermarking, page organization, color enhancement, and N-up processing. It also provides file conversion and image-related operations.

6.5 Combine & Convert

The **Combine & Convert** section brings multiple file operations into one place. Users can work with PDF, Word, and image files according to the selected operation. For merge-based tools, files can be arranged before processing so that the output follows the required order.

6.6 Smart Tools

Quill includes utility tools for different practical calculations and conversions. These are organized into categories such as **Number, Currency, Unit, Data, Electrical, Health, and Specific** tools.

6.7 History and Activity

The History section keeps track of supported user activities and previously processed work. Users can search and filter records, preview available information, download generated files, and remove individual or stored records according to the available options.

6.8 User Settings

Quill provides settings for controlling user preferences such as default language behavior, translation options, appearance, notifications, privacy-related settings, and account preferences.

6.9 Authentication and Account Management

The application includes user account functionality so that user-specific settings and supported activity history can be associated with an account. This also provides a structured way to manage access to the application.

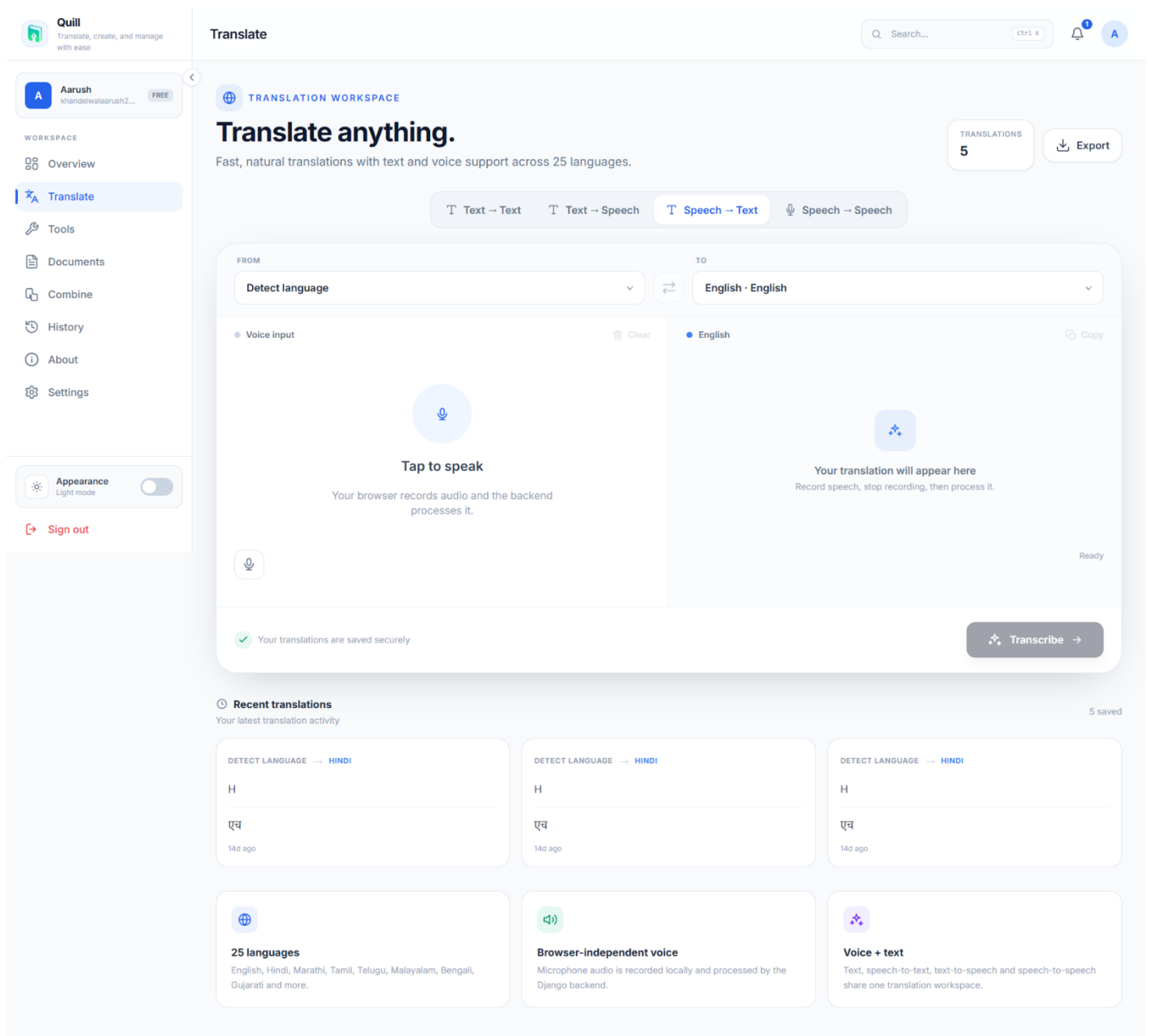
6.10 Cloud Deployment and Monitoring

Quill is deployed as a web application using a separate frontend and backend setup. The frontend is hosted on **Vercel**, the Django backend is hosted on **Render**, and **UptimeRobot** is used for availability monitoring.

7. Translation Module

The **Translation Module** is one of the main features of Quill. It is designed to handle different types of translation tasks within the same workspace. Instead of limiting the user to simple text translation, Quill supports both **text-based and voice-based translation workflows**. The translation workspace includes four main modes: **Text → Text**, **Text → Speech**, **Speech → Text**, and **Speech → Speech**.

The module is designed so that the user can select the required languages, provide the input, process the translation, and review the output without leaving the application.



7.1 Translation Modes

Quill provides four translation modes:

1. Text → Text

The user enters written text and receives the translated text in the selected target language.

2. Text → Speech

The entered text is translated and the resulting text can be converted into speech using the selected voice.

3. Speech → Text

The user provides speech through the microphone or an audio input. The speech is converted into text and processed through the translation workflow.

4. Speech → Speech

This workflow combines speech recognition, translation, and speech generation. The user can provide spoken content and receive translated speech as the final output.

7.2 Language Selection

Quill's translation interface supports **25 languages** in its application UI. These include English, Hindi, Marathi, Tamil, Telugu, Malayalam, Bengali, Gujarati, Spanish, French, German, Italian, Portuguese, Russian, Japanese, Korean, Chinese, Arabic, Turkish, Dutch, Polish, Swedish, Indonesian, Vietnamese, and Ukrainian. The exact availability can depend on the selected translation or speech service.

The translation workspace also supports **automatic source-language detection**, while the target language can be selected by the user. This reduces manual setup when the source language is not known in advance.

7.3 Translation Preferences

The module provides several controls so that the result can be adjusted according to the user's requirement.

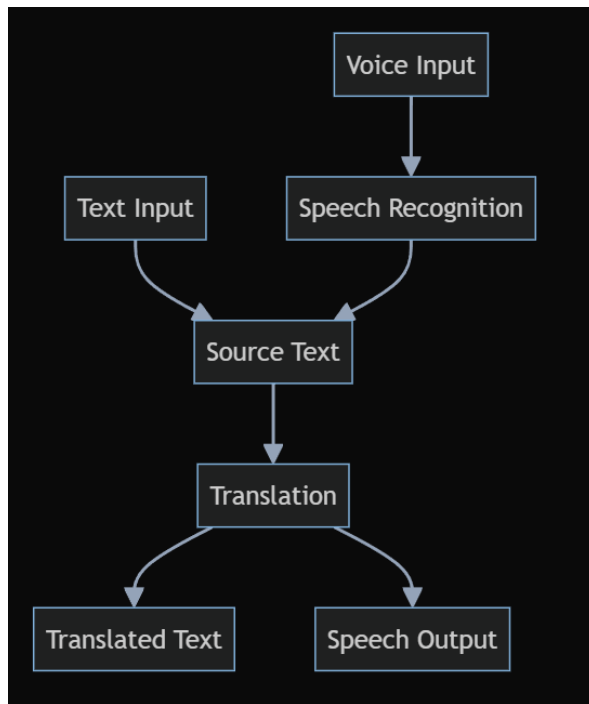
These include:

- **Translation style**
- **Formality**
- **Translation speed**
- **Preserve formatting**
- **Original text visibility**
- **Voice selection for speech output**

These settings make the translation workflow more flexible than a basic input-and-output translator.

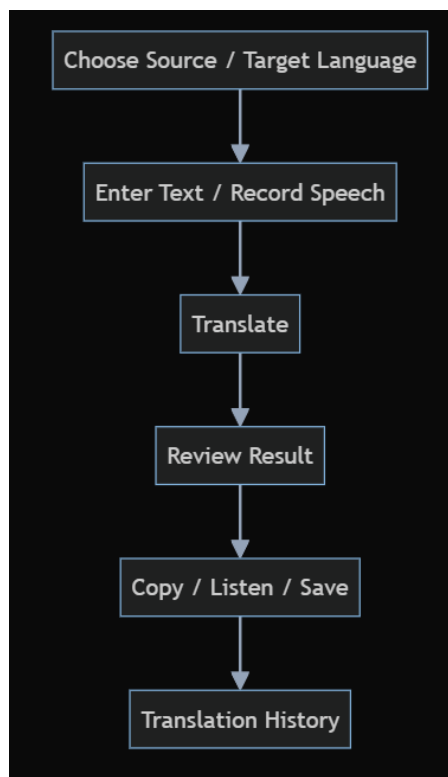
7.4 Voice Translation Support

For voice-based workflows, Quill provides microphone recording and audio-upload support. The backend can normalize audio using **FFmpeg**, process the speech through speech recognition, use an automatically selected or explicitly selected speech locale, and generate multilingual text-to-speech output. The resulting audio can be returned through an output URL and the activity can be connected with translation history.



7.5 Translation History

Quill connects translation activity with the signed-in user's history. Recent translation cards can also be displayed within the workspace, allowing users to refer back to previous translation activity.



8. Document Translation Module

The **Document Translation Module** is designed for users who need to translate an entire document instead of copying and translating its content manually. Quill provides a dedicated workspace where a user can upload a document, select or detect the source language, choose the target language, process the document, review the translated content, and finally download the generated file. The module is intended for **multi-page documents** and supports both document processing and translation in the same workflow.

The main purpose of this module is to make document translation more practical. Instead of translating page by page outside the application, the complete document can be processed through a structured workflow.

The screenshot displays the Quill Document Translation Module interface. On the left, a sidebar contains a 'Quill' logo and a 'Documents' section with a 'Sign out' button. The main area is titled 'Documents' and features a 'Book Page' view. A document titled 'AARUSH KHADELWAL' is shown, with a 'View Original' button. The document content includes a profile summary, education, technical skills, work experience, and projects. The interface is divided into three main steps: 'STEP 1 - UNIVERSAL UPLOADER', 'STEP 2 - LANGUAGE ENGINE', and 'STEP 3 - PROCESS & SECURE DOWNLOAD'. Step 1 shows the document being uploaded. Step 2 shows the language selection process, with 'Auto (Detected)' selected. Step 3 shows the document being processed, with a '100%' completion indicator. The 'Recent Document History' section at the bottom shows no documents yet.

Documents

Upload, translate and securely download multi-page documents in 24+ languages.

Book Page View Original View Translated

STEP 1 - UNIVERSAL UPLOADER

Upload Your Files

Drag & drop here, or [browse from file explorer](#)

Aarush_Khandelwal_Resume.pdf

235.6 KB · 1 page

Multi-page 1-1 Text Ready

STEP 2 - LANGUAGE ENGINE

Original

Search languages

Auto (Detected)

Search for a language..

Translate To:

English Hindi Spanish

French German Italian

Portuguese Russian Japanese

Korean Chinese Arabic

1 Target Language Selected: Hindi

STEP 3 - PROCESS & SECURE DOWNLOAD

100%

✓ Processing complete — downloads ready.

Whole document translated — layout, borders & alignment preserved

DOWNLOAD OPTIONS

PDF DOCX TXT HTML

Re-process Document Download All Formats

Recent Document History

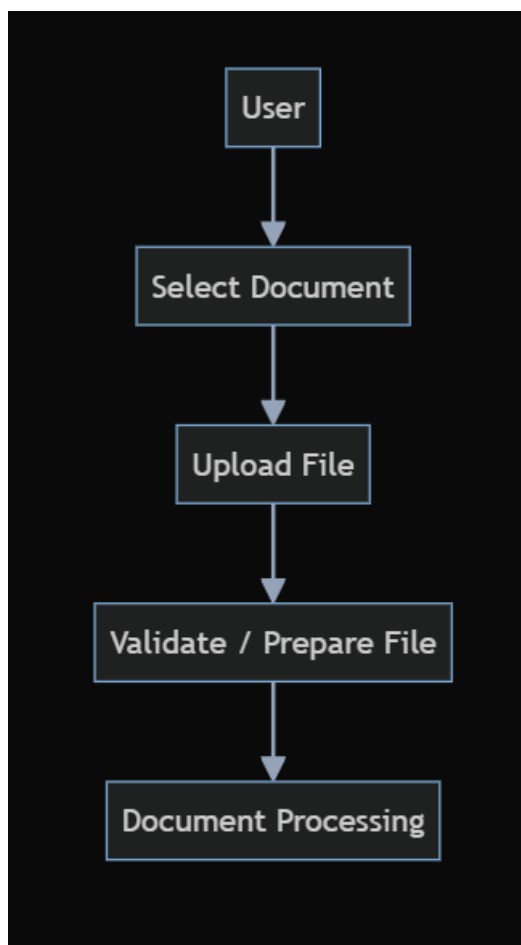
No document history yet.

Upload and translate a document to see it here.

8.1 Document Upload

The workflow starts when the user selects a document from the device and uploads it to Quill.

After the file is selected, the application can validate the input and prepare it for processing. The document is then passed to the backend for further processing.

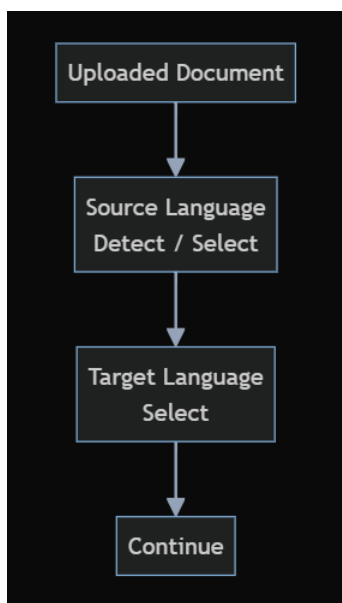


This first step provides a clear starting point for the complete document translation process.

8.2 Language Detection and Selection

After uploading the document, Quill provides a workflow for identifying or selecting the language.

The system supports **automatic source-language detection**, while the user can also select the required target language. This is useful when the source language is already known or when the user wants to explicitly control the translation direction.



This keeps the translation direction clear before the actual processing begins.

8.3 Document Text and Page Processing

Once the language settings are confirmed, the document enters the processing stage.

According to the documented workflow, Quill extracts the required **pages/text**, translates the content, and then builds the output document. The module is designed for multi-page documents, so the processing is not limited to a single page.

The basic internal flow is:

Uploaded File



Extract Pages / Text



Process Document Content



Translate Content

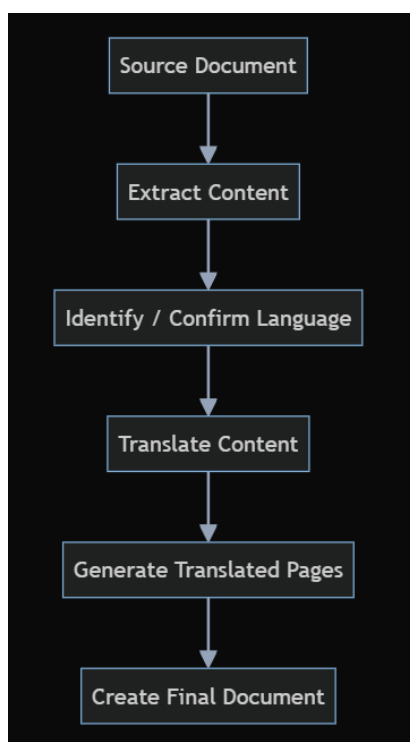


Build Translated Document

This separation between extraction, translation, and output generation makes the document workflow easier to manage.

8.4 Translation of the Complete Document

Quill supports a **whole-document translation workflow**, allowing the uploaded content to be translated as a complete document rather than treating each piece of text as a separate manual task.



The module also supports **page-based translation**, which is useful for multi-page documents where the user may need to review content page by page.

8.5 Original and Translated Content

A useful part of the document workspace is the ability to work with the **original and translated versions** of the document content.

The documentation specifies original/translated page viewing, allowing the processed result to be reviewed before downloading.

This is important because the user can check whether:

- the required content was translated,
- the target language is correct,
- the translated pages are properly generated,
- the document is ready for final download.

Therefore, the workflow is not simply:

Upload → Translate → Download

It is:

Upload → Process → Translate → Review → Download

8.6 Layout and Formatting Preservation

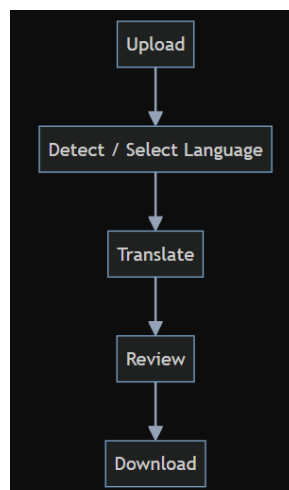
Document translation is different from simple text translation because the structure of the original document also matters.

Quill's documented document workflow includes **layout-aware processing** and a **formatting preservation** option.

The purpose of this stage is to keep the generated document closer to the structure of the source document while replacing or generating the translated content.

8.7 Review and Download

After processing is complete, the translated document is reviewed before it is downloaded.



Quill's documented output formats include:

- PDF
- DOCX
- TXT
- HTML

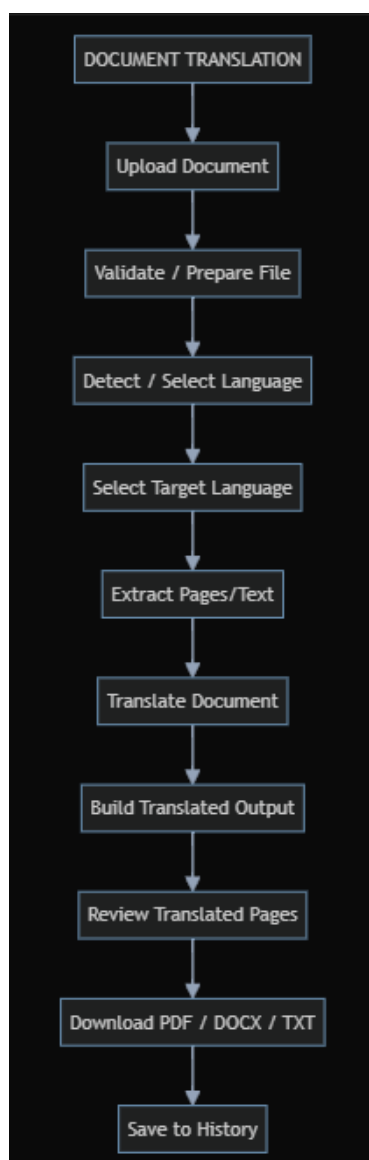
This allows the processed content to be exported in different formats depending on the user's requirement.

8.8 Document Translation and History

After a document has been processed, the activity can become part of the user's recent document history. This provides continuity between document processing and the History section of Quill.

This is useful when a user needs to return to previously processed documents instead of repeating the same workflow.

Complete Document Translation Workflow



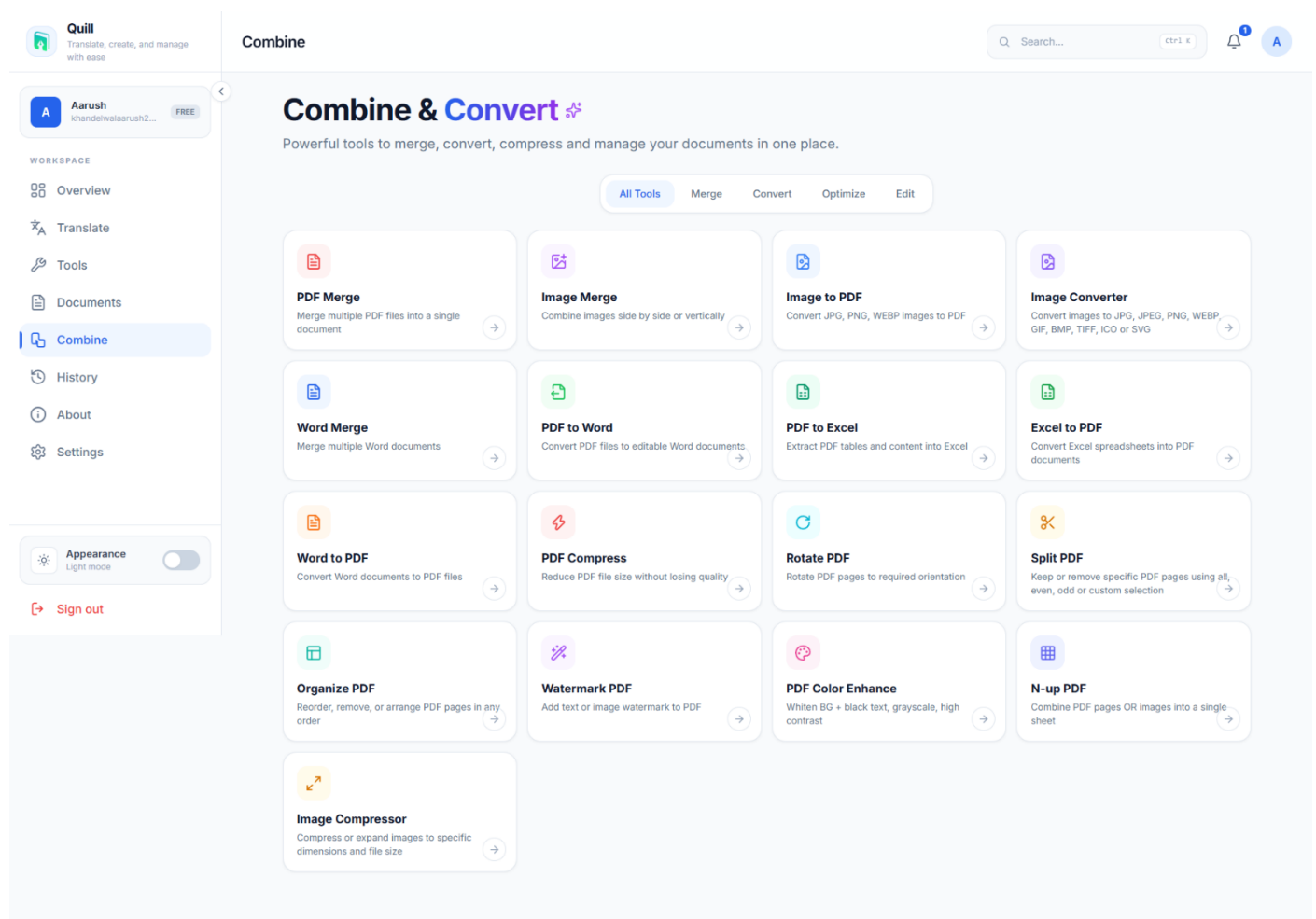
9. Combine & Convert Module

The **Combine & Convert Module** is one of the largest functional sections of Quill. It is designed to handle common operations related to **PDF, Word, and image files** from a single workspace. Instead of using separate applications for merging documents, converting formats, compressing PDFs, rotating pages, or organizing files, these operations are available together inside Quill.

The module contains tools for **merging, converting, compressing, rotating, splitting, organizing, watermarking, enhancing, and arranging documents and images**. The project documentation groups these operations under PDF, image, and document workflows.

A typical file operation in Quill follows the sequence:

Choose Tool → Upload Files → Validate → Arrange / Configure → Process → Download Result → History.



9.1 PDF Merge

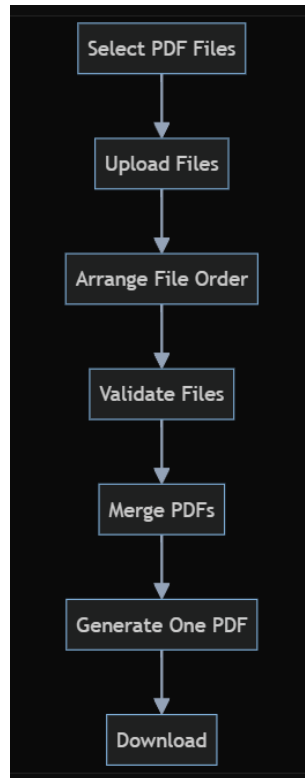
PDF Merge allows multiple PDF files to be combined into a **single PDF document**.

This is useful when a user has separate documents such as:

- multiple assignment files,
- different chapters of a report,
- scanned document sections,

- invoices or records,
- project documentation.

The user selects the required PDF files and can arrange their order before processing. The files are then sent to the backend, where they are combined according to the selected order.



The order of the selected files is important because it determines the order of pages in the final document.

9.2 Image Merge

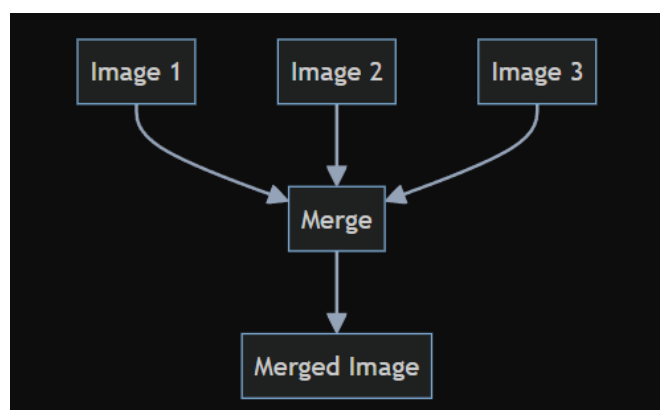
Image Merge combines multiple images into a single output image.

The tool supports arranging images either:

- **Side by Side**
- **Vertically**

This can be useful for creating comparison images, combining screenshots, preparing a collage-like output, or placing multiple images into one file.

Example

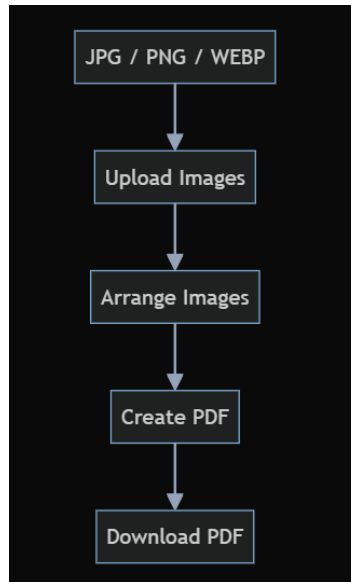


The user can upload multiple images, select the required arrangement, process them, and download the generated image.

9.3 Image to PDF

The **Image to PDF** tool converts image files into PDF documents.

Supported input formats include: **JPG, PNG, and WEBP**



A typical use case is converting scanned pages or photographs into a single PDF document.

This is especially useful when several scanned images need to be submitted as one document.

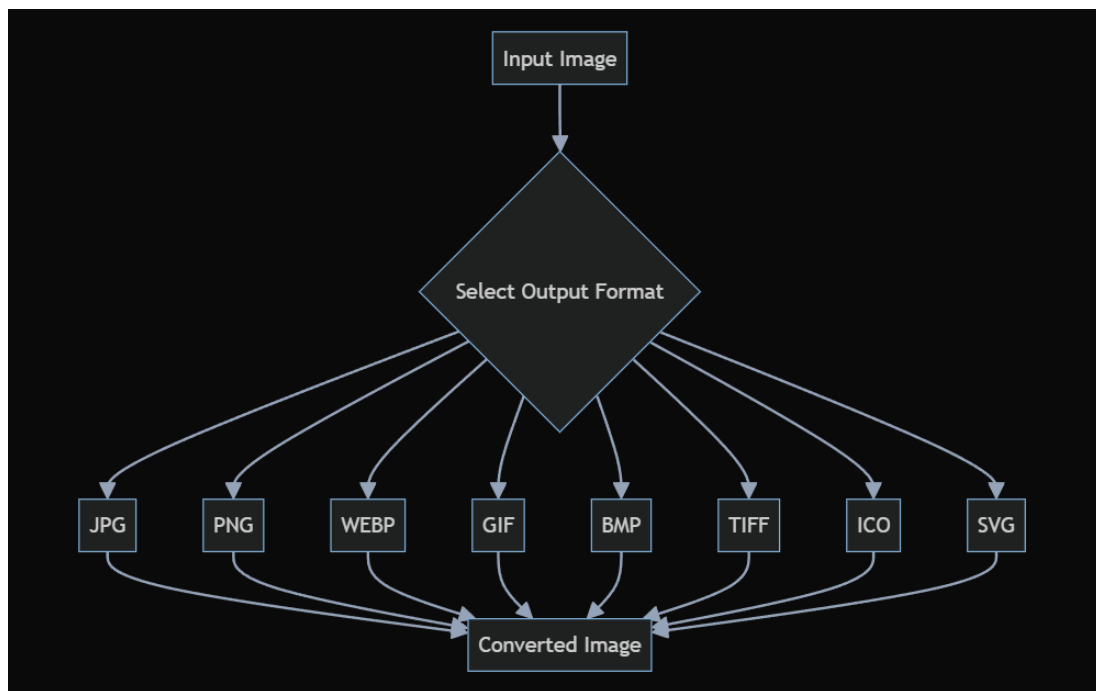
9.4 Image Converter

The **Image Converter** provides format conversion between different image types.

Quill supports conversion to formats such as:

- JPG
- JPEG
- PNG
- WEBP
- GIF
- BMP
- TIFF
- ICO
- SVG

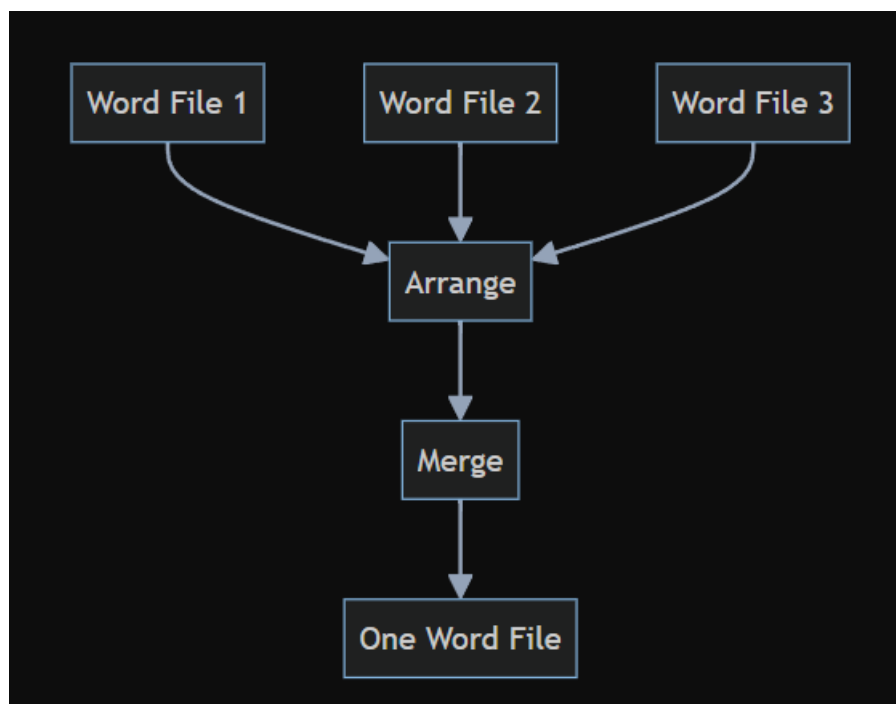
This allows users to change an image into a format that is more appropriate for a particular application or workflow.



9.5 Word Merge

The **Word Merge** tool combines multiple Word documents into a single document.

It can be useful when separate sections of a report, assignment, or project documentation have been prepared as individual Word files.

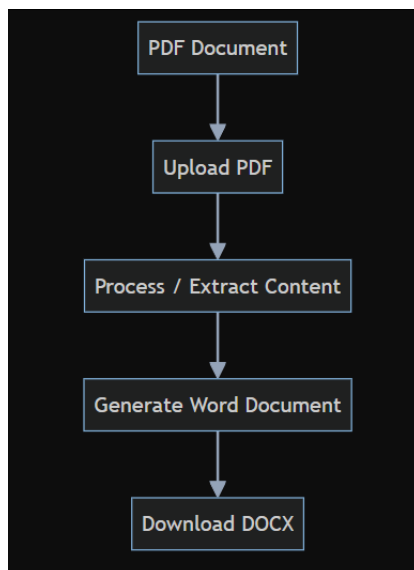


As with PDF merging, arranging the files before processing is important because the selected order determines how the final document is assembled.

9.6 PDF to Word

The **PDF to Word** tool converts a PDF document into an editable Word document.

This is useful when a user receives information in PDF format but needs to make further edits in Microsoft Word or another compatible word-processing application.



The tool is intended to reduce the need for manually recreating document content.

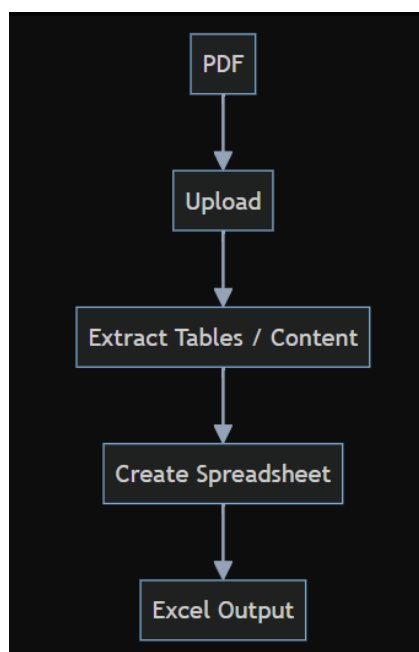
9.7 PDF to Excel

The **PDF to Excel** tool is intended for documents containing structured information such as tables.

It extracts relevant PDF content and prepares it for use in an Excel spreadsheet.

This can be useful for:

- tabular reports,
- financial records,
- lists,
- structured data,
- table-based PDF documents.



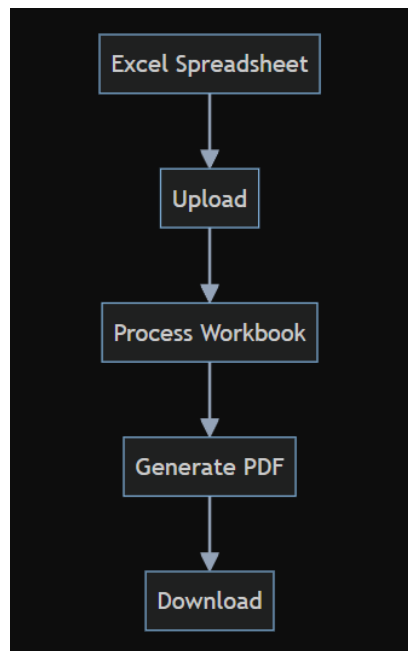
The main benefit is moving information from a fixed document format into a spreadsheet-oriented format.

9.8 Excel to PDF

The **Excel to PDF** tool converts spreadsheet files into PDF documents.

This can be useful when an Excel report needs to be:

- shared,
- submitted,
- printed,
- archived,
- viewed without spreadsheet editing software.



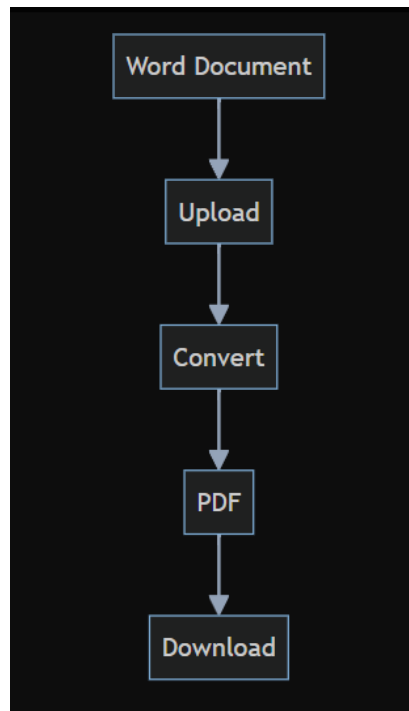
The resulting PDF provides a document-oriented version of the spreadsheet.

9.9 Word to PDF

The **Word to PDF** tool converts Word documents into PDF files.

This is commonly useful for final submission of:

- reports,
- resumes,
- assignments,
- project documentation,
- official documents.

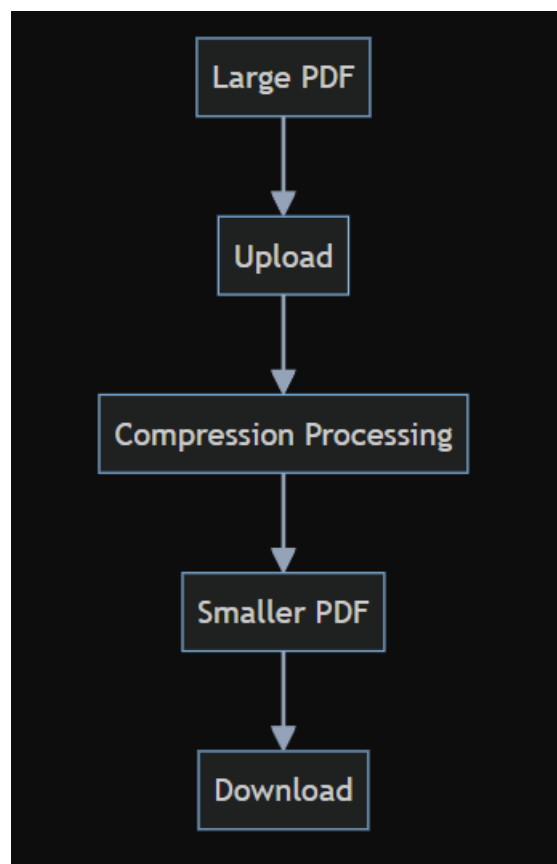


This provides a simple way to move from an editable document to a more fixed, shareable format.

9.10 PDF Compress

The **PDF Compress** tool reduces the file size of a PDF.

Large PDF files can be difficult to upload, share, store, or send through communication platforms. Compression helps make these files more manageable.

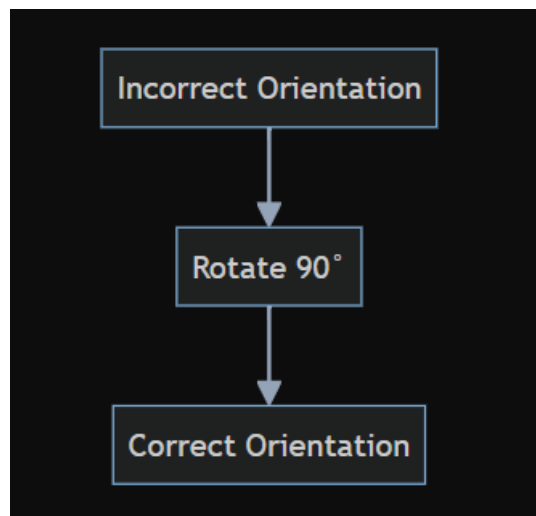


The purpose of this tool is to reduce unnecessary file size while maintaining usable document quality.

9.11 Rotate PDF

The **Rotate PDF** tool changes the orientation of PDF pages.

This is useful when scanned or imported pages appear sideways or upside down.



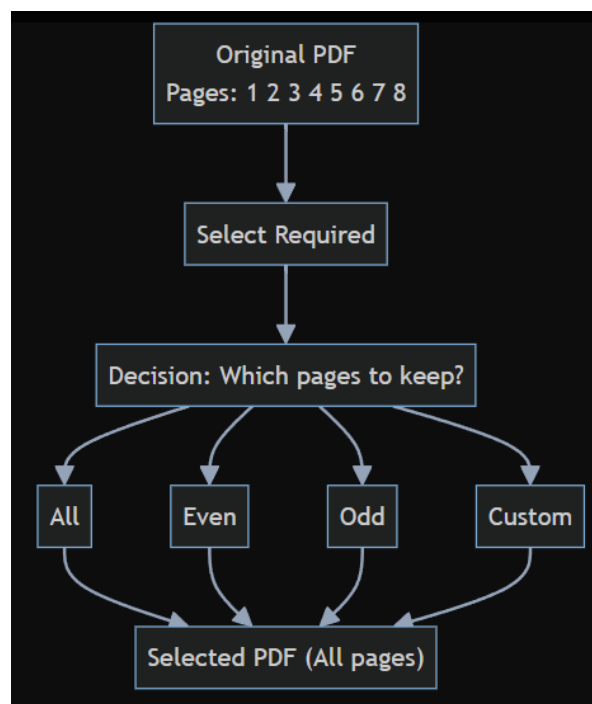
The operation can be applied to the PDF pages according to the selected rotation requirement.

9.12 Split PDF

The **Split PDF** tool allows users to extract specific parts of a PDF rather than keeping the complete document.

The module supports different page-selection approaches, including:

- All pages
- Even pages
- Odd pages
- Custom page selection



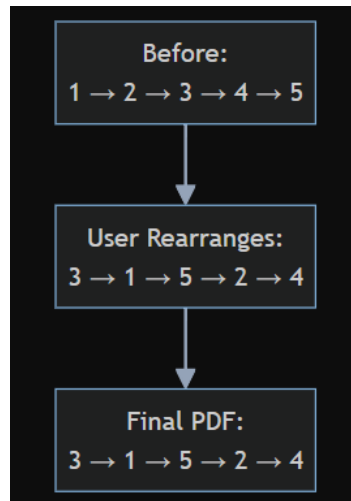
This provides flexibility when the user needs only particular pages from a larger document.

9.13 Organize PDF

The **Organize PDF** tool is used to rearrange and manage pages inside a PDF.

Users can:

- reorder pages,
- remove pages,
- arrange pages into a required sequence.

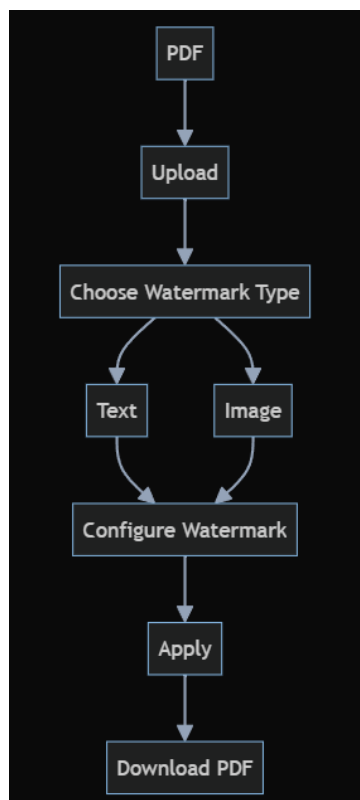


This is particularly useful for reports, scanned documents, and mixed page collections where the page order needs to be corrected.

9.14 Watermark PDF

The **Watermark PDF** tool adds a watermark to a PDF document.

The watermark can be used to identify or mark a document. Quill supports adding **text or image-based watermarks**.



Possible uses include adding ownership information, document labels, or other identifying marks.

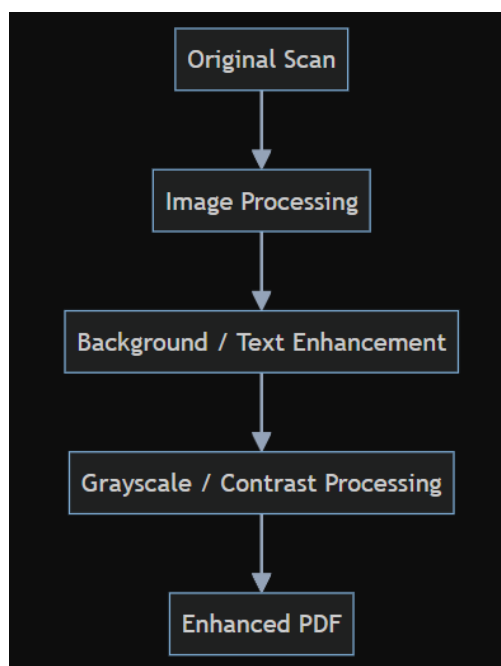
9.15 PDF Color Enhance

The **PDF Color Enhance** tool is intended to improve the visual appearance of document pages.

The available processing includes options such as:

- **Whitening the background**
- **Darkening / emphasizing black text**
- **Grayscale**
- **High contrast**

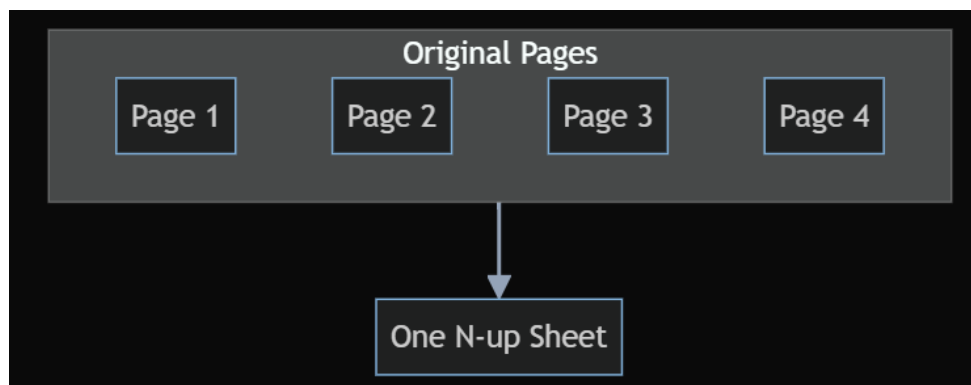
This can be especially useful for scanned documents where the background may appear grey or the text may not be sufficiently clear.



9.16 N-up PDF

The **N-up PDF** tool places multiple PDF pages or images onto a **single sheet**.

This is useful when the user wants to reduce the number of printed pages or create a compact document layout.

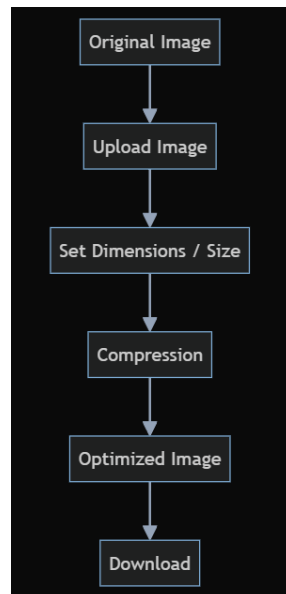


The tool can arrange multiple pages according to the selected N-up configuration.

9.17 Image Compressor

The **Image Compressor** provides image-size processing for cases where an image needs to fit a particular size or dimension requirement.

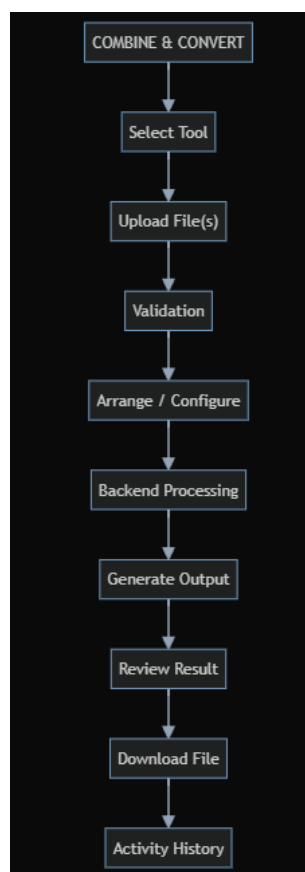
The tool can be used when an image needs to be compressed or adjusted to specific dimensions and file-size requirements.



This is useful for online forms, profile images, document submissions, and other situations where file-size restrictions apply.

9.18 Common Combine & Convert Workflow

Although every tool performs a different operation, most tools in this module follow a common processing pattern:

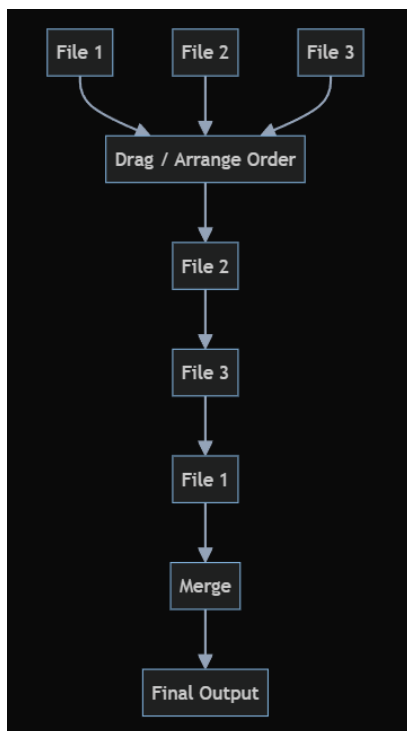


The project documentation similarly describes the file workflow as choosing a tool, uploading files, validating them, arranging/converting/optimizing, processing, downloading the result, and recording the activity in history.

9.19 File Reordering in Merge Operations

For merge-based operations such as **PDF Merge**, **Word Merge**, and **Image Merge**, the order of the selected files matters.

A user can organize the files before starting the merge:



This makes the final output predictable because the backend processes the files according to the order supplied by the frontend.

9.20 Module Summary

The Combine & Convert module therefore covers three major types of operations:

Category	Main Operations
PDF	Merge, Split, Compress, Rotate, Organize, Watermark, Color Enhance, N-up
Images	Merge, Image to PDF, Image Converter, Image Compressor
Documents	Word Merge, PDF to Word, PDF to Excel, Excel to PDF, Word to PDF

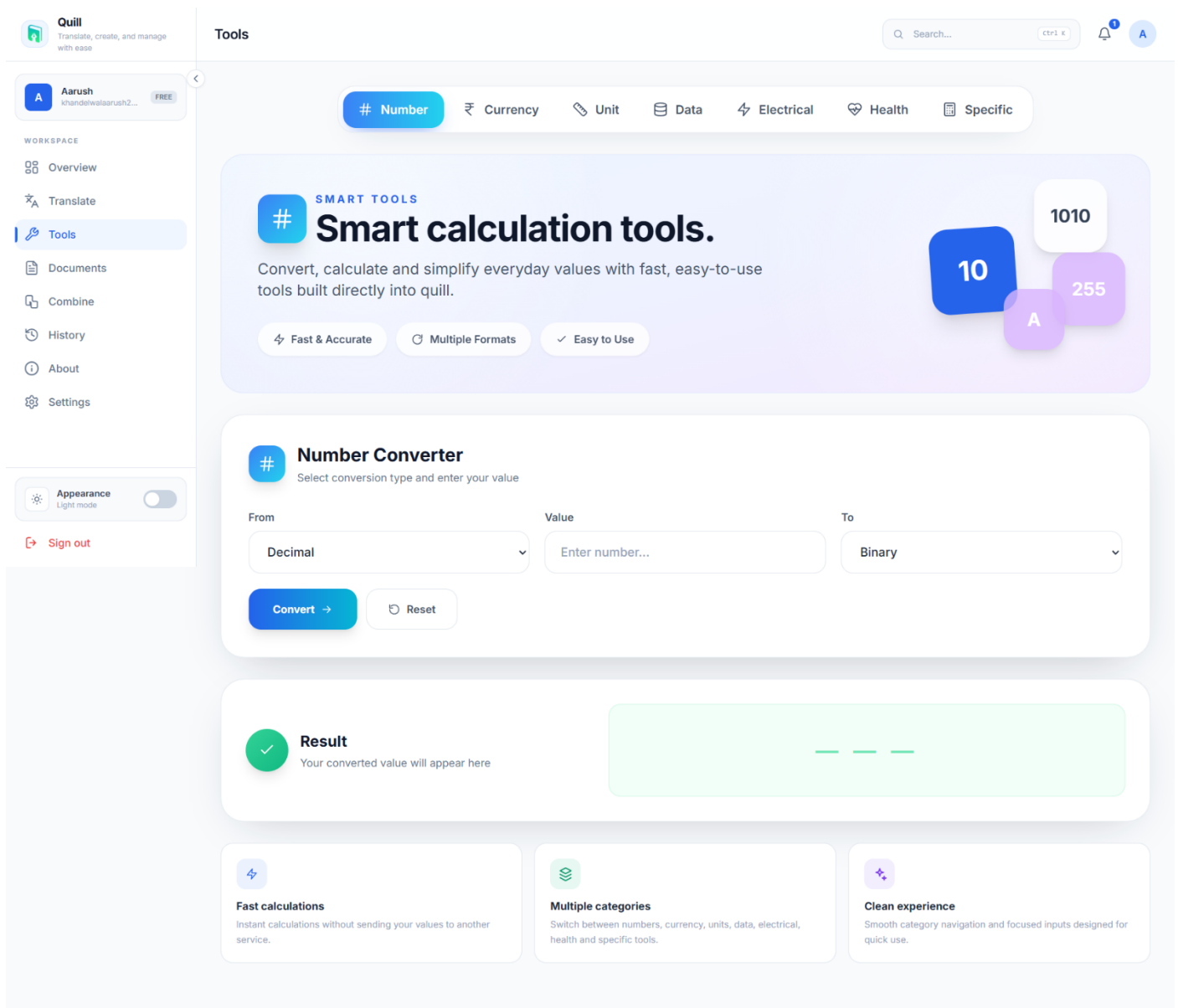
These tools are documented as part of Quill's Combine & Convert feature set

10. Smart Tools & Utility Module

The **Smart Tools & Utility Module** is another important part of Quill. While the Translation and Document modules focus mainly on language and document workflows, the Smart Tools section provides a collection of **practical utility tools for everyday calculations, conversions, and specific use cases**.

Quill organizes these utilities into separate categories so that users can find the required tool more easily instead of searching through unrelated options. The documented categories are **Number, Currency, Unit, Data, Electrical, Health, and Specific**.

The main idea behind this module is to keep frequently required utility operations available inside the same Quill workspace.

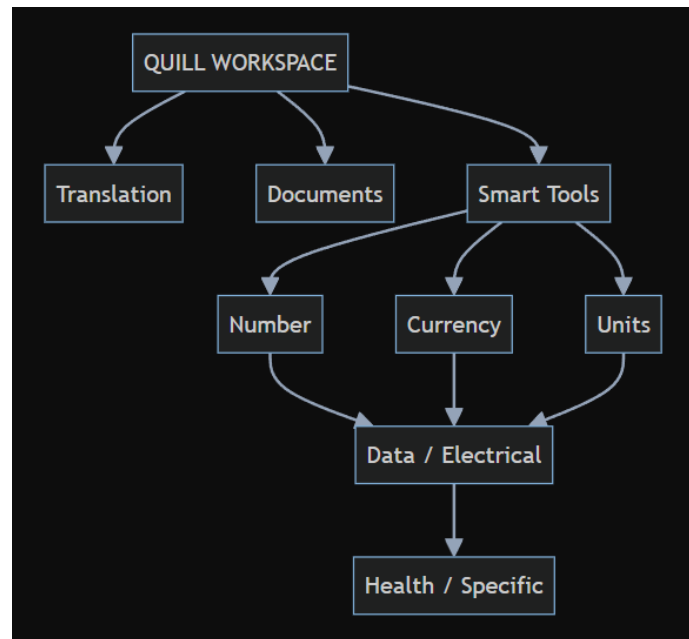


10.1 Purpose of the Smart Tools Module

The Smart Tools module extends Quill beyond translation and document processing.

For example, a user working on a document or project may also need to perform a calculation or conversion during the same task. Instead of leaving Quill and opening another website, the required utility can be accessed from the Smart Tools section.

The module therefore follows the same overall idea of Quill:

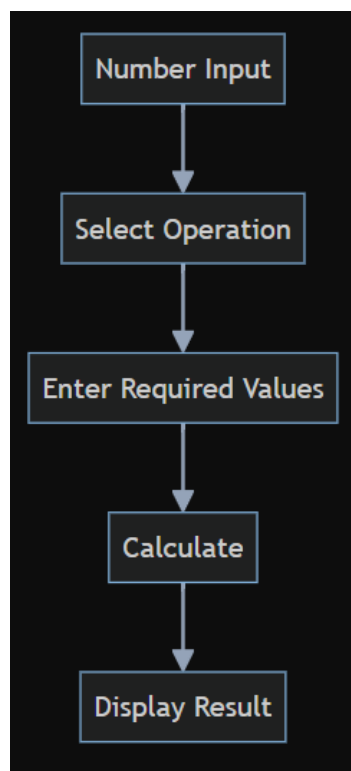


10.2 Number Tools

The **Number** category contains tools related to numerical operations.

This category is intended for situations where the user needs to work with numerical values as part of a calculation or utility workflow.

Instead of having number-based utilities distributed across different pages, Quill keeps them under one category.

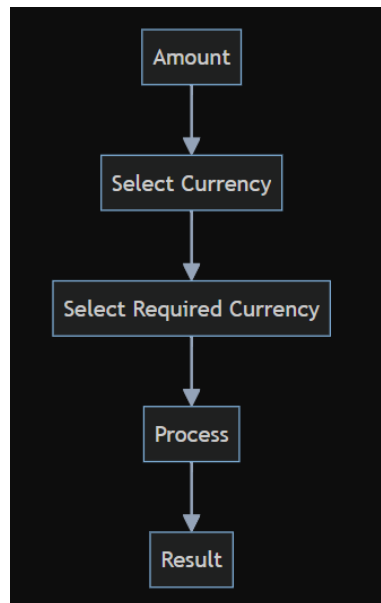


The result can then be used by the user as part of the current task.

10.3 Currency Tools

The **Currency** category is designed for utility operations involving monetary values and currencies.

This is useful when working with values that need to be interpreted or converted between different currency representations.

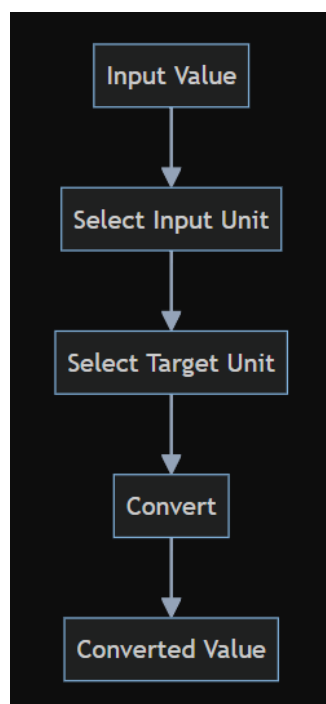


Keeping currency-related functionality inside Quill is useful because financial values can often appear alongside documents, reports, business information, and other productivity tasks.

10.4 Unit Tools

The **Unit** category groups utilities related to measurement units.

This allows unit-related operations to be handled through a dedicated utility section rather than mixing them with other calculations.

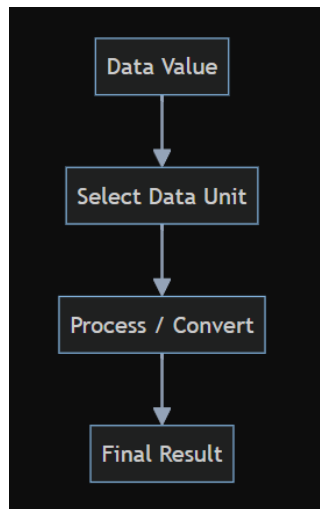


This category makes the utility section useful for users who frequently work with values expressed in different measurement systems.

10.5 Data Tools

The **Data** category is intended for utilities related to data measurements and data-related values.

This is particularly relevant when working with digital files, storage values, computing-related quantities, or other situations where data units need to be handled.

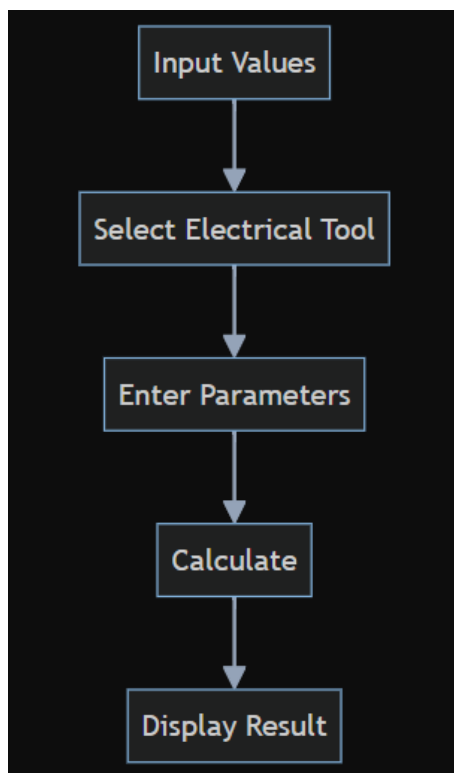


By grouping these operations under **Data**, Quill keeps computing-related utilities separate from general measurement and currency utilities.

10.6 Electrical Tools

The **Electrical** category provides utilities intended for electrical and electronics-related calculations or operations.

This makes the Smart Tools module more useful for technical users who may need quick electrical calculations while working on a project or document.

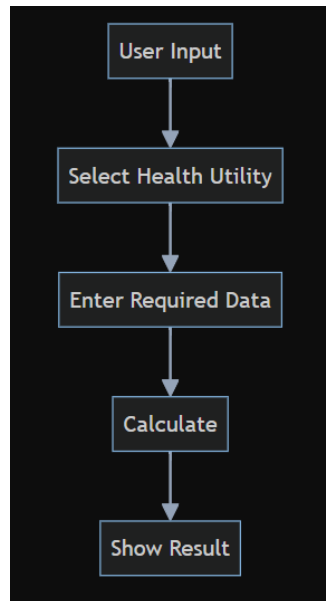


The important design point is that electrical utilities are kept inside their own category rather than mixed with general-purpose tools.

10.7 Health Tools

The **Health** category is intended for utility calculations related to health-oriented use cases.

These tools are grouped separately so users can quickly identify them within the Smart Tools workspace.

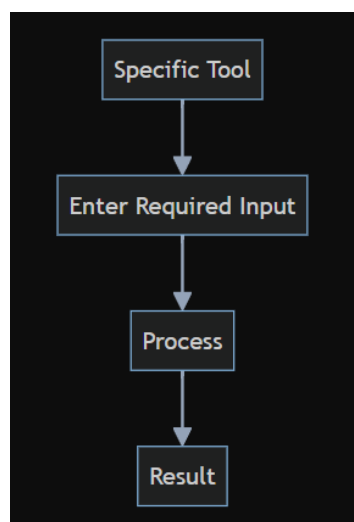


For the project report, this category should be described as a **utility section**, not as a replacement for professional medical advice.

10.8 Specific Tools

The **Specific** category is used for utilities that do not naturally fit into the general Number, Currency, Unit, Data, Electrical, or Health groups.

It provides a separate place for more specialized utility requirements.



This category also gives the overall Smart Tools section room to accommodate additional practical utilities without changing the main category structure.

10.9 Category-Based Organization

One of the useful design decisions in this module is the **category-based organization**.

Instead of displaying every utility in one large list, Quill separates them into logical groups:

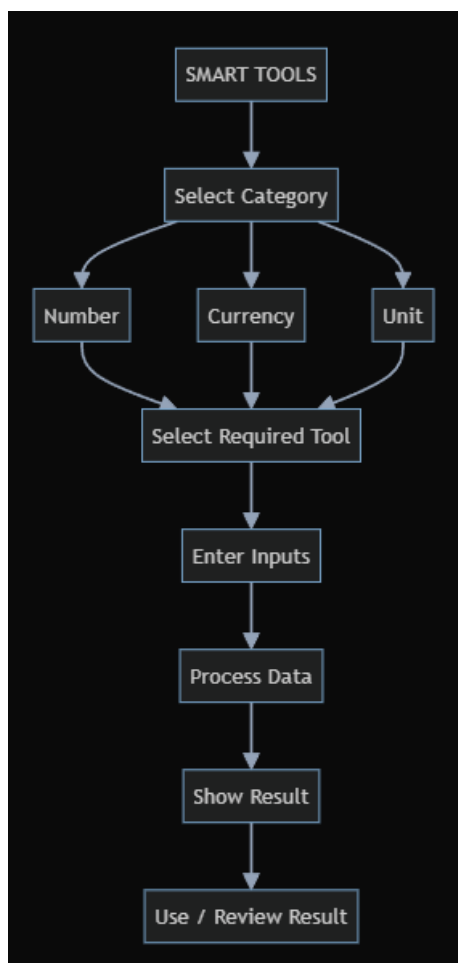
Category	Purpose
Number	Numerical utility operations
Currency	Currency and monetary-value utilities
Unit	Measurement-related operations
Data	Digital data-related utilities
Electrical	Electrical and technical utilities
Health	Health-oriented utility calculations
Specific	Specialized utilities that do not fit other groups

These seven categories are documented as the main Smart Tools groups in Quill.

This organization helps reduce the time required to find a particular tool.

10.10 General Smart Tool Workflow

Although different utility categories perform different tasks, the overall interaction remains similar.

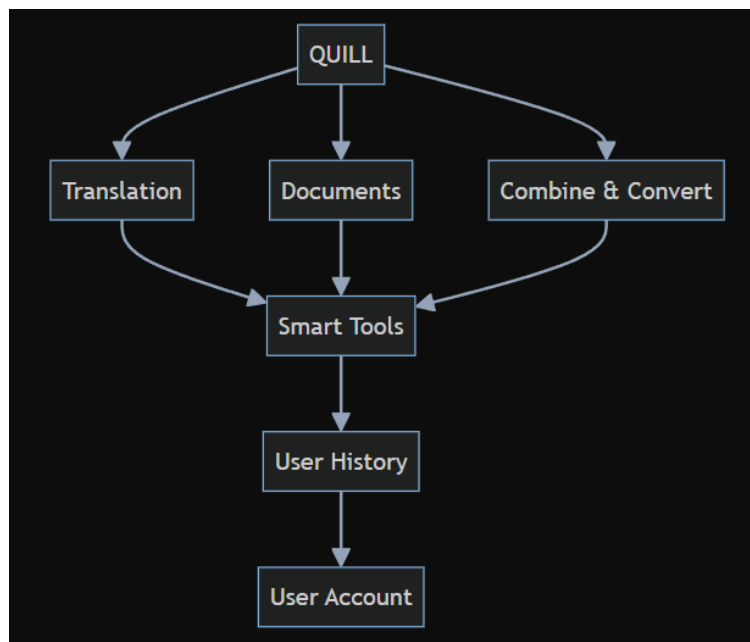


The interaction pattern is intentionally simple: **select** → **enter input** → **process** → **view result**.

10.11 Smart Tools as Part of Quill

The Smart Tools section is not an isolated utility website inside Quill. It is connected to the wider application.

The overall Quill workspace contains:



This means the utility module fits into the same workspace as the translation, document, and file-processing features. Quill's documented product overview also places Smart Tools alongside Translation, Documents, Combine & Convert, History, Settings, and Account features.

10.12 Benefits of the Smart Tools Module

The Smart Tools module provides several practical benefits within the overall design of Quill.

Centralized utilities

Different utility categories are available from the same application instead of requiring separate websites.

Clear organization

The categories make it easier to locate tools based on the type of task.

Consistent interface

The utility section follows the same workspace concept used throughout Quill.

Faster workflow

A user can perform a utility operation while already working with documents, translations, or files.

Expandable structure

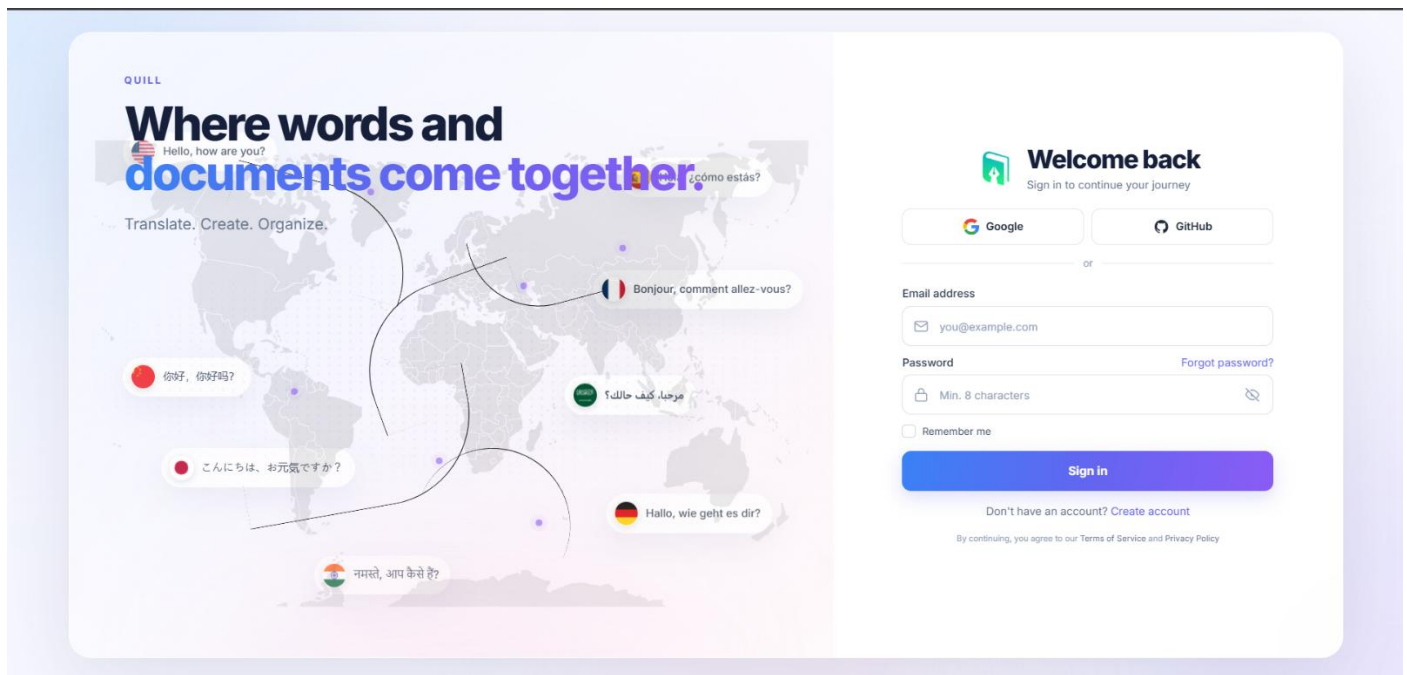
The category-based design provides a clear structure for adding more utility tools in the future.

11. User Authentication & Security

Quill includes a user authentication system to make the workspace account-based and to protect features that depend on the logged-in user. Authentication is handled through the Django backend, while the frontend manages the authenticated session and uses tokens when communicating with protected APIs.

The documented authentication flow includes **registration, email OTP verification, login, optional two-factor authentication, JWT access and refresh tokens, and token refresh during an active session.**

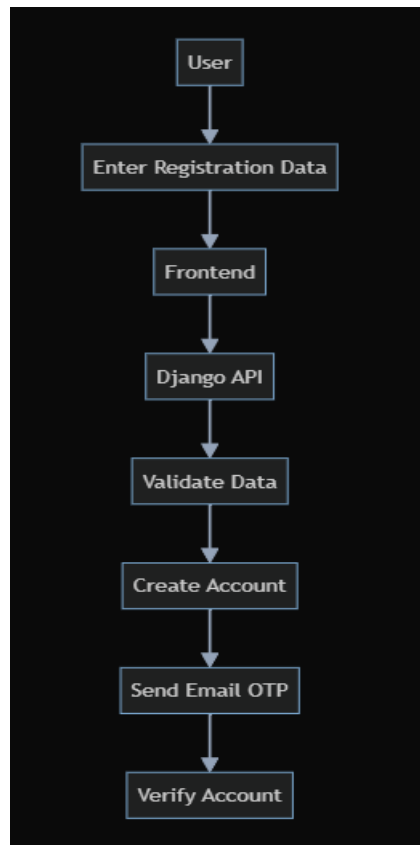
The main purpose of this system is to ensure that user-specific information such as account settings and activity history is associated with the correct user.



11.1 User Registration

The registration process starts when a new user provides the required account information through the registration interface.

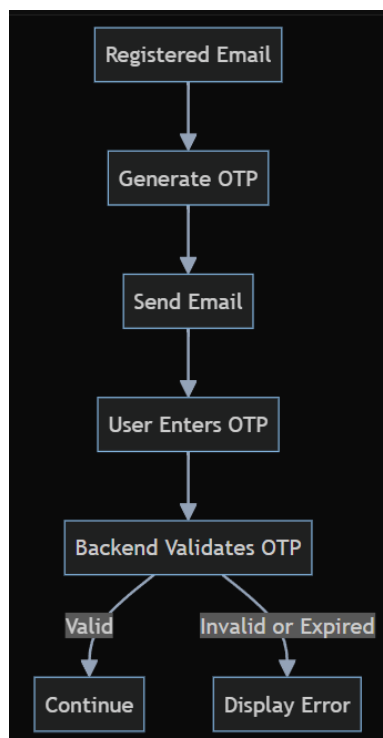
The backend validates the submitted information using a registration serializer. If the information is valid, the user account is created and an OTP is sent to the registered email address for verification. The account creation event is also recorded as user activity.



11.2 OTP-Based Verification

Quill uses **One-Time Password (OTP)** verification as part of its authentication workflow.

During registration, the user receives an OTP through email. The submitted OTP is verified by the backend before completing the corresponding verification flow. The same OTP mechanism can also be used for other account-related authentication operations.

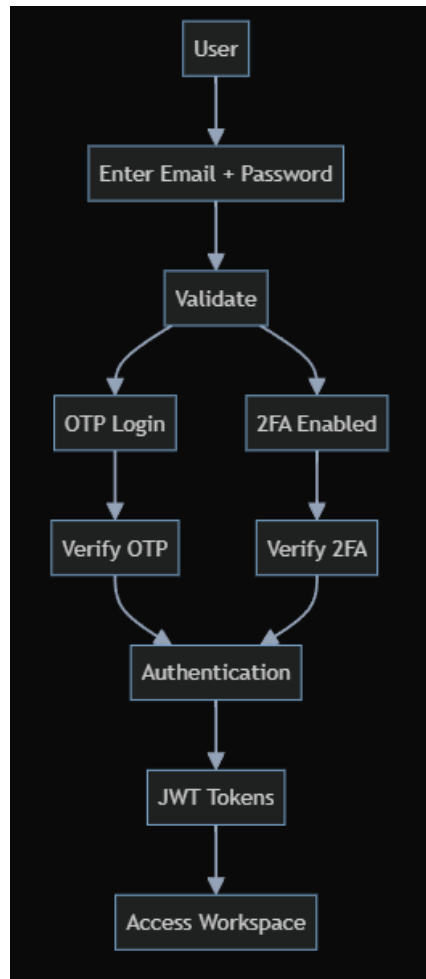


The backend's OTP verification code checks the OTP against the user's configured secret and rejects invalid or expired codes.

11.3 User Login

The login workflow begins with the user's credentials.

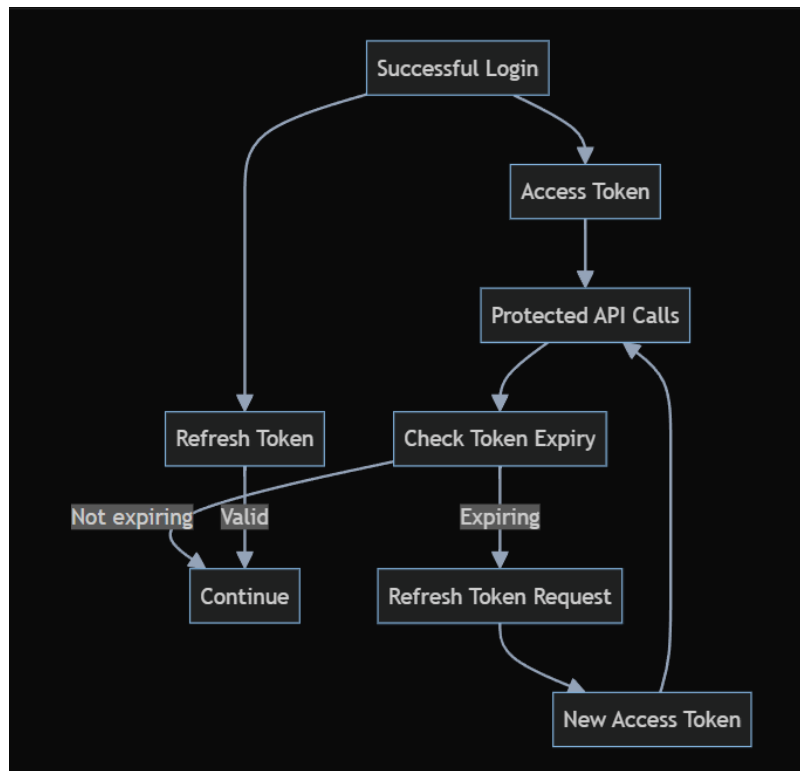
The backend validates the login information. Depending on the account state, Quill can then require an email OTP or proceed through the configured two-factor authentication flow.



11.4 JWT Authentication

After successful authentication, Quill uses **JWT access and refresh tokens**.

The access token is used for authenticated API requests, while the refresh token is used to obtain a new access token when the current access token is close to expiry. This approach allows an active session to continue without requiring the user to log in again every time an access token expires.

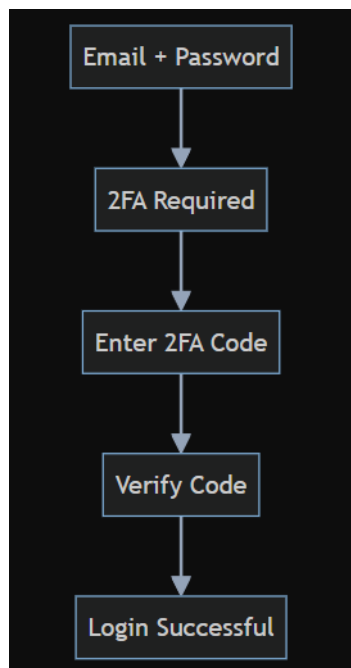


The frontend includes logic to check token expiry and request token refresh when appropriate.

11.5 Two-Factor Authentication

Quill also supports **optional Two-Factor Authentication (2FA)**.

The backend provides a 2FA setup workflow that creates a secret for the user and generates a provisioning URI using pyotp. This allows the account to be configured for time-based one-time-password authentication.



The login endpoint explicitly checks whether two-factor authentication is enabled for the account and returns a response indicating that 2FA verification is required.

Quill also contains an email fallback mechanism for sending a current 2FA code to the registered email address when the fallback flow is used.

11.6 Enabling and Disabling 2FA

The account can enable or disable 2FA through the authenticated backend.

When enabling 2FA, the backend creates an OTP secret if required, marks OTP as enabled, and records the event as user activity. When disabling it, the OTP configuration is removed from the account state.

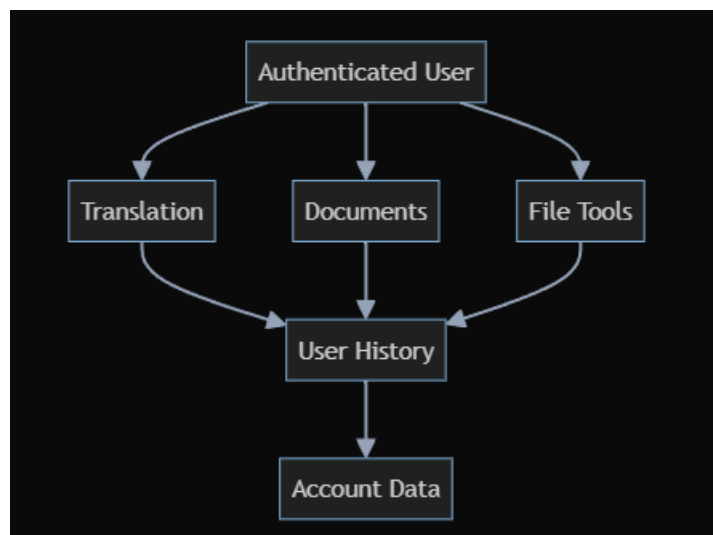
This gives the user control over whether two-factor authentication is active on the account.

11.7 Authentication and User-Specific Data

Authentication is also important for Quill's account-linked features.

The documented system states that authenticated history and account actions are protected by user identity. This allows activity such as translations, document processing, and combine operations to be associated with the appropriate account.

Conceptually:

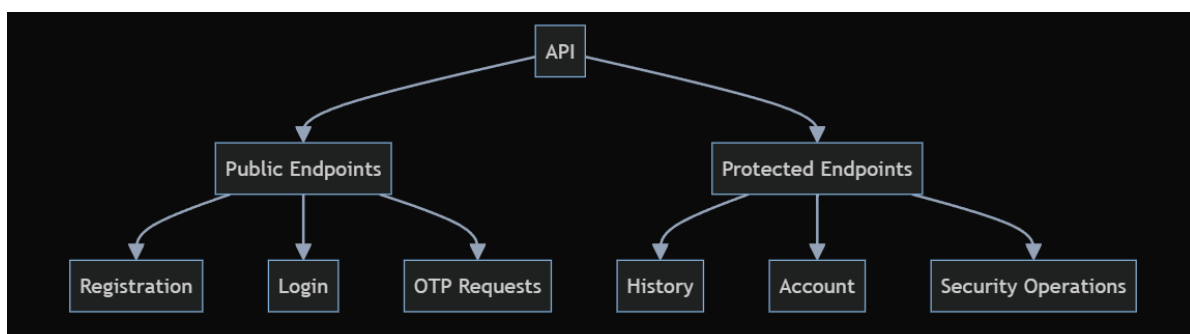


This is important because the History module should not treat all application activity as belonging to a single shared user.

11.8 API Access Control

Protected backend operations use authenticated access control. The backend code contains views using authenticated permissions such as `IsAuthenticated`, which ensures that certain account-specific operations can only be accessed by an authenticated user.

For example, the 2FA setup endpoint requires an authenticated user, while public registration and login endpoints are available without an existing authenticated session.



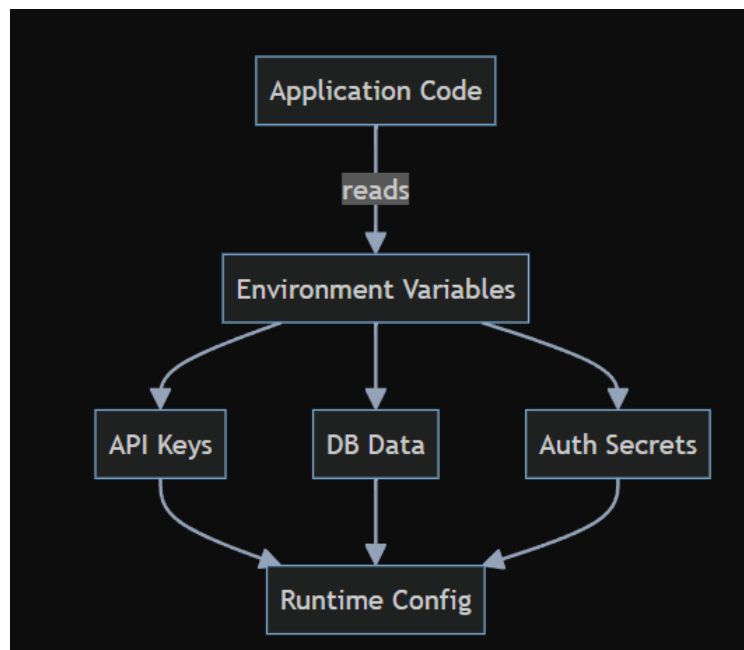
11.9 Secure Handling of Secrets

Quill uses **environment-based configuration** for sensitive credentials instead of placing these values directly in source code.

The documented sensitive configuration includes items such as:

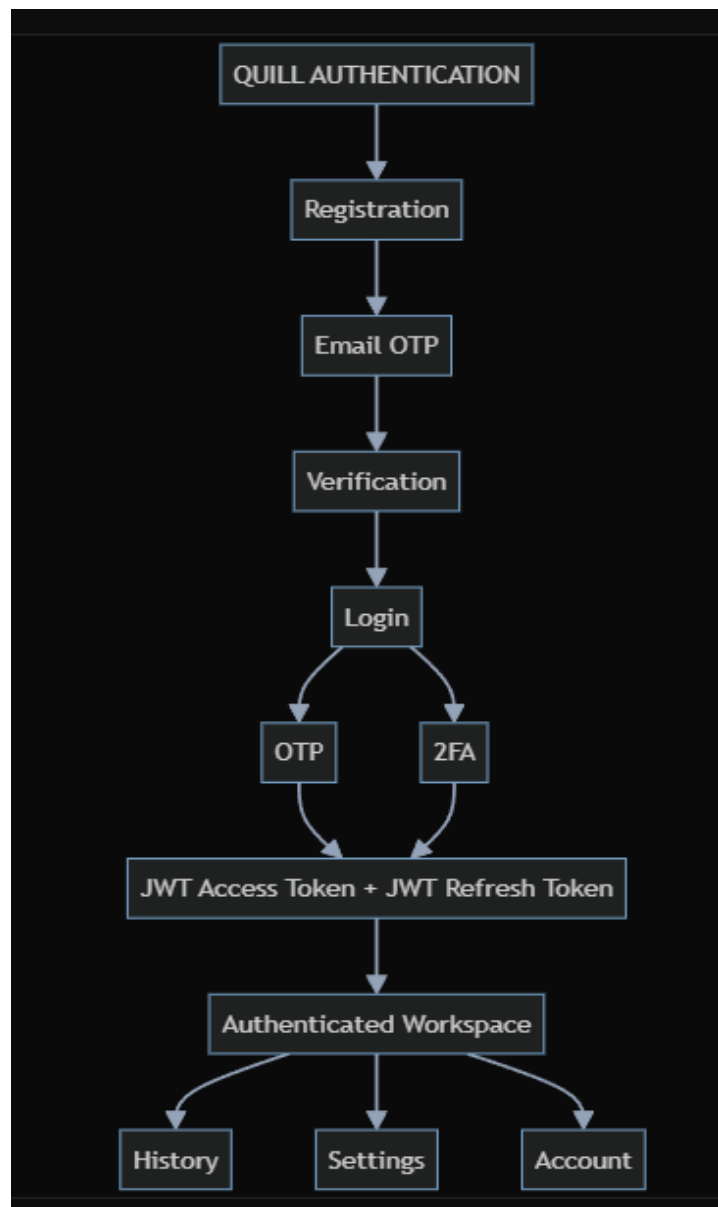
- API keys
- SMTP credentials
- OAuth secrets
- JWT secrets
- database passwords
- private VAPID keys

The project documentation specifically states that .env files and these credentials should not be committed to the Git repository. Instead, secrets should be supplied through the hosting environment.



11.10 Authentication Security Flow

The complete authentication process can be summarized as:



12. History, Activity & User Management

The **History, Activity & User Management** section in Quill provides a central place where users can review their previous work and manage account-related activity. Since Quill contains several independent modules such as translation, document processing, and file operations, keeping their activities in one place makes it easier for users to find previous work.

Quill's documented History feature covers **translation, tools, documents, and Combine & Convert activity**. Users can search and filter records, preview information, download generated files, delete history entries, and use history export functionality. The history is linked to the authenticated user's account.

The screenshot displays the Quill application interface, specifically the 'History' section. On the left is a sidebar with navigation links: Overview, Translate, Tools, Documents, Combine, History (selected), About, and Settings. Below the sidebar are options for 'Appearance' (Light mode) and a 'Sign out' button. The main content area is titled 'Your Complete Activity Timeline'. It features a search bar, a 'Filters' button, and a refresh icon. Below these are five summary cards: 'ALL ACTIVITY' (5 Total activities), 'TRANSLATE' (4 Completed translations), 'TOOLS' (0 Tool operations), 'DOCUMENTS' (0 Document activities), and 'COMBINE' (1 Combined operations). A tabbed interface below the cards shows 'All Activity' (5), 'Translate' (4), 'Tools' (0), 'Documents' (0), and 'Combine' (1). The 'All Activity' tab is active, showing an 'Activity timeline' with 'Showing 5 activities'. It includes 'Export PDF' and 'Delete All' buttons. The activity list is a table with columns: ACTIVITY, DETAILS, CATEGORY, DATE & TIME, and ACTIONS. It contains five entries: four 'Document Translation' activities and one 'Image Compressor' activity. Each entry has a 'Translate' or 'Combine' button and a 'Live' status. At the bottom, a status message says 'History is synced with your account.' and '5 results' are shown.

ACTIVITY	DETAILS	CATEGORY	DATE & TIME	ACTIONS
Document Translation	Translated page 1 from auto to hi auto → hi Document Translation 843 chars	Translate	Sep 17, 2026 05:02 PM	Translate, Live, Eye, Delete
Document Translation	Translated page 1 from auto to hi auto → hi Document Translation 852 chars	Translate	Sep 17, 2026 05:02 PM	Translate, Live, Eye, Delete
Document Translation	Translated page 1 from auto to hi auto → hi Document Translation 862 chars	Translate	Sep 17, 2026 05:02 PM	Translate, Live, Eye, Delete
Document Translation	Translated page 1 from auto to hi auto → hi Document Translation 860 chars	Translate	Sep 17, 2026 05:02 PM	Translate, Live, Eye, Delete
Image Compressor	Processed automeld_logo.png to automeld... Image Compressor 200.1 KB	Combine	Sep 14, 2026 11:01 PM	Combine, Live, Eye, Download, Delete

12.1 Purpose of the History Module

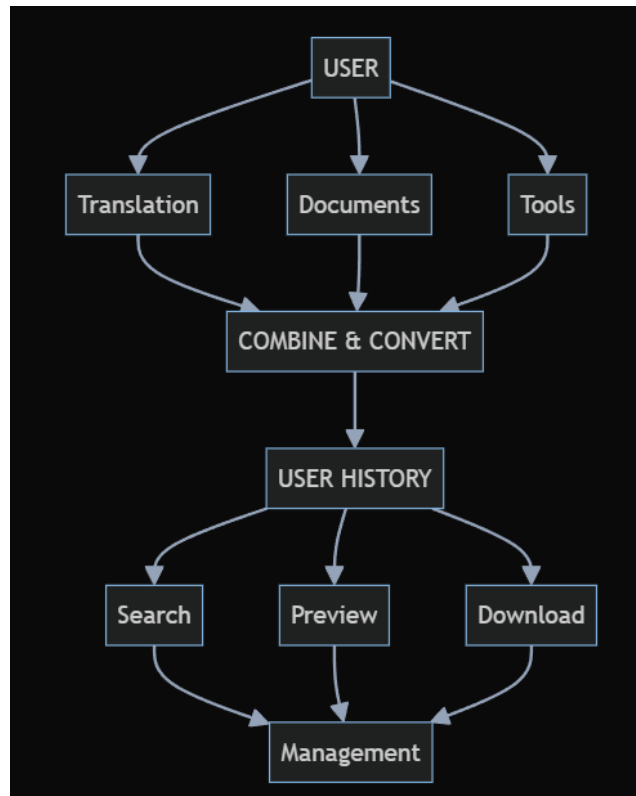
The main purpose of History is to maintain a record of work performed inside Quill.

For example, a user may perform:

- a text translation,
- a document translation,
- a PDF merge,
- a Word conversion,
- an image conversion,
- another supported tool operation.

Instead of losing track of these activities after leaving the page, the operations can appear in the user's history.

The overall idea is:



12.2 Types of Activity Stored in History

Quill's History section is designed around several major activity categories.

Translation Activity

This includes supported translation-related operations performed through the Translation workspace.

Examples include text translation and supported voice/translation workflows.

Document Activity

Document-related processing can be shown in the history so that users can return to previously processed documents.

The document workspace itself retrieves document history through an authenticated API request, which demonstrates the connection between document processing and account-based history.

Tool Activity

Operations completed through the Smart Tools section can form part of the user's activity record.

Combine & Convert Activity

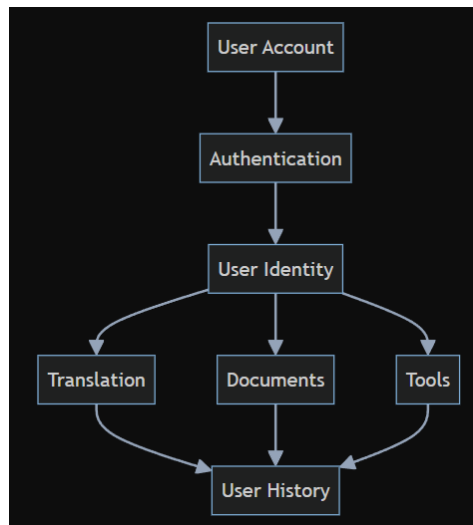
File-processing operations such as merge and conversion can also be associated with history records.

The technical documentation explicitly groups **Translate**, **Tools**, **Documents**, and **Combine** categories under the History feature.

12.3 Account-Linked History

History in Quill is associated with the authenticated user rather than being treated as a common application-wide list.

This is important in a multi-user application because one user's translations or file operations should not appear in another user's workspace.



The technical documentation states that authenticated history and account actions are protected by user identity.

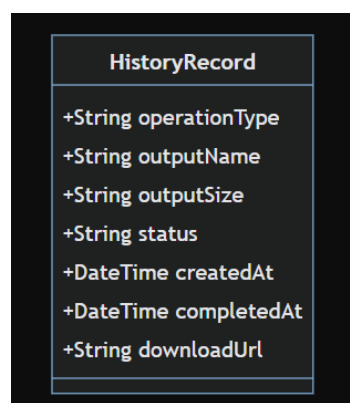
12.4 History Records

A history record contains information about the processed operation and its result.

The available file-processing code defines history information such as:

- operation type,
- output filename,
- output size,
- processing status,
- creation time,
- completion time,
- download URL.

This provides enough information for the interface to identify what operation was performed and what output was generated.

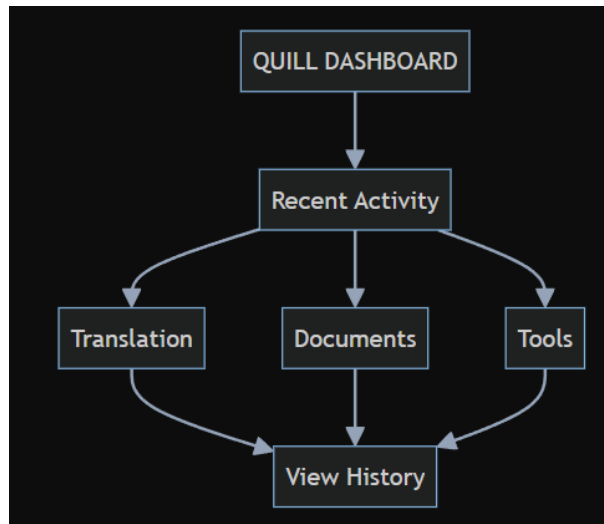


12.5 Recent Activity

Quill also provides a **Recent Activity** view on the dashboard.

This gives the user a quick view of the latest workspace actions without opening the complete History page. The dashboard includes a "Recent activity" section and provides a **View history** action for opening the full history workspace.

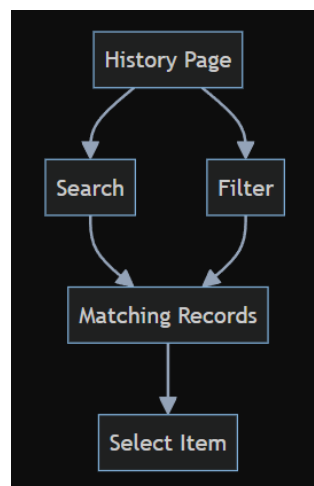
The dashboard activity is organized by type, including translation, document, and tool activity.



12.6 Search and Filtering

As the number of history records grows, searching and filtering become important.

The documented History feature supports **search and category filters**.

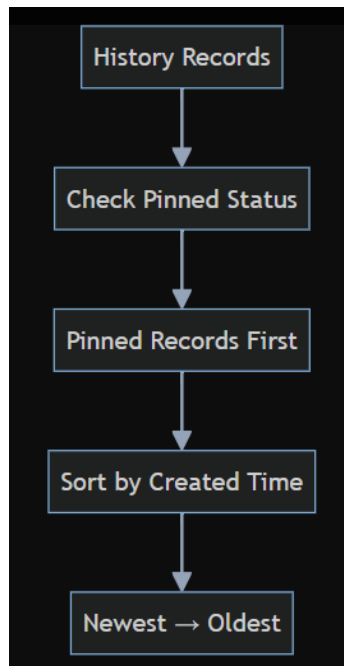


For example, users can narrow their history to a particular category instead of viewing all activities together.

12.7 Sorting History

The frontend source also maintains a sorted history list.

The current implementation first gives priority to **pinned records**, and then sorts remaining records using their creation time so newer activities appear before older ones.



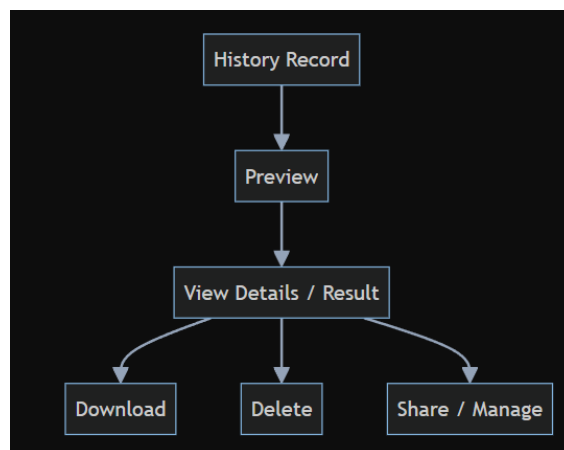
This makes frequently important records easier to access while keeping recent activities visible near the top.

12.8 Previewing History

The History section is not limited to displaying a filename.

The documented feature set includes **preview** functionality, allowing users to inspect information about a stored record before deciding what action to take.

A typical interaction can be:

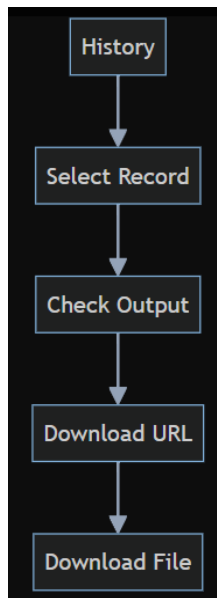


The exact available actions can vary depending on the history item and application state.

12.9 Downloading Previous Results

For operations that produce files, the history record can contain a **download URL**.

This allows the user to access the generated output again from the History section rather than repeating the complete operation. The file-processing history structure includes `download_url` for records where an output file is available.

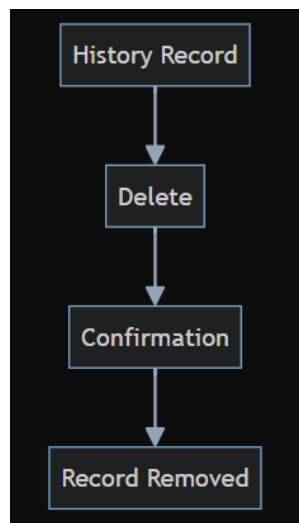


This is especially useful for PDF, Word, Excel, image, and other file-processing operations.

12.10 Deleting History

Quill also allows users to remove history records.

The documented feature set includes deletion of history entries, including support for deleting individual or stored records according to the available history interface.

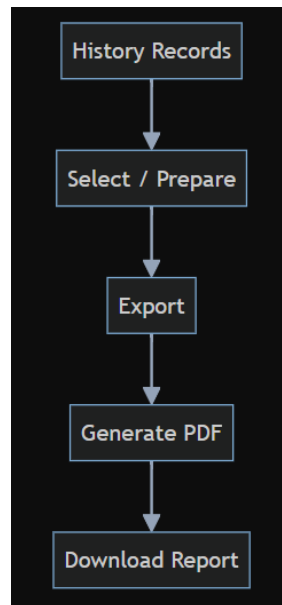


This gives users control over the information displayed in their activity history.

12.11 Exporting History

Quill's documented History feature also includes **PDF export functionality**.

This can be represented as:



This provides a convenient way to keep an external copy of activity information.

12.12 Account Activity

History is closely related to broader **account activity**.

In addition to file-processing records, the backend includes an account-activity endpoint that retrieves authenticated account activities in reverse chronological order. The implementation limits the returned list to the latest 50 records.

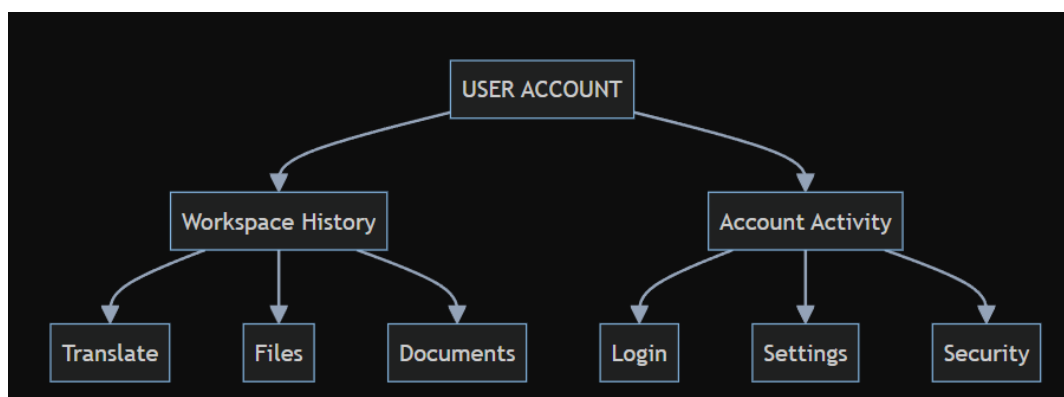
This means Quill can distinguish between:

Tool/File History

Activities related to translation, documents, conversions, and similar workspace operations.

Account Activity

Account-level actions such as login, settings changes, security actions, and other account events.



12.13 Activity Logging

Account-related events are logged by the backend through the application's activity-logging mechanism.

For example, the authentication implementation records events such as:

- account creation,
- login-related activity,

- successful verification,
- two-factor authentication changes,
- account settings changes.

The registration and login implementations explicitly call the activity logging function during these operations.

Similarly, enabling or disabling two-factor authentication also creates activity records.

This provides an additional account-level record separate from file-processing history.

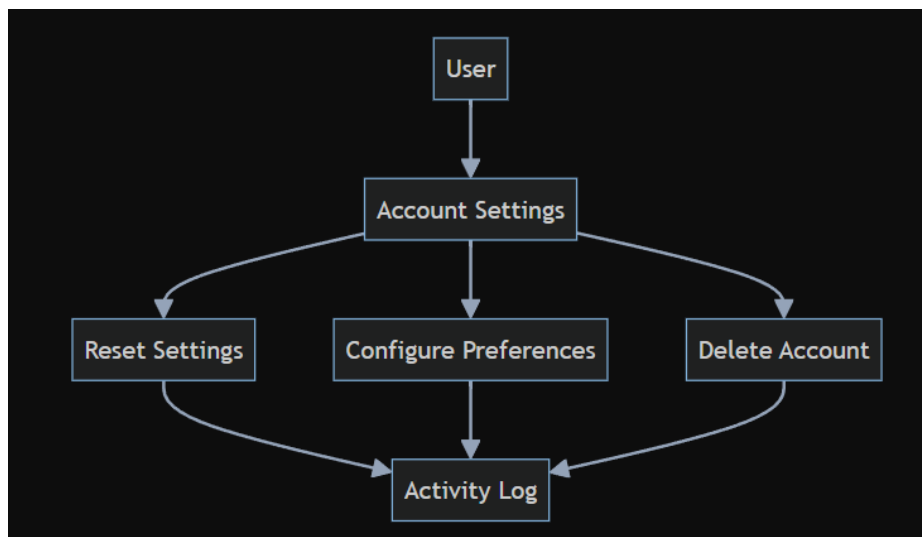
12.14 User Management

The user-management part of this section covers actions related to the user's own account and preferences.

The available backend includes authenticated account operations such as:

- resetting account settings,
- changing notification-related settings,
- managing account configuration,
- deleting the account.

For example, the backend includes an authenticated settings-reset endpoint and an authenticated account-deletion endpoint. Both operations record the corresponding activity before or during the operation.



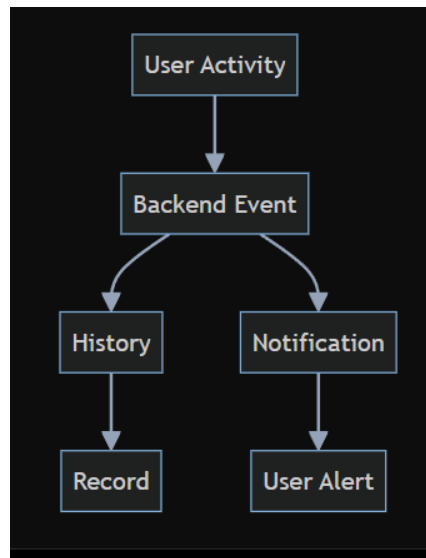
12.15 Notifications and Activity

Quill also connects activity with its notification system.

The project documentation describes in-app completion notifications and configurable notification preferences.

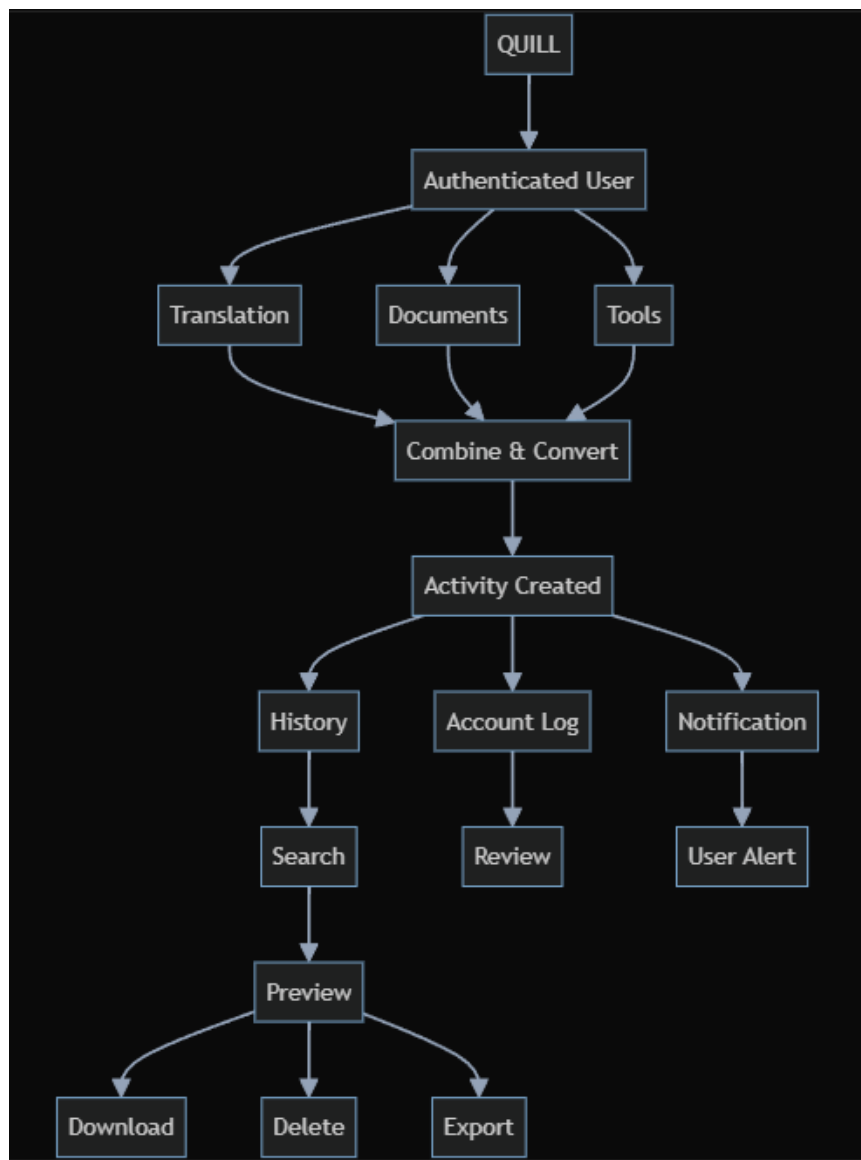
The backend also contains notification logic that can create account-aware notifications for events such as successful login verification.

This creates a useful relationship:



12.16 Overall History and User Management Flow

The complete system can be summarized as:



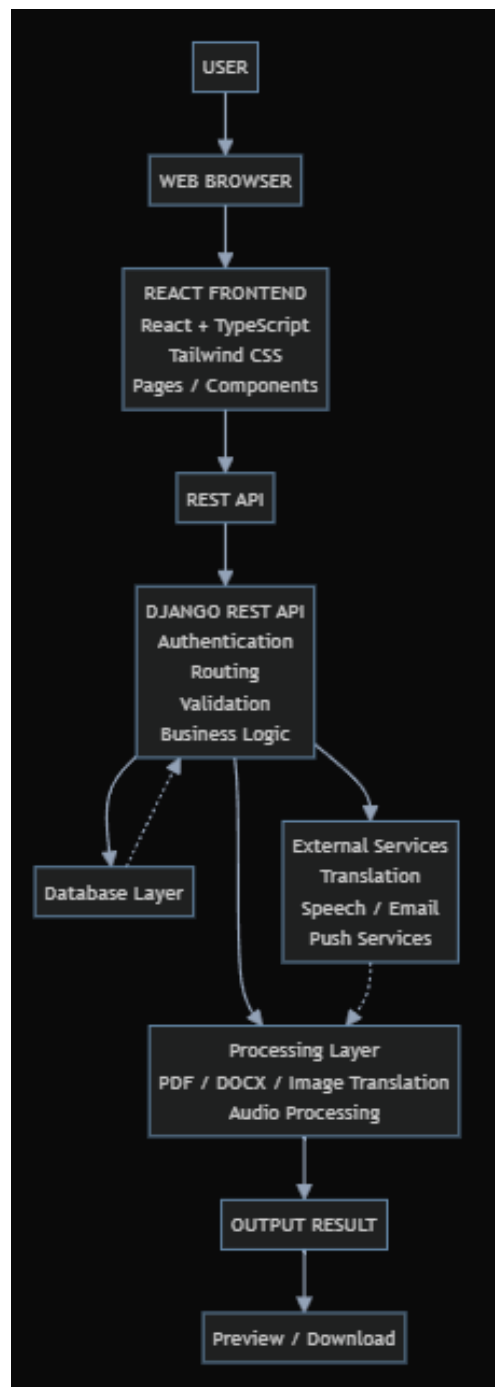
13. System Architecture

The architecture of Quill follows a **layered client-server structure**. The system separates the user interface, API and application logic, file and document processing, data persistence, and external services into different layers. This separation makes the application easier to organize because each layer has a specific responsibility.

In Quill, the **React frontend** manages the user experience and client-side interactions, while the **Django REST backend** handles authentication, API routing, validation, business logic, and processing. Database persistence and external services are accessed behind the API layer.

13.1 High-Level Architecture

The overall architecture can be represented as:



This follows the architecture described in the project's technical documentation.

13.2 Presentation Layer

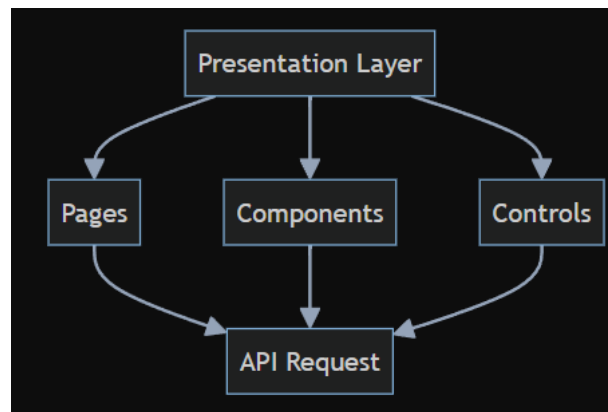
The first major layer is the **Presentation Layer**.

This is the part directly visible to the user. It is implemented using **React, TypeScript/JavaScript, and Tailwind CSS** and contains pages, upload interfaces, controls, feedback elements, and other UI components.

The presentation layer is responsible for:

- displaying the available Quill modules,
- accepting user input,
- uploading files,
- selecting languages and options,
- showing processing states,
- displaying results,
- providing preview and download controls.

Conceptually:



The frontend does not directly perform every backend operation itself. Instead, it communicates with the Django API.

13.3 API / Application Layer

The second layer is the **API and Application Layer**, implemented using **Django and Django REST Framework**.

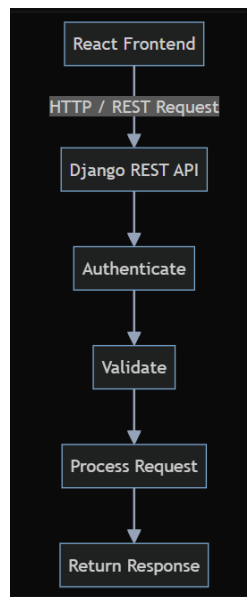
This layer acts as the main communication point between the frontend and the backend processing system.

Its responsibilities include:

- receiving REST requests,
- authentication,
- API routing,
- request validation,
- business logic,
- coordinating processing,
- returning responses to the frontend.

The technical documentation specifically assigns authentication, API routing, validation, and processing responsibilities to Django REST.

The communication flow is:



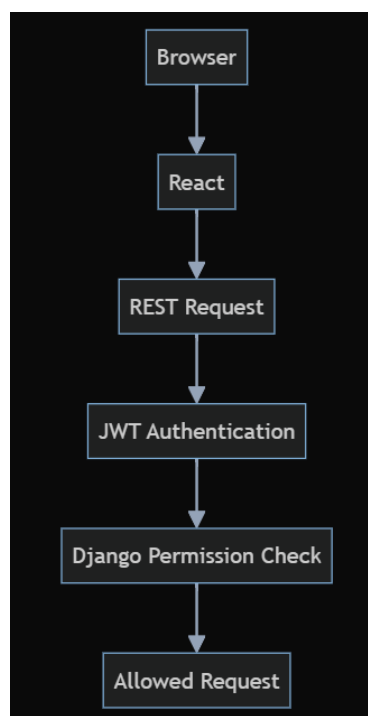
13.4 Authentication Layer

Authentication is closely connected with the API layer.

When a request requires an authenticated account, the backend verifies the user's identity before allowing protected operations.

Quill's documented authentication architecture uses JWT access and refresh tokens, while OTP and optional 2FA are also present.

This means the request path can be represented as:



For account-specific features such as History and account operations, this layer ensures that the request is associated with the authenticated user.

13.5 Processing Layer

After a request is validated, the required operation moves into the **Processing Layer**.

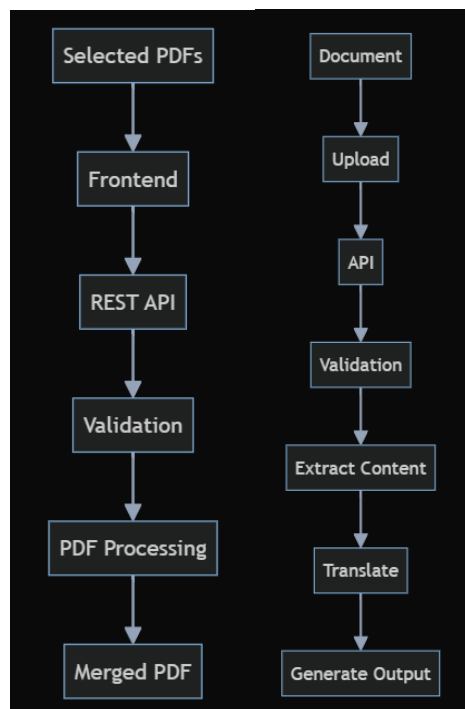
This layer handles operations such as:

- PDF processing,
- DOCX processing,
- image processing,
- translation,
- audio normalization,
- output generation.

These responsibilities are explicitly listed in the technical documentation.

For example, a PDF merge request can follow:

Similarly, a document translation request can enter the document-processing pipeline:



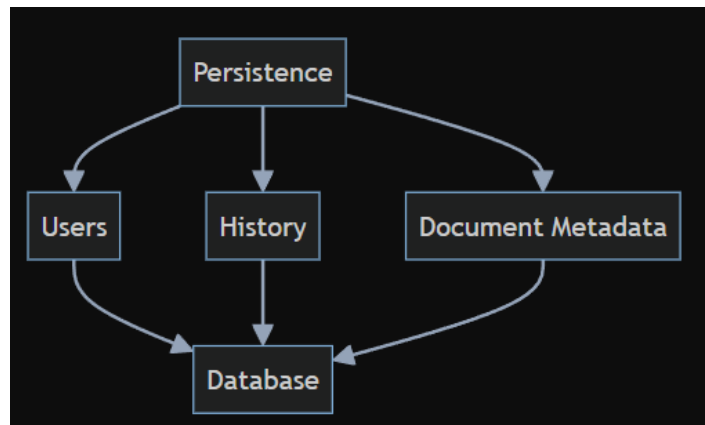
This separation keeps the heavy processing logic away from the presentation layer.

13.6 Persistence Layer

The **Persistence Layer** is responsible for storing application data.

The architecture documentation identifies this layer as containing the **relational database layer, user state, history, and document metadata**.

The persistence layer can therefore be represented as:



Examples of information that needs persistence include:

- user information,
- account state,
- history records,
- document metadata,
- processing-related information.

This layer allows information to remain available beyond a single browser session.

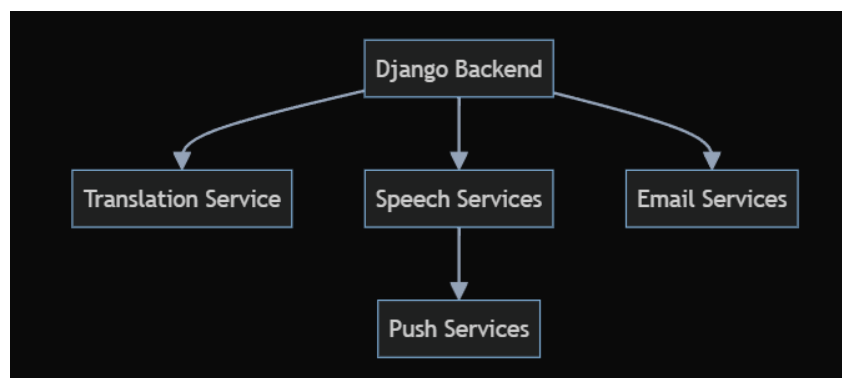
13.7 External Services Layer

Quill also interacts with external services for selected capabilities.

The technical documentation identifies external integrations for:

- **Translation**
- **Speech**
- **Email**
- **Push notifications**

The architecture therefore becomes:



The API layer acts as the controlled point through which these services are used.

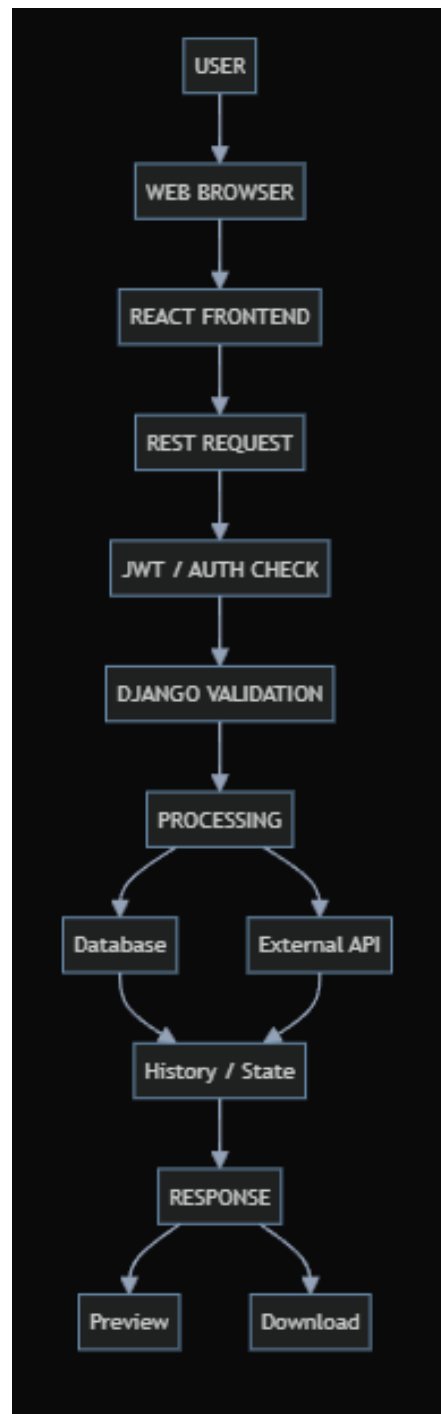
13.8 Core Request Lifecycle

A normal request in Quill follows a predictable path.

The technical documentation defines the core lifecycle as:

Browser → React → REST request → JWT/auth → Django validation → processing → persistence/history → response → download/preview.

For the report, this can be shown as a separate detailed flow:

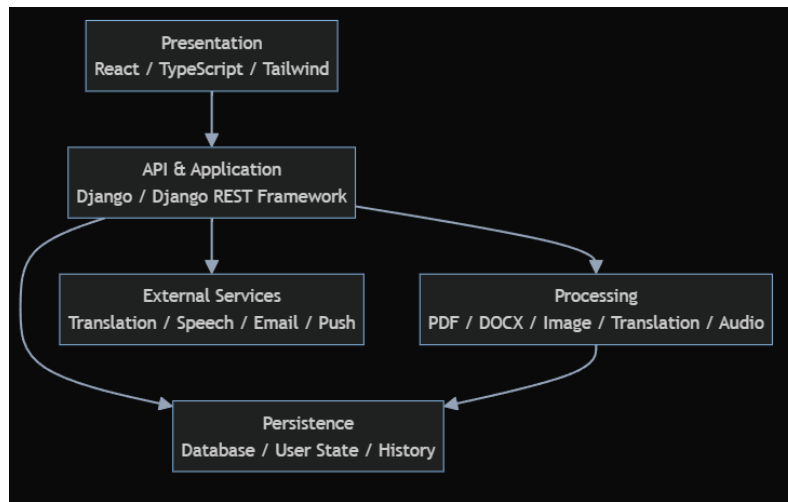


13.9 Why a Layered Architecture is Used

The layered structure keeps different responsibilities separate.

For example, the frontend should primarily handle the user's interaction, while the backend controls authentication, validation, and processing. Database operations remain behind the backend rather than being directly exposed to the browser.

This separation gives Quill a structure like:



This organization makes it easier to understand where a particular responsibility belongs.

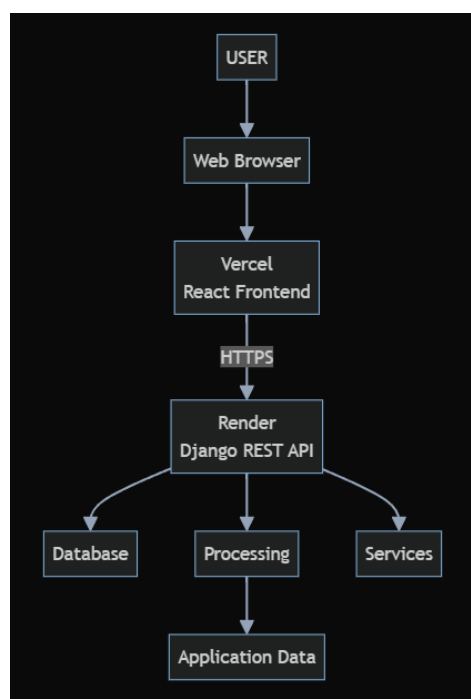
13.10 Production Deployment Architecture

The logical architecture above runs in a cloud deployment setup.

Quill's documented production deployment separates the frontend and backend:

Service	Role
Vercel	React frontend hosting
Render	Django backend/API hosting
GitHub	Source control
UptimeRobot	Availability monitoring
Browser	End-user runtime

The deployment structure can be shown as:



- GitHub → Source Control
- UptimeRobot → Availability Monitoring

This deployment diagram should be explained further in **Page 29 — Deployment**, so on Page 13 keep it as a high-level connection diagram rather than discussing deployment configuration in depth.

13.11 Architecture Summary

The complete Quill architecture can therefore be understood through five main functional layers:

1. **Presentation Layer** — handles the user interface and interaction.
2. **API/Application Layer** — manages authentication, routing, validation, and application logic.
3. **Processing Layer** — performs translation and file/document/audio operations.
4. **Persistence Layer** — stores user state, history, and document-related metadata.
5. **External Services Layer** — connects the application with translation, speech, email, and push services.

Together, these layers allow Quill to provide multiple workflows through one web application while keeping its frontend, backend, processing, and data responsibilities clearly separated.

14. Frontend Architecture

The frontend of Quill is responsible for everything the user interacts with in the browser. It provides the application interface, manages user input and UI state, communicates with the backend APIs, performs selected browser-side processing, and displays processing results.

Quill's frontend is built primarily with **React, TypeScript/JavaScript, and Tailwind CSS**. It also uses the Lucide icon ecosystem, client-side state management, React Query, and Fetch/Axios-style API communication.

The frontend can therefore be considered the **presentation and interaction layer** of the application.

14.1 Frontend Technology Stack

The major frontend technologies used in Quill are:

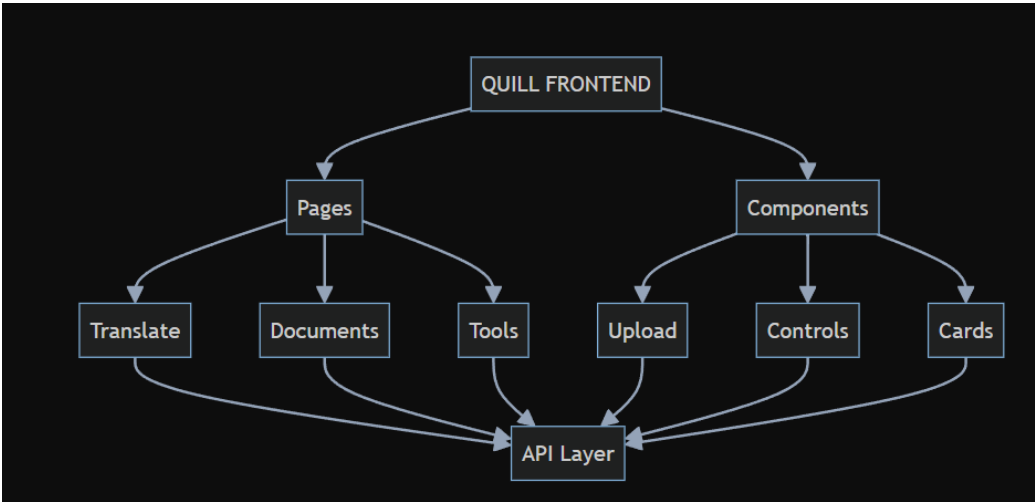
Technology	Role in the Frontend
React	Builds the component-based user interface
TypeScript / JavaScript	Application logic and component behavior
Tailwind CSS	Styling, layout and responsive UI
Lucide / React Icons	Interface icons and visual controls
Client-side state management	Maintains active UI and workflow state
React Query	API/data interaction support documented by the project
Fetch / Axios-style integration	Communication with Django REST APIs

The use of these technologies allows the frontend to support the different workflows of Quill while maintaining a common interface style.

14.2 Component-Based Architecture

Quill's frontend is based on React's component-oriented approach.

Instead of creating the complete interface as one large page, the application can divide the user experience into pages and reusable interface components.



This structure helps keep individual UI responsibilities separated.

14.3 Main Frontend Workspaces

The frontend provides separate workspaces for the major modules of Quill.

These include:

Translation Workspace

Provides text and voice translation controls, source/target language selection, translation options, and result display.

Documents Workspace

Provides document upload, language selection, page viewing, processing progress, translation review, and download options.

Combine & Convert Workspace

Provides tool selection, file upload, file ordering, operation-specific configuration, processing feedback, and output download.

Smart Tools Workspace

Provides categories and interfaces for utility operations.

History Workspace

Displays previous user activity, filtering, preview, download, and management controls.

The frontend therefore presents different functional areas while preserving a common application structure.

14.4 State Management

State management is important because many Quill workflows are interactive.

For example, the Combine & Convert page maintains several pieces of state, including:



The current CombinePage implementation maintains values such as selectedFiles, activeTool, toolOptions, processing status, generated output, and history information.

This state determines what the user sees and what operation is performed when they interact with the interface.

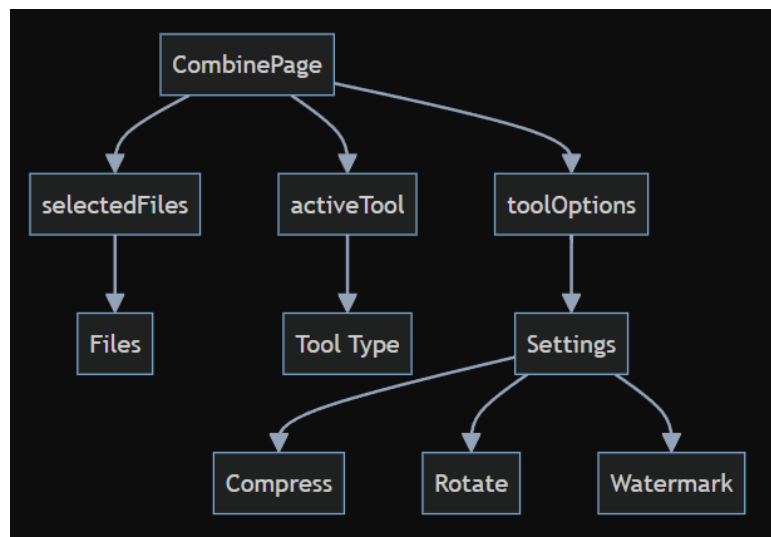
14.5 File State in Combine & Convert

One of the clearest examples of frontend state management is the Combine & Convert page.

The page stores uploaded files in:

selectedFiles: File[]

The currently selected tool is maintained as tool metadata, while tool-specific configuration is stored inside a shared options object. The documented examples include compression, rotation, split, watermark, and N-up options.



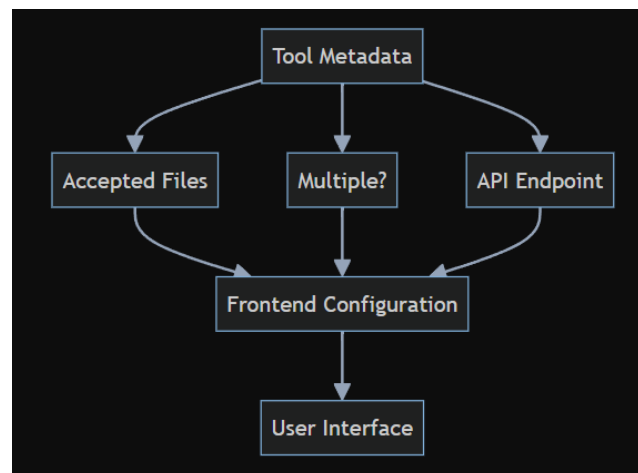
14.6 Tool Metadata Driven Interface

The Combine & Convert frontend also uses tool metadata to determine how each operation should behave.

According to the technical documentation, each tool declares information such as:

- accepted file extensions,
- whether multiple files are supported,
- the API endpoint,
- the history type.

This allows the frontend to use a common interface structure while changing the configuration according to the selected tool.

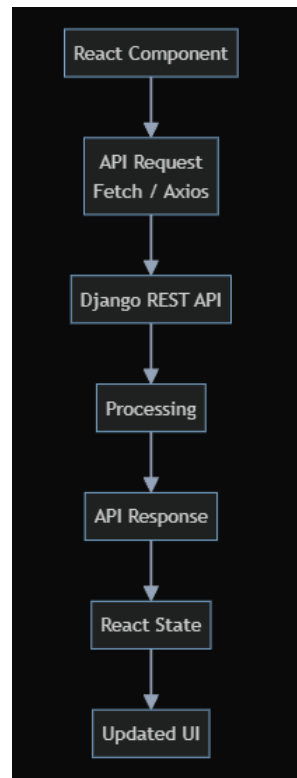


This reduces the need to create a completely separate upload and processing interface for every file tool.

14.7 API Communication

The frontend communicates with the Django backend through REST-style API requests.

The basic architecture is:



The frontend stack documentation specifically lists **Fetch / Axios-style API integration** and React Query as part of the frontend architecture.

This creates a clear separation between the UI and backend services.

14.8 Browser-Side Processing

Not every operation requires the backend.

Quill also performs selected processing directly in the browser using client-side libraries and browser APIs.

The documented browser-side helpers include:

PDF.js

Used for:

- PDF reading,
- page rendering,
- page-related processing,
- text extraction helpers.

jsPDF

Used for:

- client-side PDF creation.

mammoth

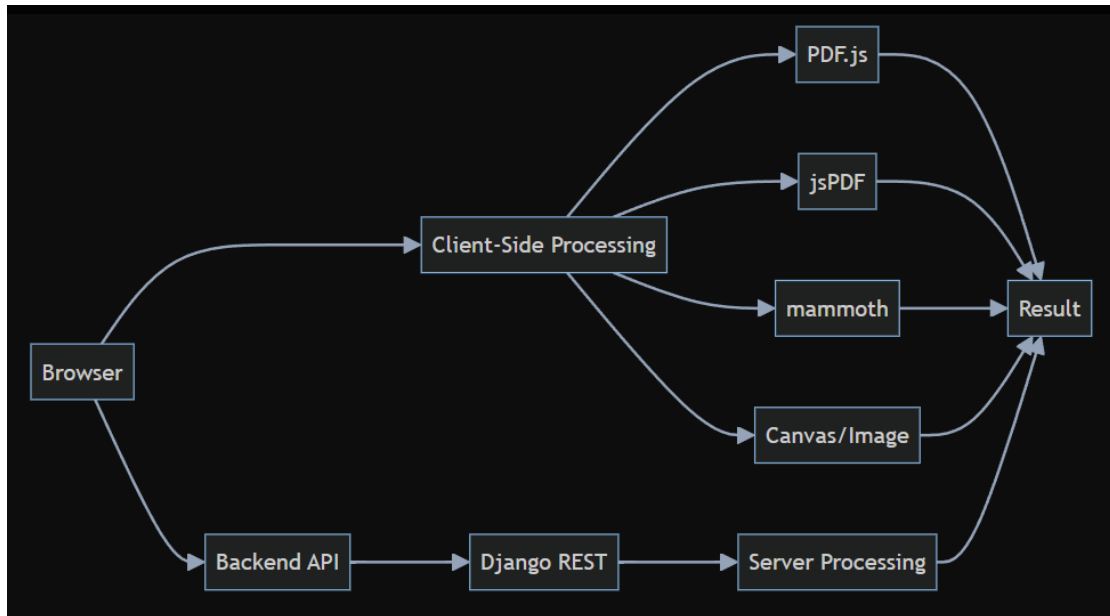
Used as a DOCX raw-text / HTML extraction helper.

Canvas / Image APIs

Used for:

- raster image conversion,
- browser-side image processing helpers.

The architecture can therefore be shown as:

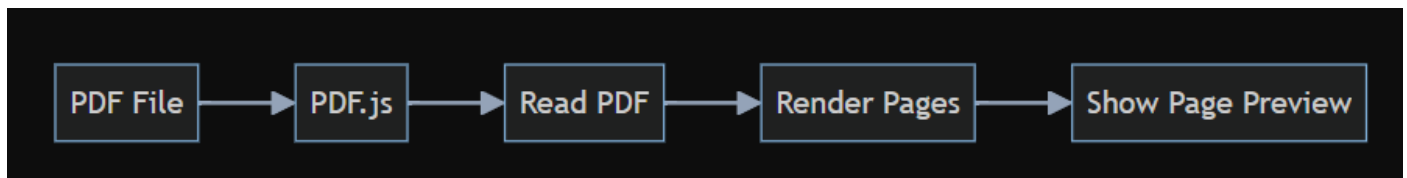


14.9 PDF.js in the Frontend

PDF.js has a specific role in Quill's browser-side PDF handling.

The technical documentation states that PDF.js is used for **PDF reading, page rendering, and text extraction helpers**.

This is particularly useful for operations where the frontend needs to display or inspect PDF pages before sending or completing an operation.



This improves the interactive experience because the browser can display document information while the user is configuring the operation.

14.10 Frontend File Reordering

One of the more important frontend algorithms is the **file reordering system** used during merge operations.

The order of the files is represented by their position in the selectedFiles array.

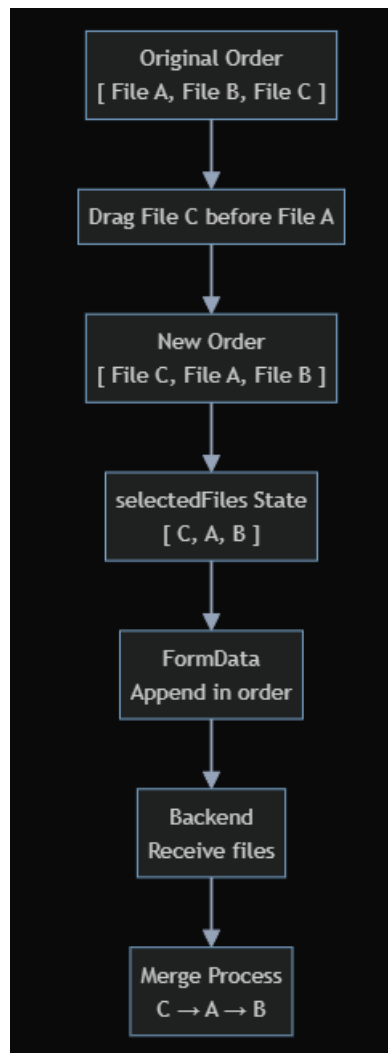
When a user drags a file from one position to another:

1. The existing array is copied.
2. The selected file is removed using splice.
3. The file is inserted at the target position.
4. The updated array becomes the new file order.

5. During processing, files are appended to FormData using this array order.

The project documentation describes this exact process.

The simplified logic is:



This is an important example of how the UI directly affects the final backend result.

14.11 Merge Reordering Algorithm

The implementation can be represented using the following logic:

```
const next = [...previous];  
const [movedFile] = next.splice(sourceIndex, 1);  
next.splice(targetIndex, 0, movedFile);  
return next;
```

The purpose of this logic is simple: remove the file from its old position and insert it into its new position.

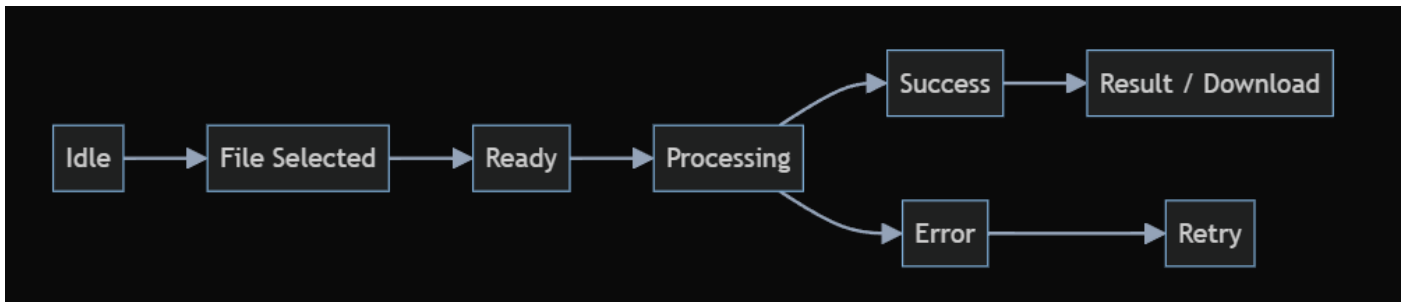
Because the processing function subsequently appends the files to FormData in array order, the final merge order follows the order selected in the UI.

This is a good example of frontend state being directly connected to backend processing behavior.

14.12 Processing and Feedback States

The frontend also needs to communicate the current state of an operation to the user.

Typical states include:



For document workflows, the frontend also maintains processing progress and stage messages, allowing the UI to communicate the current stage of the operation.

This is important for longer-running operations such as document translation and file processing because the user should know whether the request is waiting, processing, completed, or failed.

14.13 Responsive and Consistent UI

Tailwind CSS is used for styling and layout across the frontend. The overall interface follows a consistent card, control, spacing, and navigation structure across different pages.

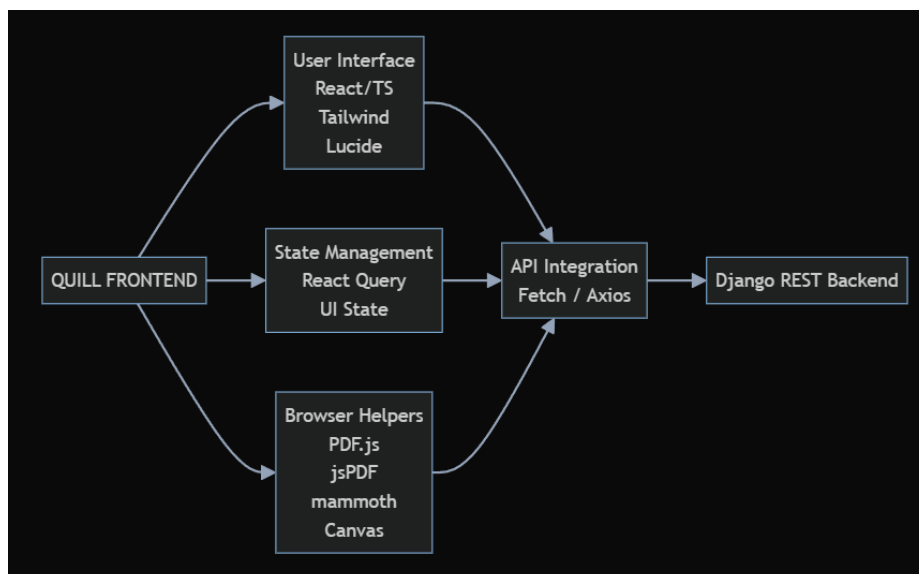
The project design principles emphasize **clarity, speed, consistency, accessibility, and practicality**.

Therefore, the frontend is not only responsible for functionality but also for maintaining a consistent experience when the user moves between:

Translation → Documents → Combine & Convert → Smart Tools → History → Settings

14.14 Frontend-to-Backend Architecture

The frontend architecture can finally be summarized as:



14.15 Summary

The Quill frontend acts as the main interaction layer between the user and the backend system.

Its main responsibilities are:

- building the user interface,
- managing component and workflow state,
- handling file uploads and user input,
- communicating with REST APIs,
- providing browser-side PDF/document/image helpers,
- maintaining file ordering during merge operations,
- displaying processing states and results,
- providing consistent navigation and visual feedback.

The frontend architecture is therefore not limited to visual design. It also contains important client-side logic that controls how files, options, API requests, previews, and processing results move through the application.

15. Backend Architecture

The backend of Quill is responsible for authentication, API communication, validation, translation processing, document handling, file operations, utility tools, and history management. It is developed using **Python, Django, and Django REST Framework (DRF)**. The React frontend communicates with the backend through REST APIs, while the backend handles the main business logic and processing tasks.

15.1 Backend Technology

Quill uses a Django-based backend that follows a modular application structure. Instead of placing all functionality in a single application, different responsibilities are separated into individual Django modules.

The backend mainly performs the following tasks:

- Authentication and user account management
- REST API routing and request handling
- Input validation
- Translation processing
- Document extraction and generation
- PDF and file operations
- Smart utility processing
- History and activity management
- User preference management
- Integration with external services

This separation makes the backend easier to maintain because each module focuses on a specific type of functionality.

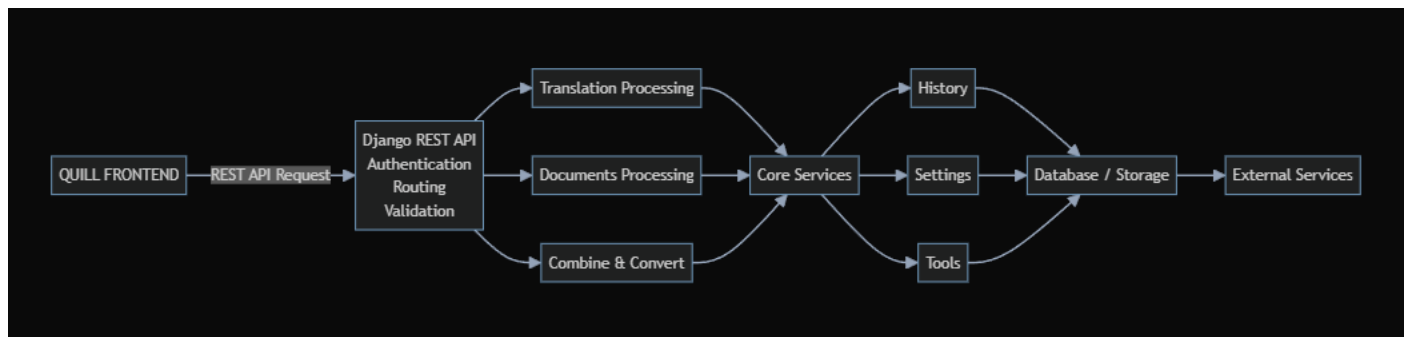
15.2 Main Django Modules

Module	Responsibility
quill	Main Django project configuration
users	Registration, login and authentication
translation	Text and translation workflows
documents	Document upload and processing
combine	File merging, conversion and PDF processing
tools	Smart utility operations
history	Activity and history storage/retrieval
settings_app	User preferences and settings
about	Product/about information

This modular design allows features to be developed and maintained independently.

15.3 Backend Architecture Diagram

The overall backend structure can be represented as:

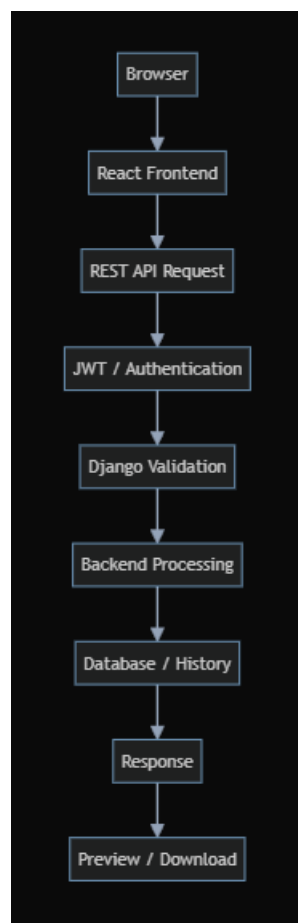


This diagram can be placed in the **center of Page 15** because it gives a quick visual explanation of how requests move from the frontend to different backend modules.

15.4 API and Application Layer

Django REST Framework acts as the main API and application layer between the frontend and backend processing logic.

When a user performs an operation in Quill, the general request flow is:

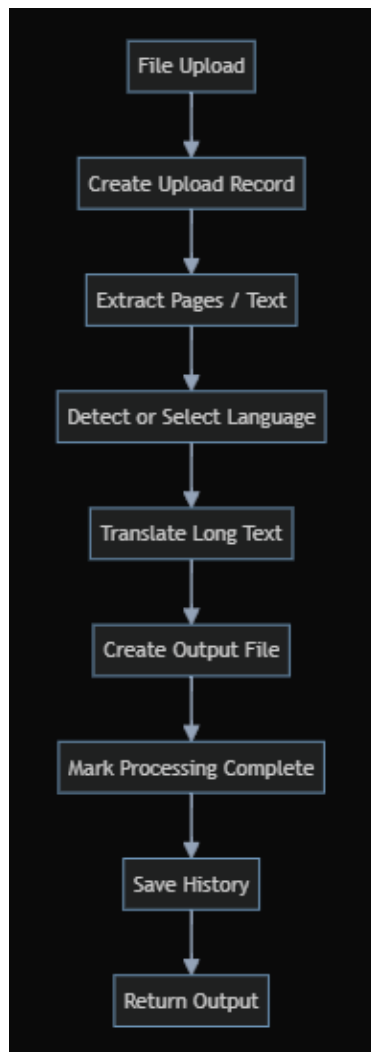


Before processing an operation, the backend validates the incoming request and checks the user's authentication state. After processing is completed, the result can be returned to the frontend for preview or download.

15.5 Document Processing Pipeline

One of the important responsibilities of the backend is document translation.

The document workflow follows a sequence of processing stages:



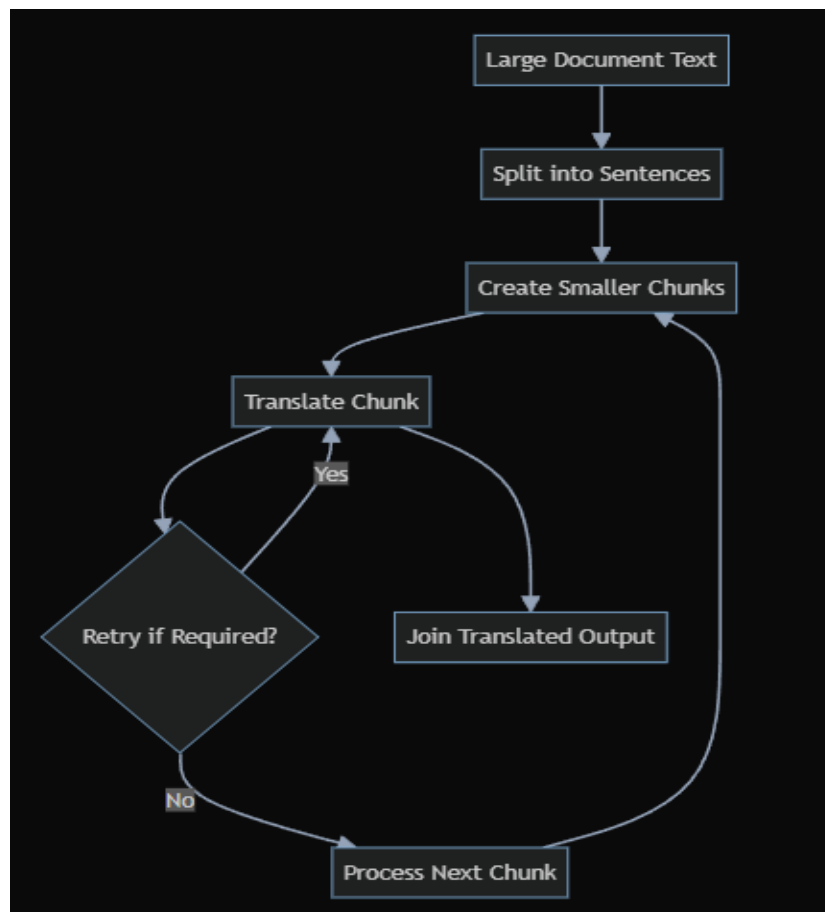
The backend uses page-oriented processing for PDF files and document-specific handling for formats such as DOCX. The documented output workflow supports **PDF, DOCX, TXT and HTML** formats.

Place this diagram **below the backend module diagram** or on the **right half of the page** if space is limited.

15.6 Translation Processing

The translation module handles translation-related backend workflows.

For larger documents, the backend does not treat the entire text as one large request. The documented processing uses sentence-aware chunking. Text is divided into smaller groups, translated sequentially, and failed chunks can be retried.

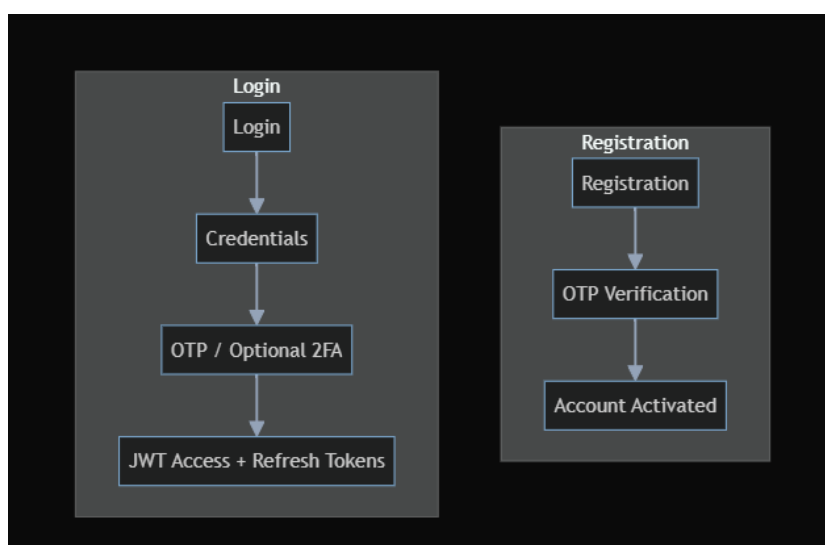


The current implementation describes a target of roughly **450 characters per chunk**, while preferring sentence boundaries and handling unusually long sentences separately.

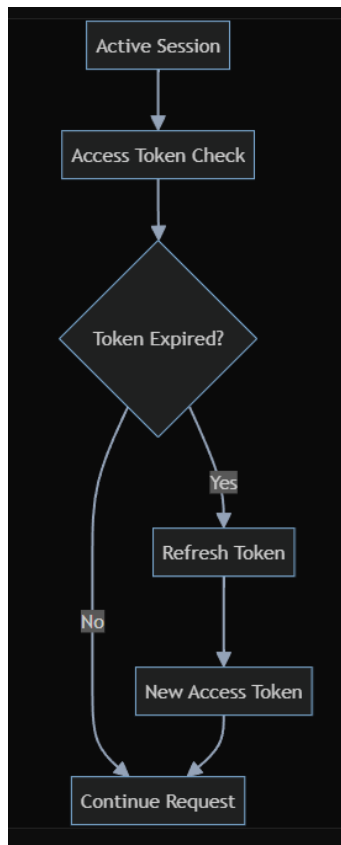
15.7 Authentication in the Backend

Authentication is also handled by the Django backend.

The documented authentication flow is:



For an active session, the frontend checks access-token validity. When the access token expires, the refresh-token mechanism can be used to obtain a new access token.



JWT access and refresh tokens, OTP verification, and optional two-factor authentication are documented in the project backend.

15.8 External Service Integration

The backend also works with external services for tasks such as:

- Translation
- Speech-related processing
- Email
- Push notifications

The project documentation describes multiple translation-service fallbacks and environment-based service credentials. The exact translation provider can vary according to workflow and runtime configuration.

This allows Quill to continue supporting its workflows without making the frontend responsible for service-specific processing.

15.9 Database and History Layer

The backend also connects application processing with persistence.

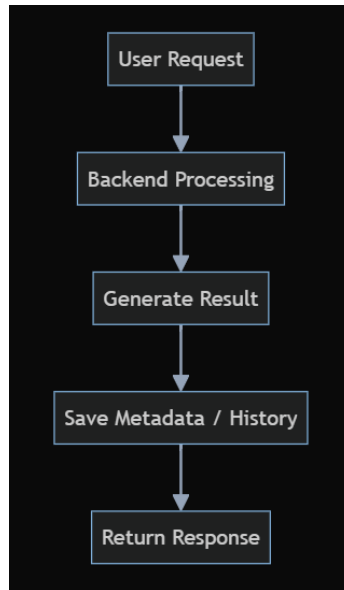
The persistence layer is responsible for information such as:

- User state
- History records
- Document metadata

- Activity information

After an operation is completed, the backend can save the corresponding history record and then return the result to the frontend.

For example:



This is especially important for Quill because translation, document, tools, and combine operations can appear in the user's activity history.

15.10 Why the Backend Is Modular

The modular Django structure provides several practical advantages.

Separation of responsibilities: translation, documents, combine operations, authentication, and history are maintained independently.

Maintainability: changes to one feature can be made without restructuring the complete backend.

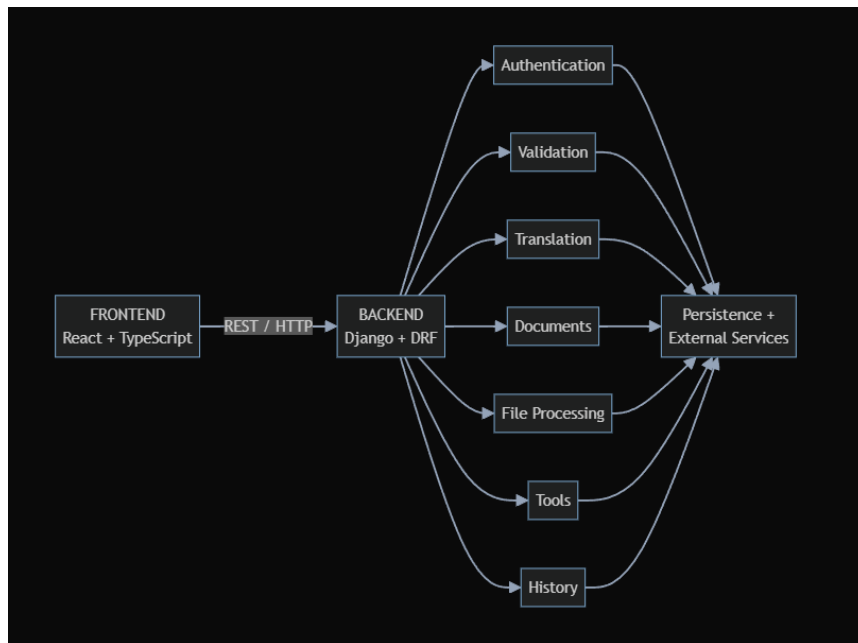
Scalability: additional processing modules or external integrations can be added without redesigning the complete application.

Clear API structure: frontend requests are routed through the API layer before reaching the corresponding processing module.

Better debugging: errors can be traced to a particular application or processing stage.

15.11 Backend and Frontend Relationship

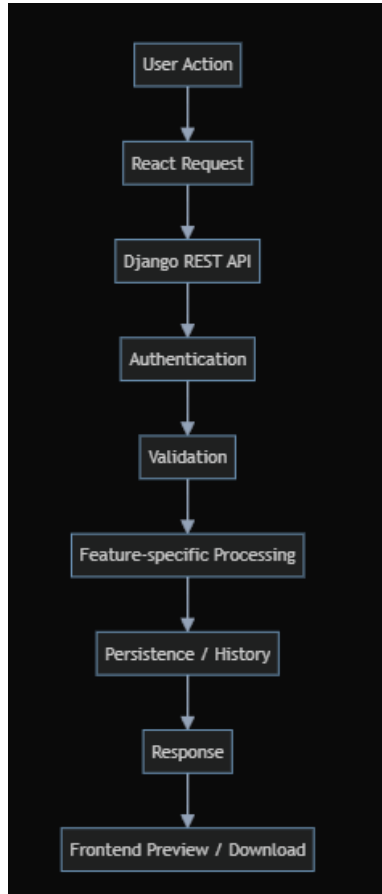
The frontend and backend have clearly separated responsibilities.



The frontend primarily manages the user interface and client-side helpers, while the Django backend owns authentication, API routing, validation and major processing operations.

15.12 Backend Architecture Summary

The Quill backend follows a **layered and modular architecture**. Django REST Framework provides the API and application layer, while separate Django modules handle authentication, translation, document processing, file operations, smart tools, history, and settings.



This architecture keeps the processing logic centralized in the backend while allowing the React frontend to remain focused on the user interface and interaction experience.

16. Database Design

The database layer of Quill is responsible for storing the persistent information required by the application. It supports user state, document metadata, activity history, preferences, and other account-related information used by the backend.

Quill follows a layered architecture in which the Django backend processes requests and communicates with the persistence layer when information needs to be stored or retrieved. The project documentation describes this as a relational database layer behind the API and processing layers.

Important: The available project documentation describes the database at the logical level, but it does not provide a complete SQL schema or an authoritative list of physical table names. Therefore, the design below uses logical entities and relationships rather than claiming undocumented table names.

16.1 Role of the Database

The database provides persistent storage for information that should remain available after a user closes or refreshes the application.

The main responsibilities include:

- Maintaining user/account-related state
- Maintaining document metadata
- Storing translation and tool activity history
- Storing account-linked operation records
- Maintaining user preferences and settings
- Supporting retrieval of previous activities and generated results

The backend communicates with this persistence layer after processing requests so that completed operations can be associated with the appropriate user and history records.

16.2 Logical Database Entities

At a logical level, the Quill database can be understood through the following major entities:

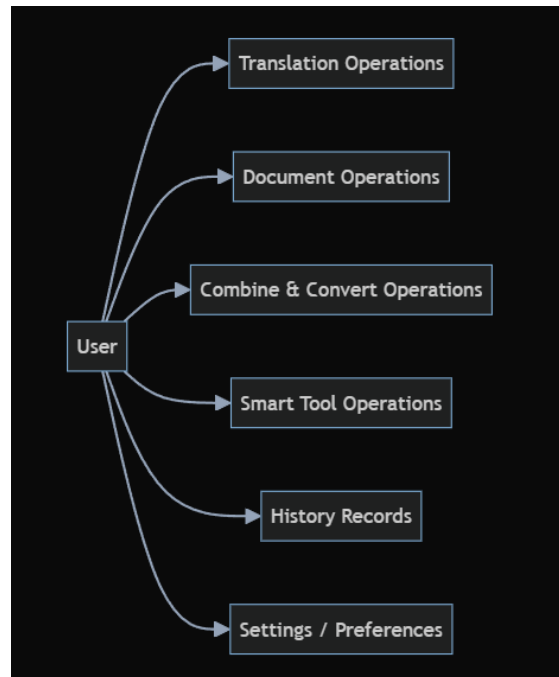
Entity	Main Purpose
User	Stores account identity and user-related information
Authentication State	Supports account/session and authentication-related state
Document Metadata	Stores information related to uploaded/processed documents
History Record	Stores completed user activities and generated outputs
User Settings	Stores preferences and application settings
Notification/Preference Data	Stores account-related notification preferences where applicable

These entities are connected through the user's account and the operations performed inside Quill.

16.3 User Entity

The **User** entity acts as the central account reference for account-linked functionality.

A user can perform multiple operations inside Quill, such as:



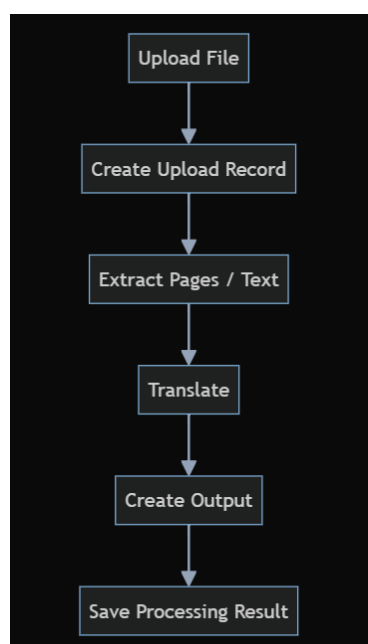
This relationship allows Quill to keep user-specific information separated between different accounts.

The documentation explicitly states that history and account actions are protected by user identity and that history is account-linked.

16.4 Document Metadata

When a document is uploaded, the backend creates an upload record before processing the document.

The documented workflow is:



The database therefore needs to maintain the **metadata associated with the document workflow**, while the actual file processing is handled by the backend.

The documentation specifically identifies **document metadata** as part of the persistence layer and describes an upload-record-first document pipeline.

16.5 History Entity

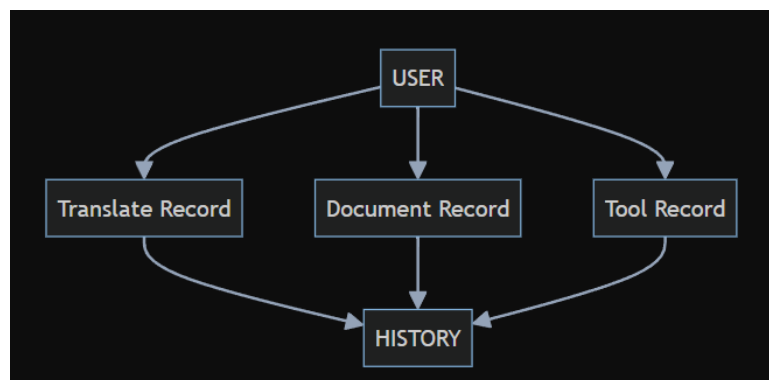
History is an important part of Quill because operations performed by the user can be accessed later.

The history layer covers activities from:

- Translation
- Tools
- Documents
- Combine & Convert

Users can search and filter history, preview records, download results, and delete individual or multiple history records.

A logical relationship can therefore be represented as:



This is a **logical relationship diagram**, not a claim that all four concepts are implemented as separate physical SQL tables.

16.6 Settings and Preferences

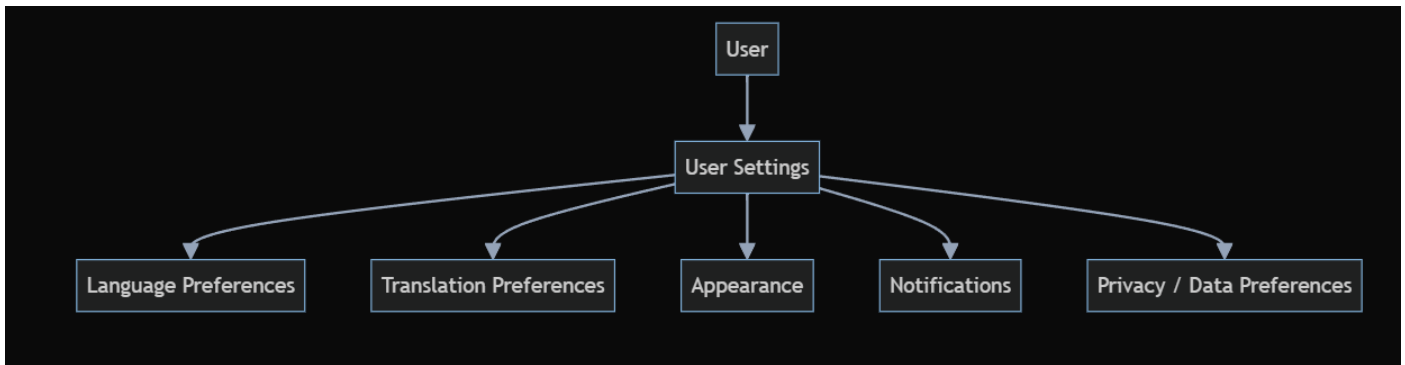
Quill also maintains user preferences through the settings functionality.

The documented settings include preferences related to:

- Default source and target language
- Auto swap
- Auto translate
- Preserve formatting
- Translation style
- Translation formality
- Translation speed
- Appearance
- Notification preferences

- Privacy/data and account settings

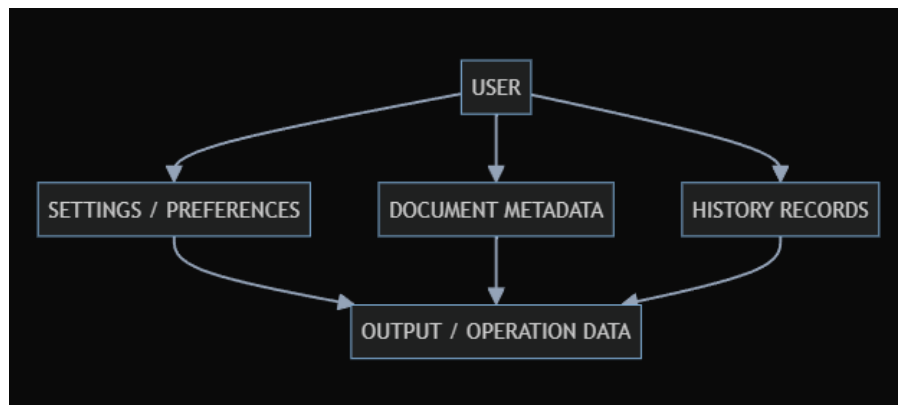
Conceptually:



This allows the application to load the user's preferences when required rather than keeping those settings only in the browser.

16.7 Logical Entity Relationship

The overall logical database design can be represented as:

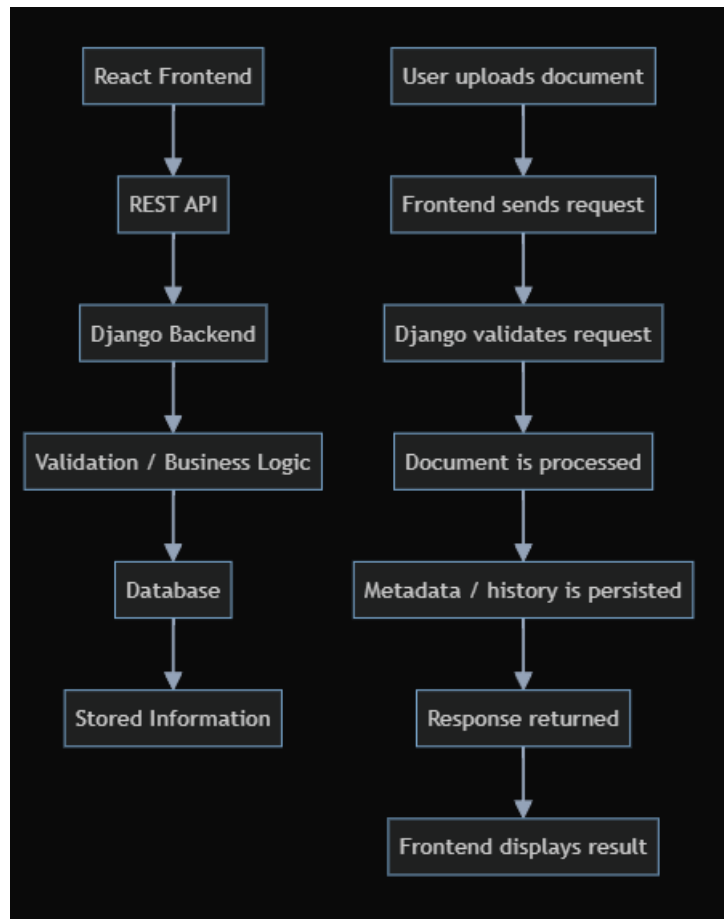


It visually explains that the user is the primary account-level reference while documents, settings and activity history support different parts of the application.

16.8 Database and Backend Relationship

The database does not directly communicate with the React frontend.

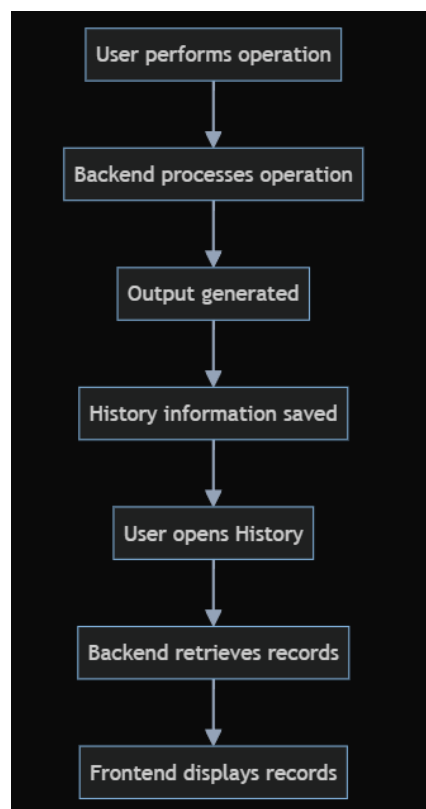
The communication follows the application layers:



This separation keeps database access inside the backend rather than exposing the persistence layer directly to the client. The documented system architecture places Django REST between the frontend and persistence layer.

16.9 History Data Flow

A typical history operation can be understood as:

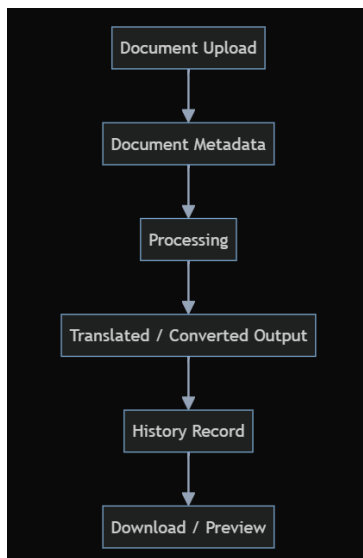


The project documentation also notes that history views prioritize **pinned records first and then newer creation timestamps**. Combine history can be retrieved according to operation type.

This makes history more than a simple list of files; it acts as an account-linked record of previous application activity.

16.10 Document-to-History Relationship

Document processing and history are connected through the completed operation.

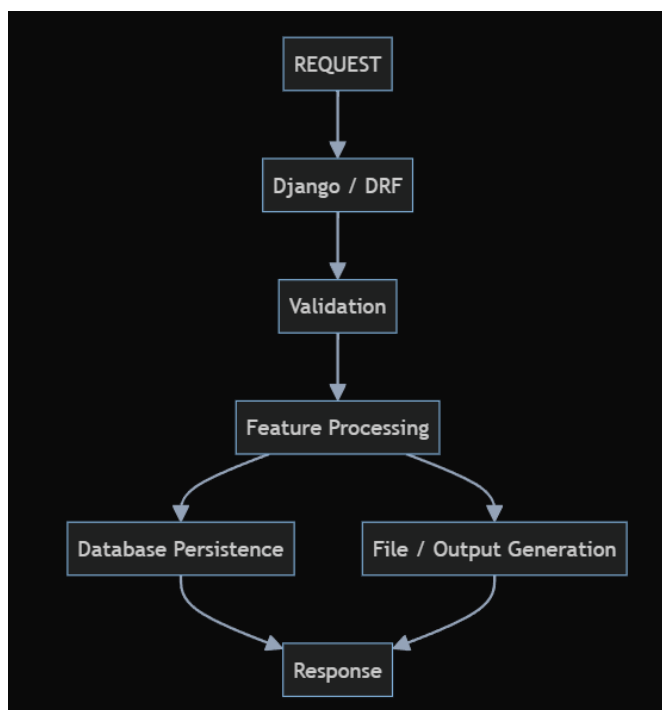


For example, after a translated document is generated, the operation can be represented in the user's history along with its associated information.

This relationship is useful because it allows the user to return to previous work without repeating the complete operation.

16.11 Data Persistence During a Request

A simplified backend request can be represented as:



The persistence layer therefore works together with file processing rather than replacing it.

For document and file workflows, the backend can create output files while separately maintaining the metadata and history information needed by the application.

16.12 Database Design Principles

The logical database design follows a few important principles.

1. Account Association

User-related records are associated with the corresponding account so that one user's history and preferences remain separate from another user's data.

2. Persistent History

Completed operations can be retained as history records for later viewing, searching, previewing, downloading or deletion.

3. Separation of Metadata and Processing

The database stores persistent information about operations and documents, while heavy translation and file processing is handled by the backend processing layer.

4. Modular Ownership

Different Django modules are responsible for different application areas, while the persistence layer supports their data requirements.

5. Backend-Only Access

The database sits behind the Django API layer rather than being directly accessed from the browser.

16.13 Example Logical Data Flow

A complete example for a translated document is:

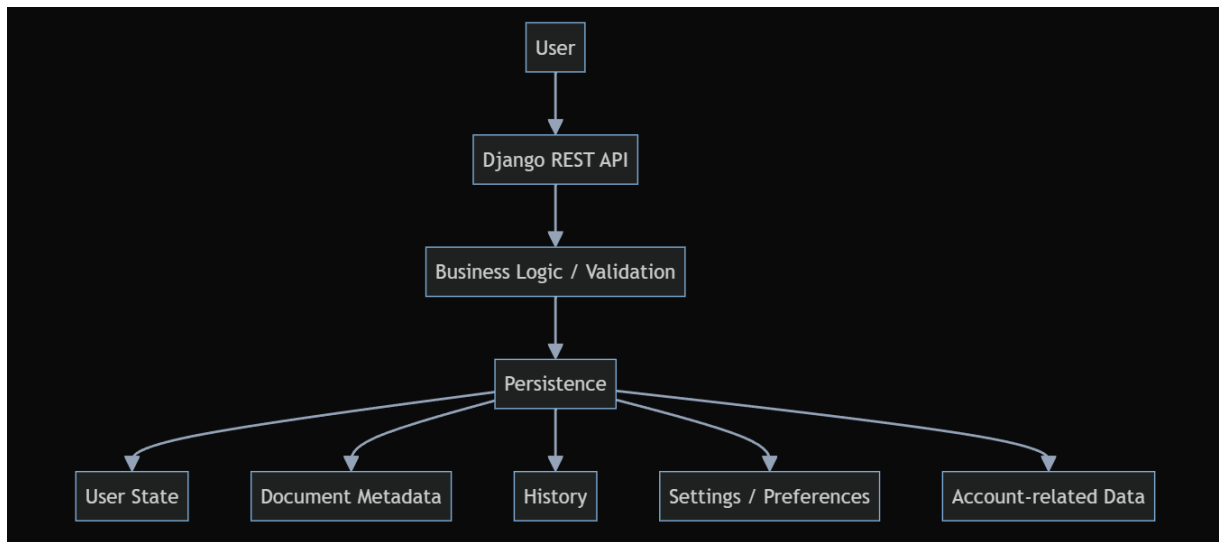


This demonstrates how a single user action can involve both document processing and persistent application data.

16.14 Database Design Summary

The Quill database layer provides the persistent foundation for the application. It supports **users, document metadata, history, settings and related account information**, while Django handles the business logic and processing between the frontend and database.

The overall design can be summarized as:



The available project documentation establishes the **relational persistence layer and its responsibilities**, but does not provide enough source evidence to claim a complete physical database schema or exact table/column definitions.

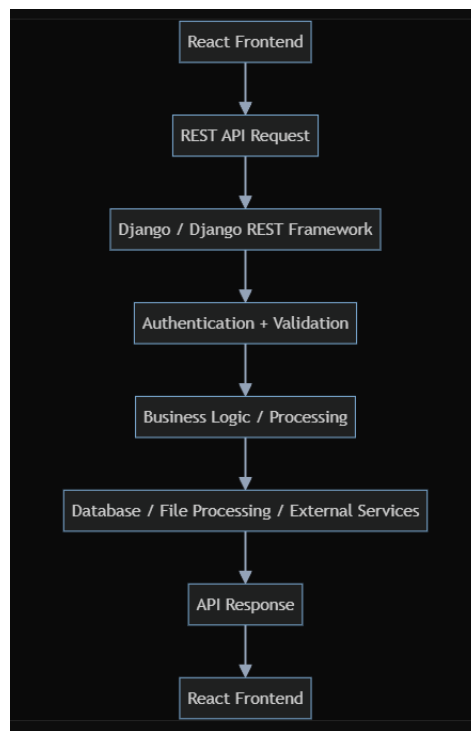
17. API Architecture & Communication

The **API layer** is the communication bridge between the Quill React frontend and the Django backend. Instead of allowing the frontend to directly access database or processing logic, Quill sends requests to the backend through REST APIs. Django REST Framework then authenticates the request, validates the input, performs the required operation, interacts with persistence or external services when required, and returns a response to the frontend.

17.1 Role of the API Layer

The API layer provides a clear separation between the **user interface** and the **backend processing system**.

The communication model is:



This architecture allows the frontend to focus on user interaction while the backend remains responsible for authentication, validation, processing, and persistence.

17.2 REST API Architecture

Quill uses a **REST-based API architecture** for communication between React and Django.

The basic structure is:

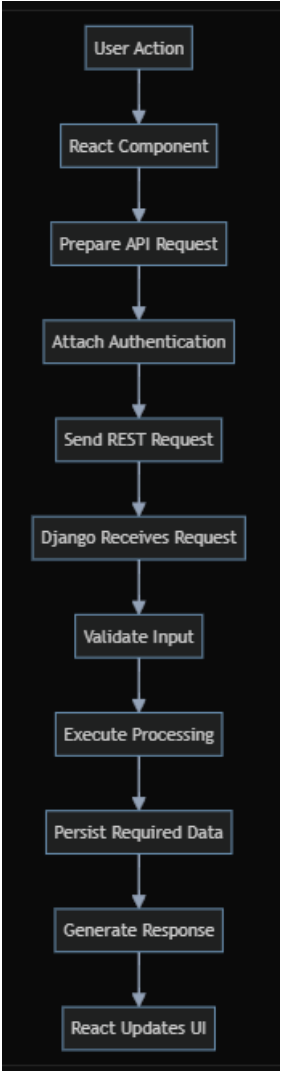
Layer	Responsibility
React Frontend	Collects user input and displays results
REST API	Transfers requests and responses
Django REST Framework	Routes and validates requests
Backend Modules	Perform feature-specific processing
Persistence Layer	Stores user state, history and metadata

External Services	Translation, speech, email and other integrations
-------------------	---

The technical documentation identifies Django REST endpoints as the application's API/application layer, with authentication, business logic and validation handled there.

17.3 Request Lifecycle

Whenever a user performs an operation that requires the backend, the request follows a common lifecycle.



The documented core request lifecycle is:

Browser → React → REST request → JWT/auth → Django validation → processing → persistence/history → response → download/preview.

Figure 17.2: API Request Lifecycle

Placement: Middle section of the page.

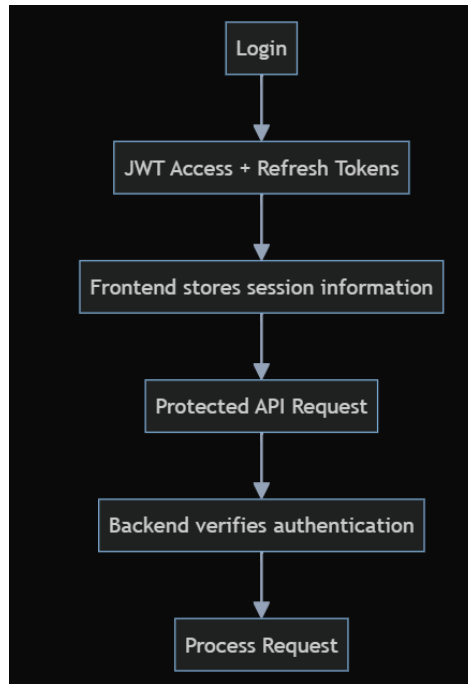
17.4 Authentication and API Communication

Quill uses **JWT-based authentication** for protected API operations.

The documented authentication model includes:

- Access token
- Refresh token
- OTP verification
- Optional two-factor authentication
- Token refresh when an access token expires

A simplified authenticated request is:



When the access token expires, the frontend can use the refresh-token mechanism to obtain a new access token.

17.5 API Request Types

Quill communicates with the backend for different categories of operations.

Translation APIs

Used for translation-related workflows such as text and document processing.

Document APIs

Used when files need to be uploaded, extracted, translated, processed, or converted.

Combine & Convert APIs

Used for PDF, image and document operations.

History APIs

Used to retrieve records associated with previously completed operations.

Account and Settings APIs

Used for account-related information and preferences.

The exact complete API surface is larger than the representative list below; the project's technical documentation explicitly provides representative Combine & Convert endpoints.

17.6 Representative Combine & Convert API Endpoints

The following endpoints are documented in Quill's API surface:

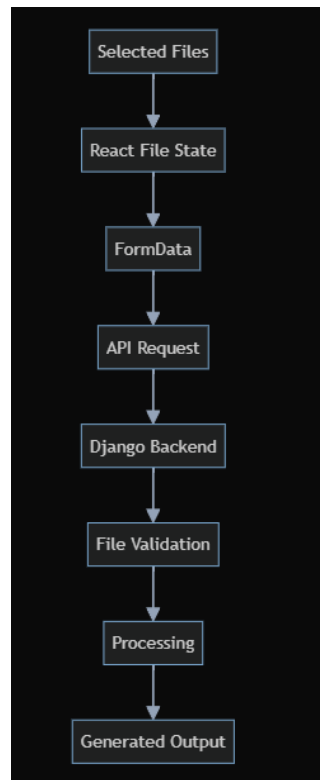
Endpoint	Purpose
/api/combine/pdf-merge/	Merge PDF files
/api/combine/image-merge/	Merge images
/api/combine/word-merge/	Merge Word documents
/api/combine/image-to-pdf/	Convert images to PDF
/api/combine/image-convert/	Convert images
/api/combine/pdf-to-word/	Convert PDF to Word
/api/combine/pdf-to-excel/	Convert PDF to Excel
/api/combine/excel-to-pdf/	Convert Excel to PDF
/api/combine/word-to-pdf/	Convert Word to PDF
/api/combine/compress-pdf/	Compress PDF
/api/combine/rotate-pdf/	Rotate PDF
/api/combine/split-pdf/	Split PDF
/api/combine/organize-pdf/	Organize PDF pages
/api/combine/watermark-pdf/	Add PDF watermark
/api/combine/pdf-color-enhance/	Enhance PDF colors/readability
/api/combine/nup-pdf/	N-up PDF processing
/api/combine/history/	Retrieve Combine history

These endpoints are taken directly from the project's documented representative API surface.

17.7 File Upload Communication

Several Quill operations work with uploaded files. For browser-based file uploads, the frontend uses **multipart request data through FormData**.

Conceptually:



The technical documentation specifically identifies FormData as the browser multipart upload payload used for files.

For example, in the Combine workflow, the order of files in the frontend selectedFiles array is preserved when files are appended to FormData. This allows the backend merge operation to follow the order selected by the user.

17.8 Input Validation

API requests should not be processed blindly.

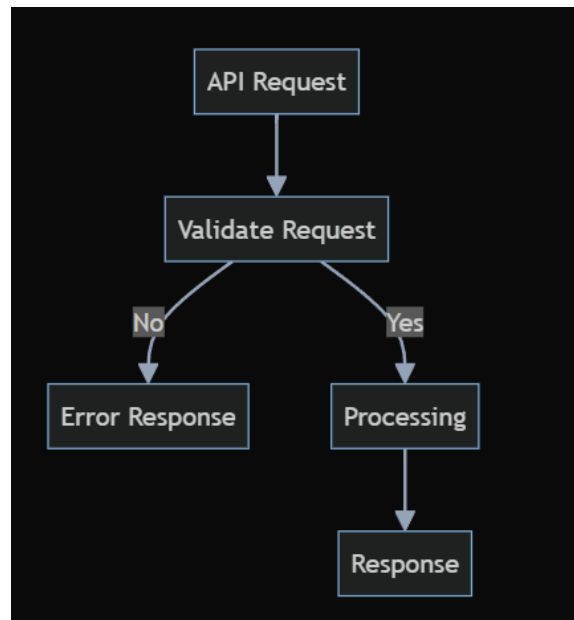
The backend validates the incoming request before performing the requested operation.

Validation may include:

- Required input/file presence
- Accepted file extensions
- Number of files
- Tool-specific configuration
- Translation/document parameters
- Authentication state
- Processing options

For example, Combine tools define accepted extensions and whether multiple files are allowed. The documented validation layer also checks tool options such as compression, rotation, splitting, watermarking and N-up settings.

A simplified flow is:



Placement: Right side of the validation section.

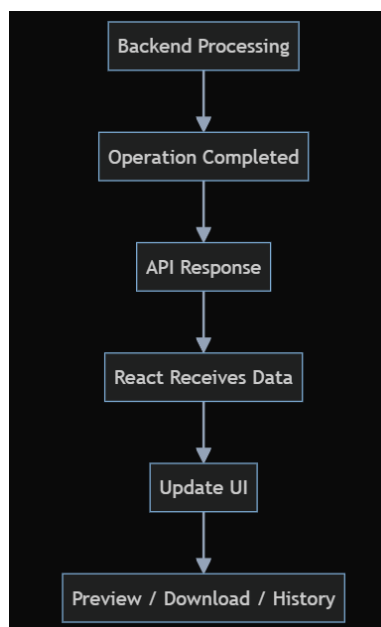
17.9 API Response Handling

After the backend completes an operation, it returns information required by the frontend.

Depending on the workflow, the frontend may receive information that allows it to:

- Display translated content
- Show processing status
- Preview a generated result
- Provide a download
- Update history
- Display an error or status message

For file-based operations, the frontend also checks the returned download information before presenting the result to the user.



17.10 Error Handling in API Communication

API communication also includes error handling.

The project documentation identifies several important cases:

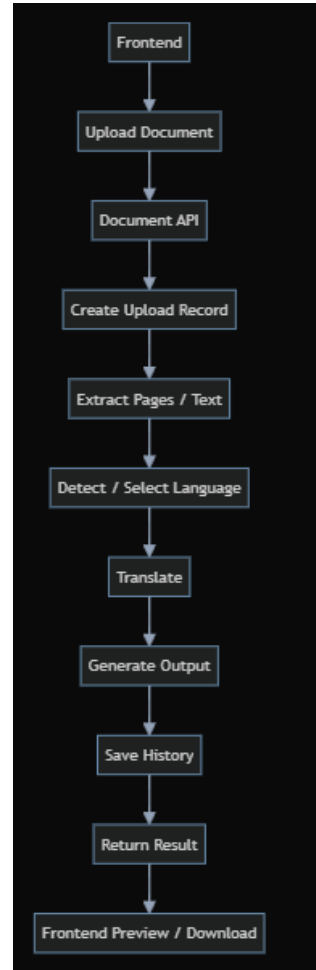
Situation	Backend/Frontend Response
Invalid tool input	Validation error
Incorrect file selection	Request rejected
Authentication problem	Authentication/session error
Expired token	Token refresh flow
Backend failure	User-visible status/toast
Invalid download response	Result is not presented as a valid download

The authentication flow can specifically trigger token refresh when a protected request encounters an expired access token.

17.11 API Communication for Document Processing

Document translation involves a longer backend workflow than a normal text request.

The API interaction can be represented as:

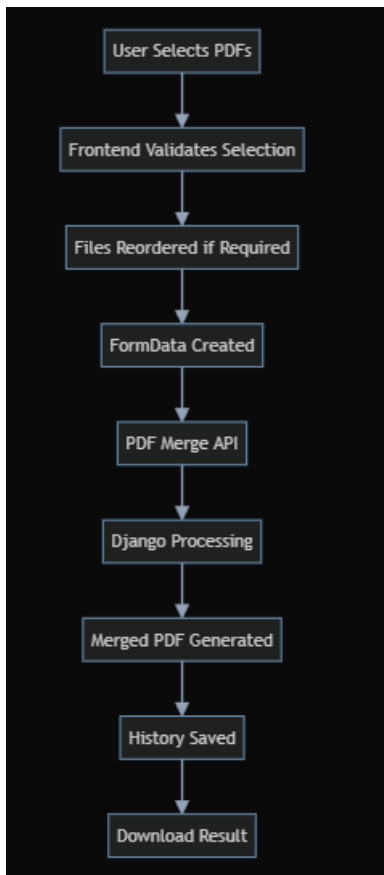


The backend documentation explicitly describes this sequence for the document translation pipeline.

17.12 API Communication for Combine & Convert

The Combine & Convert page follows a similar request model.

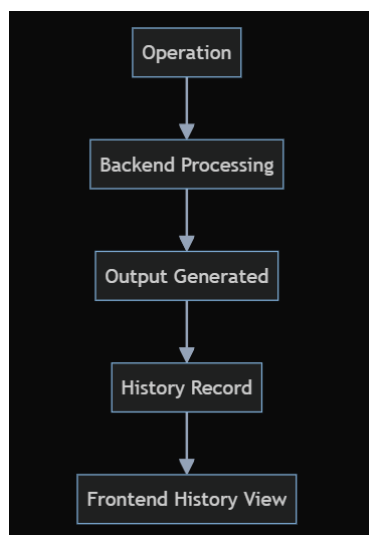
For example, for PDF Merge:



The frontend's file order is maintained when files are added to the multipart request, so the backend can process the merge according to the user's selected order.

17.13 API and History Integration

API communication is also connected to Quill's history system.



History is account-linked and covers **Translate, Tools, Documents and Combine** activities. Users can search, filter, preview, download and delete history entries.

17.14 API Architecture and Security

Security is integrated into API communication rather than treated as a separate frontend-only feature.

Important elements documented for Quill include:

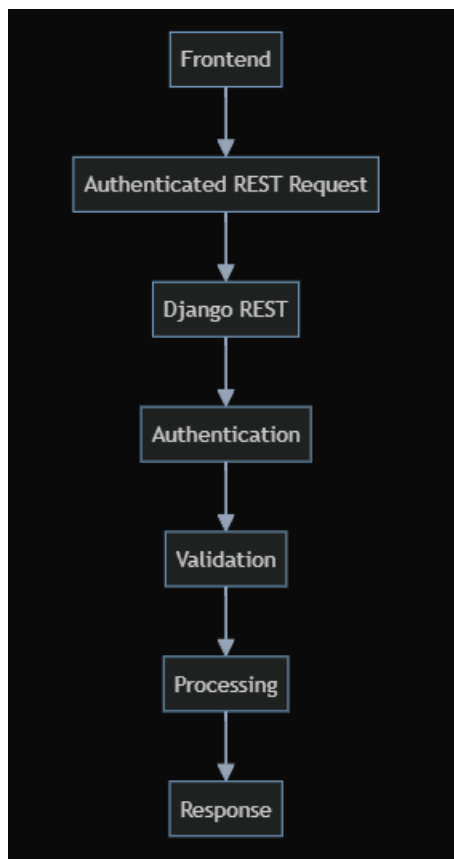
JWT authentication: protects authenticated API operations.

OTP verification: can be used during registration.

Optional 2FA: can be required depending on account state.

Environment-based secrets: API keys, SMTP credentials, OAuth secrets, JWT secrets, database passwords and private VAPID keys are kept outside source code.

Therefore, the API architecture can be summarized as:



17.15 Advantages of the API-Based Architecture

The REST API architecture gives Quill a clear separation between frontend and backend responsibilities.

Separation of concerns: React handles interface and interaction, while Django handles backend logic and processing.

Security: protected operations can be authenticated through JWT.

Reusable services: multiple frontend workspaces can communicate with backend processing through APIs.

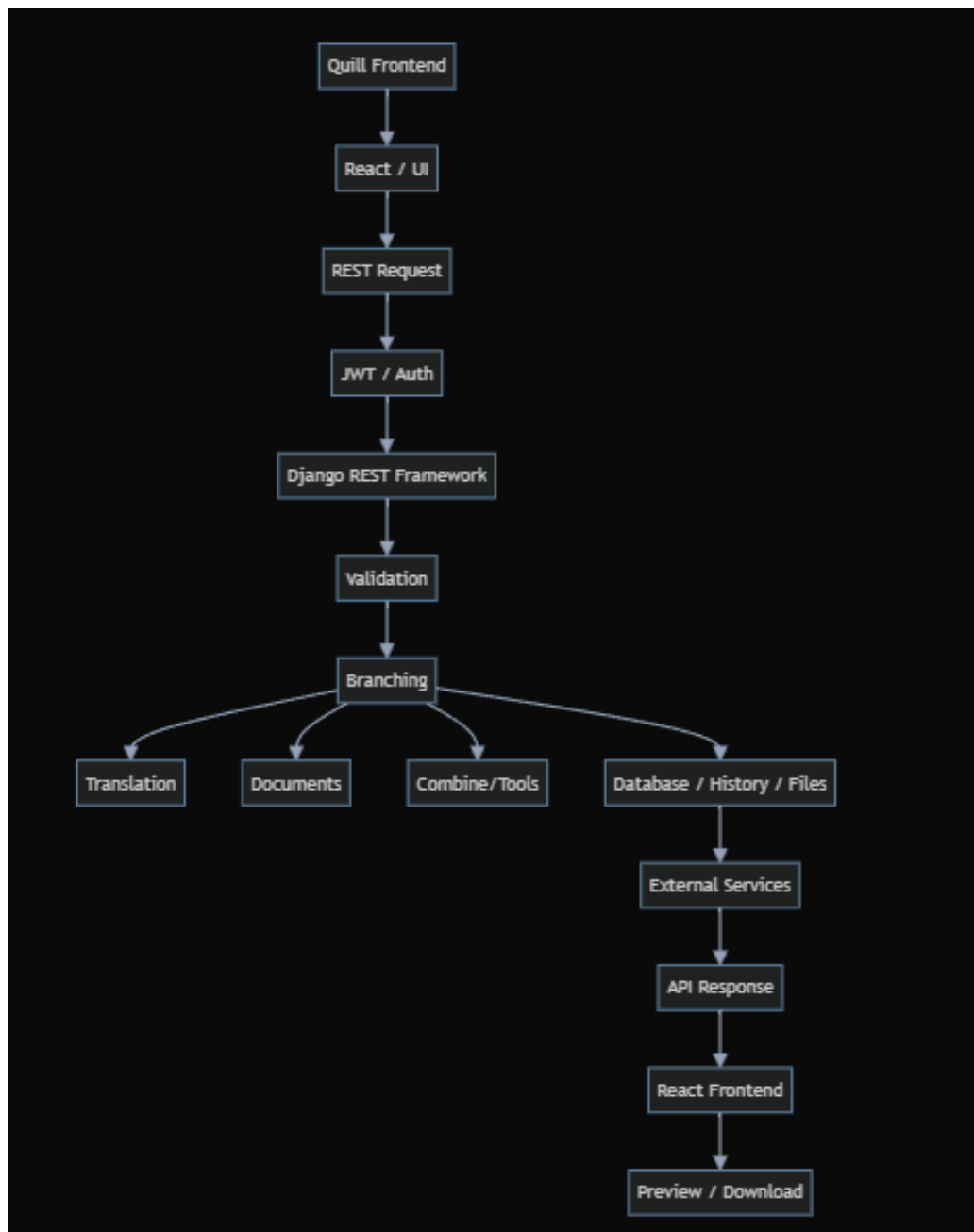
Maintainability: feature-specific backend modules remain separated.

Independent deployment: the frontend and backend can be deployed separately. Quill's production architecture uses Vercel for the React frontend and Render for the Django backend.

Future extensibility: new APIs can be added for additional translation, document, utility or file-processing features.

17.16 Overall API Architecture

The complete Quill communication model can be summarized as:



18. Important Algorithms & Processing Logic

Quill performs several operations that require more than a simple API call. Document translation, PDF processing, file ordering, N-up generation, and history retrieval all use specific processing logic to produce consistent results.

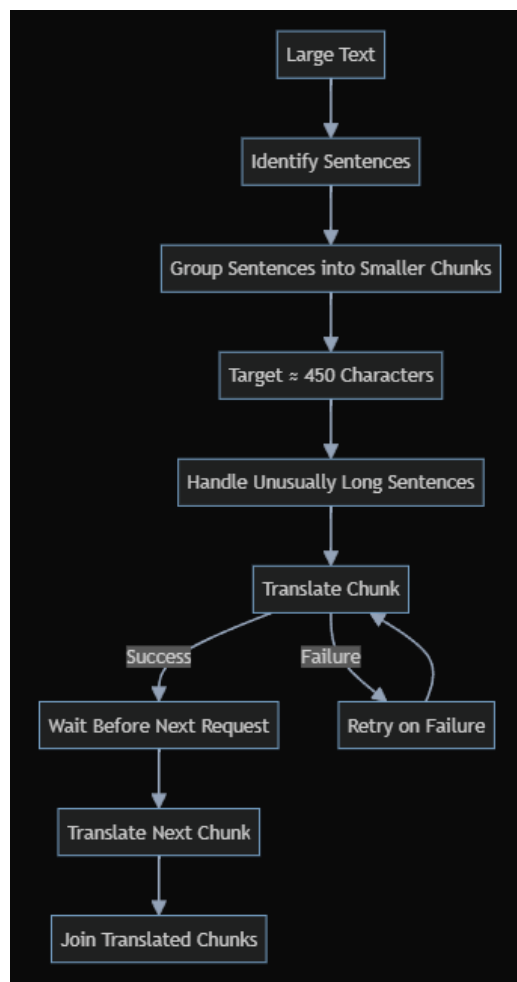
The most important algorithms and data-processing techniques documented for Quill are **sentence-aware translation chunking**, **PDF page extraction**, **N-up layout mapping**, **merge file reordering**, **history ordering**, and **PDF helper rendering**.

18.1 Sentence-Aware Translation Chunking

Large documents cannot always be sent to a translation service as one very large text request. Quill therefore uses a `splitChunks` utility for document-page translation.

The documented algorithm targets approximately **450 characters per chunk**, while trying to preserve sentence boundaries.

Processing logic



The implementation prefers sentence boundaries, slices unusually long sentences, translates chunks sequentially, retries failed chunks, and waits between requests. This reduces the chance of sending an oversized translation request.

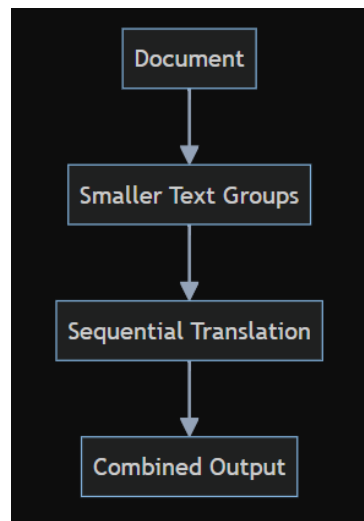
18.2 Why Chunking Is Used

The chunking approach provides a controlled way of processing long document text.

Instead of:

Entire Document → One Large Translation Request

Quill uses:



Combined Output

This also allows an individual failed chunk to be retried without requiring the entire document to be processed again.

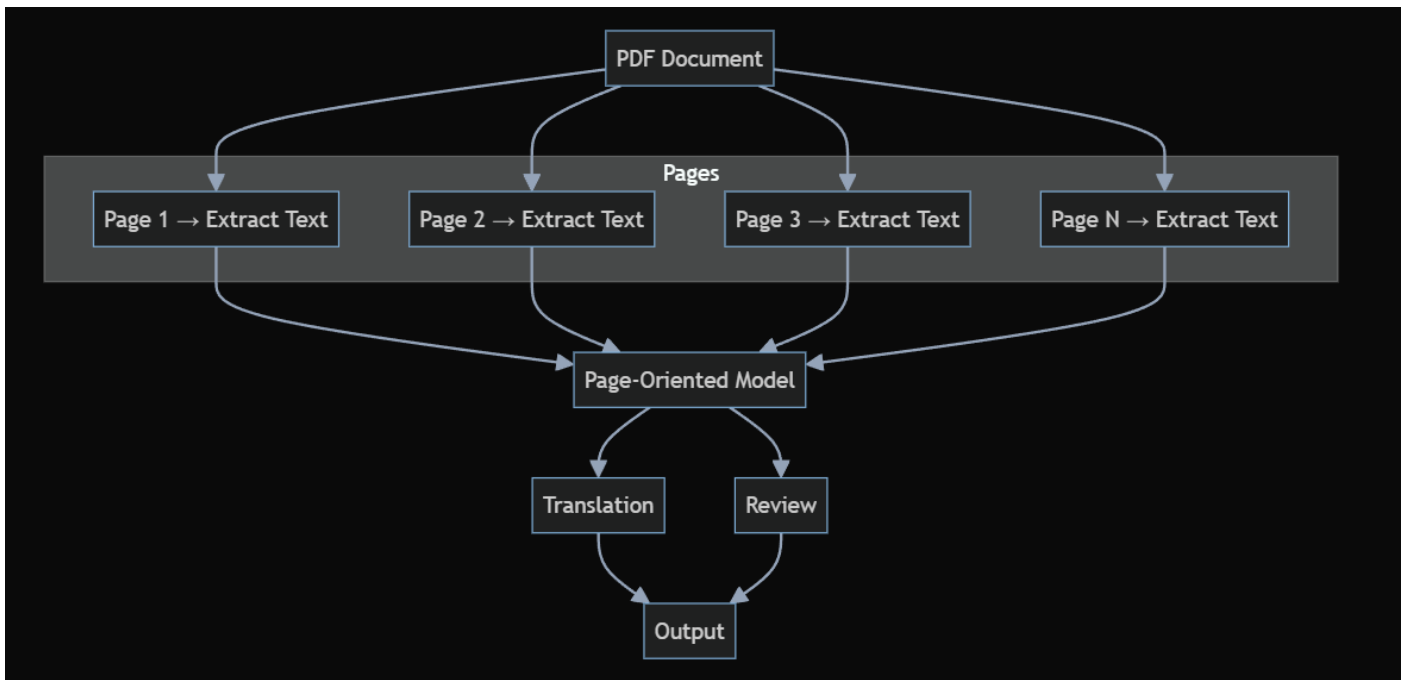
18.3 PDF Page Extraction Algorithm

Quill uses a **page-oriented document model** for PDF processing.

The documented extraction process is:

Step	Operation
1	Open PDF using a page-aware parser
2	Iterate through page numbers
3	Extract text from each page
4	Maintain page count
5	Store per-page content
6	Use the page model for translation, review and output

This approach is particularly useful for multi-page documents because each page can remain separately identifiable during later processing.



The documented implementation explicitly keeps page count and page-level content so that the collected page model can be reused during translation, review and output generation.

18.4 N-up PDF Layout Algorithm

The **N-up PDF** feature places multiple pages or images onto a single output sheet.

Quill uses a layout mapping based on the number of pages selected per sheet.

Pages per Sheet	Grid
2	2×1
3	3×1
4	2×2
6	3×2
9	3×3
Horizontal	$N \times 1$
Vertical	$1 \times N$

The selected layout is used for the preview and is passed to the backend together with options such as spacing, border, page size, orientation, margin and fit.

You can show the grids visually:



For the final report, a **clean diagram made in Word/PowerPoint** will look better than a code-style representation.

18.5 Merge File Reordering Algorithm

Quill uses an array-based approach to maintain the order of files selected for merging.

The file order is represented by the position of each file inside:

selectedFiles

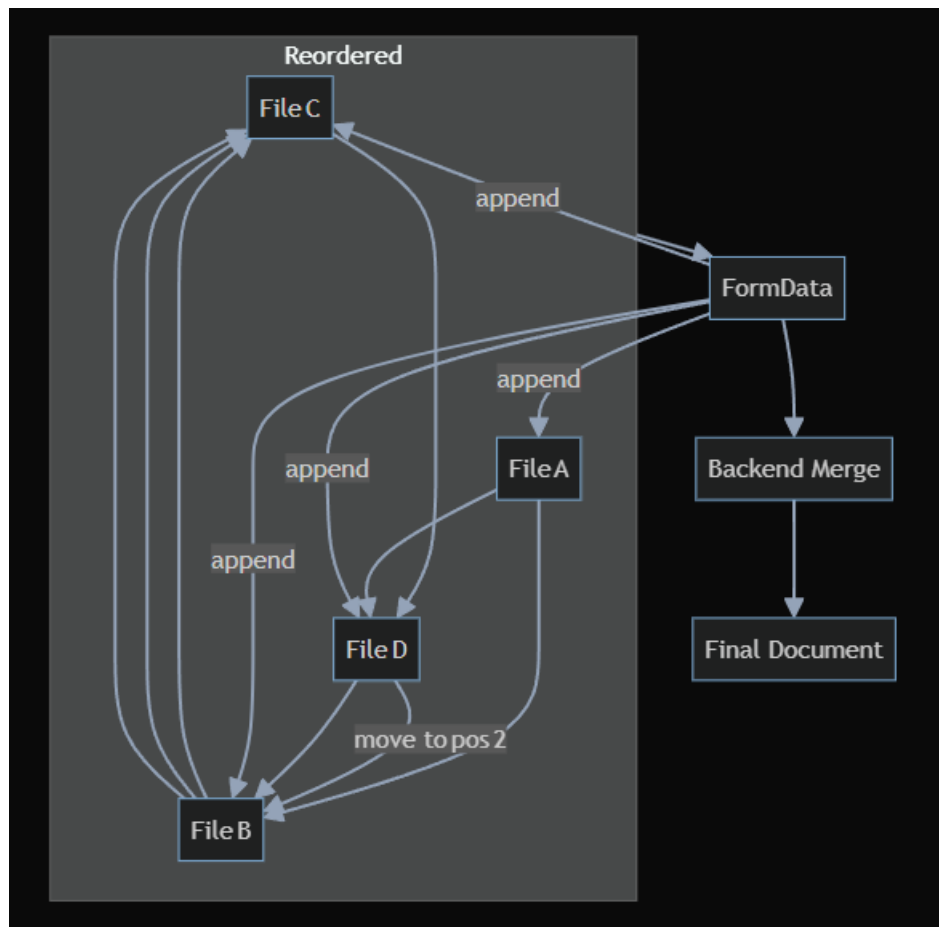
When a user drags a file to another position, the implementation:

1. Copies the existing array.
2. Removes the selected file using splice.
3. Inserts it at the target position.
4. Uses the resulting array order when constructing FormData.

Representative logic documented in the project is:

```
const next = [...previous];  
  
const [movedFile] =  
  next.splice(sourceIndex, 1);  
  
next.splice(targetIndex, 0, movedFile);  
  
return next;
```

This means the order shown in the interface becomes the order sent to the backend for merging.



This algorithm is one of the important engineering decisions in Quill because the array order directly corresponds to multipart upload order.

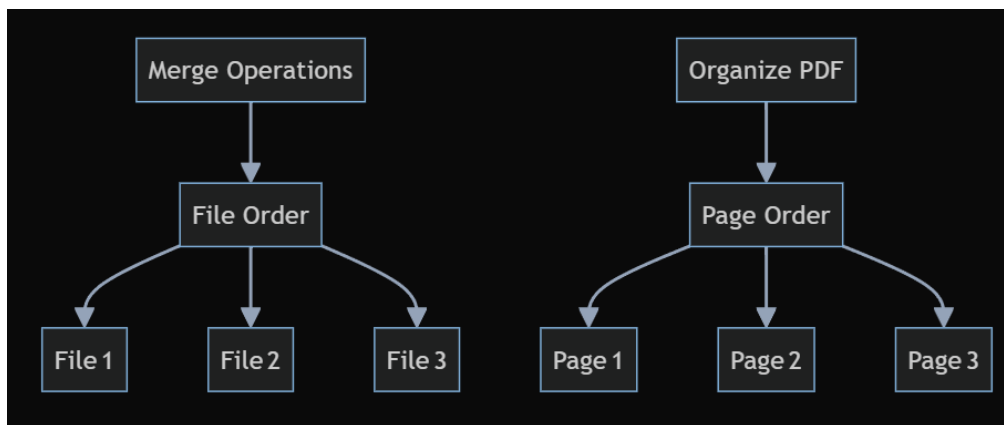
18.6 Separate File Ordering and Page Ordering

Quill maintains two different ordering concepts:

Ordering Model	Used For
File-order array	PDF, Word and image merge
Page-order array	Organize PDF

These states are intentionally kept separate so that changing the order of files does not interfere with changing the order of pages inside a PDF.

Conceptual design



18.7 History Ordering Algorithm

Quill's history interface does not simply display records in arbitrary order.

The documented history logic gives priority to:

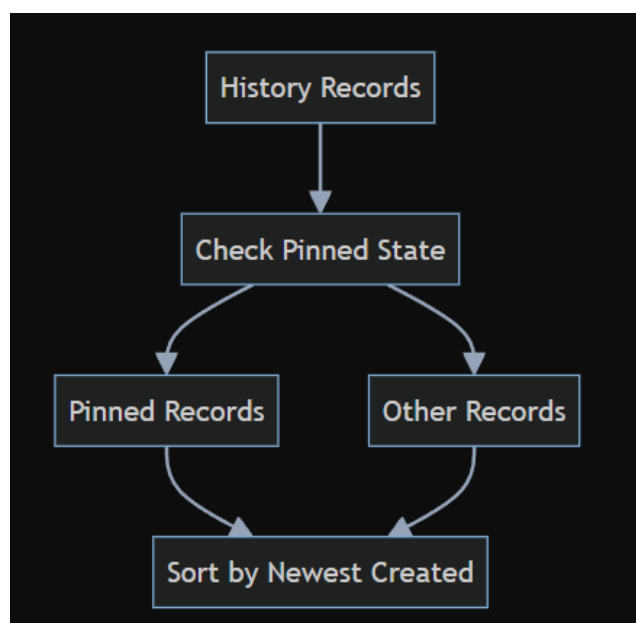
1. Pinned records

followed by

2. Newer creation timestamps

This produces a predictable ordering for frequently referenced activities while keeping recent operations visible.

A simplified representation is:



The Combine history can additionally be retrieved according to operation type, while the UI supports actions such as refresh, pin, share, delete and download in the available source variants.

18.8 PDF Rendering and Preview Logic

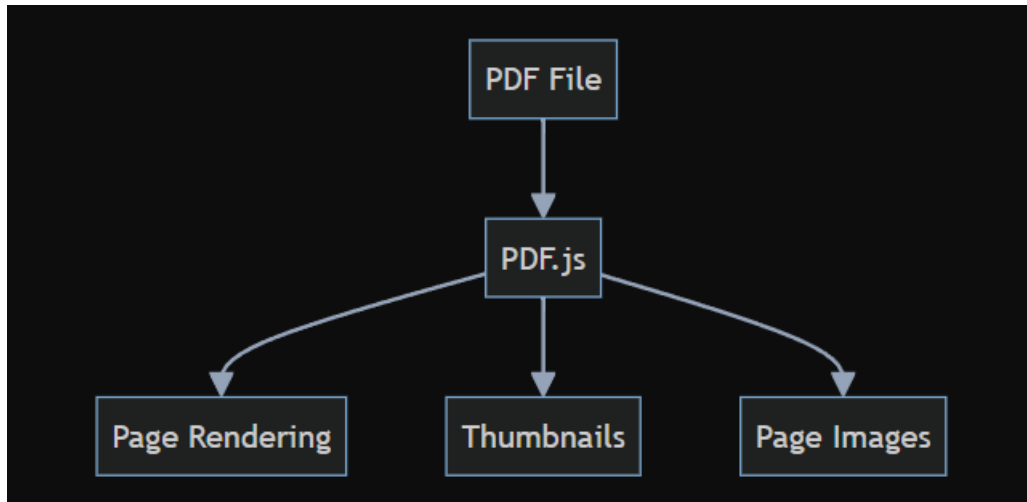
Quill also uses **PDF.js** in the browser for PDF-related visual processing.

The documented uses include:

- PDF page thumbnails

- Page images
- Browser-side PDF rendering

The Organize PDF workflow maintains the page-order array, while merge functionality maintains the file-order array. Keeping these models separate prevents the two types of ordering from interfering with one another.



18.9 Client-Side and Backend Processing

Quill deliberately divides processing between the browser and backend.

The engineering documentation identifies:

- **Client-side helpers** for preview, rendering and lightweight file interaction.
- **Backend processing** for heavier transformations.

Examples include:

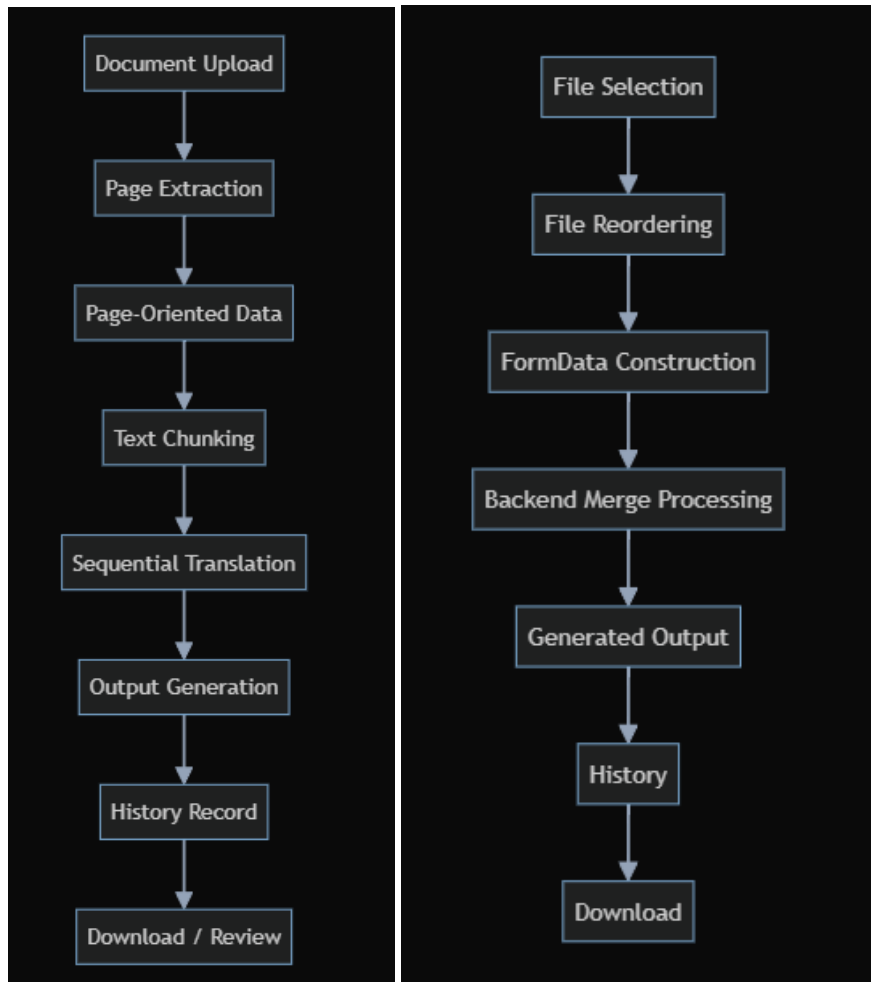
Client Side	Backend Side
PDF preview	Document translation
PDF.js rendering	Heavy PDF processing
File ordering	File merging
Browser-side preview	Output generation
Canvas/Image helpers	Backend document processing

This design keeps heavy processing behind the backend API while allowing the frontend to provide responsive previews and interactions.

18.10 Combined Processing Workflow

Several of these algorithms work together during an actual user operation.

For a document translation task:



These workflows illustrate that Quill's processing logic is composed of multiple smaller algorithms rather than one single operation.

18.11 Summary of Important Algorithms

Algorithm / Logic	Main Purpose
Sentence-aware chunking	Process long translation text in smaller groups
PDF page extraction	Maintain page-level document information
N-up layout mapping	Decide grid placement for multiple pages per sheet
File reordering	Preserve user-selected merge order
Separate file/page ordering	Prevent merge and organize states from interfering
History ordering	Prioritize pinned and recent records
PDF.js rendering	Generate browser-side PDF previews and thumbnails

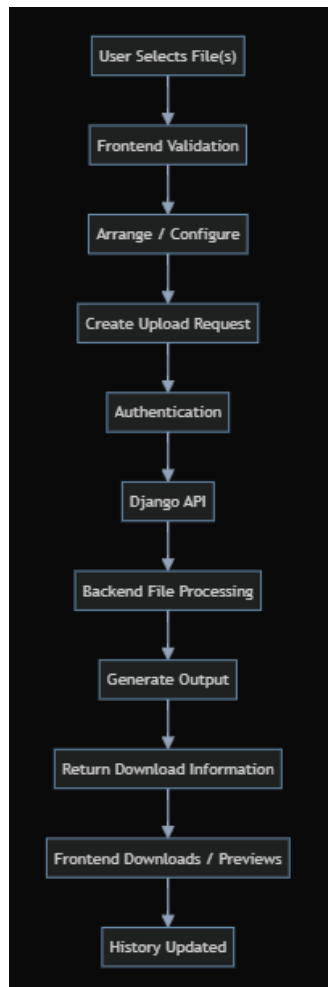
These algorithms form an important part of Quill's document and file-processing workflow.

19. File Processing Workflow

File processing is one of the central workflows in Quill. The application allows users to upload, arrange, convert, optimize, process, preview, and download different types of files. The processing is divided between the **React frontend**, which manages file selection, validation, ordering and preview helpers, and the **Django backend**, which performs the heavier processing and generates the final output.

19.1 Overview of File Processing

The general Quill file-processing workflow is:



The documented file workflow follows the sequence **choose tool → upload → validate → arrange/convert/optimize → process → download → history**.

19.2 Step 1 — Selecting Files

The workflow begins when the user selects one or more files from the Quill Combine & Convert workspace.

The frontend maintains the selected files in a `File[]` structure:

```
selectedFiles
├─ File 1
├─ File 2
├─ File 3
└─ ...
```

Different tools specify whether they support:

- Single or multiple files
- Particular file extensions
- Specific processing options
- A particular backend endpoint
- A particular history category

This information is maintained through the tool metadata used by the Combine page.

19.3 Step 2 — File Validation

Before processing begins, Quill validates the selected input.

Validation checks include:

Validation	Purpose
File type	Ensures the selected extension is supported
File count	Ensures the correct number of files is selected
Tool compatibility	Prevents incompatible files from being submitted
Tool options	Checks compression, rotation, split, watermark and N-up settings
Authentication	Ensures the user has a valid session

For example, merge operations require multiple files, while single-file tools reject invalid extra-file selections.

The validation stage prevents unnecessary API requests and gives the user feedback before heavy processing begins.

19.4 Step 3 — Arranging and Configuring Files

For tools where order matters, users can arrange files before processing.

This is especially important for:

- PDF Merge
- Word Merge
- Image Merge

The order is stored in the `selectedFiles` array. When a file is dragged, the frontend removes it from its old position and inserts it into the new position.

```
const next = [...previous];
```

```
const [movedFile] =
```

```
  next.splice(sourceIndex, 1);
```

```
next.splice(targetIndex, 0, movedFile);
```

```
return next;
```

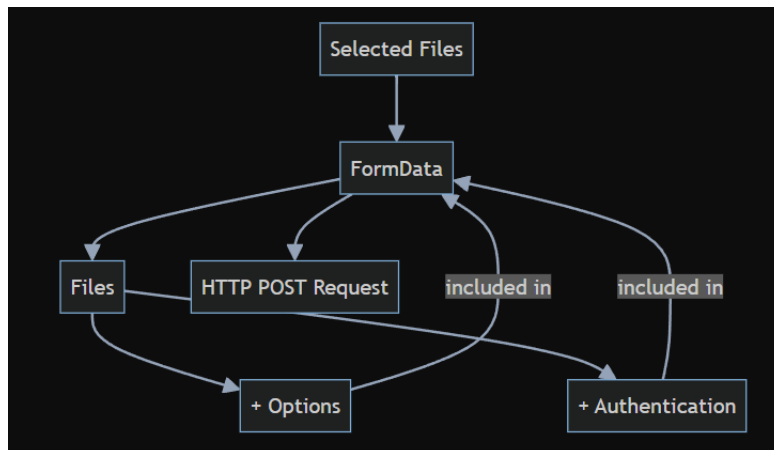
The final array order is then preserved when files are added to the multipart request. Therefore, the backend receives the files in the same sequence shown to the user.

19.5 Step 4 — Preparing the Upload Request

Once validation is complete, the frontend prepares the files for transmission to the backend.

Quill uses **FormData** for browser multipart file uploads.

Conceptually:



For multiple-file tools, the selected files are added to the request in their current order. For single-file tools, the selected file is sent individually.

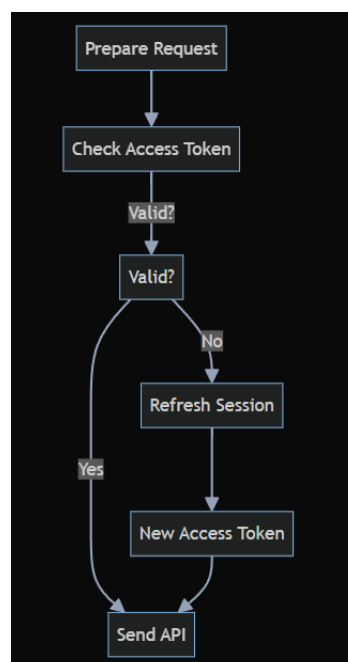
This allows the same communication pattern to support both single-file and multi-file operations.

19.6 Step 5 — Authentication

Before sending a protected processing request, Quill verifies the user's authentication state.

The API uses JWT-based authentication, including access and refresh tokens. If the active access token is no longer valid, the application can use the refresh-token mechanism before continuing.

The simplified process is:



This ensures that authenticated processing and account-linked history remain associated with the correct user.

19.7 Step 6 — Sending the File to the Backend

After authentication, the frontend sends the request to the appropriate Django REST endpoint.

The endpoint depends on the selected operation.

Examples include:

Operation	Representative API
PDF Merge	/api/combine/pdf-merge/
Image Merge	/api/combine/image-merge/
Word Merge	/api/combine/word-merge/
PDF Compress	/api/combine/compress-pdf/
PDF Rotate	/api/combine/rotate-pdf/
PDF Split	/api/combine/split-pdf/
PDF Organize	/api/combine/organize-pdf/
Watermark	/api/combine/watermark-pdf/
N-up PDF	/api/combine/nup-pdf/

These endpoints are part of the representative API surface documented for Quill.

19.8 Step 7 — Backend File Processing

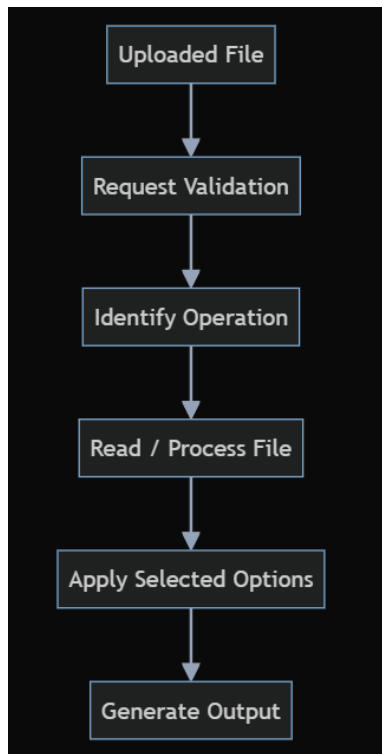
Once Django receives the request, the backend performs the actual file operation.

The backend contains separate processing modules for:

- Document processing
- PDF operations
- File conversion
- Translation
- Combine operations
- Smart tools

The backend is responsible for the heavier transformations, while the frontend provides interaction and lightweight preview functionality.

A general backend workflow is:



19.9 File-Type Specific Processing

Different types of files require different processing logic.

PDF Files

PDF operations include:

- Merge
- Split
- Compress
- Rotate
- Organize
- Watermark
- Color enhancement
- N-up processing

PDF.js is also used in the browser for page thumbnails, rendering and page images.

Image Files

Image operations include:

- Image Merge
- Image to PDF
- Image conversion

Canvas and browser Image APIs provide client-side image conversion helpers, while heavier transformations can be handled by the backend.

Word Documents

Word-related operations include:

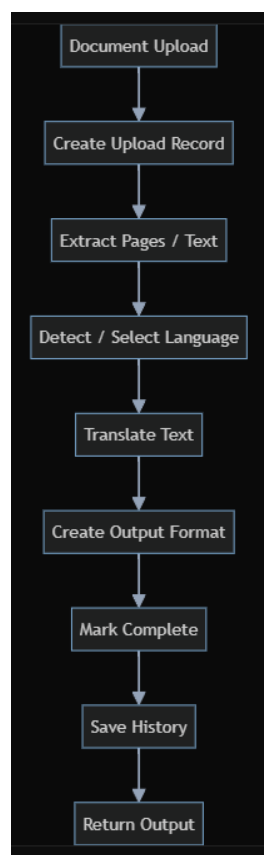
- Word Merge
- Word to PDF
- PDF to Word

The project also documents **Mammoth** as a browser-side helper for DOCX raw-text and HTML extraction.

19.10 Document Processing Workflow

The document workflow is slightly different from a simple file conversion because text may need to be extracted and translated before the output is generated.

The documented pipeline is:



The backend supports page-oriented PDF extraction and document-specific processing for formats such as DOCX. Document outputs include **PDF, DOCX, TXT and HTML** in the documented workflow.

19.11 PDF Page-Level Processing

For PDF documents, Quill uses a page-aware processing model.

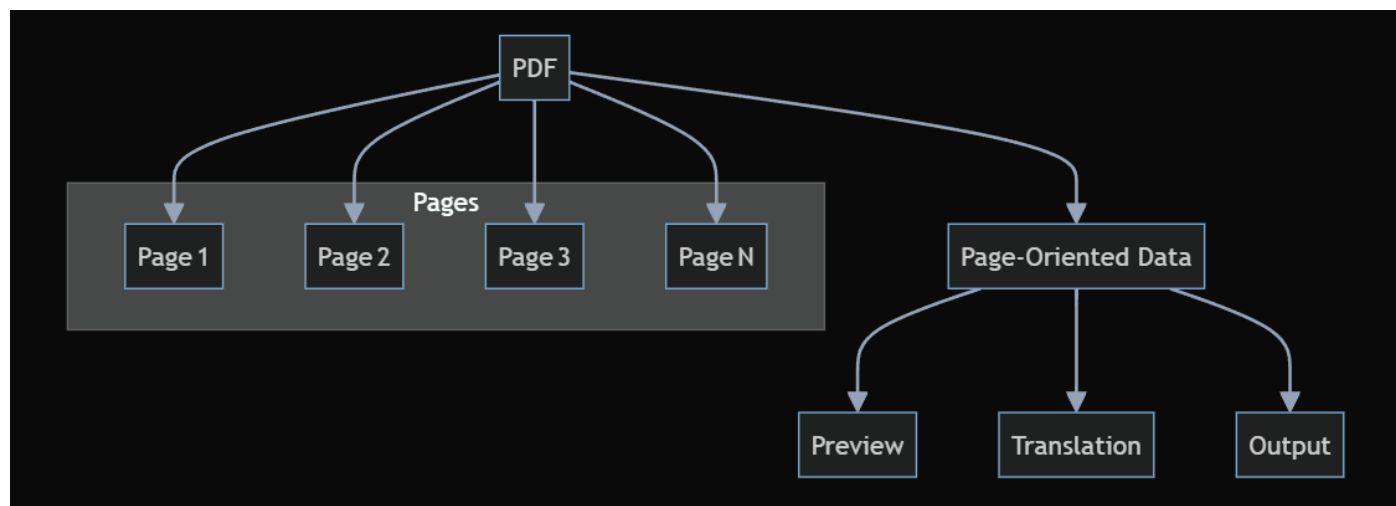
The backend:

1. Opens the PDF.
2. Iterates through its pages.
3. Extracts page text.

4. Maintains page count.
5. Maintains per-page content.
6. Uses this page model for later processing.

This structure supports page-based translation, review and output generation.

The frontend can additionally use PDF.js for page thumbnails and page images.



19.12 Step 8 — Output Generation

After the backend completes processing, an output file is created.

Depending on the selected operation, the result may be:

- PDF
- DOCX
- TXT
- HTML
- PNG
- JPG
- Another converted file format

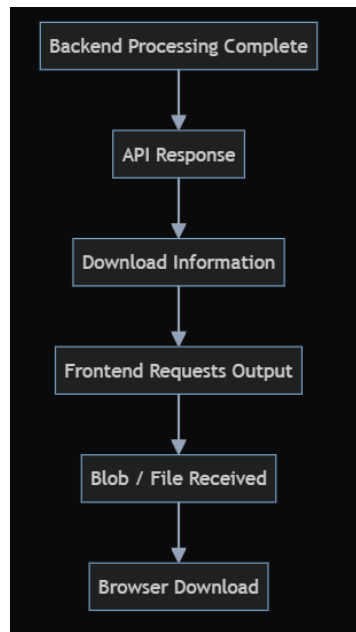
The documented Combine & Convert functionality includes multiple PDF, Word and image conversions, while document translation supports PDF, DOCX, TXT and HTML output.

19.13 Step 9 — Download Response

After processing, the backend returns information that the frontend can use to obtain the generated output.

The frontend verifies that the response is successful and that valid download information is available before attempting the download.

The workflow becomes:



For historical files, the frontend can similarly retrieve a stored download URL using authenticated API access.

19.14 Step 10 — History Integration

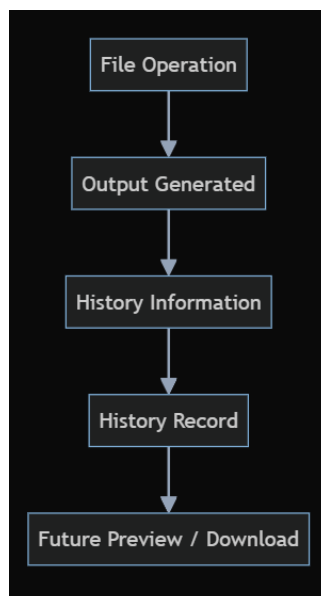
Completed file operations are connected to Quill's history system.

After successful processing, the operation can be associated with:

- Operation type
- Output name
- Output size
- Status
- Creation/completion time
- Download information

History is account-linked and supports retrieval for Combine operations as well as other Quill workflows.

Thus:



19.15 Error Handling During File Processing

Several stages can produce errors:

Stage	Possible Problem
Selection	Unsupported file
Validation	Wrong file count
Authentication	Expired/invalid session
Upload	Request failure
Backend Processing	Processing exception
Output	Invalid or missing result
Download	Download request failure

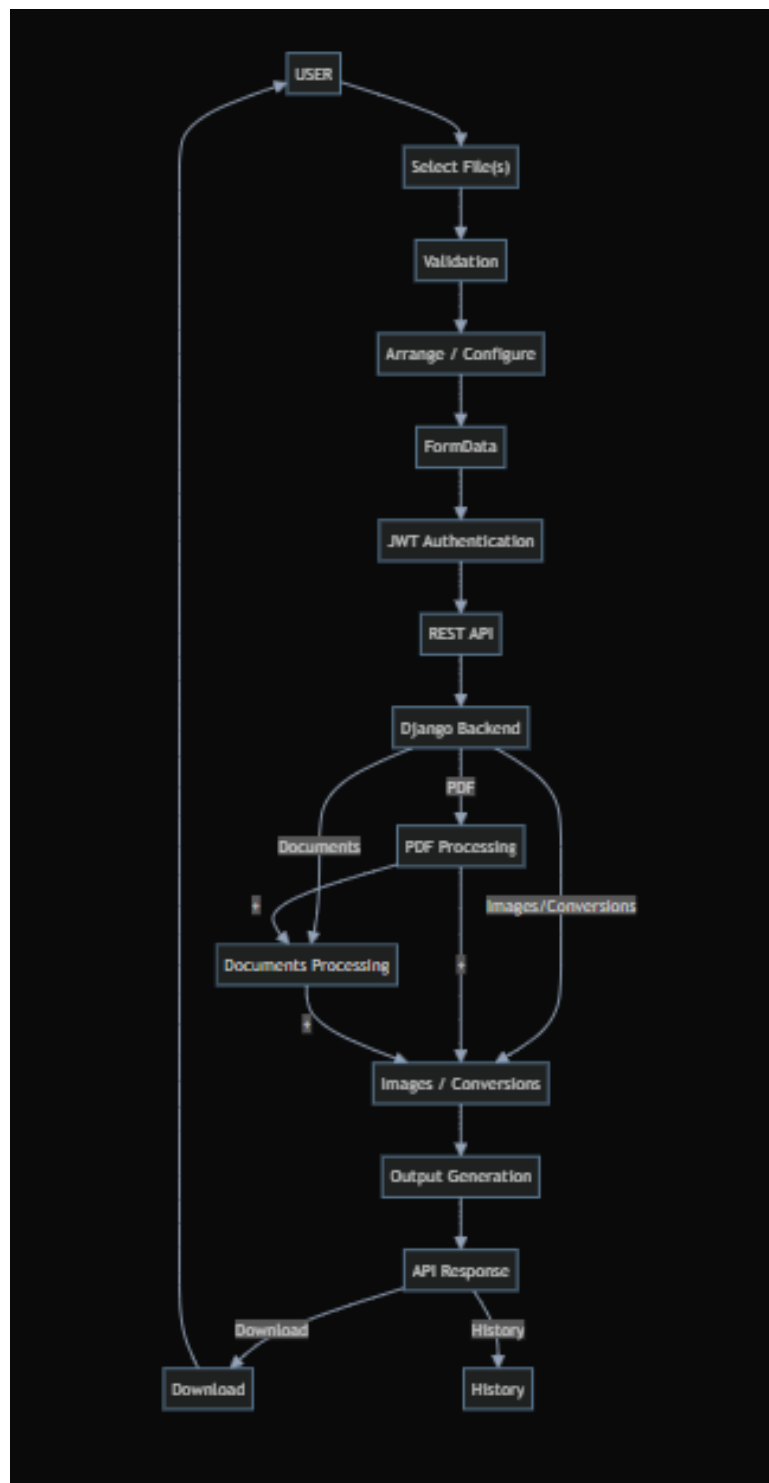
Quill provides user-visible status or toast messages for backend failures and checks the response before presenting generated files for download.

This allows the interface to communicate whether processing is:

- Preparing files
- Uploading/processing
- Preparing the download
- Completed
- Failed

19.16 Complete File Processing Architecture

The complete Quill file workflow can be represented as:



19.17 Client-Side vs Backend Processing

One important design decision in Quill is that not every file operation needs to be performed entirely on the server. The architecture separates lightweight browser-side operations from heavier backend processing.

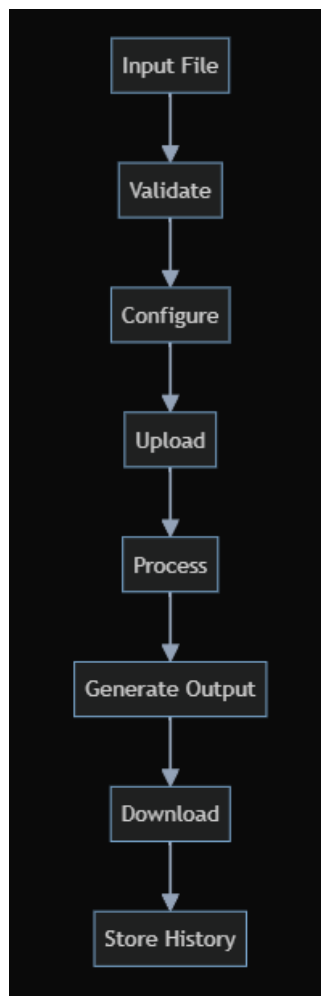
Client / Browser	Backend
File selection	Heavy file processing
File ordering	PDF transformations
PDF.js preview	Document processing

Page thumbnails	Translation
Browser-side helpers	Output generation
Canvas/Image helpers	Persistent history

The documented engineering approach is to use client-side preview helpers while keeping heavier transformations behind the backend API.

19.18 File Processing Summary

Quill's file-processing workflow is designed around a clear input-to-output pipeline:



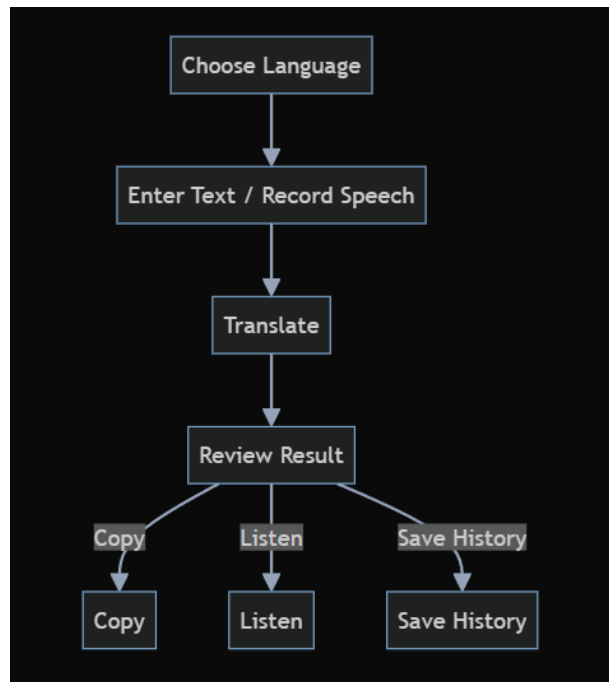
The process can change depending on the selected operation, but the overall architecture remains consistent. This allows PDF, image, Word and document workflows to share a common user experience while using different backend processing logic.

20. Translation Workflow

Translation is one of the main workflows of Quill. The application is designed to support text, voice, multilingual, and speech-based translation without requiring the user to switch to another application. The translation workflow combines **language selection, input handling, translation preferences, backend processing, result review, and history management**.

20.1 Overview of the Translation Workflow

The basic translation workflow documented for Quill is:



For documents, the workflow extends to file upload, text extraction, translation, output generation, and download.

20.2 Step 1 — Select Source and Target Language

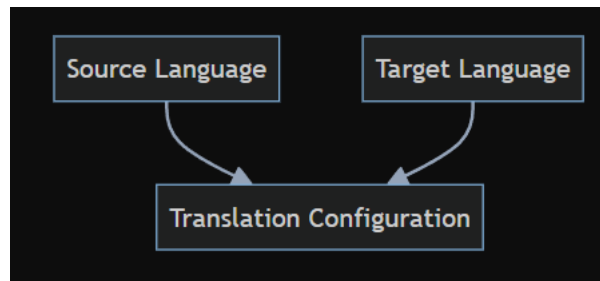
The first step is selecting the language configuration for the translation.

Quill supports:

- Source-language selection
- Target-language selection
- Automatic source-language detection
- Multiple target languages
- Auto-swap language functionality

The application is built around **25 supported languages**, including English, Hindi, Marathi, Tamil, Telugu, Malayalam, Bengali, Gujarati, Spanish, French, German, Italian, Portuguese, Russian, Japanese, Korean, Chinese, Arabic, Turkish, Dutch, Polish, Swedish, Indonesian, Vietnamese, and Ukrainian. Exact availability can depend on the selected translation service.

A simple representation is:



20.3 Step 2 — Provide Translation Input

Quill supports multiple types of translation input.

Text Input

The user enters the original content directly into the translation workspace.

Speech Input

The user can record speech through the microphone. Quill's voice workflow includes speech recognition and locale selection.

Voice and Speech Translation

The broader translation workspace supports four documented modes:

Mode	Input	Output
Text → Text	Written text	Translated text
Text → Speech	Written text	Spoken translation
Speech → Text	Spoken input	Translated text
Speech → Speech	Spoken input	Spoken translation

These four modes are part of Quill's documented translation feature set.

20.4 Step 3 — Configure Translation Preferences

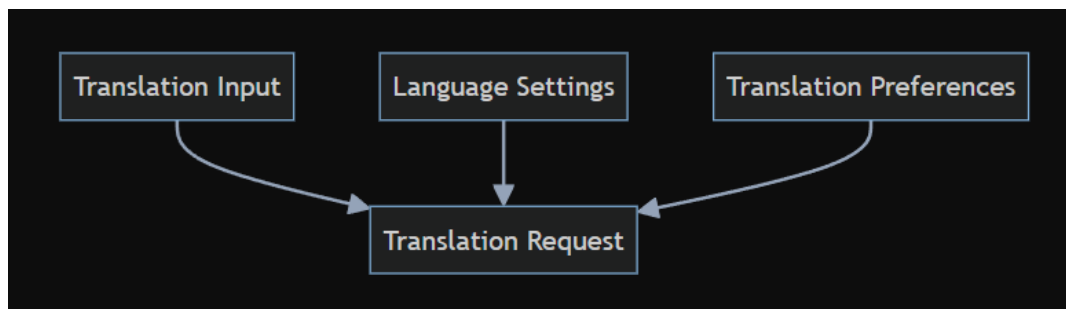
Before processing, users can control how the translation should behave.

Available documented preferences include:

- Translation style
- Formality
- Translation speed
- Preserve formatting
- Original-text visibility
- Voice selection for speech output

These preferences allow the same translation workspace to support different types of output requirements.

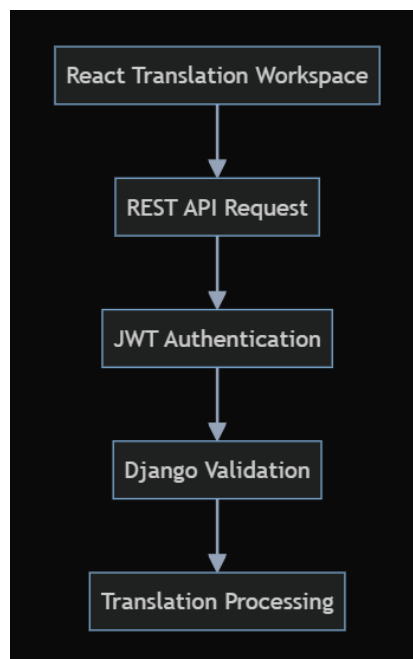
Conceptually:



20.5 Step 4 — Send Translation Request

The frontend prepares the selected input and sends the translation request to the Django REST backend.

The general architecture is:



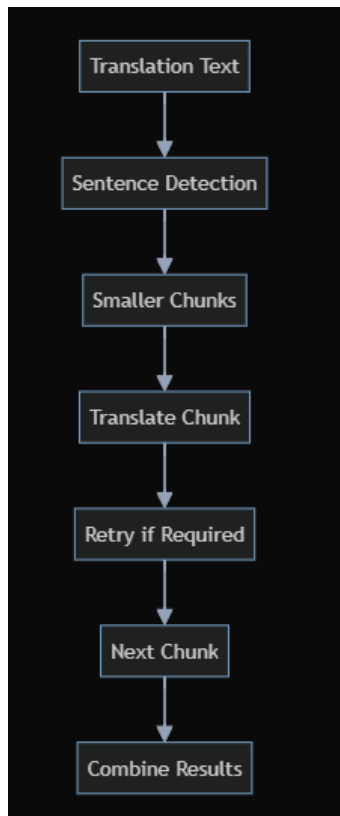
The Quill architecture places authentication, API routing, validation, and processing inside the Django REST backend.

20.6 Step 5 — Backend Translation Processing

The backend receives the request and performs the required translation operation.

For longer text, Quill uses a **sentence-aware chunking approach**. The documented `splitChunks` utility targets roughly **450 characters**, prefers sentence boundaries, handles unusually long sentences, translates chunks sequentially, retries failed chunks, and waits between requests.

The processing can therefore be represented as:



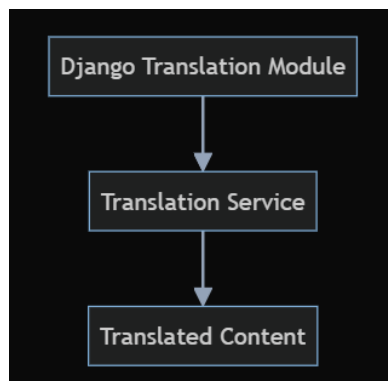
This prevents a very long document from being treated as one oversized translation request.

20.7 Step 6 — Translation Service Integration

Quill's documentation describes multiple translation-service fallbacks with credentials supplied through environment configuration.

The exact translation provider can vary depending on the workflow and runtime configuration.

The logical structure is:

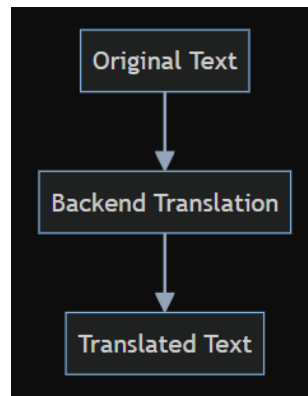


The frontend does not need to manage provider-specific credentials because the translation processing is handled through the backend.

20.8 Step 7 — Build the Translation Result

After translation processing is completed, the backend returns the translated content to the frontend.

For a normal text translation:



The frontend can then display the result for the user to review.

For document translation, the backend performs additional output-generation steps so that the translated document can be reviewed and downloaded.

20.9 Step 8 — Review the Translation

After processing, the user can review the generated translation.

The translation workspace supports:

- Viewing the translated content
- Viewing the original text
- Copying the result
- Listening to speech output where available
- Reviewing the result before saving it

The documented translation workflow explicitly includes **review** before copying, listening, or saving the translation to history.

20.10 Step 9 — Speech Output

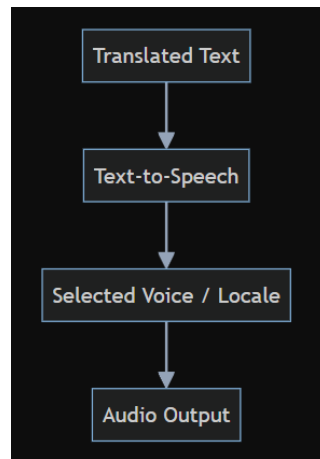
When a speech-based translation mode is selected, the translated text can be converted into spoken output.

The voice workflow includes:

- Multilingual TTS voices
- Locale selection
- Audio output URLs
- Translation history integration

Quill also uses audio processing support such as FFmpeg-based normalization in its voice workflow.

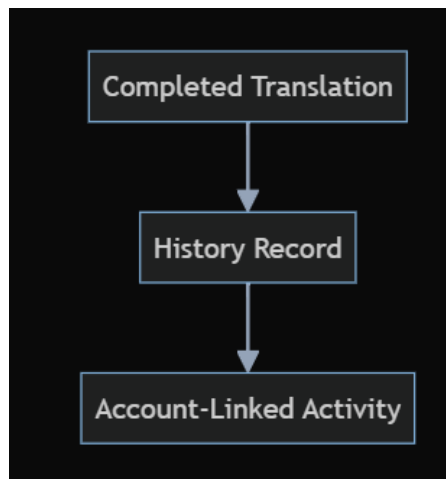
The conceptual workflow is:



20.11 Step 10 — Save to Translation History

After completing a translation, the user can save the operation into the account-linked history.

The history system supports translation-related activity along with document, tool, and Combine operations. Users can later search, filter, preview, download, or delete their history records.

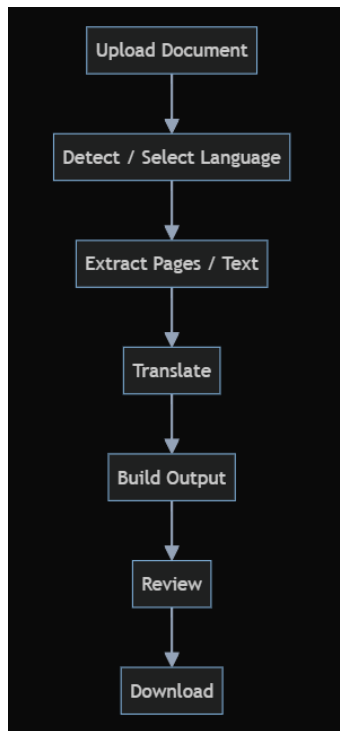


This makes previous translations discoverable from the History workspace.

20.12 Document Translation Workflow

Quill also supports translation of multi-page documents.

The documented document workflow is:



The backend uses page-oriented extraction and document-specific handling, with documented output formats including **PDF, DOCX, TXT and HTML**.

20.13 Page-Based Document Translation

For PDF documents, Quill maintains page-level information during processing.

The backend:

1. Opens the document.
2. Iterates through pages.
3. Extracts page text.
4. Maintains page count and per-page content.
5. Uses the page model during translation, review, and output generation.

This approach supports original and translated page viewing and page-based translation.

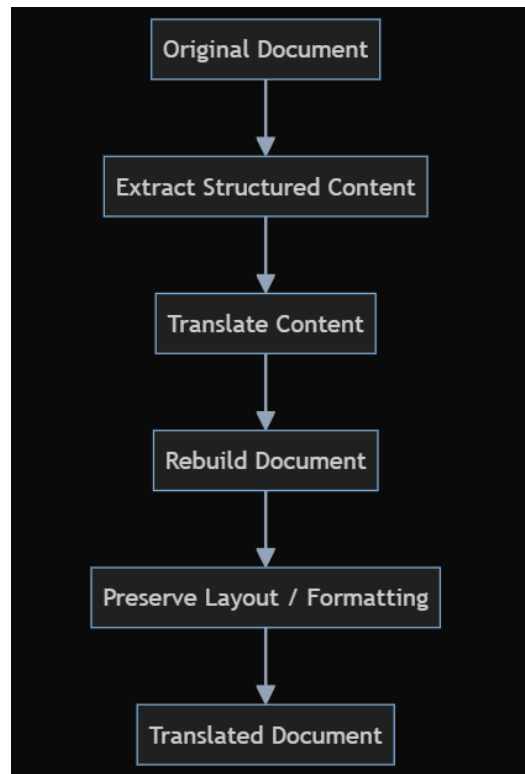
20.14 Layout and Formatting Preservation

Quill's document workflow includes:

- Layout-aware processing
- Formatting preservation
- Original/translated page viewing
- Page-based processing

The application therefore attempts to keep the translated document structure close to the original during output generation.

Conceptually:



20.15 Translation Error Handling

Translation is an external-service-dependent workflow, so the processing logic includes retry handling for failed translation chunks.

The documented chunking implementation:

- Splits the text into smaller chunks.
- Processes them sequentially.
- Retries failed chunks.
- Waits between calls.
- Continues with the next chunk.
- Joins the translated results.

This makes long-form translation processing more controlled than sending the whole document in one request.

20.16 Complete Translation Architecture

The overall Quill translation workflow can be represented as:



20.17 Translation Workflow Summary

The complete workflow can be summarized as:

Stage	Main Activity
1. Language Setup	Select or detect source and target languages
2. Input	Enter text or provide speech
3. Preferences	Style, formality, speed, formatting and voice
4. API Request	Send authenticated request to Django
5. Processing	Translate content and handle long text
6. Output	Return translated text or speech
7. Review	Check the generated result
8. History	Save the completed translation

Quill extends the same concept to documents by adding upload, page extraction, output generation, review, and download.

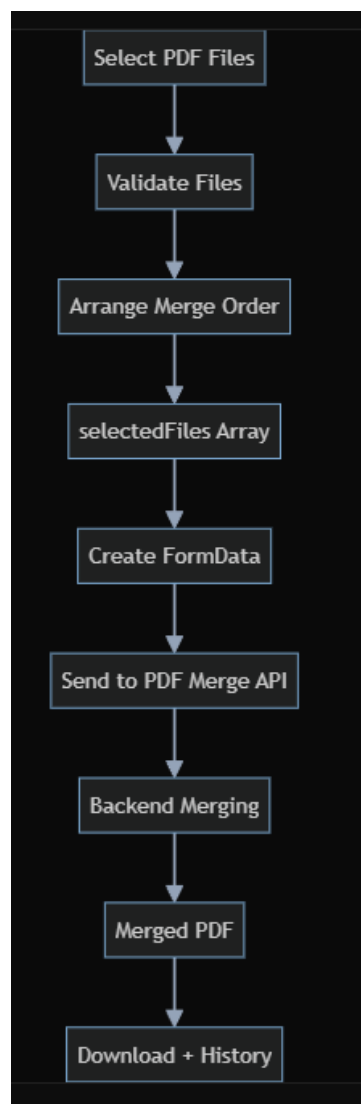
21. PDF Merge & File Reordering Algorithm

PDF Merge is one of the main operations in Quill's **Combine & Convert** workspace. It allows users to select multiple PDF files, arrange them in the required sequence, and send them to the backend for merging into a single document.

The important part of this workflow is that the **order selected by the user is preserved throughout processing**. Quill represents the merge order using the `selectedFiles` array, provides both drag-and-drop and move-up/move-down controls, and then appends the files to `FormData` in exactly that array order.

21.1 PDF Merge Overview

The PDF Merge workflow can be represented as:



Quill's documented API surface includes the `/api/combine/pdf-merge/` endpoint for PDF merging.

21.2 Selecting PDF Files

The PDF Merge tool is defined as a **Merge** operation that accepts multiple PDF files.

The current source identifies the tool as:

PDF Merge

Category: Merge

Description: Merge multiple PDF files into a single document

Only merge tools receive file-level drag reordering in the current source. This includes:

- PDF Merge
- Word Merge
- Image Merge

Other tools such as N-up and Image to PDF use their own processing behavior and do not use this particular file-level merge ordering mechanism.

21.3 Merge File Validation

Before the merge request is processed, Quill validates the selected files.

The current source defines:

```
const MIN_MERGE_FILES = 2;
```

```
const MAX_MERGE_FILES = 10;
```

Therefore, in this source snapshot:

- At least **2 files** are required for a merge.
- At most **10 files** are accepted by the frontend validation.
- A selection outside this range is rejected before processing.

The source also applies the maximum limit while adding multiple selected files:

```
return [  
  ...previous,  
  ...array,  
].slice(  
  0,  
  MAX_MERGE_FILES  
);
```

This keeps the number of files inside the configured frontend limit.

Report note: The reviewed source snapshot uses `MAX_MERGE_FILES = 10`. The project documentation separately mentions a requested future increase to 30, but this page should describe the verified source value as **10** rather than presenting 30 as an already deployed limit.

21.4 selectedFiles — Core Data Structure

The most important data structure for PDF merging is:

```
const [selectedFiles, setSelectedFiles] =  
  useState<File[]>([]);
```

The array stores the currently selected files in their **final merge order**.

For example:

selectedFiles

Index 0 → File A

Index 1 → File B

Index 2 → File C

Index 3 → File D

The position of each file is therefore meaningful.

If the user changes the order to:

File A

File D

File B

File C

the array becomes:

Index 0 → File A

Index 1 → File D

Index 2 → File B

Index 3 → File C

This array becomes the source of truth for the final merge order.

21.5 Drag-and-Drop Reordering

Quill provides a dedicated drag-and-drop algorithm for changing the position of merge files.

The implementation maintains two indexes:

draggedFileIndex

dragOverFileIndex

draggedFileIndex identifies the file currently being moved, while dragOverFileIndex identifies the position over which the file is being dragged.

The drag operation can be divided into three stages.

Stage 1 — Drag Start

When the user starts dragging a file:

```
setDraggedFileIndex(index);
```

```
setDragOverFileIndex(index);
```

The browser drag operation is configured with:

```
event.dataTransfer.effectAllowed = "move";
```

The source index is also placed into the drag data using text/plain.

21.6 Drag Over

While the user moves the file over another position, Quill checks whether reordering is allowed.

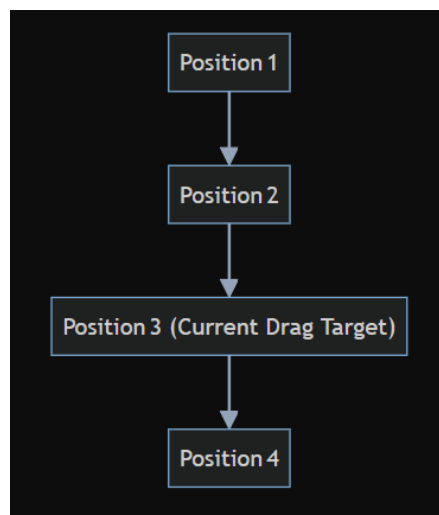
The source verifies:

```
canReorderFiles
```

and that a valid dragged file exists.

The target index is then stored as the current drop position.

Conceptually:



21.7 Drop Algorithm

When the user releases the file, Quill obtains the original source index from the drag data.

The implementation first validates:

- Whether reordering is allowed
- Whether the source index is valid
- Whether the target index is valid
- Whether the source and target are different

If the indexes are valid and different, the array is reordered.

The core algorithm is:

```
const next = [...previous];
```

```
const [movedFile] =
```

```
  next.splice(sourceIndex, 1);
```

```
next.splice(  
    targetIndex,  
    0,  
    movedFile  
);
```

```
return next;
```

This is the central **PDF merge file-reordering algorithm** used by Quill.

21.8 Understanding the splice() Algorithm

Suppose the current order is:

A B C D

The user moves **D** from index 3 to index 1.

Step 1 — Remove D

```
next.splice(3, 1);
```

Result:

A B C

The removed file is stored in:

movedFile

Step 2 — Insert D at index 1

```
next.splice(1, 0, movedFile);
```

Result:

A D B C

Therefore:

Before: A B C D

After: A D B C

The resulting array is returned as the new selectedFiles state.

21.9 Move Up and Move Down Controls

Quill does not depend only on drag-and-drop.

The selected-file interface also provides **Move Up** and **Move Down** controls for merge files. The buttons are disabled when a file is already at the first or last position respectively.

The interface therefore gives two ways to change order:

Method	Purpose
Drag & Drop	Quickly move a file to another position
Move Up / Down	Precisely shift a file one position at a time

Both methods ultimately modify the same `selectedFiles` array.

21.10 Merge Position Indicator

The UI also displays the position of each file during merge operations.

For example:

- 1 document-a.pdf
- 2 document-d.pdf
- 3 document-b.pdf
- 4 document-c.pdf

The source displays a **Merge Position #** value based on the current array index.

This gives the user a direct visual confirmation of the sequence that will be used during merging.

21.11 FormData Construction

After the final order has been established, the PDF files are added to a `FormData` object.

The processing logic explicitly documents that `selectedFiles` is **already reordered** at this stage.

The files are appended sequentially:

```
selectedFiles.forEach(
  (file) => {
    formData.append(
      "files",
      file
    );
  }
);
```

Therefore, if:

`selectedFiles =`

`[A, D, B, C]`

then the multipart request receives:

files = A

files = D

files = B

files = C

The backend therefore receives the files in the same order chosen by the user.

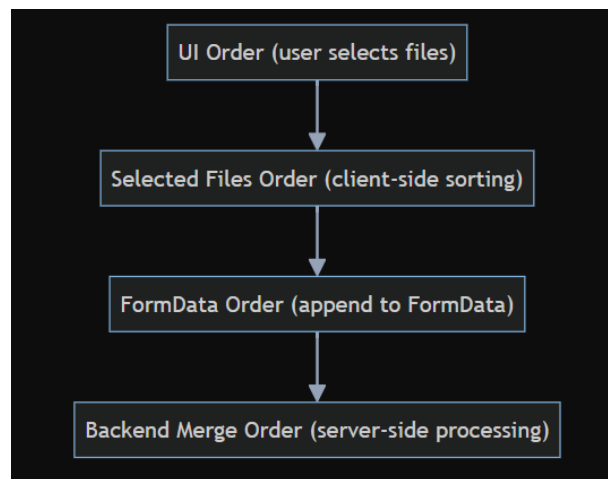
21.12 Why Array Order Is Important

The important design decision is that Quill does not maintain a separate merge-order structure.

Instead:

Array position = merge position

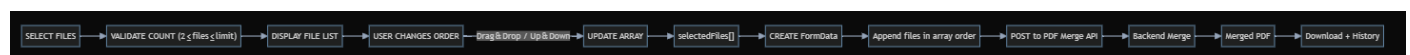
This provides a simple deterministic relationship between the UI and backend request.



The technical documentation explicitly identifies this array-based file ordering as an engineering decision because it maps directly to multipart upload order.

21.13 Complete PDF Merge Algorithm

The entire algorithm can be summarized as:



21.14 Example Dry Run

Consider four PDF files:

A.pdf

B.pdf

C.pdf

D.pdf

Initial array:

Index 0 → A.pdf

Index 1 → B.pdf

Index 2 → C.pdf

Index 3 → D.pdf

The user drags D.pdf to position 2.

Before

1. A.pdf

2. B.pdf

3. C.pdf

4. D.pdf

Source and target

sourceIndex = 3

targetIndex = 1

Remove D

A.pdf

B.pdf

C.pdf

Insert D at index 1

A.pdf

D.pdf

B.pdf

C.pdf

Final array

selectedFiles = [

A.pdf,

D.pdf,

B.pdf,

C.pdf

]

FormData order

files → A.pdf

files → D.pdf

files → B.pdf

files → C.pdf

Backend merge order

A.pdf

D.pdf

B.pdf

C.pdf

Thus, the document produced by the backend follows the exact order selected in the interface. This is the key property of the algorithm.

21.15 Validation Before Processing

The merge operation does not immediately call the backend after files are selected.

The frontend first checks the validation state:

```
if (  
  !activeTool ||  
  selectedFiles.length === 0 ||  
  !validation.valid  
) {  
  return;  
}
```

This prevents processing when:

- No tool is selected
- No file is selected
- The selected file count is invalid
- The selected operation fails validation

This is important because validation occurs before the expensive backend operation begins.

21.16 PDF Merge API Communication

Once validation succeeds, Quill creates the request and sends the files to the configured Combine API.

The documented representative endpoint is:

`/api/combine/pdf-merge/`

The frontend gets its API base from the configured environment variable and sends the multipart request to the selected tool's endpoint.

The communication model is:

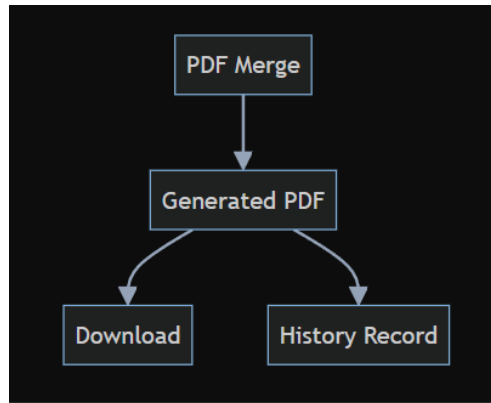


21.17 History After PDF Merge

Once processing succeeds, the generated result can be associated with the user's history.

The Combine workspace maintains account-linked history information and supports retrieving Combine history through the backend.

Therefore:



This allows the user to access previously processed results from the History workspace.

21.18 Complexity of the Reordering Algorithm

The important operation in the reordering algorithm is:

`splice()`

Removing and inserting an item into an array can require shifting other elements.

For an array containing **n files**, the reordering operation is therefore generally **$O(n)$** in the worst case.

The merge-order state itself requires **$O(n)$** space because the `selectedFiles` array contains the selected file references.

The actual PDF merge processing is separate from this frontend ordering algorithm and may have its own cost depending on PDF size, number of pages, and backend processing.

21.19 Engineering Decision

The array-based approach was chosen because it creates a direct relationship between:



The project documentation explicitly identifies **array-based file ordering** as an engineering decision because it is deterministic and maps directly to multipart upload order.

This makes the behavior easier to reason about and reduces the need for an additional synchronization mechanism between UI order and request order.

21.20 PDF Merge Algorithm Summary

Stage	Data / Logic
File Selection	PDFs stored in <code>selectedFiles</code>

Validation	Minimum and maximum merge count checked
Drag Start	Store source index
Drag Over	Track target index
Drop	Remove and reinsert using splice()
Manual Reorder	Move Up / Move Down controls
Final Order	selectedFiles becomes source of truth
Request	Files appended to FormData sequentially
API	/api/combine/pdf-merge/
Backend	Files merged according to request order
Output	Merged PDF
History	Operation can be retained in Combine history

The core idea is simple: **Quill preserves user-selected order from the interface all the way to the backend merge request.**

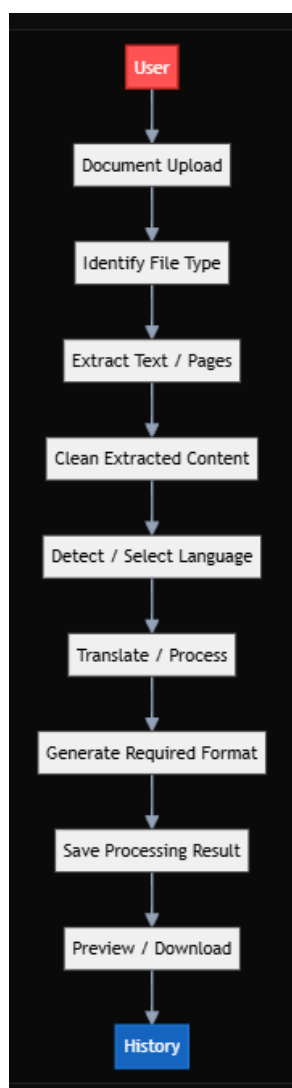
22. Document Processing & Conversion Logic

Quill provides a dedicated document-processing workflow for handling **PDF, DOCX, TXT, HTML and related document formats**. The processing system is designed to separate document extraction, text cleaning, translation, page handling, output generation, and download.

The backend uses a page-aware document model so that multi-page files can be processed while retaining page boundaries. It also contains format-specific output logic for generating PDF, DOCX, TXT and HTML files.

22.1 Overview of Document Processing

The complete document-processing workflow can be represented as:



The documented Quill document workflow follows upload → detect/select language → extract pages/text → translate → build output → review/download.

22.2 File-Type Detection

The backend first determines how the uploaded file should be processed based on its format.

The reviewed document-processing source supports different extraction strategies for:

File Type	Processing Method
-----------	-------------------

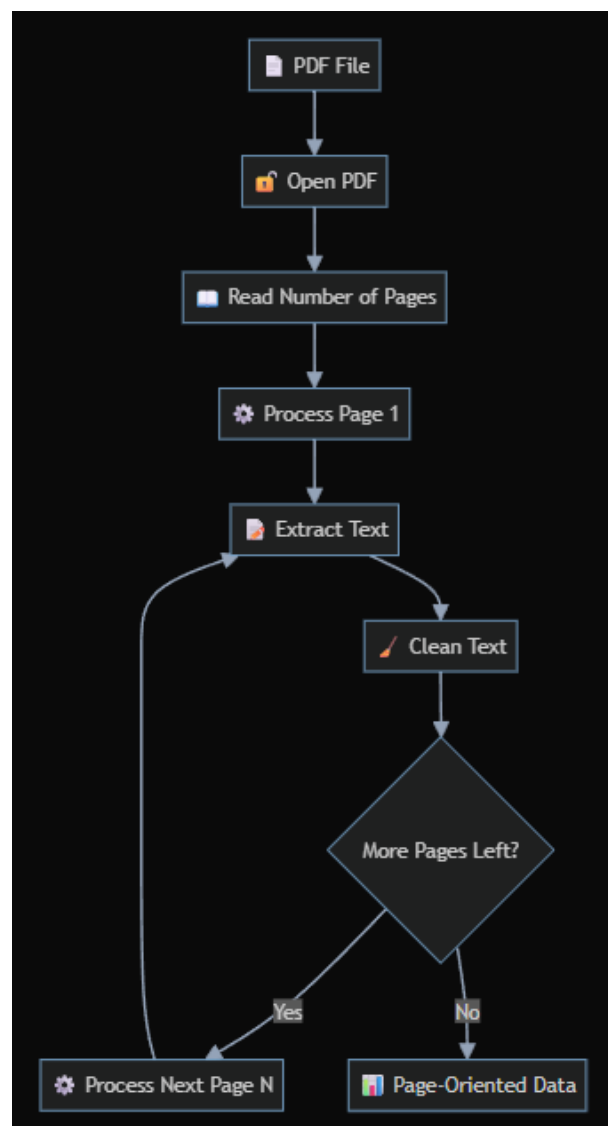
PDF	Page-by-page PDF extraction
TXT	Plain-text reading
MD	Plain-text/markdown reading
RTF	Text extraction with page-break handling
DOCX	Paragraph and explicit page-break processing
HTML	HTML parsing followed by text extraction

The backend's `DocumentProcessor.extract_pages()` method uses different logic depending on the detected file type.

22.3 PDF Extraction Logic

For PDF files, the backend uses a page-aware parser and iterates over the actual PDF pages.

The process is:



The implementation uses PdfReader, obtains the real page count, processes each page separately, and stores the cleaned text for every page.

A notable implementation detail is that **PDF page count is based on the actual PDF page structure rather than estimating pages from character length.**

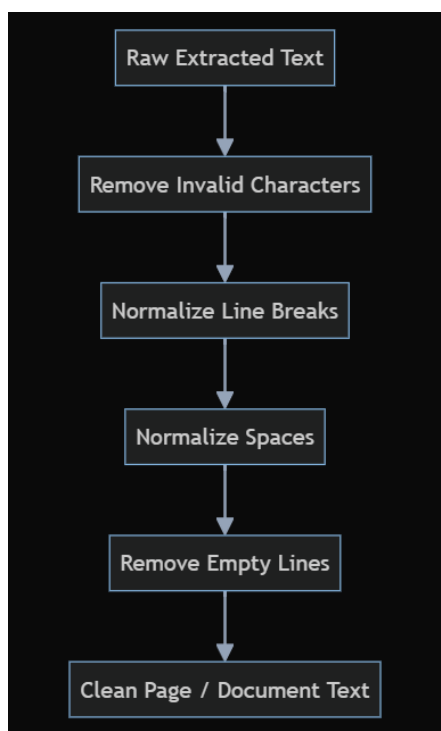
22.4 Text Cleaning

Extracted text is cleaned before further processing.

The backend removes unwanted control characters, normalizes line endings, removes unnecessary spaces, and removes empty lines.

For normal whole-document text, the cleaned lines are combined into a readable text representation.

For page-level processing, however, line breaks are preserved so that the page boundary and internal structure remain available.



22.5 TXT, Markdown and RTF Processing

For text-oriented formats such as TXT, MD and RTF, the backend reads the file as text.

The implementation also preserves explicitly supplied page breaks.

Recognized page separators include:

--- PAGE BREAK ---

and form-feed page breaks.

The resulting sections are cleaned individually and represented as separate page entries.

This allows even a simple text document to participate in the same page-oriented processing model used by the application.

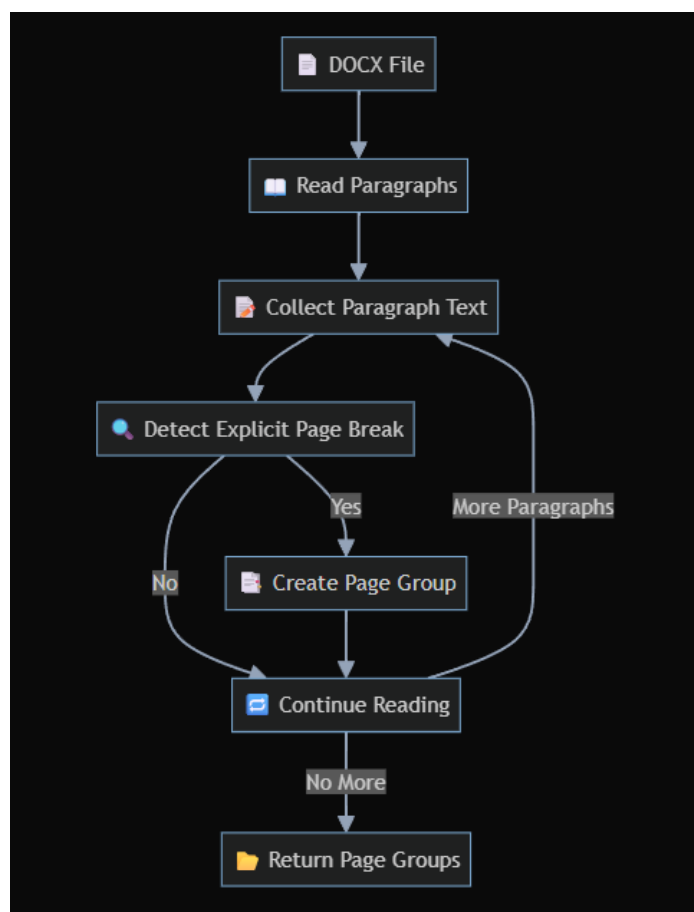
22.6 DOCX Processing Logic

DOCX processing is handled differently from PDF extraction.

The backend reads the Word document using the document structure and processes its paragraphs.

When explicit page breaks are present, the source detects them within Word document runs and creates separate page groups.

The logic is:



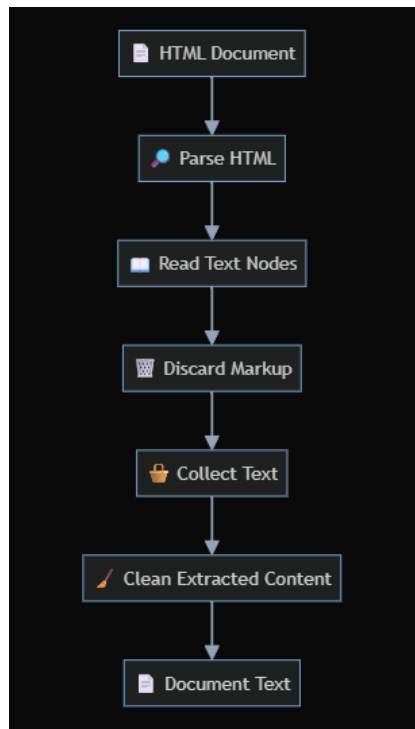
When no explicit page break is detected, the extracted paragraphs can be combined into a single document-text representation.

22.7 HTML Processing Logic

HTML documents are processed by parsing the HTML structure and extracting textual content.

The backend uses an HTML parser whose data handlers collect meaningful text content while ignoring HTML markup itself.

The conceptual process is:



This allows HTML content to enter the same downstream processing pipeline as other text-based documents.

22.8 Page-Oriented Document Model

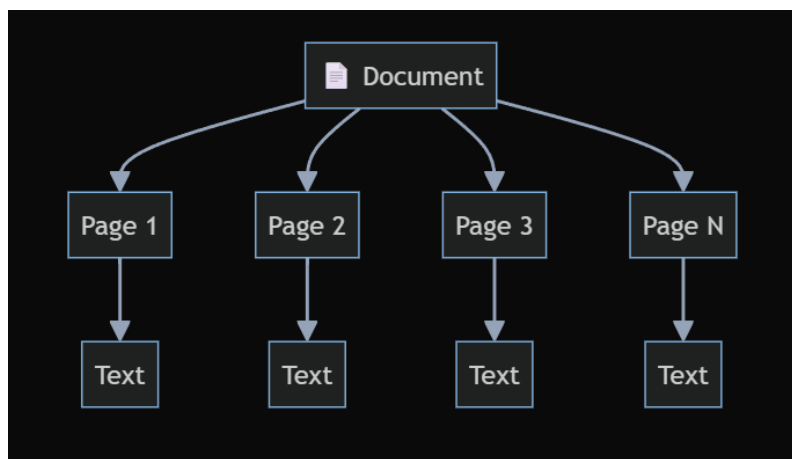
After extraction, Quill maintains a list in which each item corresponds to a source page where applicable.

This page-oriented model is useful for:

- Page-by-page translation
- Original/translated page viewing
- Output reconstruction
- PDF generation
- Review workflows

The documentation explicitly describes the backend as using page-oriented PDF extraction and document-specific handling for formats such as DOCX.

The resulting structure can be visualized as:

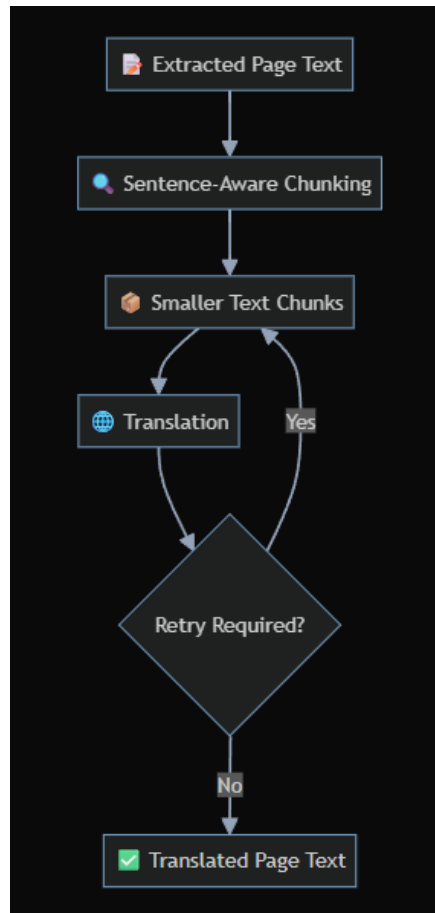


22.9 Translation Within Document Processing

After extraction, document text can enter the translation pipeline.

For larger text, Quill uses sentence-aware chunking rather than sending the complete document as one oversized request.

The documented process uses roughly **450-character target chunks**, prefers sentence boundaries, handles unusually long sentences, retries failed chunks, and processes chunks sequentially.



The translated page content can then be passed to the output-generation stage.

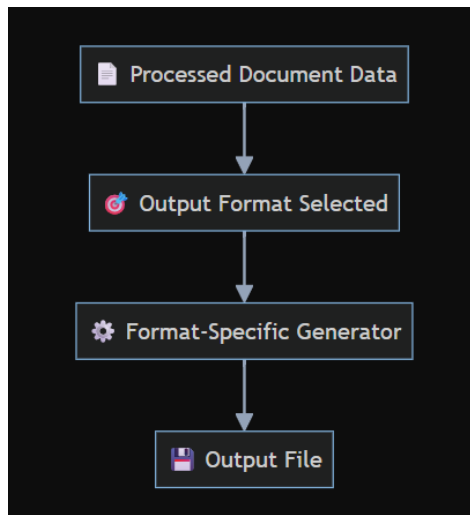
22.10 Output Conversion Logic

Once processing is complete, the backend generates the requested output format.

The reviewed `DocumentProcessor.create_output()` implementation explicitly handles:

- TXT
- DOCX
- PDF
- HTML

The general conversion architecture is:



22.11 TXT Output Generation

For TXT output, the backend creates a plain-text representation.

When page data exists, each page is represented using a page marker such as:

--- PAGE 1 ---

Page content...

--- PAGE 2 ---

Page content...

The backend then writes the resulting text to the requested output file.

This preserves the logical separation between pages even though TXT itself is not a visual page-layout format.

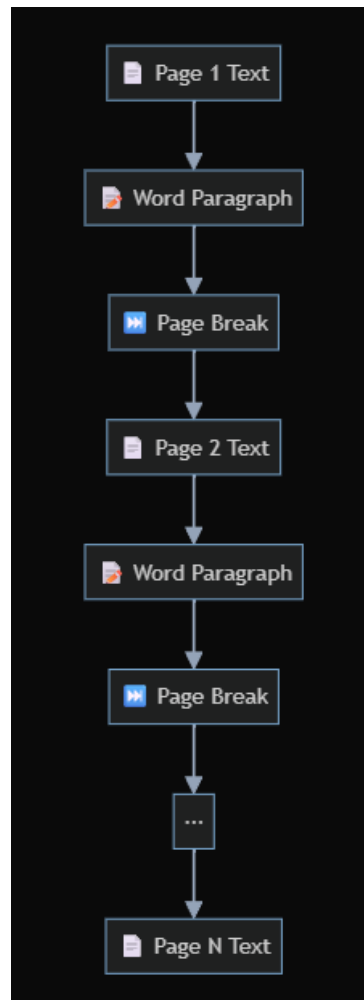
22.12 DOCX Output Generation

For DOCX output, the backend creates a new Word document.

When multiple pages are present:

- Each page's text is added as document content.
- An explicit Word page break is inserted between pages.
- The final document is saved as a .docx file.

Conceptually:

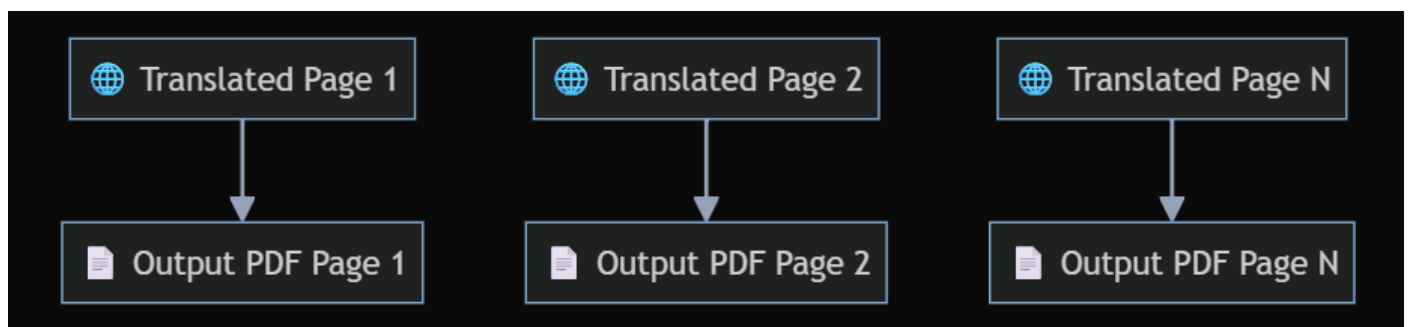


22.13 PDF Output Generation

PDF generation contains additional logic because the system needs to create an actual page-based document.

For PDF input, the backend attempts to preserve the original page dimensions when generating the translated output. It reads the original PDF's page sizes and applies the corresponding size to the output page where possible.

The implementation creates **exactly one output page for each translated source page** in the page-based PDF-generation path.



22.14 PDF Text Fitting Algorithm

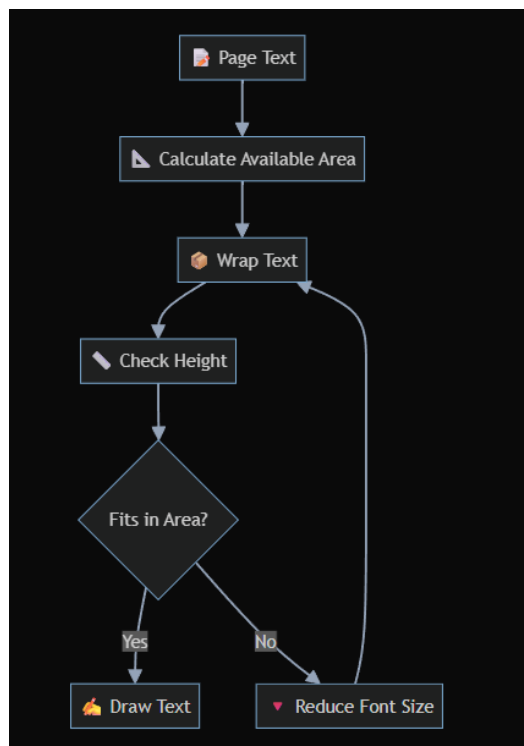
The PDF generator also contains a text-fitting mechanism.

The backend:

1. Defines page margins.

2. Calculates the available page width and height.
3. Wraps text according to the available width.
4. Starts from a readable font size.
5. Reduces the font size when the content does not fit.
6. Continues until the text fits or reaches the safety limit.
7. Draws the resulting lines onto the PDF page.

This logic can be illustrated as:



This is a good technical diagram for Page 22 because it shows an actual implementation detail rather than only the high-level workflow.

22.15 Unicode and Multilingual PDF Handling

The PDF-generation code attempts to register an available Unicode-capable font before drawing the translated content.

The source checks several font candidates and falls back to Helvetica if an appropriate font cannot be loaded.

This is particularly relevant to Quill because its translation workflow supports multiple languages, including scripts that require Unicode font support.

The purpose of this processing stage is to make multilingual output safer than relying only on a basic Latin-oriented font.

22.16 HTML Output Generation

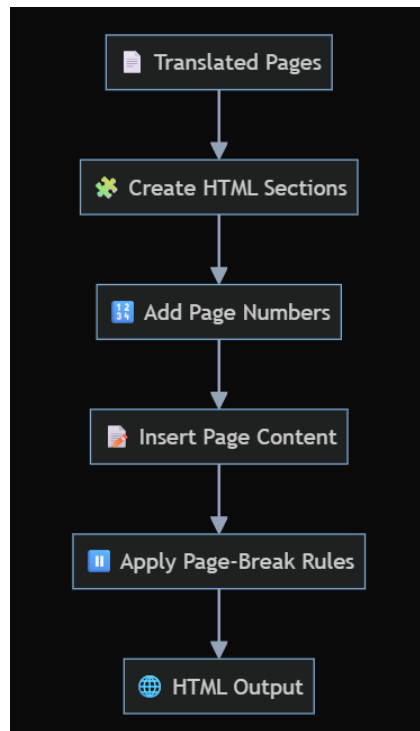
For HTML output, the backend generates an HTML document containing separate page sections when page data is available.

Each generated section can contain:

- Page number
- Page content
- Page separation
- Basic document styling

The output uses explicit page-oriented HTML sections and print-related page-break rules.

The logical structure is:



22.17 Conversion Between Different Formats

Quill's broader Combine & Convert workspace supports document conversions such as:

Conversion	Purpose
PDF → Word	Convert PDF content into editable Word documents
PDF → Excel	Extract PDF content/tables into Excel
Excel → PDF	Convert spreadsheet content into PDF
Word → PDF	Convert Word documents into PDF
Word Merge	Combine multiple Word documents

These conversion tools are documented as part of Quill's Combine & Convert feature set.

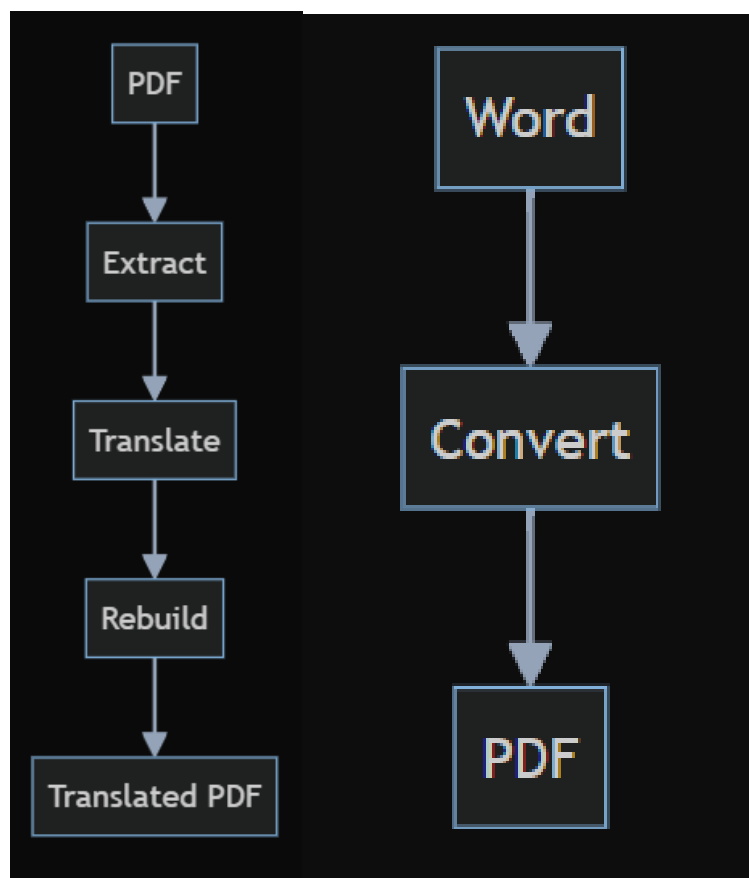
The important distinction is that **document translation output generation** and **general file conversion** are related but separate workflows within the application.

22.18 Document Processing vs File Conversion

Document Processing	File Conversion
Extract content	Transform existing file format
Detect/select language	Select destination format
Translate text	Convert representation
Maintain page model	Generate converted file
Review translated content	Download converted result

For example:

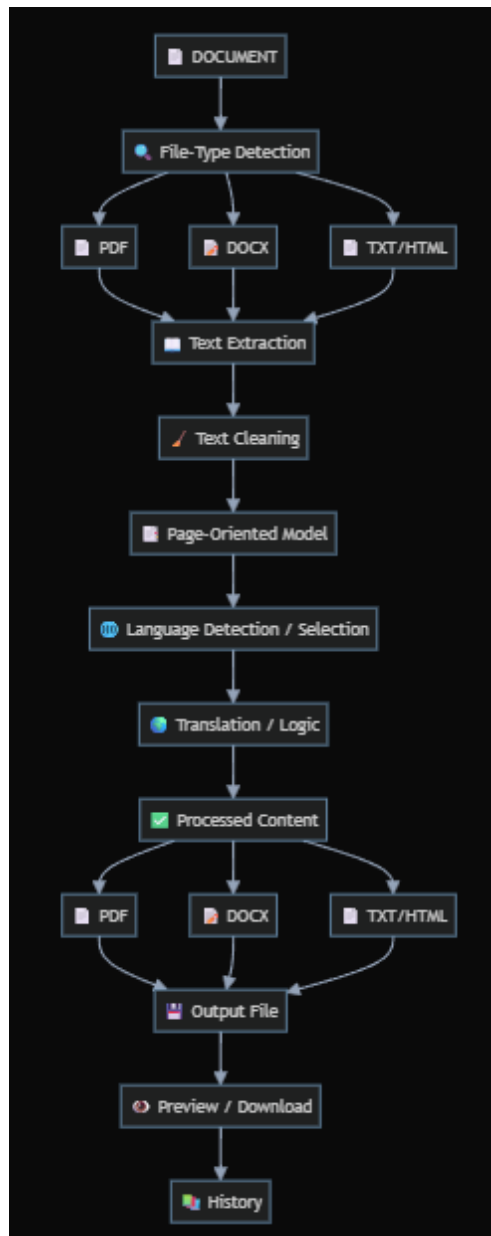
Document translation:



This distinction is useful because translation modifies the **content**, while conversion primarily changes the **file representation**.

22.19 Complete Document Processing Architecture

The complete processing architecture can be represented as:

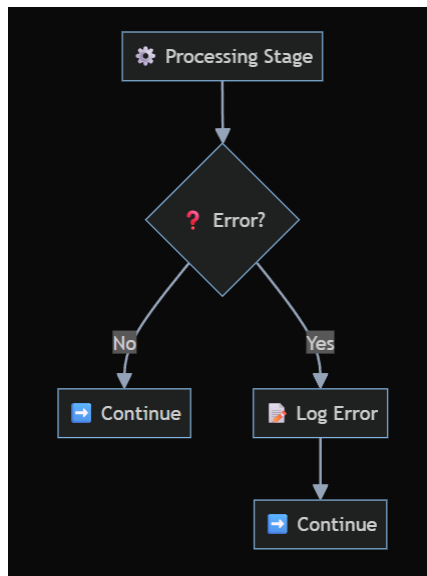


22.20 Error Handling During Processing

Document processing can fail at several stages, so the backend contains error handling around extraction and output operations.

For example, PDF page text extraction is wrapped in exception handling. If extraction fails for a particular page, the implementation logs the problem and continues with an empty page-text value rather than crashing the entire page loop.

This provides a controlled processing path:



The same overall application also checks processing and download responses at the API layer before presenting the final output.

22.21 Document Processing Summary

The Quill document-processing system can be summarized through five major stages:

Stage	Main Work
Extraction	Read document content according to file type
Cleaning	Normalize and clean extracted content
Processing	Translate or transform document data
Generation	Create PDF, DOCX, TXT or HTML output
Delivery	Preview/download result and connect it to history

The actual implementation provides separate handling for PDF, DOCX, text-oriented and HTML documents, while the page-oriented model supports structured processing and output generation.

23. Error Handling & Validation

Error handling and validation are important parts of Quill because the application accepts user text, documents, PDFs, images, configuration options, authentication credentials, and downloadable outputs. Validation prevents invalid requests from reaching expensive processing stages, while error handling gives the user a clear indication of what went wrong.

The documented validation strategy covers **tool input, file count, authentication, processing failures, download responses, and tool-specific options**.

23.1 Purpose of Validation

Validation is performed before an operation is processed.

Its main purpose is to ensure that:

- A required file is present.
- The selected number of files is valid.
- The selected tool supports the chosen input.
- Tool-specific options are valid.
- The user has a valid authenticated session.
- A generated output is actually available before download.

This reduces unnecessary backend processing and allows problems to be reported close to the point where they occur.

23.2 Validation at Different Layers

Quill uses validation at more than one stage.

Layer	Main Responsibility
Frontend	File selection, file count, tool compatibility and basic option validation
API / Backend	Authentication, request validation and processing validation
Processing	Detect processing failures and invalid input conditions
Download	Verify that a valid output response is available

The architecture places Django REST between the frontend and processing layers, with authentication, routing, validation and business logic handled by the backend.

23.3 Frontend File Validation

The Combine & Convert workspace calculates a validation state before processing.

The current source checks whether:

selectedFiles.length

is valid for the active tool.

A simplified example is:

```
const validation = useMemo(() => {  
  if (!activeTool) {  
    return { valid: true, message: "" };  
  }  
  
  const count = selectedFiles.length;  
  const isMerge = activeTool.category === "Merge";  
  
  if (count === 0) {  
    return {  
      valid: false,  
      message: "Please upload at least 1 file."  
    };  
  }  
  
  // additional validation...  
}, [activeTool, selectedFiles]);
```

The source then applies different rules depending on whether the selected tool is a merge, a multi-file operation, or a single-file operation.

23.4 File Count Validation

File count validation is particularly important for Combine & Convert operations.

The current implementation uses:

```
const MIN_MERGE_FILES = 2;  
const MAX_MERGE_FILES = 10;
```

For merge tools:

- Fewer than 2 files produces a validation error.
- More than the configured maximum produces a validation error.

For single-file tools, selecting more than one file is rejected.

For other multi-file tools, the maximum file limit is also checked.

The project's technical documentation describes this as:

Validation Case	Current Pattern
No file	Request cannot be processed
Merge below minimum	Merge rejected
Merge above maximum	Merge rejected
Single-file tool with multiple files	Extra selection rejected
Multi-file tool over configured maximum	Selection rejected

Note for the report: the reviewed source snapshot uses a maximum of **10 files**. The documentation separately mentions a requested future increase to 30, but 30 should not be described as an already verified deployed limit.

23.5 Tool-Specific Validation

Different tools require different kinds of input.

For example:

Tool Type	Validation Consideration
PDF Merge	Multiple PDF files
Word Merge	Multiple Word files
Image Merge	Multiple image files
PDF Compress	PDF file + compression option
Rotate PDF	PDF + rotation setting
Split PDF	PDF + split configuration
Watermark PDF	PDF + watermark configuration
N-up PDF	Supported files + pages-per-sheet settings

The frontend tool metadata defines accepted extensions, whether multiple files are supported, the backend endpoint, and the history type.

This lets the application perform validation according to the selected tool instead of applying exactly the same rules to every operation.

23.6 Validation of Processing Options

Quill also validates tool-specific settings before processing.

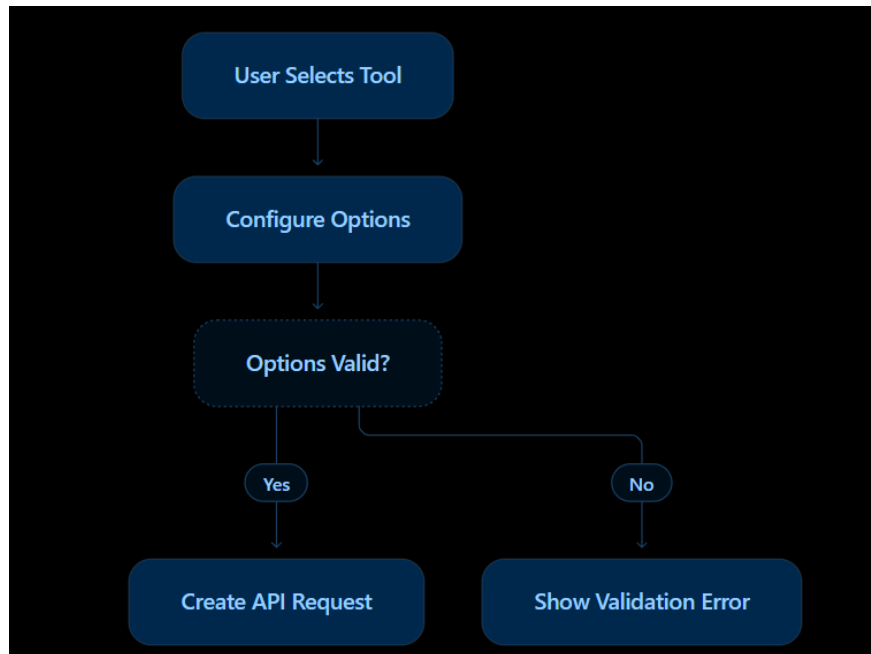
The documented validation area includes:

- Compression settings
- Rotation settings
- Split settings

- Watermark settings
- Image settings
- N-up settings

For example, the frontend places selected operation options into the multipart request before sending it to the backend.

Conceptually:



23.7 Authentication Validation

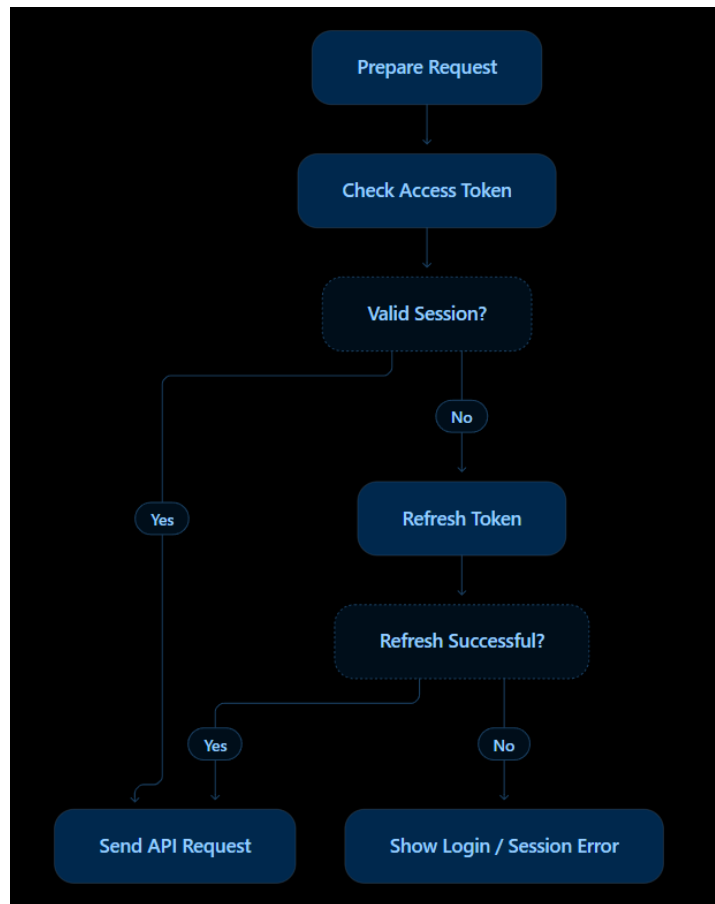
Protected operations require an authenticated session.

The documented authentication architecture uses:

- JWT access tokens
- JWT refresh tokens
- OTP verification
- Optional two-factor authentication

Before a protected processing request is sent, the frontend obtains a valid access token.

The processing code uses a flow equivalent to:



23.8 API-Level Authentication Errors

The frontend explicitly handles an HTTP **401** response from the processing API.

The current source uses logic equivalent to:

```
if (processRes.status === 401) {  
    setStatusMsg({  
        type: "error",  
        text: "Authentication failed. Please login again."  
    });  
  
    setIsProcessing(false);  
    return;  
}
```

This prevents the application from treating an unauthenticated request as a successful processing operation.

The broader technical documentation also identifies **401 handling and token refresh/session errors** as part of the validation and error-handling strategy.

23.9 API Processing Failure

A request can fail even after authentication succeeds.

The frontend therefore checks the HTTP response status:

```
if (!processRes.ok) {  
  const errData =  
    await processRes.json().catch(() => ({}));  
  
  throw new Error(  
    errData.message ||  
    `Processing failed (${processRes.status})`  
  );  
}
```

This allows a backend error message to be surfaced when one is available. Otherwise, the HTTP status is used to create a useful fallback message.

23.10 Invalid or Missing Processing Response

A successful HTTP request does not automatically guarantee a valid application result.

Quill checks whether the returned response contains the expected success state and download information:

```
if (  
  !processData.success ||  
  !processData.download_url  
) {  
  throw new Error(  
    processData.message ||  
    "No download URL received from server"  
  );  
}
```

This adds another validation layer between the backend response and the final download step.

Therefore, Quill checks both:

HTTP-level success

and

application-level success.

23.11 Download Validation

After processing, the frontend makes a separate request to retrieve the generated output.

The download response is checked:

```

if (!downloadRes.ok) {
    throw new Error(
        `Download failed (${downloadRes.status})`
    );
}

```

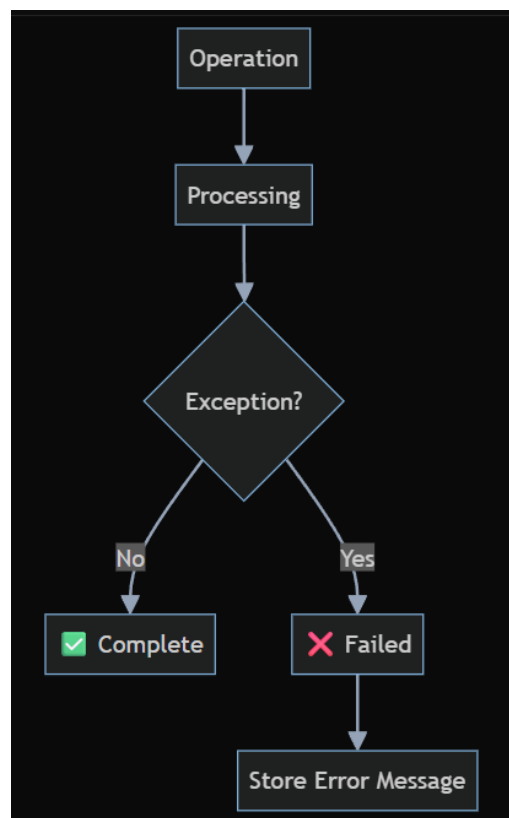
Only after a successful response does the frontend convert the response into a Blob and prepare the downloaded file.

This prevents an unsuccessful download response from being treated as a valid document.

23.12 Backend Processing Errors

The backend also records processing failures.

For example, the documented PDF merge backend creates an operation record and updates its status if an exception occurs:



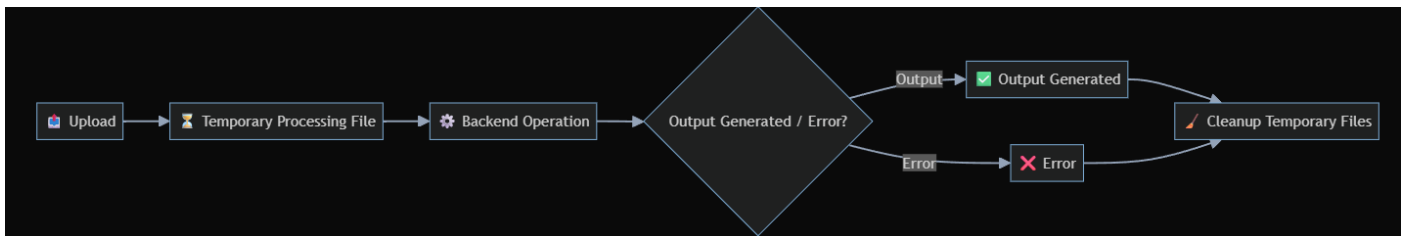
The implementation in the reviewed backend sets an operation status to "failed" and stores the error message when PDF merge processing raises an exception.

23.13 Temporary File Cleanup

File-processing operations may create temporary files during backend processing.

The reviewed PDF merge implementation keeps track of temporary input paths and performs cleanup in a finally block.

Conceptually:



The cleanup step is placed in the finalization stage so that temporary input files are removed after processing, including after an exception.

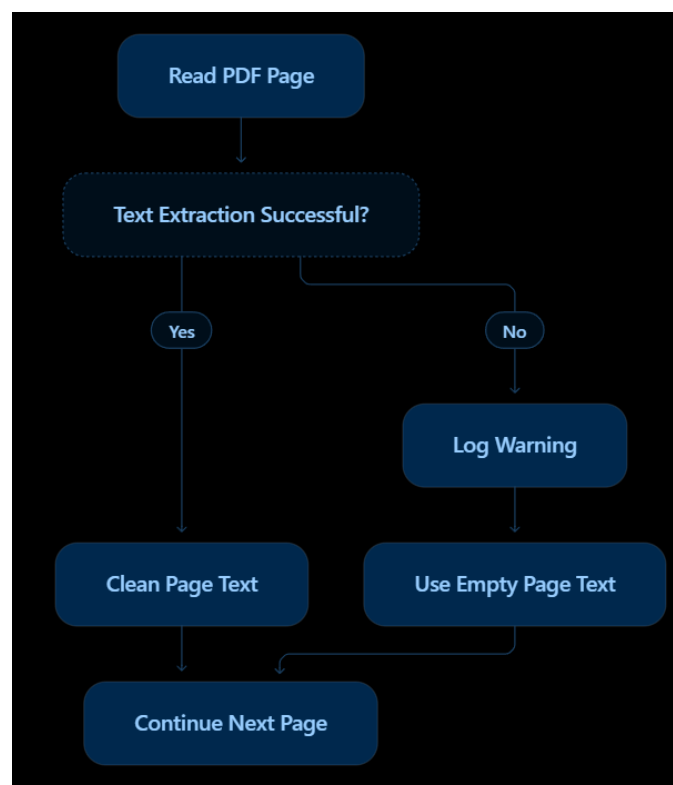
This is important for file-based applications because temporary processing artifacts should not accumulate indefinitely.

23.14 Document Extraction Error Handling

Document processing has its own error handling.

During PDF extraction, the backend wraps individual page extraction in exception handling. If text extraction fails for a particular page, the implementation logs a warning and continues with an empty page-text value for that page.

The approach can be represented as:



This is useful because an extraction problem on one page does not necessarily stop the complete page-processing loop.

23.15 User-Visible Error Messages

Errors should be visible in the interface rather than remaining only in browser or server logs.

The current Combine implementation maintains a status message state:

```

setStatusMsg({
  type: "error",

```

```
text: err.message ||  
    "Failed to process files."  
});
```

The application therefore has a mechanism for showing processing errors directly to the user.

The technical documentation describes these as **user-visible status/toast messages** for backend failures.

23.16 Processing State and Feedback

Quill also maintains a processing state to prevent the interface from behaving as though a request has finished when processing is still underway.

The current source updates a processing step such as:

Preparing files...

and later:

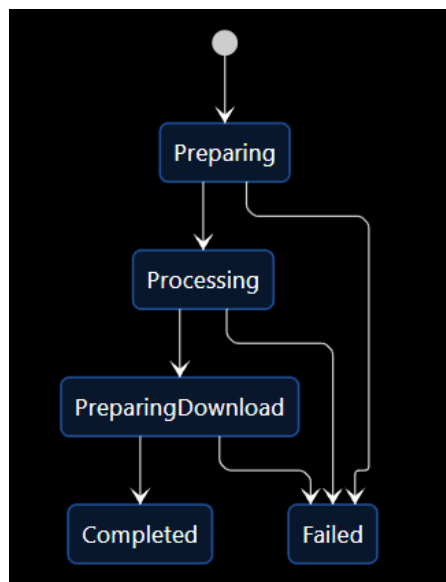
Uploading & processing files...

followed by:

Preparing download...

These stages provide feedback during the operation rather than leaving the user without any indication of the current state.

A clean conceptual model is:



23.17 Successful Processing Validation

Quill also validates the success response before updating the final interface state.

After receiving a valid output:

1. The response is checked.
2. The download file is retrieved.

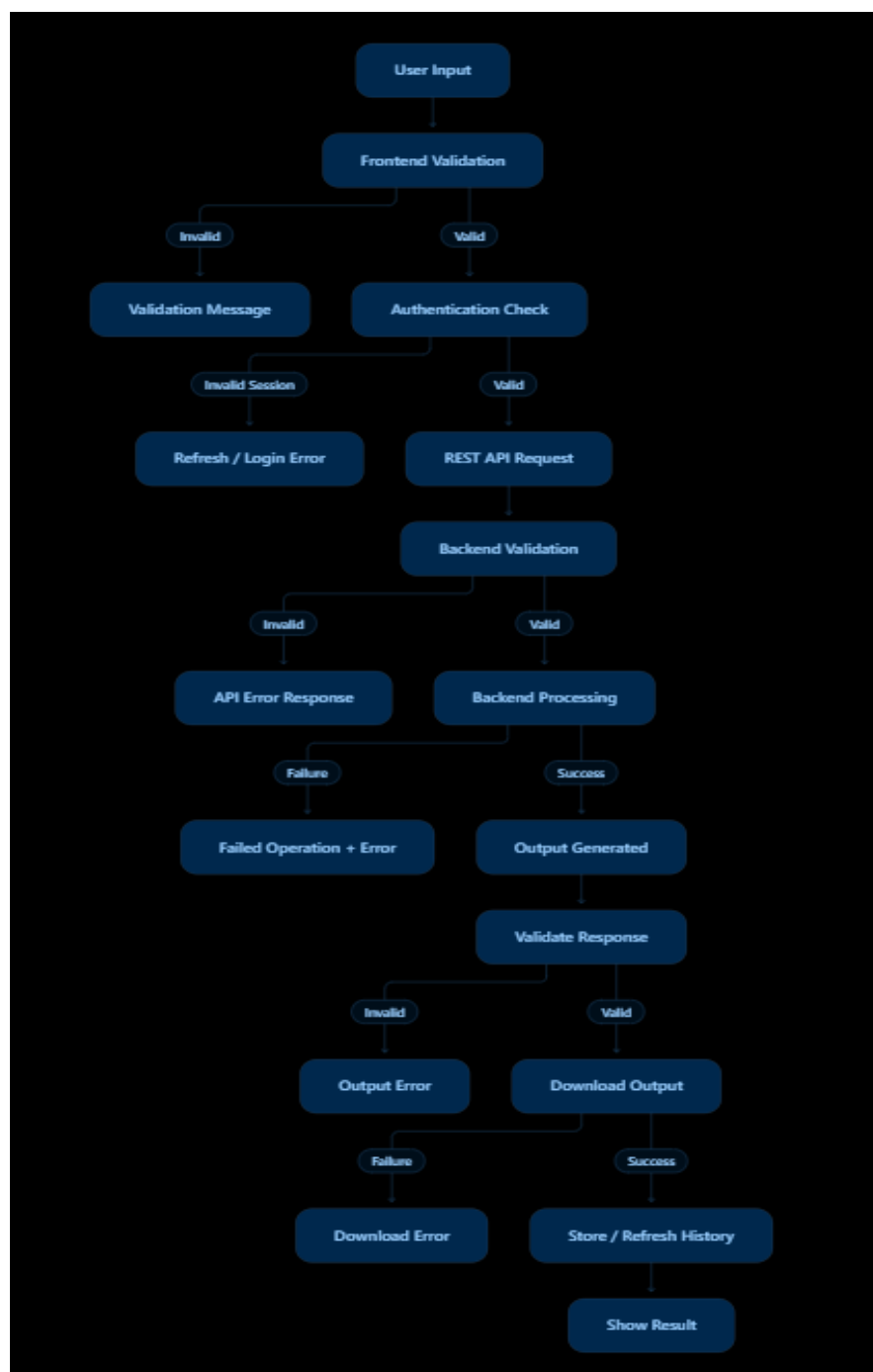
3. The filename is determined.
4. The resulting blob is stored.
5. A success message is displayed.
6. Selected files are cleared.
7. History is updated or refreshed.

The current source creates a history item when an operation_id is returned; otherwise it refreshes history from the backend.

This prevents the UI from showing a successful operation without a usable output.

23.18 Error Handling Across the Complete Workflow

The complete validation and error-handling architecture can be represented as:



23.19 Validation and Error Handling Table

Stage	Validation / Error Check	Result
File Selection	File exists and is compatible	Continue / show validation
File Count	Minimum/maximum limits	Continue / reject
Tool Type	Single vs multi-file operation	Continue / reject
Tool Options	Operation-specific settings	Continue / reject
Authentication	Access token/session	Continue / login error
API Request	HTTP status	Continue / processing error
API Response	success + download_url	Continue / invalid response
Download	HTTP status	Receive file / download error
Backend Processing	Exceptions	Completed / failed operation
Document Extraction	Page-level exceptions	Extract / log and continue
Temporary Files	Cleanup after processing	Remove temporary artifacts

This table is directly aligned with the documented current patterns for tool validation, authentication, backend failure handling, and download verification.

23.20 Importance of Layered Validation

The main idea behind Quill's error-handling approach is that **validation is distributed across the complete processing lifecycle**.

A file can be invalid at the frontend, a session can be invalid at authentication, an API request can fail at the backend, a processing operation can fail internally, or a generated result can fail during download.

By handling these separately, Quill can provide a more precise response for each failure stage instead of treating every problem as a generic error.

The architecture therefore follows:



23.21 Summary

Quill's error-handling and validation system protects the complete file-processing workflow.

The main mechanisms include:

- Frontend file-count and tool validation
- Tool-specific option validation
- JWT/session validation

- HTTP status checking
- Application-level response checking
- Download-response validation
- Backend exception handling
- Failed-operation recording
- Document page-level extraction handling
- Temporary-file cleanup
- User-visible status and error messages

This layered approach ensures that invalid input can be rejected early, backend failures can be recorded, and successful output is only presented after the required processing and download checks have passed.

24. Performance & Optimization

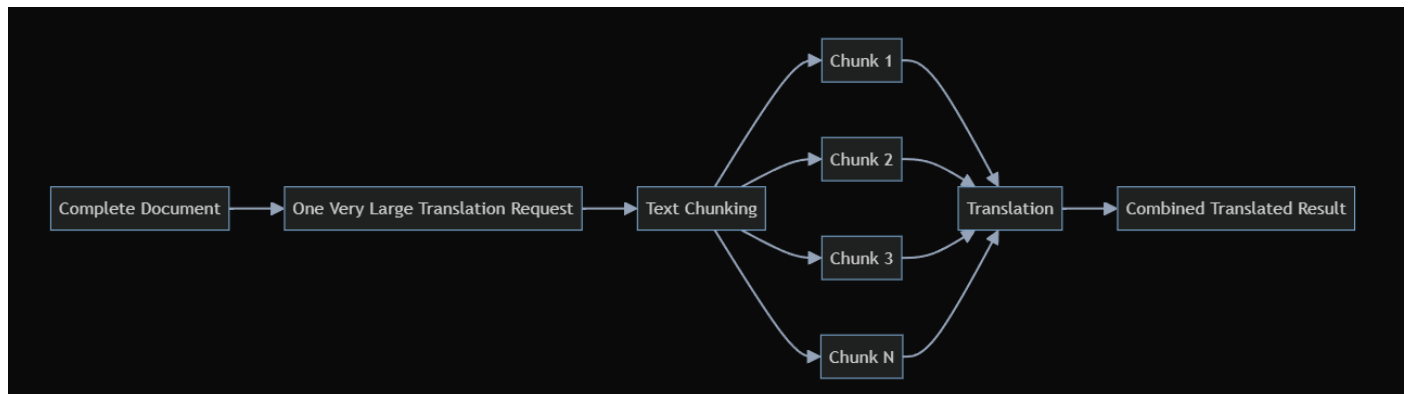
Performance is an important part of Quill because the application handles operations such as **long-text translation, PDF rendering, document processing, file conversion, merging, and history retrieval**. Instead of trying to optimize everything at once, Quill uses different strategies for different types of workloads.

The main objective is to keep the application responsive for the user while moving heavier processing to the backend and avoiding unnecessarily large requests or responses. The documented performance approach includes translation chunking, page-by-page PDF rendering, paginated history, backend-based heavy processing, and independent frontend/backend deployment.

24.1 Translation Request Optimization

Long documents should not be sent to the translation service as one extremely large request. Quill therefore uses a **chunking approach** for long translation requests.

Instead of:



This makes large translation workloads easier to process and prevents one oversized request from becoming the single bottleneck. The technical documentation specifically identifies chunking long translation requests as a performance strategy.

Large Text → Chunking → Multiple Translation Requests → Combined Output

24.2 Page-by-Page PDF Rendering

PDF previews and thumbnails can become expensive when an entire document is rendered at once. Quill uses **page-by-page rendering** when thumbnails or page previews are required.

The browser uses **PDF.js** for PDF page thumbnails and page images. This also fits the application's page-aware document model, where individual pages can be processed and displayed separately.

The basic approach is:

PDF Document



Page 1 → Render

Page 2 → Render

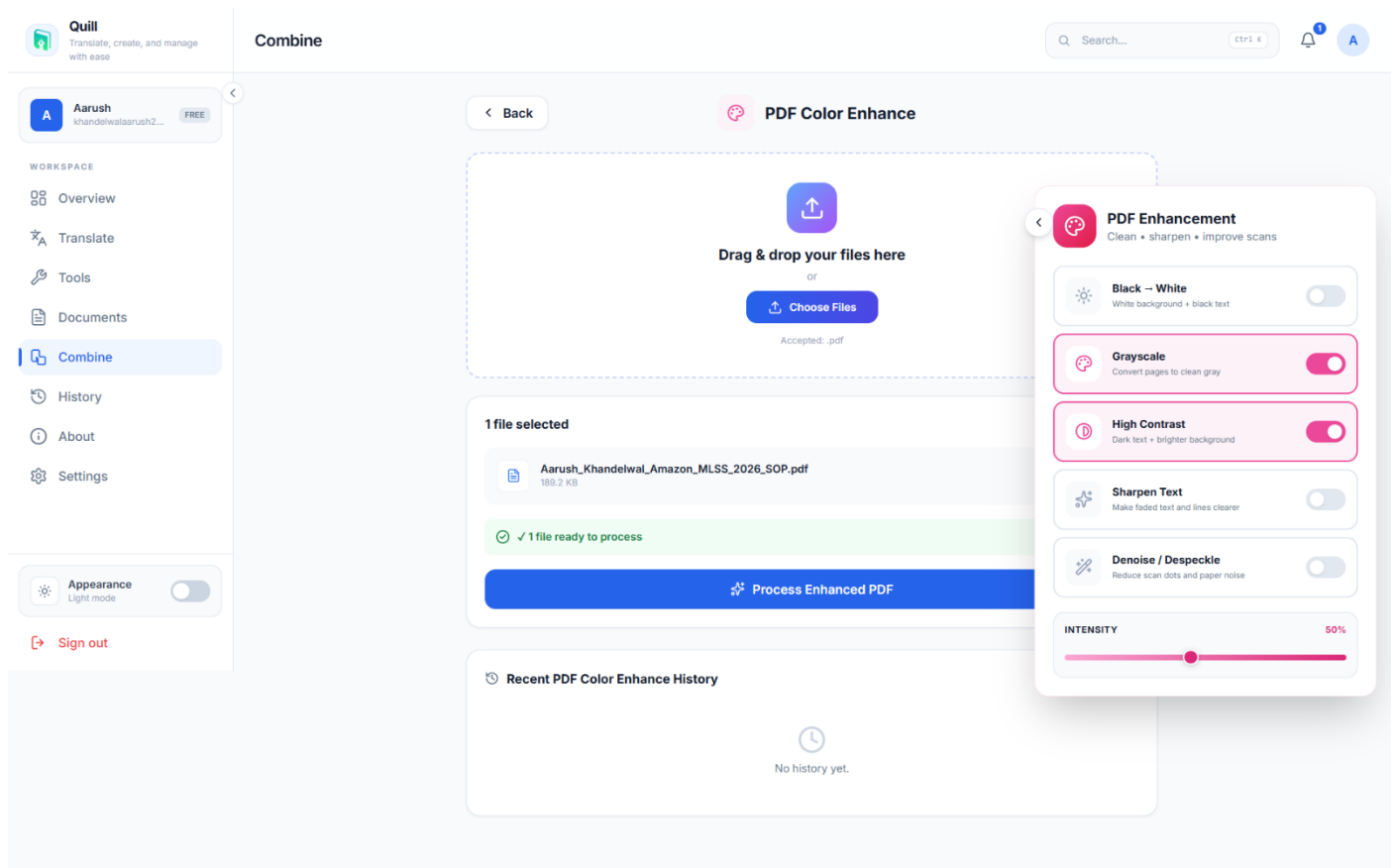
Page 3 → Render

...

Page N → Render

This avoids treating the complete PDF as a single rendering operation and allows the interface to work with individual pages.

PDF → PDF.js → Page 1 / Page 2 / Page 3 / ... → Thumbnail / Preview



24.3 History Pagination

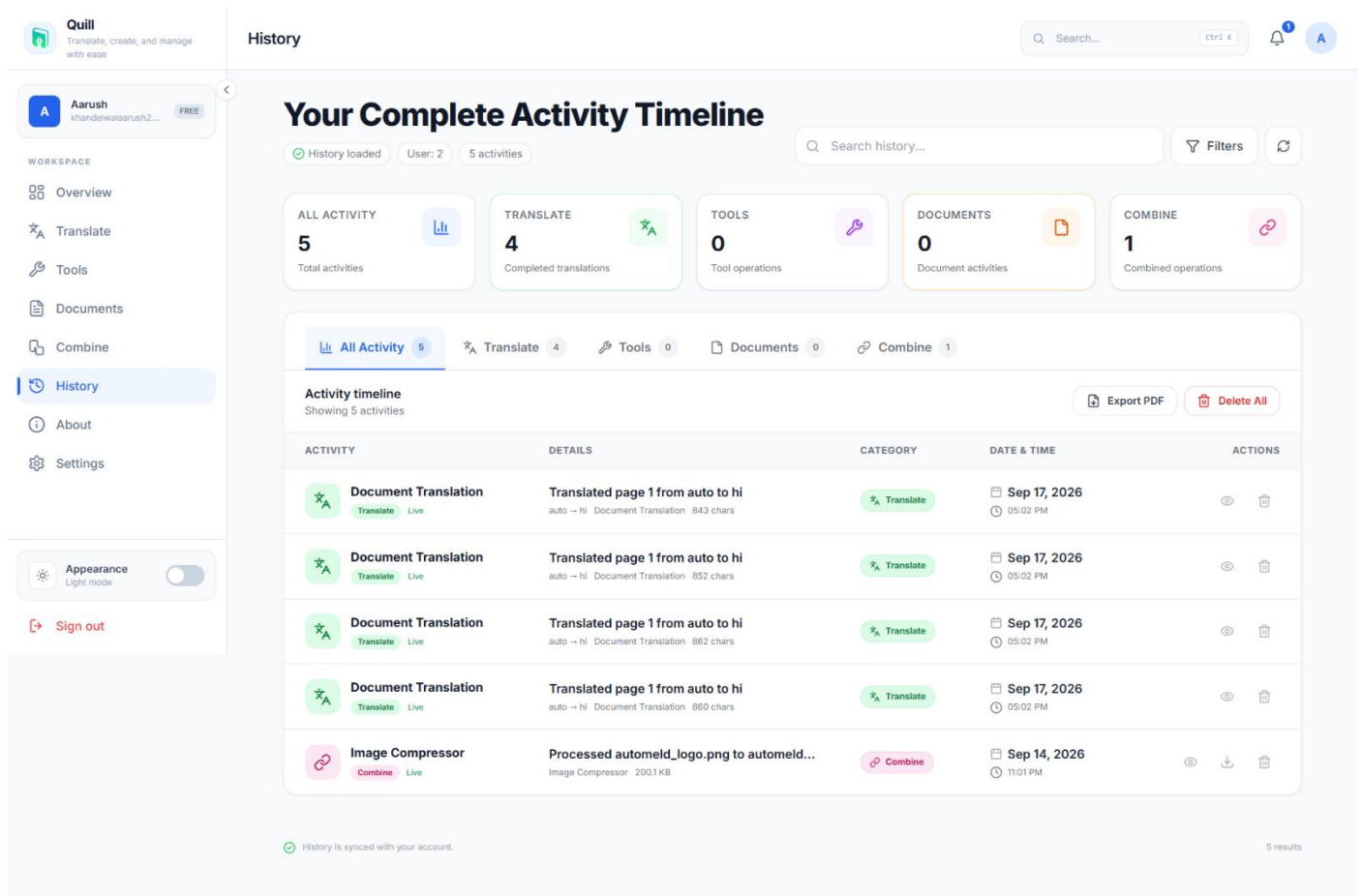
History can continuously grow as users perform more translations and file operations. Loading an unlimited number of records in a single request would increase the amount of data transferred and processed by the frontend.

Quill therefore uses **pagination for history instead of loading an unbounded list**.

For example:

Request	Records Loaded
Page 1	First set of records
Page 2	Next set of records
Page 3	Next set of records
Page N	Remaining records

This keeps the initial history request smaller and allows the user interface to retrieve additional records when required.



24.4 Backend Processing for Heavy Operations

Quill follows a clear separation between **client-side interaction** and **heavy file processing**.

The frontend is mainly responsible for the user interface, file selection, previews, configuration, and request handling. Heavier transformations are kept behind the Django backend API.

Typical heavy operations include:

Operation	Processing Location
File selection and UI interaction	Frontend
PDF preview rendering	Browser / PDF.js
Large document processing	Backend
PDF merge and conversion	Backend
Document translation processing	Backend
Output file generation	Backend

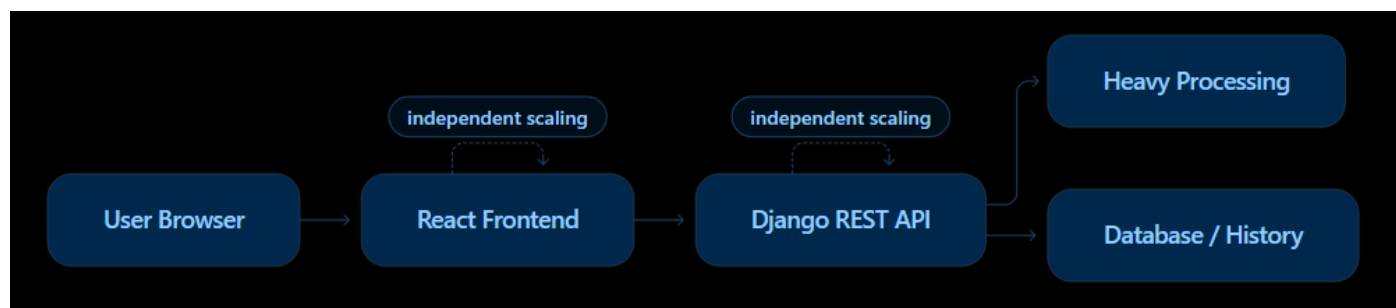
This architecture prevents large processing tasks from being handled entirely inside the browser.

24.5 Independent Frontend and Backend Scaling

Quill is deployed with the frontend and backend as separate services. This means the two parts of the system can be scaled independently according to their own workload.

The documentation identifies **independent frontend/backend scaling in production** as part of the performance and scalability strategy.

For example, increased file-processing demand can affect backend resources without requiring the frontend application itself to be redesigned.



24.6 Performance Measurement

Optimization should be based on actual system behavior rather than assumptions. The current engineering documentation recommends **measuring real throughput before introducing more complex worker or queue infrastructure**.

At present, the reviewed project material does **not define formal throughput benchmarks, background job queues, or horizontal autoscaling rules**. These areas are therefore left open for future capacity planning rather than being presented as already implemented features.

Performance Strategy Summary

Area	Optimization Approach
Long Translation	Chunk large requests
PDF Preview	Render pages individually
History	Paginated loading
Heavy Processing	Backend API
Deployment	Independent frontend/backend scaling
Future Capacity Planning	Measure real throughput first

25. Technology Stack

Quill is developed as a **full-stack web application** with a React-based frontend and a Django REST backend. Different technologies are used for user interface development, API communication, authentication, speech and audio processing, document handling, database interaction, deployment, and source control.

The technology stack is organized so that the frontend focuses mainly on the user experience while the backend handles authentication, API requests, translation, document processing, file operations, and other heavier workloads.

25.1 Technology Stack Overview

Layer	Technologies
Frontend	React, TypeScript, JavaScript
Styling	Tailwind CSS
UI Icons	Lucide / React icon ecosystem
State & Data	React Query, client-side state management
HTTP Communication	REST APIs, Axios / Fetch
Backend	Python, Django, Django REST Framework
Authentication	JWT-based authentication
Speech	SpeechRecognition, edge-tts, gTTS
Audio Processing	FFmpeg, FFprobe, Pydub
Translation	Multiple translation-service fallbacks
Documents	PDF, DOCX and image-processing workflows
Database	Relational database layer
Notifications	In-app, email and push workflows
Frontend Hosting	Vercel
Backend Hosting	Render
Source Control	Git and GitHub

This stack is documented in the Quill project README and technical documentation.

25.2 Frontend Technologies

1. React

React is used to build the main user interface of Quill. The application is divided into reusable pages and components so that features such as Translation, Documents, Combine, Tools, History, and Settings can be developed independently.

2. TypeScript & JavaScript

Quill uses **TypeScript and JavaScript** for frontend development. TypeScript provides type-oriented development for the React application, while JavaScript is also used across the frontend codebase.

3. Tailwind CSS

Tailwind CSS is used for styling the application. It provides utility classes for creating the responsive layouts, spacing, typography, buttons, forms, cards, and other interface elements.

4. Lucide / React Icon Ecosystem

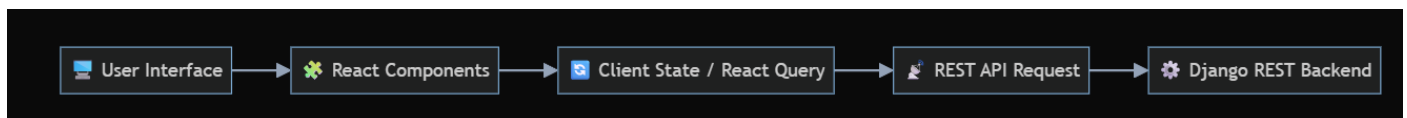
Icons are used throughout the application for navigation, actions, file operations, settings, status indicators, and controls. The documented stack includes the **Lucide / React icon ecosystem**.

25.3 State Management and API Communication

Quill uses **React Query and client-side state management** to manage application data and UI state.

REST APIs form the communication layer between the frontend and backend. **Axios / Fetch-style HTTP requests** are used to send authentication requests, text data, files, processing options, and other application data to the backend.

A simplified communication model is:



25.4 Backend Technologies

1. Python

Python is the primary backend programming language used in Quill. It is particularly suitable for the project's document processing, translation, audio processing, and file manipulation workloads.

2. Django

Django provides the main backend framework and application structure.

3. Django REST Framework

Django REST Framework (DRF) is used to expose backend functionality through REST APIs. The backend contains separate modules for areas such as users, translation, documents, combine operations, tools, history, and settings.

25.5 Authentication Technology

Quill uses **JWT-based authentication** for protected API access.

The documented authentication architecture includes:

- Access and refresh JWT tokens
- Token-expiry checking
- Token refresh
- OTP-based verification
- Optional two-factor authentication

- pyotp for 2FA-related functionality

Authenticated history and account-related operations are associated with the logged-in user.



25.6 Speech and Audio Technologies

Quill's voice-translation functionality uses several technologies.

1. SpeechRecognition

Used to process spoken input and convert speech into text.

2. edge-tts and gTTS

These technologies support **Text-to-Speech (TTS)** functionality, allowing translated text to be converted back into spoken audio.

3. FFmpeg, FFprobe and Pydub

These tools are used for audio handling and normalization workflows. FFmpeg/FFprobe can inspect or convert media formats, while Pydub supports audio manipulation.

25.7 Translation and Document Processing

Quill uses **multiple translation-service fallbacks**, allowing translation workflows to use alternative services depending on the available configuration and runtime environment.

For document processing, the application works with:

- PDF files
- DOCX documents
- Text-based files
- HTML
- Images

The project also uses browser-side helper technologies such as **PDF.js**, **jsPDF**, and **Mammoth** for specific preview, PDF-generation, and document-extraction tasks.



25.8 Database and Notification Layer

Quill uses a **relational database layer** for persistent application data. User-related records, document information, history and settings are maintained through the backend.

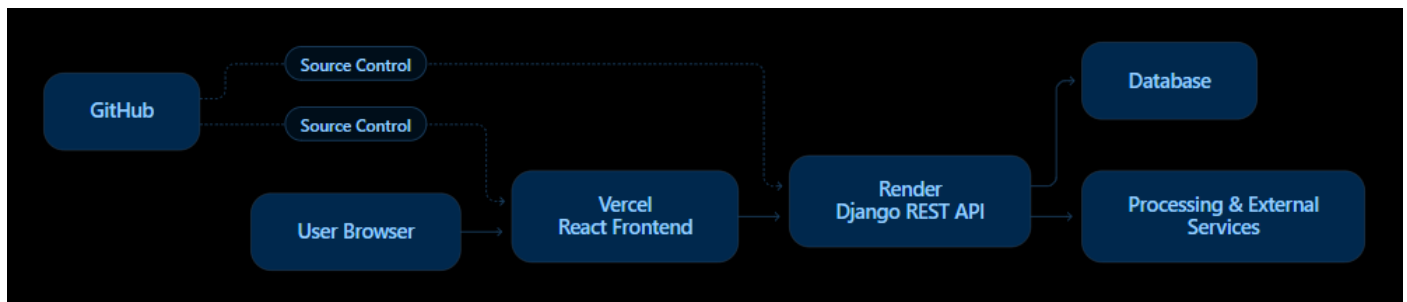
The application also contains notification workflows covering **in-app**, **email** and **push notifications**.

25.9 Deployment and Development Tools

The production environment separates frontend and backend hosting:

Component	Platform
Frontend	Vercel
Backend	Render
Source Code	Git + GitHub

The documented production setup uses Vercel for the React frontend and Render for the Django API, while Git and GitHub are used for version control and project collaboration.



25.10 Why This Stack Fits Quill

The technologies are divided according to the type of work they perform:

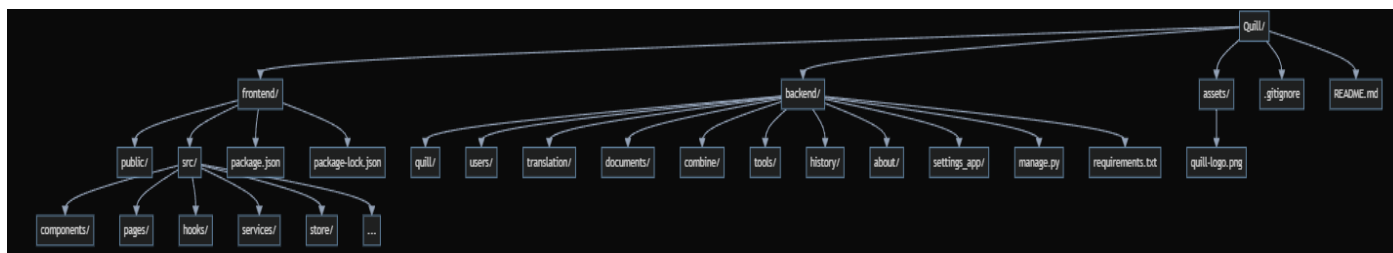
React + Tailwind handle the interactive product interface, **Django + DRF** provide the API and backend application layer, while specialized tools handle speech, audio, documents, PDFs, and translation. Vercel and Render provide separate deployment environments for the two main application layers.

26. Project Folder Structure

Quill follows a **separate frontend and backend folder structure** so that the user interface, API logic, document processing, authentication, history, and supporting services remain organized and maintainable.

The repository is divided mainly into frontend, backend, and assets, along with project-level configuration and documentation files. The exact contents can evolve as new features are added.

26.1 Overall Project Structure



This structure is documented in the project README and technical documentation.

26.2 Frontend Folder

The frontend folder contains the React application and everything required for the user interface.

public/

The public directory contains static frontend resources that are served directly by the web application.

src/

The main application code is maintained inside src.

It is further organized into several logical sections:

Folder	Purpose
components/	Reusable interface components
pages/	Main application pages and feature screens
hooks/	Reusable React hooks and frontend logic
services/	API and service-related functions
store/	Client-side application state
...	Other supporting frontend files

This separation prevents the entire frontend from becoming one large collection of files and makes individual features easier to maintain.

26.3 Components

The components directory contains reusable UI elements.

Instead of rebuilding the same interface element on every page, shared components can be used wherever required.

Examples of responsibilities include:

- Navigation and layout elements
- Buttons and controls
- File upload interfaces
- Reusable dialogs and status elements
- Shared cards and UI sections

The folder therefore acts as the reusable UI layer of the frontend.

26.4 Pages

The pages folder contains the major screens through which users interact with Quill.

The project feature structure includes pages for areas such as:

- Translation
- Documents
- Combine & Convert
- Smart Tools
- History
- Settings
- Authentication and account-related screens

Each major feature can therefore have its own page-level implementation while sharing common components.

26.5 Hooks, Services and Store

hooks/

The hooks directory contains reusable React logic that can be shared between components and pages.

services/

The services directory is used for application-service and API-related code. This provides a separation between the UI layer and the functions responsible for communicating with the backend.

store/

The store directory contains client-side state management logic. This helps maintain application state independently from individual UI components.

The technical documentation describes the frontend architecture as a combination of React components, pages, hooks, services, and client-side state management.

26.6 Backend Folder

The backend directory contains the Django application and the REST API implementation.

Its main structure is:

Folder / File	Responsibility
quill/	Main Django project configuration
users/	User and authentication functionality
translation/	Translation-related backend logic
documents/	Document processing and translation
combine/	File merging and conversion operations
tools/	Smart tools and utility operations
history/	Activity and operation history
about/	About-related backend functionality
settings_app/	Application settings functionality
manage.py	Django management entry point
requirements.txt	Python backend dependencies

These modules keep different backend responsibilities separated instead of placing all API and processing logic inside one application.

26.7 Main Backend Modules

users/

Responsible for user-related functionality and authentication flows such as registration, login, OTP and related account operations.

translation/

Contains backend functionality related to text and voice translation and communication with configured translation services.

documents/

Handles document-oriented workflows, including extraction, translation, and output generation for supported document types.

combine/

Contains PDF, image and Word merge/conversion functionality and other PDF operations.

tools/

Contains the application's utility tools.

history/

Handles account-linked operation history and related API functionality.

settings_app/

Provides backend support for user/application settings.

The separation of these modules follows the project's overall modular backend architecture.

26.8 Project-Level Files

Outside the two major application folders, Quill contains several important project-level files.

assets/

Contains project assets such as the Quill logo.

assets/

└─ quill-logo.png

.gitignore

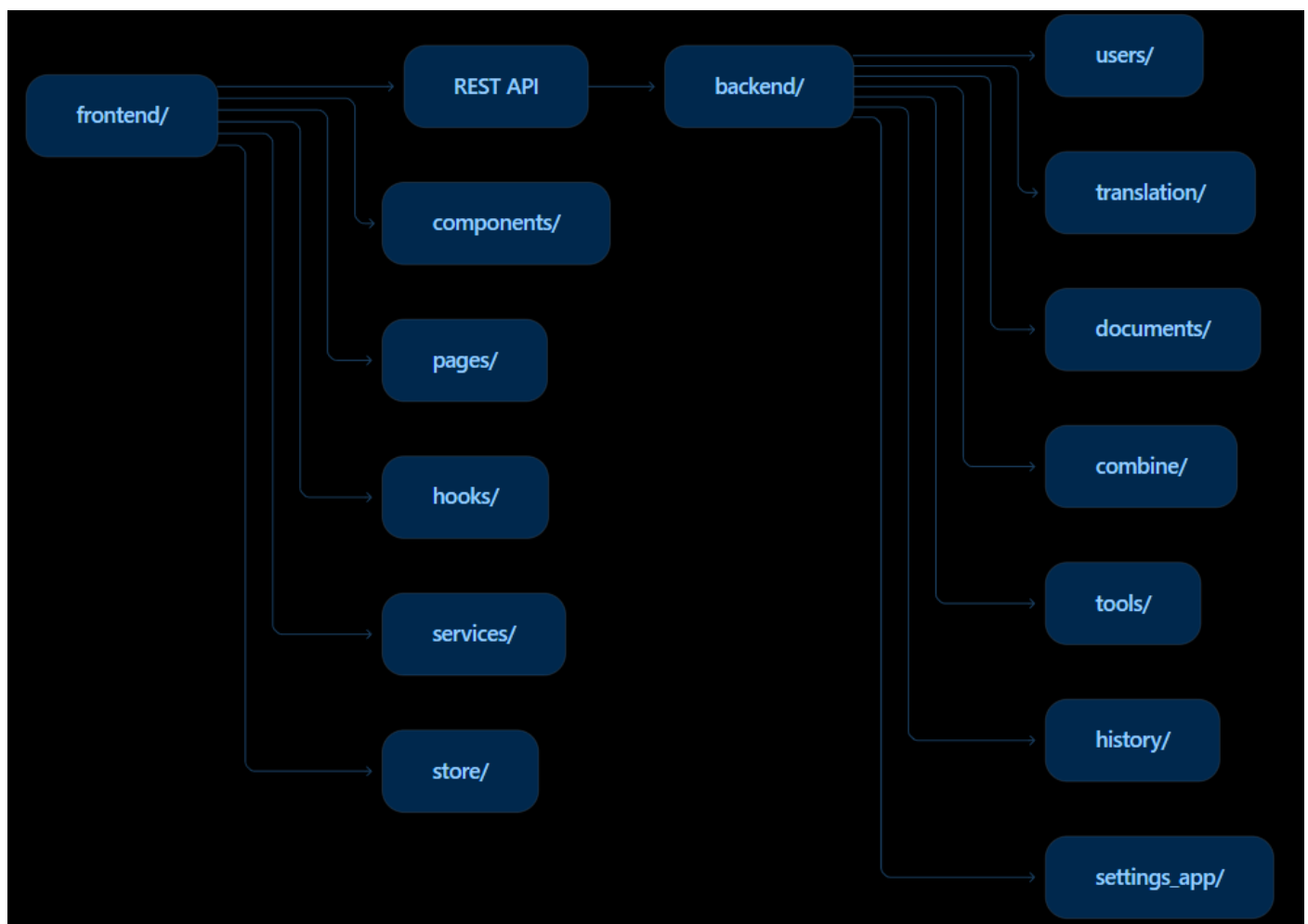
Specifies files and directories that should not be committed to the Git repository. This is particularly important for environment files and other sensitive or unnecessary local files.

README.md

Provides project information including features, architecture, technology stack, setup instructions, deployment information, and screenshots.

26.9 Relationship Between Frontend and Backend Folders

The two major folders work together but have different responsibilities.



26.10 Advantages of the Folder Structure

The Quill folder structure provides several practical benefits:

- **Separation of responsibilities:** Frontend interface code and backend processing code remain independent.
- **Maintainability:** Related functionality is grouped together, making files easier to locate and update.
- **Scalability:** New modules can be added without restructuring the complete application.
- **Reusability:** Common frontend components, hooks, and services can be reused across multiple pages.
- **Clear deployment boundary:** The frontend and backend can be developed, built, deployed, and scaled independently.

27. Development Process

The development of Quill was organized around its main product modules and the separation between the React frontend and Django REST backend. The available project documentation does not define a formal methodology such as Scrum or Agile, so this section describes the **practical development process visible from the project's architecture, engineering decisions, local setup, testing plan, deployment setup, and subsequent feature improvements**.

27.1 Project Planning and Requirement Identification

The first stage was to identify the common tasks that users would need in one workspace.

Instead of creating separate applications for each task, Quill was structured as a unified workspace containing:

- Translation
- Voice translation
- Document processing
- Combine & Convert
- Smart Tools
- History
- Settings
- Authentication

This unified-workspace decision is explicitly documented as one of the project's engineering decisions.

The requirements were then divided into independent modules so that each major feature could be developed and maintained separately.

User Requirements → Translation / Documents / Combine / Tools / History / Settings → Complete Quill Workspace

27.2 System and Technology Planning

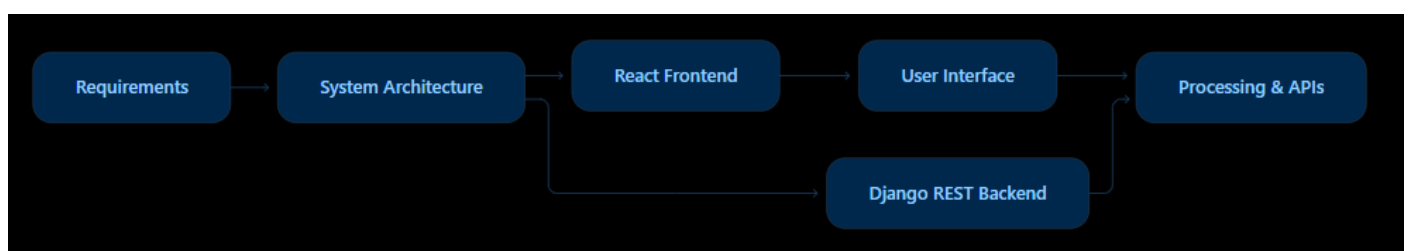
After identifying the features, the project was divided into two primary application layers:

Frontend: React + TypeScript + Tailwind CSS

Backend: Python + Django + Django REST Framework

The backend provides authenticated API services and processing, while the frontend provides the user-facing workspace. Production deployment is also separated between Vercel and Render.

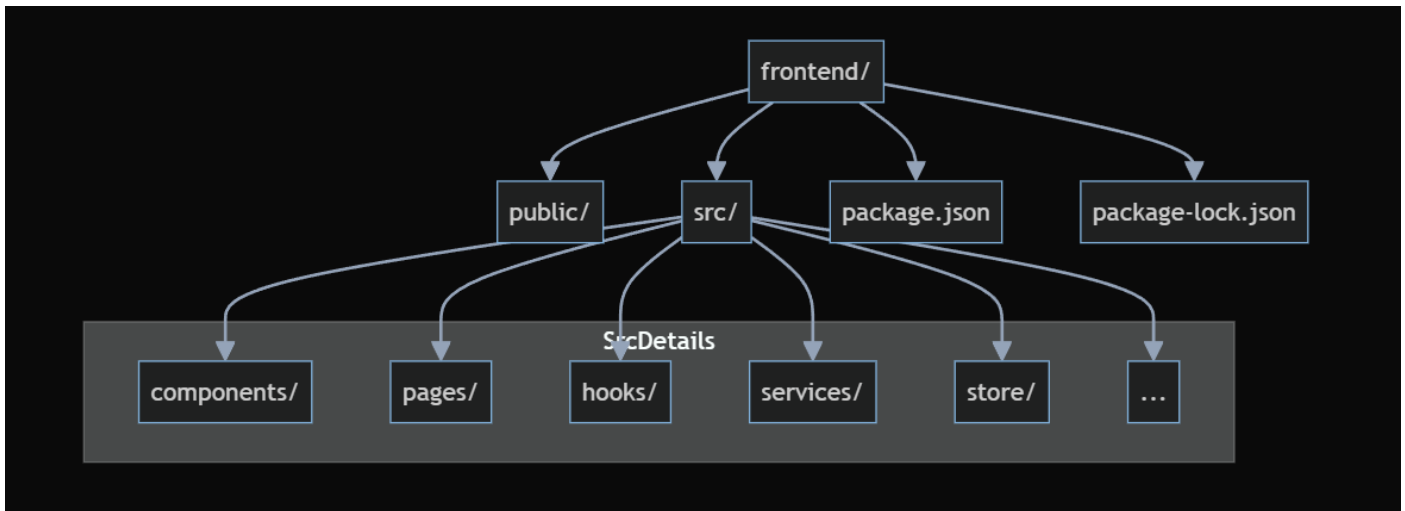
This separation established a clear boundary between interface development and server-side processing.



27.3 Frontend Development

Frontend development was organized around reusable components and feature-specific pages.

The documented frontend structure contains:

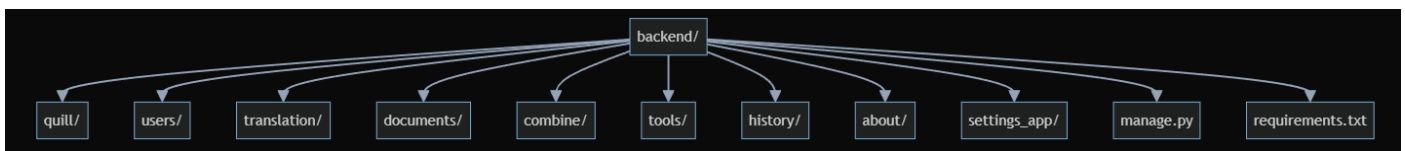


This structure separates reusable interface elements from pages, hooks, services, and application state.

During frontend development, feature screens were connected to the backend through REST API requests. Client-side helpers were also used where appropriate, while heavier processing remained behind the backend API.

27.4 Backend Development

The backend was developed as a set of Django modules, with different responsibilities assigned to different applications.



The modular structure allows authentication, translation, document processing, file operations, utility tools, and history functionality to remain separated.

For example:

users/ → authentication and account functionality

translation/ → translation workflows

documents/ → document extraction, translation and output generation

combine/ → merging, conversion and PDF operations

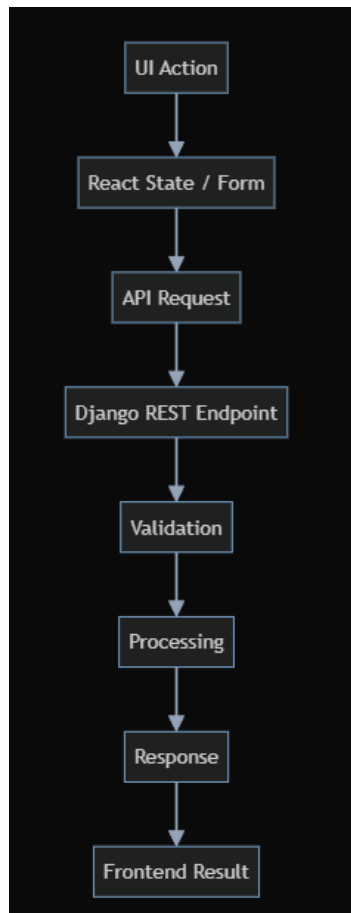
tools/ → smart utility functions

history/ → stored operation history

27.5 Frontend–Backend Integration

Once the major frontend and backend functionality was available, the modules were connected through REST APIs.

A typical feature development flow was:



Authentication is included in this communication through the project's JWT-based access and refresh-token mechanism.

For file-based features, the frontend sends the selected files and processing options to the backend, where the requested operation is performed and the resulting file is returned to the user.

React UI → REST API → Django → Processing → Output → React UI

27.6 Testing and Validation During Development

Testing was considered across the major feature areas rather than treating the complete application as one single unit.

The documented testing strategy covers:

Area	Main Checks
Authentication	Register, OTP, login, refresh, 2FA, logout
Translation	Translation modes, languages, style/formality, history
Documents	PDF/DOCX/TXT/HTML extraction, translation, download
Combine	Merge, reorder, validation, output integrity
PDF Tools	Split, compress, rotate, organize, watermark, color enhancement, N-up
Smart Tools	Utility categories
History	Search, filters, pin, share, download, delete, export
Deployment	Frontend, API, authentication, downloads, uptime

The documentation explicitly describes this as a **recommended QA matrix**, not a claim that every test is already automated.

27.7 Debugging and Feature Refinement

During development, problems were handled by isolating the affected layer and correcting the relevant logic.

Quill's architecture supports this by separating:

Frontend validation → API communication → Backend validation → Processing → Output/download

For example, file-related features have validation for file type, file count, tool-specific options, authentication and download responses.

Feature refinement was also performed after the initial implementation. One documented example is the addition of **drag-and-drop ordering for PDF, Word and image merge tools**, while keeping file-level ordering separate from page-level ordering in Organize PDF.

Implement → Test → Identify Issue → Fix → Re-test → Refine

27.8 Local Development

The project documentation provides separate development procedures for the frontend and backend.

Frontend

```
git clone https://github.com/Aarush005coder/Quill.git
```

```
cd Quill/frontend
```

```
npm install
```

```
npm start
```

Production build:

```
npm run build
```

Backend

```
cd Quill/backend
```

```
python -m venv venv
```

Windows:

```
.\venv\Scripts\Activate.ps1
```

Then:

```
pip install -r requirements.txt
```

```
python manage.py migrate
```

```
python manage.py runserver
```

These documented commands show that frontend and backend development can be run and tested independently before production deployment.

27.9 Version Control and Deployment

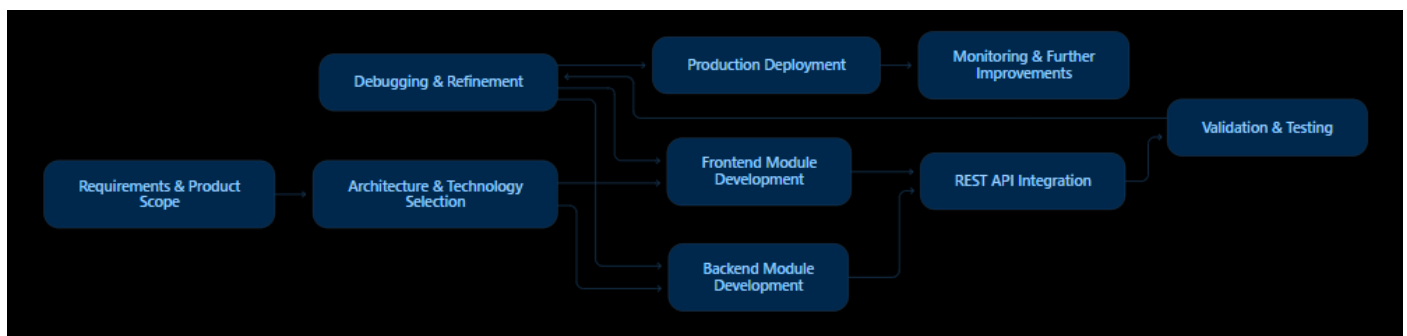
Git and GitHub were used for source control, while the production system uses separate hosting platforms for the two major application layers.

Stage	Tool / Platform
Source Control	Git + GitHub
Frontend Deployment	Vercel
Backend Deployment	Render
Availability Monitoring	UptimeRobot

The documented production process deploys the React frontend to Vercel and the Django API to Render, with secrets supplied through platform environment variables.

27.10 Overall Development Flow

Combining the documented architecture, testing strategy, deployment process and engineering decisions gives the following overall development flow:



Development Process Summary

The development of Quill can therefore be understood as a sequence of practical stages:

Requirement identification → architecture planning → module development → frontend/backend integration → validation and testing → debugging and refinement → deployment → monitoring and further development.

The project documentation also leaves several future engineering improvements open, including automated API/end-to-end testing, background processing queues, resumable uploads, output integrity checks, formal API schemas, and telemetry.

28. Testing & Debugging

Testing and debugging are important parts of the Quill development process because the application contains several interconnected workflows, including authentication, translation, document processing, PDF operations, file conversion, smart tools, history, and deployment.

The project documentation defines a **QA matrix based on the current feature surface**. It is described as a test plan rather than a claim that every test is already automated.

28.1 Testing Approach

Testing in Quill is organized around the major application areas rather than testing only the user interface.

The main testing areas are:

Module	Main Testing Checks
Authentication	Registration, OTP, login, refresh, 2FA, logout
Translation	Four translation modes, languages, style/formality, history
Documents	PDF/DOCX/TXT/HTML extraction, translation, download
Combine	PDF/Image/Word merge, reordering, limits, output integrity
PDF Tools	Split, compress, rotate, organize, watermark, color enhancement, N-up
Smart Tools	Number, currency, unit, data, electrical, health, specific tools
History	Search, filters, pin, share, download, delete, export
Deployment	Frontend, API, authentication, downloads, uptime

These checks are directly listed in the documented testing strategy.

28.2 Authentication Testing

Authentication is tested across the complete account lifecycle.

The test cases include:

Registration → OTP verification → Login → Token refresh → Optional 2FA → Logout

Important checks include whether valid users can authenticate correctly, whether OTP verification works, whether expired access tokens can be refreshed, and whether protected account operations remain associated with the authenticated user.

The testing matrix explicitly includes registration, OTP, login, refresh, 2FA, and logout.

Register → OTP → Login → JWT → API Access → Refresh / Logout

28.3 Translation Testing

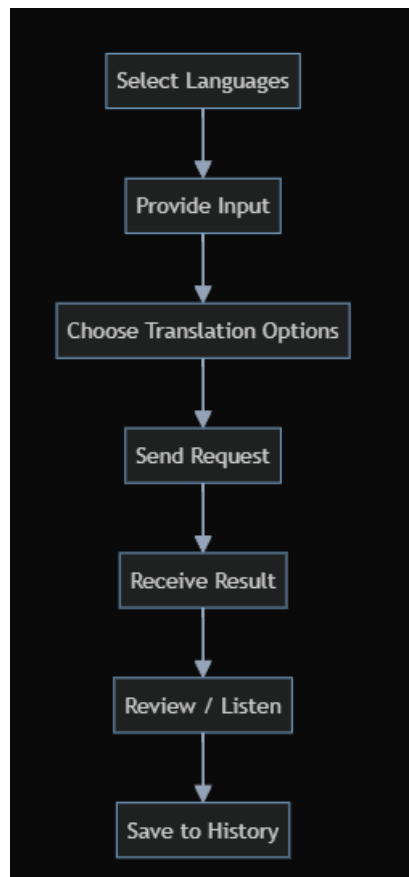
The Translation module has multiple input and output modes, so testing covers the supported translation workflows.

The documented test plan includes:

- Text → Text
- Text → Speech
- Speech → Text
- Speech → Speech
- Supported languages
- Translation style and formality
- History integration

The purpose is to check both the translation request and the surrounding workflow, including options and history behavior.

A typical test can be structured as:



28.4 Document Testing

Document processing requires testing different file formats because each format can have different extraction and output behavior.

The documented test plan includes:

PDF / DOCX / TXT / HTML

Testing covers:

File upload → Text extraction → Translation → Output creation → Download

For PDF-based workflows, page handling is particularly important because Quill uses a page-aware processing model.

28.5 Combine and PDF Tool Testing

File operations are tested for both **functional correctness** and **validation**.

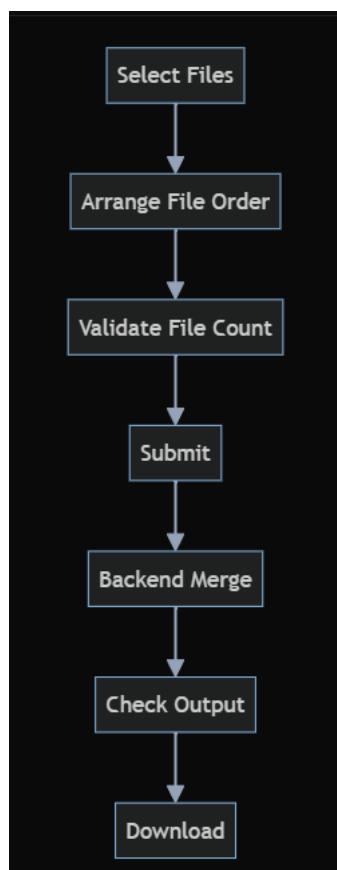
For Combine, the documented checks include:

- PDF merge
- Image merge
- Word merge
- File reordering
- Minimum/maximum file validation
- Output integrity

The PDF tool category additionally covers:

- Split
- Compress
- Rotate
- Organize
- Watermark
- Color enhancement
- N-up PDF

For example, PDF Merge can be tested using:



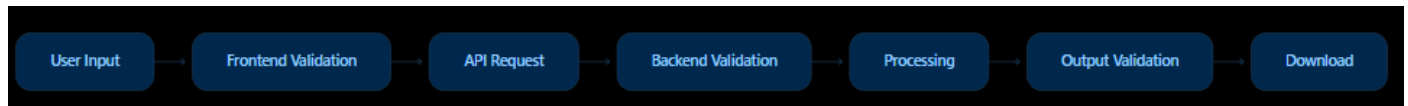
The output is not considered successful merely because the API returns a response; the resulting file also needs to be checked for expected output behavior.

28.6 Validation Testing

Validation is performed at multiple points in the application.

The documented validation patterns include accepted file extensions and multiplicity, minimum file requirements for merge operations, rejection of invalid extra-file selections, and validation of tool-specific options such as compression, rotation, split, watermark, image, and N-up settings.

A simplified validation structure is:



28.7 Error Testing

Debugging is closely connected with error handling. Quill checks errors at different stages instead of assuming that a successful request always produces a valid result.

The documented patterns include:

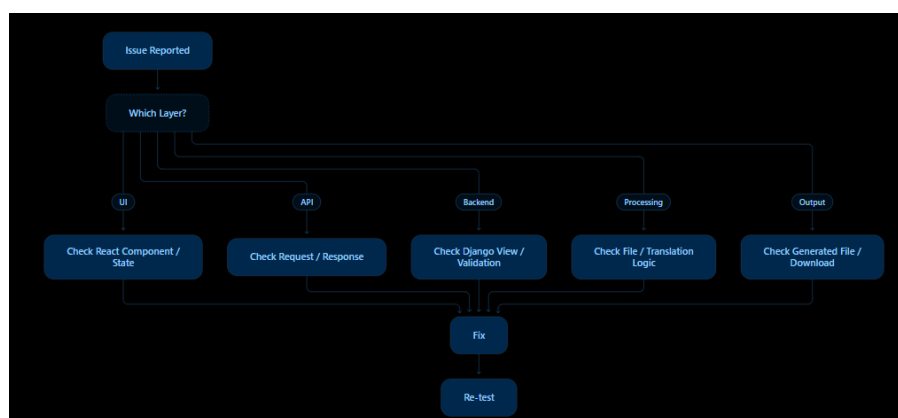
Stage	Error / Check
Input	Invalid type, missing file, incorrect file count
Authentication	Unauthorized request / token issue
API	Invalid request or backend response
Processing	File or translation processing failure
Download	Invalid or unavailable download response
Document Extraction	Individual page extraction errors

Backend operations also maintain failed status and error information when processing does not complete successfully.

The frontend uses user-visible status or toast messages to communicate failures and processing states.

28.8 Debugging Process

When an issue is found, debugging can be narrowed to the layer where the problem occurs.



This layered approach is useful because a visible frontend error may actually originate from API validation, backend processing, or generated output.

28.9 Output Integrity Testing

For file-processing features, testing also includes checking whether the generated file is usable after processing.

For example, a successful PDF merge should result in:

Valid response + accessible download + correctly generated output

Similarly, document processing should verify that the translated output corresponds to the processed source document.

The Combine testing matrix explicitly includes **output integrity** as part of the test coverage.

28.10 Deployment and Availability Testing

Testing does not stop at local development. The documented QA matrix also includes the production environment.

Deployment testing covers:

- Frontend availability
- API availability
- Authentication
- File downloads
- Uptime checks

Quill's production architecture separates the React frontend and Django backend, with UptimeRobot used for availability monitoring.

Browser → Vercel Frontend → Render Backend → API/Processing, with **UptimeRobot** monitoring availability.

28.11 Testing and Debugging Cycle

The overall process can be summarized as:

Implement → Test → Detect Issue → Debug → Fix → Re-test → Validate Output → Deploy

Testing therefore acts as a continuous part of development rather than only a final step.

The available documentation does not claim that the complete QA matrix is already automated; it is explicitly presented as a recommended test plan derived from the application's current features.

29. Deployment — Vercel & Render

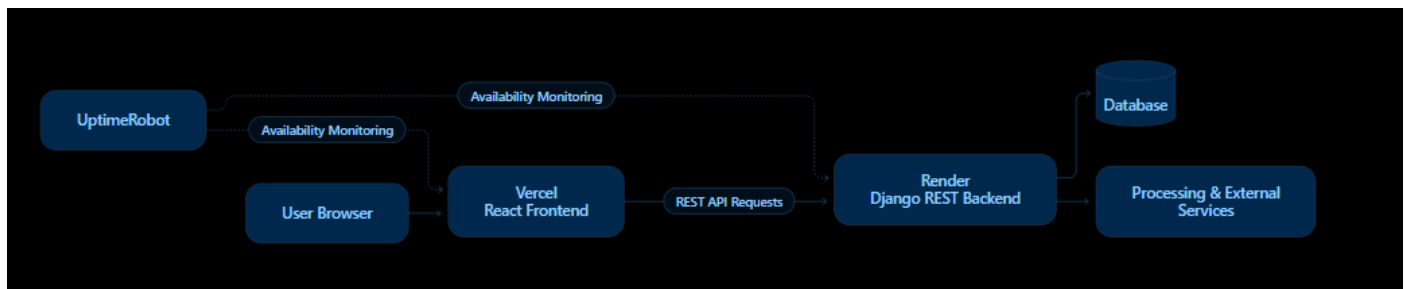
After development, testing, and debugging, Quill is deployed as a **separate frontend and backend application**. The React frontend is hosted on **Vercel**, while the Django REST backend is hosted on **Render**. This deployment structure follows the same separation used during development and allows the two application layers to operate independently. UptimeRobot is used for availability monitoring.

29.1 Production Deployment Architecture

The production deployment contains three main parts:

Component	Platform	Responsibility
Frontend	Vercel	Hosts the React application
Backend	Render	Hosts the Django REST API
Monitoring	UptimeRobot	Monitors service availability

The documented project details identify the live Quill frontend as <https://quill-one-sandy.vercel.app/> and the project repository as <https://github.com/Aarush005coder/Quill>.



29.2 Frontend Deployment on Vercel

The Quill frontend is built using React and related frontend technologies and is deployed to **Vercel**.

The production build is generated using:

```
npm run build
```

The documented local frontend workflow is:

```
cd Quill/frontend
```

```
npm install
```

```
npm start
```

For production, the frontend is built and then deployed through the Vercel environment.

The frontend is responsible for:

- User interface and navigation
- Translation workspace
- Document workspace

- Combine & Convert interface
- Smart Tools interface
- History and Settings
- Client-side previews and interactions
- Sending requests to the backend API

29.3 Backend Deployment on Render

The backend is implemented using **Python, Django and Django REST Framework** and is deployed to **Render**.

The documented backend setup is:

```
cd Quill/backend
```

```
python -m venv venv
```

On Windows:

```
.\venv\Scripts\Activate.ps1
```

Dependencies are installed using:

```
pip install -r requirements.txt
```

Database migrations are then applied:

```
python manage.py migrate
```

During local development, Django is started with:

```
python manage.py runserver
```

The production deployment moves this Django API from the local environment to Render.

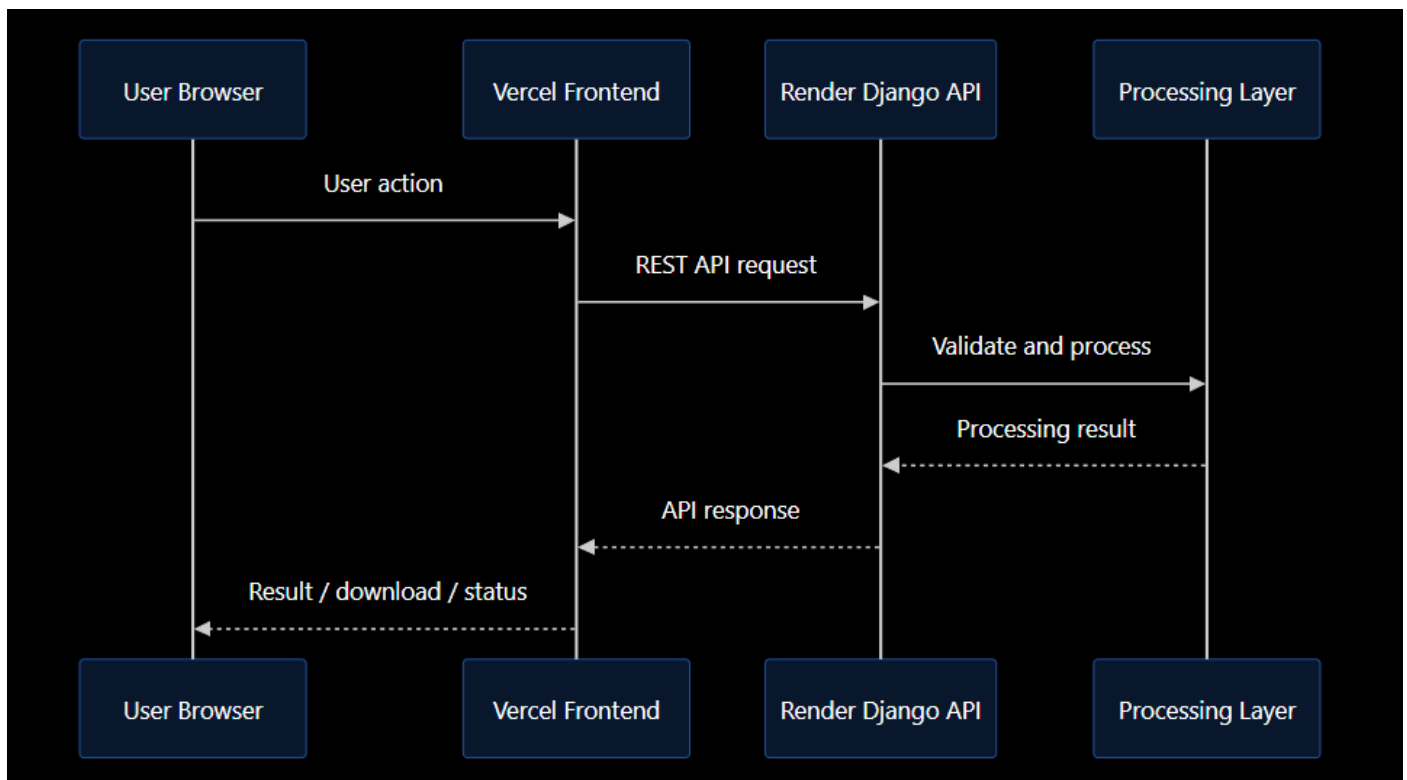
The backend handles the heavier application responsibilities, including authentication, translation processing, document operations, file conversion, Combine operations, history, and related API services.

29.4 Frontend–Backend Communication in Production

Once both services are deployed, the browser communicates with the React application hosted on Vercel.

When a user performs an operation such as translation or PDF processing, the request travels from the frontend to the Django REST API hosted on Render.

This keeps the frontend responsible for user interaction while the backend performs server-side processing.



29.5 Environment Configuration

Production credentials and sensitive configuration are not intended to be stored directly in the source code.

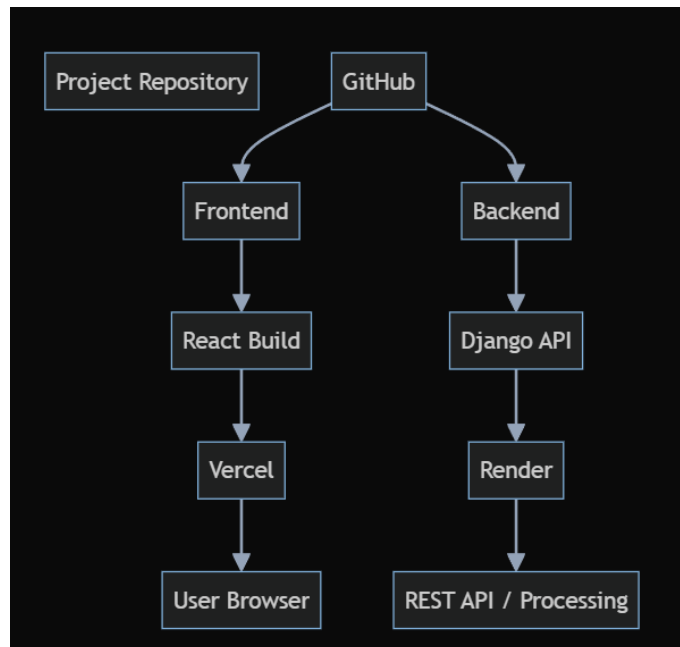
The project documentation specifies that secrets should be provided through **platform environment variables**. Examples include:

- SECRET_KEY
- DATABASE_URL
- Translation API Keys
- SMTP Credentials
- OAuth Credentials
- JWT Secrets
- Private VAPID Keys

The documentation specifically instructs that .env files, API keys, SMTP credentials, OAuth secrets, JWT secrets, database passwords, and private VAPID keys should not be committed to the repository.

29.6 Production Build and Deployment Flow

The overall deployment process can be represented as:



The documented production procedure is to deploy the frontend to Vercel, deploy the Django API to Render, keep sensitive configuration in platform environment variables, and use UptimeRobot for availability monitoring.

29.7 Deployment Verification

After deployment, the production setup needs to be checked across the complete application path.

Important verification areas documented in the testing strategy include:

Verification Area	Example Check
Frontend	Application loads correctly
API	Backend API responds
Authentication	Login and protected requests work
Downloads	Generated files can be downloaded
Integration	Frontend can communicate with backend
Availability	Monitoring detects service availability

The project testing strategy explicitly includes frontend, API, authentication, downloads, and uptime checks.

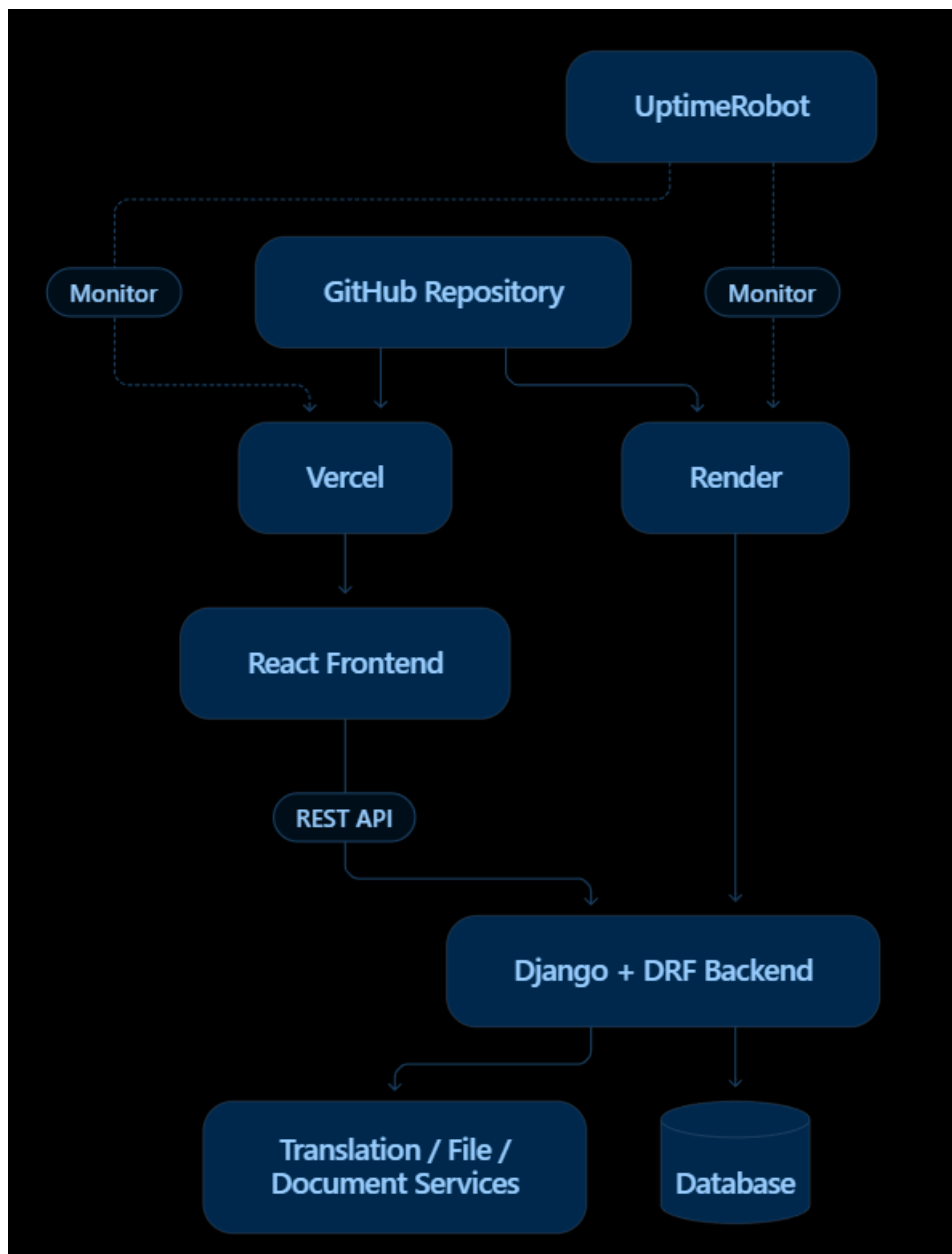
29.8 Production Deployment Benefits

Separating Vercel and Render provides a clear deployment boundary between the two major parts of Quill.

The frontend can be developed and deployed independently from backend processing, while the Django API remains responsible for server-side operations. The project's engineering decisions explicitly identify **independent frontend/backend hosting** as part of the system design.

This structure also supports the performance strategy discussed earlier, where frontend and backend resources can be handled separately.

29.9 Final Deployment Structure



Deployment Summary

Quill uses a **split production architecture**:

GitHub → Vercel → React Frontend

GitHub → Render → Django REST Backend

UptimeRobot → Availability Monitoring

This architecture keeps the frontend and backend deployment responsibilities separate while allowing them to communicate through REST APIs. The documented production setup confirms Vercel for the frontend, Render for the Django API, environment variables for secrets, and UptimeRobot for monitoring.

30. Uptime Monitoring with UptimeRobot

After deploying Quill, keeping the application available is important because the project depends on **both the frontend and backend services**. Quill uses **UptimeRobot for availability monitoring** as part of its production setup. The technical documentation specifically lists UptimeRobot as the monitoring service used alongside the Vercel frontend and Render backend.

30.1 Purpose of Uptime Monitoring

Uptime monitoring helps detect whether a deployed service is reachable and responding.

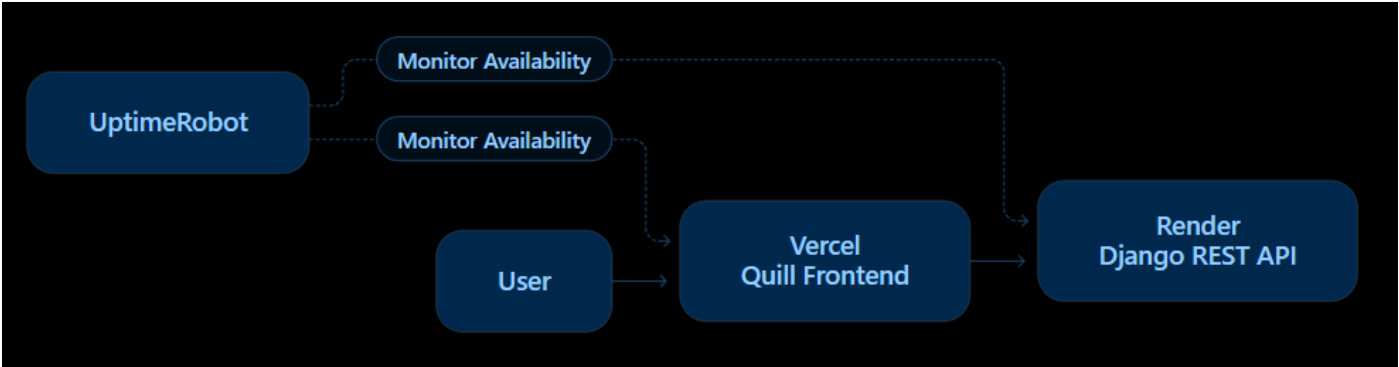
For Quill, the production architecture contains:

Service	Platform	Monitoring Purpose
React Frontend	Vercel	Check application availability
Django REST API	Render	Check backend availability
Quill Production System	Vercel + Render	Monitor overall service accessibility

The documented deployment process explicitly includes **UptimeRobot for availability monitoring**.

30.2 Quill Monitoring Architecture

The monitoring setup can be represented as:



This shows that monitoring is positioned outside the application and observes the production services independently.

30.3 Monitoring the Frontend

The Quill React frontend is deployed on Vercel. Uptime monitoring can be associated with the deployed frontend so that availability of the public application can be checked.

The monitoring relationship is:

UptimeRobot → Vercel-hosted Quill application

The purpose is to identify situations where the production frontend cannot be reached.

30.4 Monitoring the Backend

The Django REST backend is deployed on Render and provides the API used by the frontend.

Monitoring the backend is important because the frontend may still load even when API-based features are unavailable.

The monitoring relationship is:

UptimeRobot → Render-hosted Django API

This provides a separate availability check for the backend service.

A backend-specific monitor is useful because it distinguishes API availability from frontend availability.

30.5 Monitoring Frontend and Backend Separately

Since Quill uses separate Vercel and Render deployments, monitoring them separately gives a clearer indication of where a problem may exist.

For example:

Frontend	Backend	Interpretation
Available	Available	Main services reachable
Available	Unavailable	Frontend may load, but API features can fail
Unavailable	Available	Public frontend is unavailable
Unavailable	Unavailable	Both monitored services are unavailable

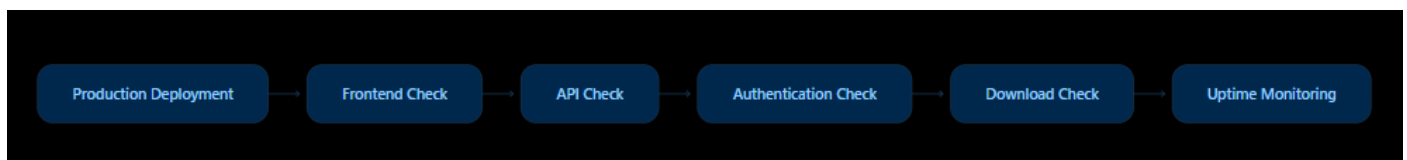
This follows Quill's documented independent frontend/backend deployment structure.

30.6 Monitoring and Production Reliability

Uptime monitoring complements the deployment and testing process.

The documented production testing strategy includes **frontend availability, API availability, authentication, downloads, and uptime checks**.

A simplified production verification flow is:



30.7 Alerts and Failure Detection

UptimeRobot is used for **availability monitoring**, so it provides an external monitoring layer separate from the Quill application itself.

The basic idea is:

Service available → monitor records normal availability

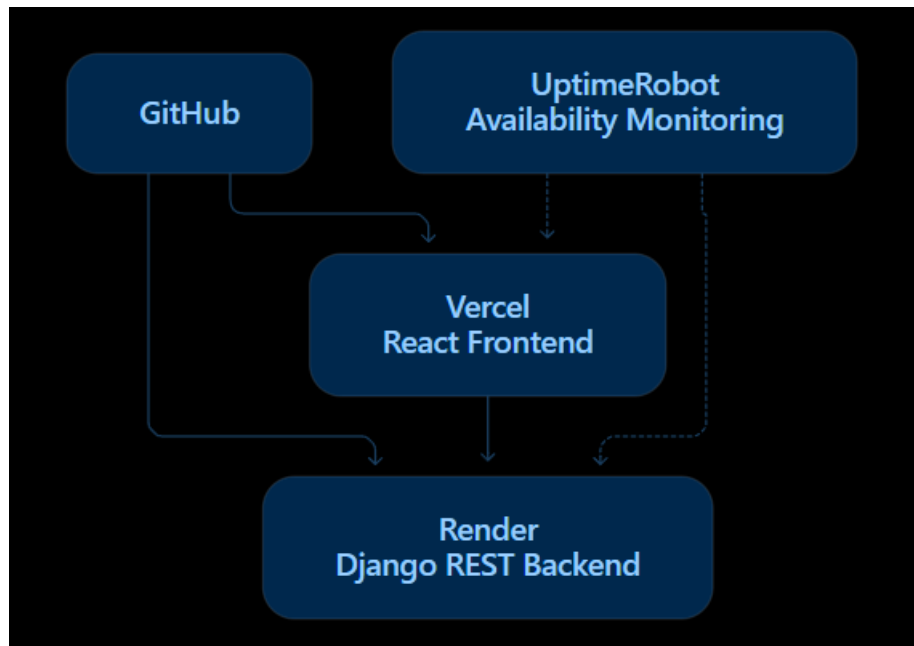
Service unavailable → monitor detects the failure

This means service availability can be observed independently of the application interface.

The available Quill documentation does **not specify the exact UptimeRobot monitor interval, alert channels, threshold values, or monitor URLs**, so those details should not be presented as fixed project settings.

30.8 Role in the Quill Deployment

UptimeRobot is therefore a supporting component of the production architecture:



30.9 Monitoring Summary

UptimeRobot adds an external availability-monitoring layer to the Quill production environment.

The overall production setup is:

GitHub → Vercel → React Frontend

GitHub → Render → Django REST Backend

UptimeRobot → Availability Monitoring

The technical documentation confirms UptimeRobot as the monitoring component of the production deployment and includes uptime checks in the deployment testing strategy.

31. Security & Environment Configuration

Security is an important part of Quill because the application handles **user accounts, authentication tokens, uploaded documents, generated files, translation services, and external API credentials**. Quill therefore uses authentication controls and environment-based configuration to separate sensitive information from the application source code.

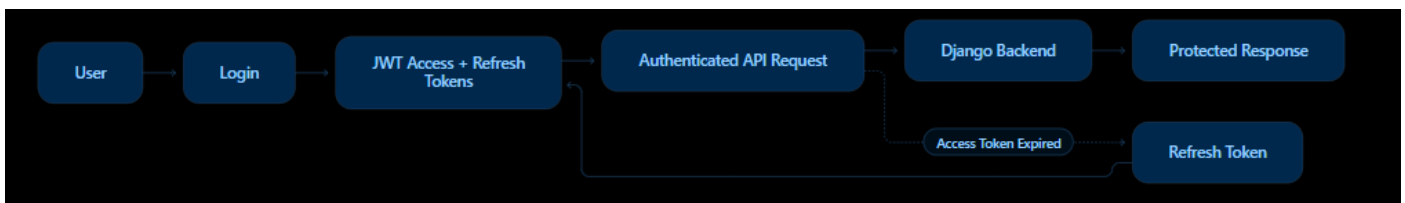
31.1 Authentication Security

Quill uses **JWT-based authentication** for protected API access.

The documented authentication mechanism includes:

- JWT access tokens
- JWT refresh tokens
- Frontend token-expiry checking
- Refreshing access tokens when required
- User-identity-based protection for history and account operations

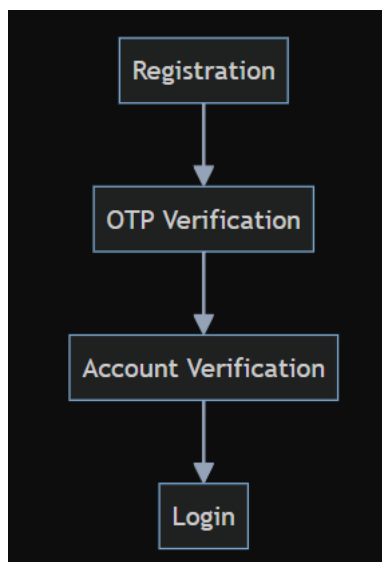
This means authenticated API operations are associated with the logged-in user rather than being treated as anonymous requests.



31.2 OTP Verification

Quill also supports **email OTP verification** as part of account authentication.

During registration, an OTP can be sent for verification. This provides an additional verification step before the account is fully used.



The authentication documentation confirms that registration can trigger email OTP verification.

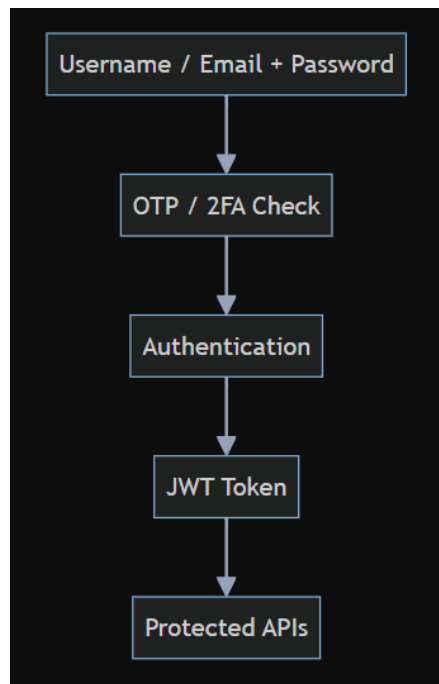
Register → Send OTP → Enter OTP → Verify → Account Ready

31.3 Two-Factor Authentication

Quill also includes support for **two-factor authentication (2FA)** depending on the user's account state.

The backend uses **pyotp** for two-factor setup and provisioning URI generation.

A simplified login flow is:



31.4 Environment-Based Configuration

Sensitive configuration is not intended to be stored directly in the source code.

Quill uses **environment variables** so credentials can be supplied separately in development and production environments.

The documented security rules specifically state:

.env files should not be committed to the repository.

The project also instructs that API keys, SMTP keys, OAuth secrets, JWT secrets, database passwords, and private VAPID keys should not be committed.

Instead, sensitive values are injected through the hosting platform's environment configuration.

31.5 Sensitive Information Protected by Environment Variables

Examples of configuration that should remain outside source control include:

Configuration	Purpose
SECRET_KEY	Django application security

DATABASE_URL	Database connection
Translation API keys	External translation services
SMTP credentials	Email/OTP functionality
OAuth credentials	Google/GitHub authentication
JWT secrets	Token authentication
VAPID private keys	Push notification functionality

The project README provides environment-based configuration examples and specifically warns against committing these types of credentials.

31.6 Development vs Production Configuration

Quill uses different environments for local development and production.

Local Development

Developers can use a local .env file for values that are required during development, while ensuring that the file remains outside version control.

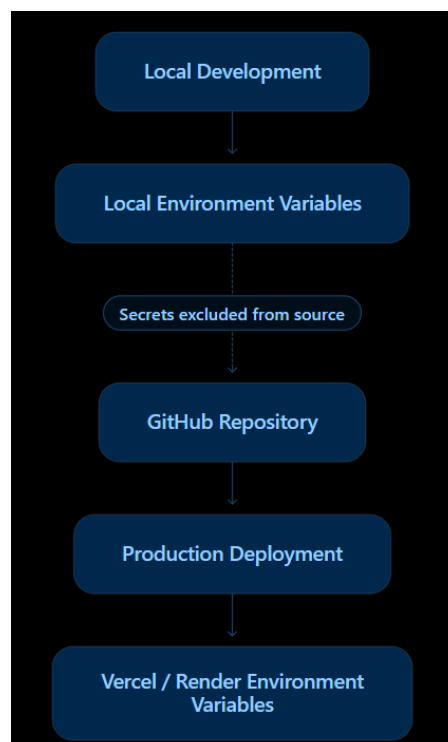
Production

For production deployment, environment variables are configured on the hosting platforms rather than being embedded in the GitHub repository.

The documented deployment architecture uses:

- **Vercel → React Frontend**
- **Render → Django REST Backend**

with secrets supplied through the hosting environment.



31.7 API and Account Protection

Security also extends to authenticated application features.

History and account-related actions are protected by the identity of the authenticated user. This helps ensure that user-specific operations are associated with the correct account.

The API layer also performs authentication and validation before processing protected operations, which connects the security layer with the error-handling and API architecture described earlier.

31.8 File and Processing Security

Quill processes uploaded PDFs, Word documents, images, audio, and other files. These operations therefore depend on controlled backend processing and validation.

The documented validation architecture includes checks for:

- Accepted file extensions
- File multiplicity
- Tool-specific options
- Authentication state
- Backend processing failures
- Download response validity

This provides a second layer of protection beyond authentication by validating what the application is being asked to process.

User Upload → Frontend Validation → Authenticated API → Backend Validation → Processing → Output

31.9 Security and Secret-Management Rules

The main security rules followed by the project are:

Rule	Purpose
Do not commit .env files	Prevent accidental credential exposure
Do not commit API keys	Protect external services
Do not commit SMTP credentials	Protect email infrastructure
Do not commit OAuth secrets	Protect social authentication
Do not commit JWT secrets	Protect authentication
Do not commit database passwords	Protect database access
Do not commit private VAPID keys	Protect push notification credentials
Use hosting environment variables	Separate secrets from source code

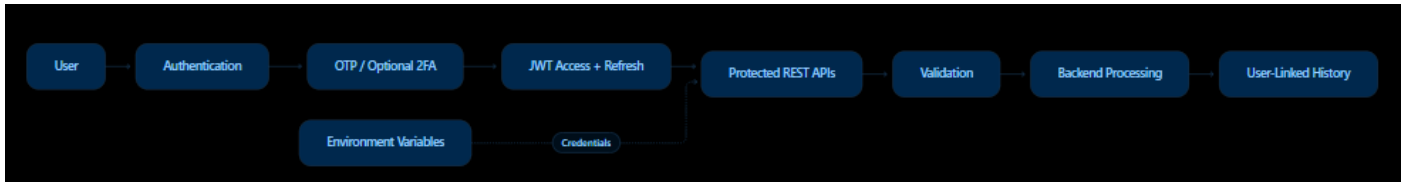
These rules are explicitly documented in the Quill technical documentation.

31.10 Security Limitation

It is important to document what the available project sources **do not establish**.

The reviewed sources do not specify a particular **encryption-at-rest implementation**. Therefore, the report should not claim that database or stored files are encrypted at rest unless such an implementation is separately verified.

This distinction keeps the project report technically accurate and avoids presenting an assumed security mechanism as an implemented feature.



32. Challenges Faced & Solutions

During the development of Quill, several technical challenges had to be addressed because the application combines **translation, voice processing, document handling, PDF operations, file conversion, authentication, history, and deployment** in one system.

The challenges described below are based on the reviewed Quill source code and technical documentation. Where the documentation does not explicitly state that an issue was encountered as a historical incident, the section presents the **engineering challenge addressed by the implementation** rather than inventing a specific failure event.

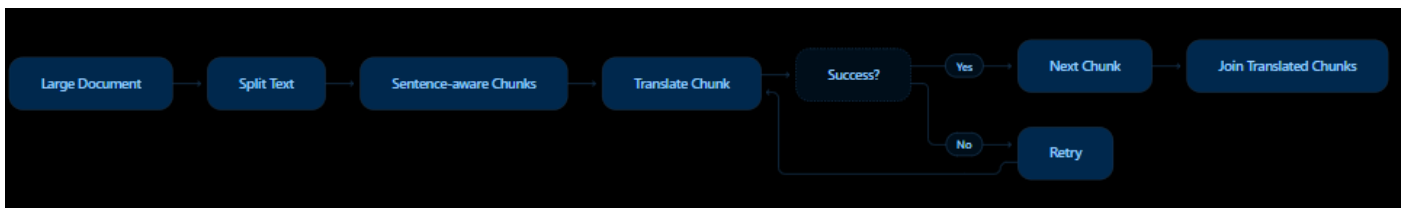
32.1 Handling Long Translation Requests

Challenge

Large documents can contain much more text than can be safely handled as one translation request. Sending the complete content at once can create oversized requests and makes individual failures harder to recover from.

Solution

Quill uses a **sentence-aware chunking approach**. The current document-processing source targets roughly **450 characters per chunk**, prefers sentence boundaries, handles unusually long sentences, processes chunks sequentially, retries failed chunks, and waits between calls.



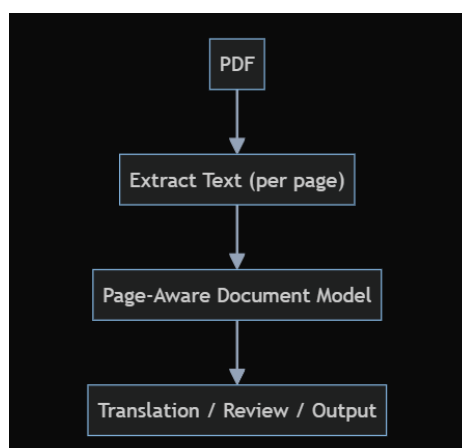
32.2 Maintaining PDF Page Structure

Challenge

A PDF is not simply one large text block. Each page can contain different content, and page structure needs to be retained when the document is translated or converted.

Solution

Quill uses a **page-oriented processing model**. The backend opens the PDF, iterates through its pages, extracts text page-by-page, preserves page count and page content, and passes the collected page model to later translation, review, and output-generation stages.



32.3 Preserving File Order During Merge

Challenge

For PDF, Word, and image merging, the output order must exactly match the order selected by the user in the interface.

A mismatch between the UI order and backend upload order could produce an incorrectly arranged final file.

Solution

Quill uses the `selectedFiles` array as the source of truth. Drag-and-drop removes the selected file using `splice()` and inserts it at the target position. During processing, files are appended to `FormData` in the same array order, so the backend receives them in the order shown by the UI.

For example:

Before:

A → B → C → D

Move D to position 2

After:

A → D → B → C

This approach keeps the UI order and backend merge order synchronized.

32.4 Supporting Multiple Translation Services

Challenge

Translation functionality may depend on external services, and a single service may not always be suitable for every workflow or runtime configuration.

Solution

The Quill architecture uses **multiple translation-service fallbacks** with environment-based credentials. The exact provider can vary depending on the workflow and runtime configuration.

This approach separates the application logic from a single fixed external translation dependency.

Quill Backend → Translation Service A / Service B / Service C → Translated Output

32.5 Managing Authentication and Session Security

Challenge

A modern application must maintain user sessions while handling registration verification, login, token expiry, and optional second-factor authentication.

Solution

Quill uses a layered authentication flow:

Registration → OTP verification

Login → Credentials → OTP / Optional 2FA → JWT access + refresh

Active session → Access-token expiry check → Refresh-token request → New access token

The backend also uses pyotp for two-factor setup and provisioning.

32.6 Handling Temporary Files and Processing Failures

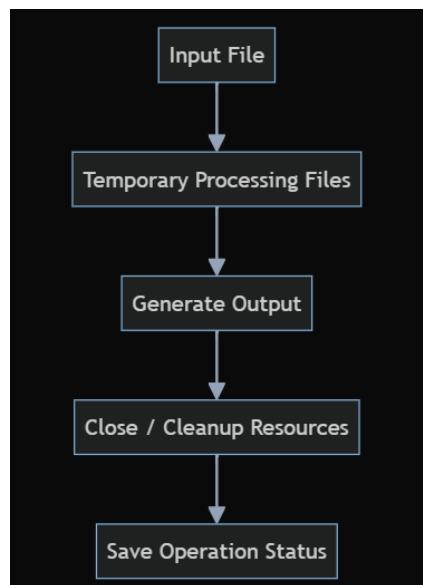
Challenge

File-processing operations often create temporary files and intermediate objects. Leaving these resources behind after an error can create unnecessary storage usage and complicate later processing.

Solution

The backend processing code uses explicit cleanup for temporary files and intermediate resources. In the PDF enhancement workflow, for example, temporary page images are tracked and removed, document objects are closed, and error handling records the failed operation before returning an error response.

The processing pattern is:



When an exception occurs, cleanup is also performed before the failed operation is stored.

32.7 Maintaining Correct Output and Error States

Challenge

A file-processing request should not be considered successful only because an API request reaches the backend. The output file, operation status, and download information also need to be handled correctly.

Solution

Quill records operation state in the backend. For successful processing, the output path/name/size and completion status are stored. For failures, the operation is marked as failed and an error message is saved.

The frontend also validates the API response and download result before presenting the output to the user. The documented validation layer includes backend failures and download-response checking.

Request → Processing → Success → Save Output → Download

Request → Processing → Error → Cleanup → Failed Status → Error Message

32.8 Connecting Frontend, Backend and Production Services

Challenge

Quill contains a browser application, backend APIs, processing services, and persistent data. These components must work together without mixing their responsibilities.

Solution

The system was divided into independent frontend and backend layers. The React application handles the user-facing workspace and communicates with the Django REST backend through APIs. Production deployment keeps these layers separate, using Vercel for the frontend and Render for the backend.

This architecture also makes it easier to isolate problems:



32.9 Protecting Sensitive Configuration

Challenge

Quill requires credentials for services such as translation, email, OAuth, JWT, and database access. Keeping these values directly in the source code would make accidental exposure easier.

Solution

The project uses **environment-based configuration**. The documentation explicitly instructs developers not to commit .env files, API keys, SMTP keys, OAuth secrets, JWT secrets, database passwords, or private VAPID keys, and instead to inject them through the hosting environment.

32.10 Summary of Challenges and Solutions

Challenge	Solution Used in Quill
Long translation requests	Sentence-aware chunking, sequential processing and retry
PDF page structure	Page-by-page extraction and page-aware model
Merge ordering	Array-based file ordering and ordered FormData
Translation dependency	Multiple translation-service fallbacks
Authentication	JWT, OTP and optional 2FA
Temporary processing files	Explicit cleanup and resource closing
Processing failures	Operation status and stored error information
Frontend/backend integration	Separate React and Django REST layers
Sensitive credentials	Environment-based configuration

The engineering decisions documented for Quill also emphasize **array-based file ordering, client-side preview helpers with backend processing, a page-aware document model, account-linked history, environment-based configuration, and independent frontend/backend hosting.**

33. Future Improvements

Quill already combines translation, voice workflows, document processing, file conversion, smart utilities, history, authentication, and deployment monitoring in one workspace. The next stage of development can focus on **scalability, reliability, automation, larger file workflows, better observability, and AI-powered features**.

The documented future-improvement plan already includes a background job queue, automated API and end-to-end testing, property-based ordering tests, resumable uploads, output integrity checks, OpenAPI support, telemetry, and completion of the planned 30-file merge-limit rollout.

33.1 Background Processing and Job Queue

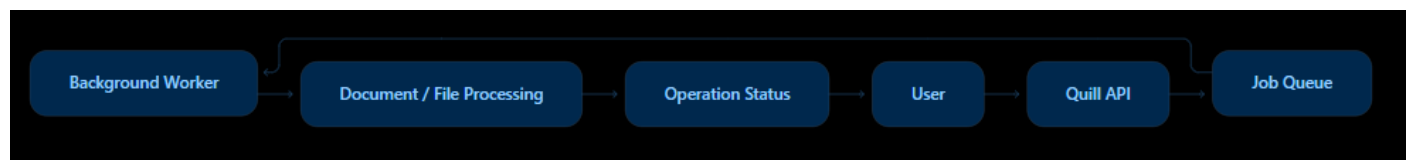
Long document translation and heavy file operations can take more time than ordinary API requests.

A future **background job queue** can move lengthy processing into asynchronous jobs.

This would allow the interface to show states such as:

Queued → Processing → Completed / Failed

The background queue is explicitly listed as a future improvement in the technical documentation.



33.2 AI Integration

Proposed AI-Powered Quill Assistant

A major future direction for Quill can be the integration of **Generative AI / Large Language Models (LLMs)** into the existing workspace.

This is a **proposed future enhancement**, not a feature that the current project documentation establishes as already implemented.

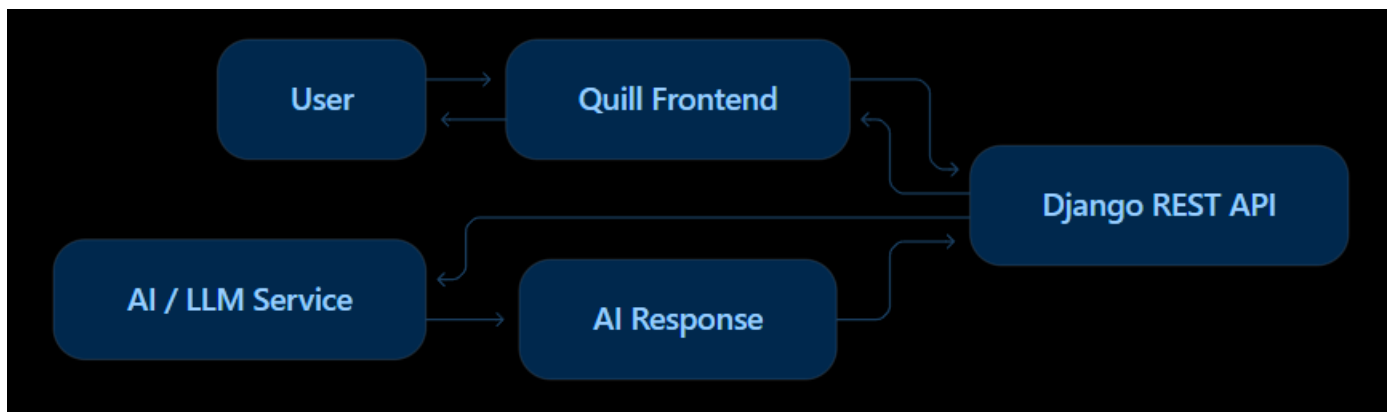
The AI layer could work alongside the existing translation and document-processing systems rather than replacing them.

Possible AI Features

AI Feature	Possible Use
AI Translation Refinement	Improve fluency, tone, grammar and context after translation
Document Summarization	Generate short summaries of uploaded documents
Ask Your Document	Allow users to ask questions about an uploaded document
AI Writing Assistant	Rewrite, simplify, expand or improve text
Tone Conversion	Convert text into formal, professional, academic or casual styles
Key Information Extraction	Extract important points, names, dates and sections
Multilingual AI Chat	Conversational assistance across supported languages

Voice AI Assistant	Combine speech input with AI responses and TTS
Smart File Insights	Describe or summarize the contents of processed files

A possible architecture is:

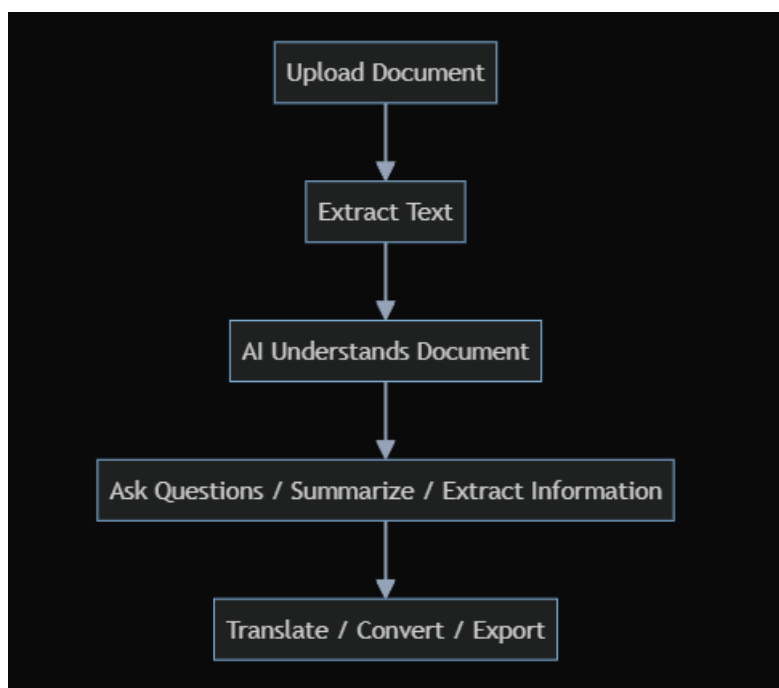


33.3 AI Document Assistant

The existing document workflow already follows:

Upload → Detect/Select Language → Translate → Review → Download

A future AI assistant could add another step after upload:



For example, a user could upload a long PDF and request:

“Summarize this document in Hindi.”

or:

“Find all important dates mentioned in this document.”

The existing page-aware document model provides a useful foundation for such an enhancement because document content is already handled page-by-page.

33.4 Automated API and End-to-End Testing

The current testing documentation defines a broad QA matrix covering authentication, translation, documents, combine operations, PDF tools, smart tools, history, and deployment. Future work can convert more of these checks into automated tests.

The future plan specifically includes:

- Automated API tests
- End-to-end workflow tests
- Tests for major application modules
- Regression testing after feature changes

This would reduce the amount of manual verification required whenever new functionality is added.

33.5 Property-Based Testing for File and Page Ordering

Quill uses array-based file ordering for merge operations. The technical documentation identifies **property-based tests for file/page ordering** as a future improvement.

Future tests could verify properties such as:

- **No files are lost during reordering.**
- **The number of files remains unchanged.**
- **The final processing order equals the UI order.**
- **Page-level ordering remains independent from file-level ordering.**

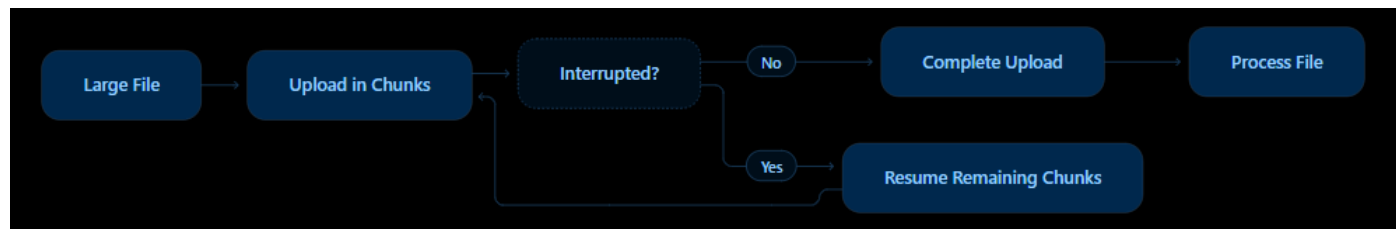
This is especially useful for complex drag-and-drop and page-management workflows.

33.6 Resumable Large-File Uploads

Large document and file-processing workflows can benefit from **resumable uploads**.

Instead of restarting a complete upload after a network interruption, future versions could continue from the point where the upload stopped.

Resumable uploads are explicitly included in the project's future-improvement list.



33.7 Output Integrity Checks

Future versions can perform stronger checks on generated files before exposing them for download.

The documented future plan includes **output integrity checks using MIME type, page count, and checksums**.

For example:

Check	Purpose
MIME Type	Confirm expected output format
Page Count	Verify expected number of pages
File Size	Detect abnormal empty or incomplete output
Checksum	Verify file consistency
Download Check	Confirm generated file is accessible

This would add another verification layer between processing and download.

33.8 Formal OpenAPI Documentation

The current backend exposes REST APIs for translation, documents, combine operations, history, and other application features.

A future improvement is to introduce a **formal OpenAPI schema and generated API client**.

This would make the API easier to:

- Document
- Test
- Integrate
- Maintain
- Consume from other clients

It would also provide a structured description of request formats, responses, parameters, authentication, and errors.

33.9 Telemetry and Performance Monitoring

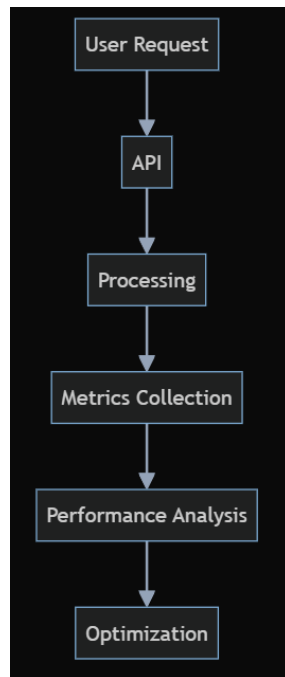
Future development can add more detailed telemetry for production systems.

The documentation specifically identifies telemetry for:

- Request duration
- File sizes
- Processing failures

This information can help identify slow operations and recurring processing problems before making larger infrastructure changes.

A future monitoring flow could be:



33.10 Increasing File Processing Limits

The reviewed source snapshot currently shows:

`MAX_MERGE_FILES = 10`

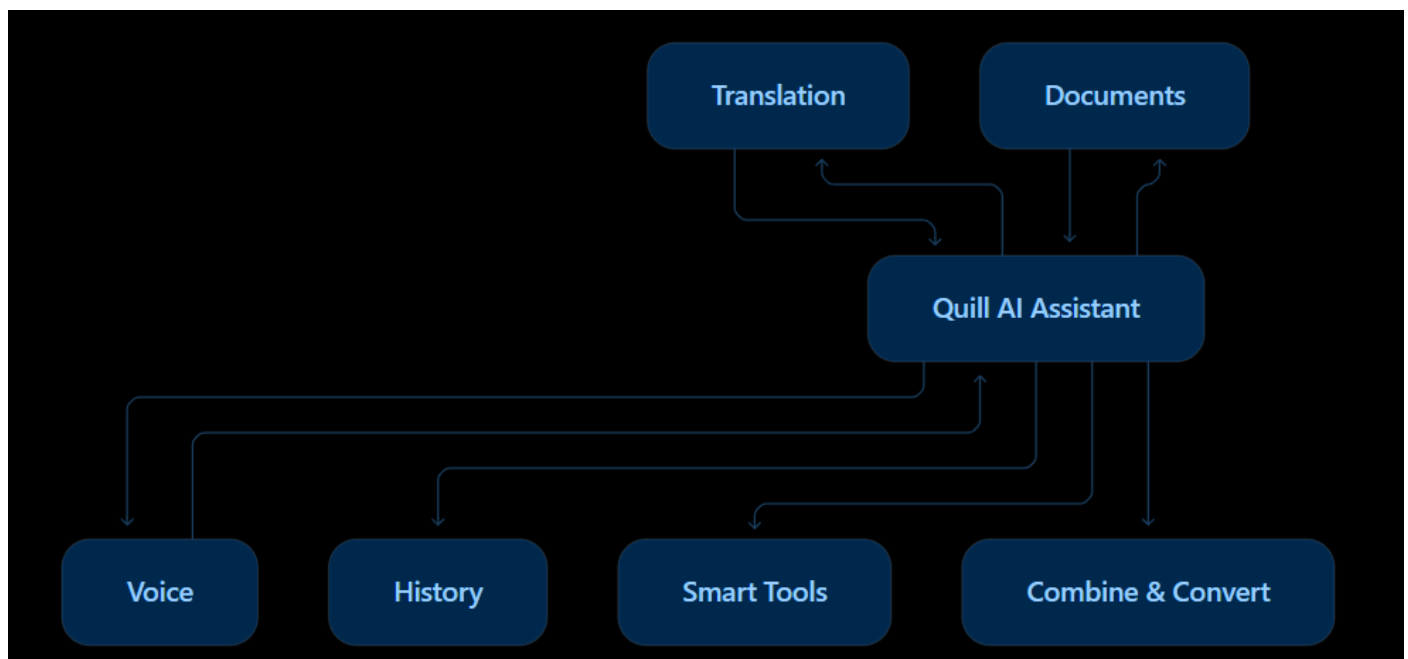
The technical documentation records a requested future change to raise the merge limit to **30 files**, with complete rollout and backend verification still listed as a future improvement.

This change would need to be verified across:

Frontend validation → API → Backend processing → Output integrity → Testing

rather than changing only the visible UI limit.

33.11 Future AI + Existing Quill Features



AI integration can eventually connect multiple existing Quill modules into one intelligent workflow.

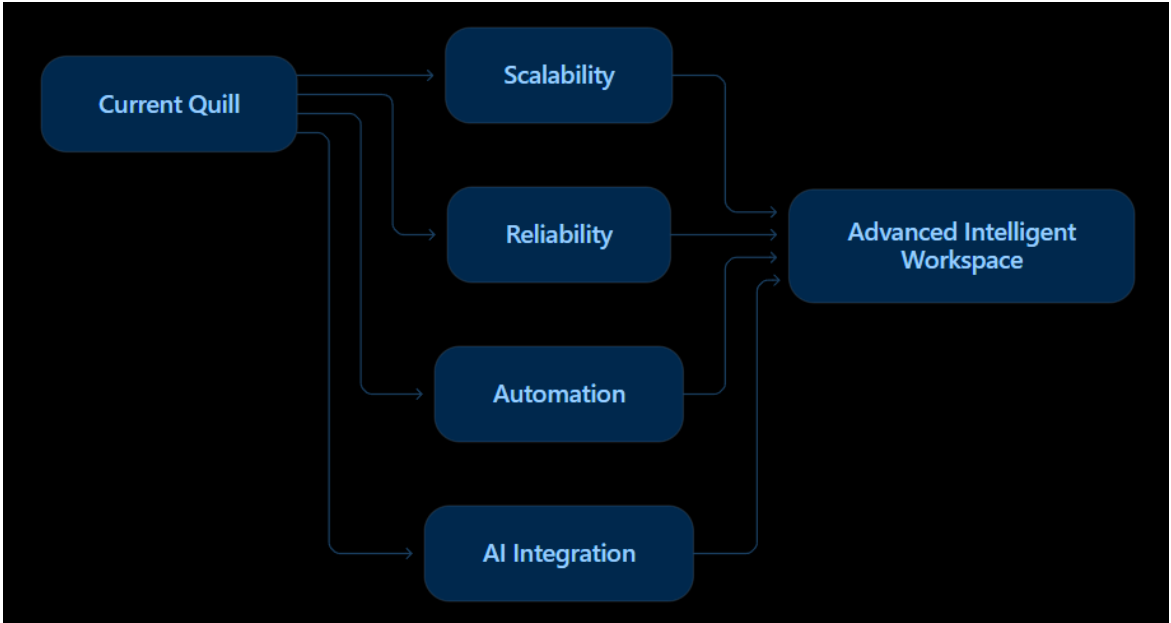
For example, the same assistant could accept a voice request such as:

“Summarize this PDF in Hindi and create a DOCX version.”

The AI assistant could interpret the request and then work with the existing document, translation, and conversion modules.

33.12 Overall Future Development Roadmap

The future direction of Quill can be summarized as:



Future Improvements Summary

Area	Planned Improvement
Scalability	Background job queue
Testing	Automated API and E2E tests
Correctness	Property-based ordering tests
File Handling	Resumable uploads
Output Reliability	MIME, page-count and checksum validation
API	OpenAPI schema and generated client
Monitoring	Detailed telemetry
Merge Capability	Complete 30-file limit rollout
AI	LLM-powered assistant, summarization, document Q&A, writing and translation enhancement

The first eight items come directly from the documented future-improvement plan; the **AI integration section is a proposed extension** added to the roadmap as requested.

34. Conclusion

Quill is a **full-stack productivity workspace** developed to bring translation, voice workflows, document processing, PDF utilities, file conversion, smart tools, history, settings, and authentication into one application. The project combines a **React/TypeScript frontend** with a **Django and Django REST Framework backend**, with production deployment separated between Vercel and Render and availability monitoring through UptimeRobot.

34.1 Project Outcome

The project brings several commonly separate tasks into a single workspace.

Users can work with:

Area	Main Capability
Translation	Text and multilingual translation workflows
Voice	Speech-to-text, text-to-speech and voice translation
Documents	PDF, DOCX, TXT and HTML processing
Combine & Convert	Merge, split, convert and optimize files
Smart Tools	Practical calculation and utility functions
History	Store and manage previous operations
Authentication	JWT, OTP and optional 2FA
Settings	User preferences and application configuration

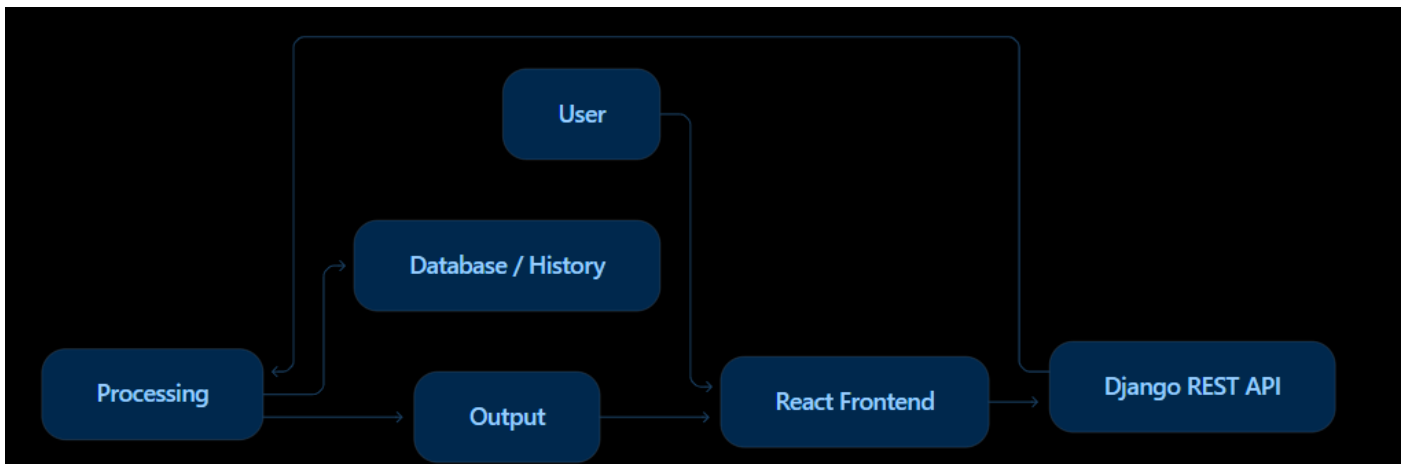
The system is designed around the idea of moving from **input to processing to output** without requiring separate products for each task.

34.2 Technical Achievement

From a technical perspective, Quill demonstrates how a complete web application can combine several different types of workloads under one architecture.

The frontend manages the interactive workspace, while the backend handles authentication, API routing, validation, document and file processing, and communication with external services. The architecture also keeps persistence and history behind the API layer.

A simplified view is:



34.3 Important Engineering Work

Several parts of the project required careful implementation rather than simple CRUD functionality.

The document workflow uses a **page-oriented processing model**, while long translation requests are divided into smaller sentence-aware chunks. PDF preview and page rendering use browser-side helpers such as PDF.js.

The Combine workflow also maintains a separate file-order model. The selected file array determines merge order, and files are added to FormData in that same order so that the backend receives the sequence selected by the user.

These implementation decisions make the project more than a collection of screens; they demonstrate practical handling of **APIs, authentication, file processing, ordering, document structure, validation, and deployment**.

34.4 Security and Reliability

Quill includes JWT access and refresh tokens, OTP verification, optional 2FA, authenticated account operations, and environment-based configuration for sensitive credentials.

The project also separates frontend and backend deployment and uses UptimeRobot for availability monitoring. Production testing covers the frontend, API, authentication, downloads, and uptime checks.

User → Vercel Frontend → Render Backend → Database/Services

with **JWT Authentication** and **UptimeRobot Monitoring** around the architecture.

34.5 Future Scope

Although the current system already covers a broad range of workflows, the project has a clear path for further development.

The documented roadmap includes background job processing, automated API and end-to-end testing, property-based testing, resumable uploads, output-integrity checks, OpenAPI support, telemetry, and the planned expansion of the merge limit.

A major additional direction is **AI integration**. As a future enhancement, Quill can incorporate an AI assistant capable of working with existing translation and document workflows.

Potential capabilities include:

- Document summarization
- Ask-your-document functionality

- AI writing and rewriting
- Translation refinement
- Tone and style transformation
- Key information extraction
- Multilingual AI assistance
- Voice-based AI interaction

This AI layer would extend Quill from a productivity workspace into a more intelligent document and language assistant. It is a **proposed future feature**, not a claim about the current implementation.

Quill AI Assistant → Translation | Documents | Voice | Writing | Summarization | Document Q&A

34.6 Learning and Development Experience

The development of Quill provided practical experience across several areas of software engineering:

- **Frontend Development:** React, TypeScript, component-based UI, state management and responsive interfaces.
- **Backend Development:** Python, Django, REST APIs, authentication and server-side processing.
- **File Processing:** PDF, DOCX, images, audio, merging, conversion and output generation.
- **Database and History:** Persistent user-linked records and operation tracking.
- **Security:** JWT, OTP, 2FA and environment-based secrets.
- **Deployment:** GitHub, Vercel, Render and UptimeRobot.

Performance: Translation chunking, page-level PDF processing, pagination and separation of frontend/backend workloads.

34.7 Final Conclusion

Quill demonstrates the development of a complete software product from **requirements and architecture to implementation, testing, deployment, monitoring, and future expansion**.

Its main achievement is not only the number of features included, but the way those features are connected through a consistent architecture. Translation, documents, file operations, authentication, history, and utilities work as parts of the same workspace rather than as unrelated tools.

The current architecture provides a foundation for continued development, while the proposed AI layer creates a path toward more intelligent translation, document understanding, and productivity workflows.

Quill brings translation, documents, conversion, utilities, and intelligent assistance together in one workspace.

35. References & Resources

The following references are the **official documentation and project resources** for the technologies and services used in, or directly related to, Quill. These are suitable for the final project report because they point to the original documentation maintained by the respective technology authors or organizations.

35.1 Official Project Resources

Resource	Purpose	Official Reference
Quill GitHub Repository	Project source code, README and development files	Quill GitHub Repository
Quill Live Application	Deployed frontend application	Quill Live Application

35.2 Frontend Technologies

Technology	Official Documentation
React	React Official Documentation
TypeScript	TypeScript Official Documentation
Tailwind CSS	Tailwind CSS Documentation
Lucide	Lucide Official Documentation
TanStack Query / React Query	TanStack Query Documentation
MDN Web Docs	MDN Web Documentation

MDN is particularly useful for the HTML, CSS, JavaScript, browser API, network-request, and web-platform concepts used by the frontend.

35.3 Backend and Database Technologies

Technology	Official Documentation
Python	Python Official Documentation
Django	Django Official Documentation
Django REST Framework	Django REST Framework Official Documentation
PostgreSQL	PostgreSQL Official Documentation

The Django and Django REST Framework documentation cover the backend framework, request handling, APIs, authentication-related integration, views, models and testing concepts used in the project.

35.4 PDF, Document and Audio Processing

Technology	Official / Maintainer Reference
PDF.js	Mozilla PDF.js Repository
jsPDF	jsPDF Project Repository
Mammoth.js	Mammoth.js Official Repository
FFmpeg	FFmpeg Official Documentation
Pydub	Pydub Project Repository
SpeechRecognition	SpeechRecognition Project Repository
edge-tts	edge-tts Project Repository
gTTS	gTTS Documentation

PDF.js is maintained by Mozilla and provides PDF parsing/rendering capabilities for web applications; jsPDF provides client-side PDF generation. FFmpeg provides official documentation for ffmpeg and ffprobe, while Pydub provides a higher-level Python interface for audio manipulation.

35.5 Authentication and Security

Technology / Standard	Official Reference
JSON Web Tokens (JWT)	JWT Official Introduction
PyOTP	PyOTP Official Documentation

PyOTP provides Python support for generating and verifying one-time passwords, including HOTP and TOTP-based authentication mechanisms.

35.6 Deployment, Monitoring and Version Control

Service	Official Reference
Vercel	Vercel Official Documentation
Render	Render Official Documentation
UptimeRobot	UptimeRobot Official Website
Git	Git Official Documentation
GitHub	GitHub Official Documentation

Vercel's documentation covers deployment of web applications and Git-connected deployments, while Render's documentation covers service deployment, environment variables, networking, scaling, health checks, and monitoring. Git's official documentation covers version control and command-line Git operations, while GitHub documents repository and collaboration workflows.

35.7 AI / Future Integration Resources

For the AI integration proposed in **Page 33**, the following official resource can be used as a reference:

Technology	Official Reference
OpenAI API	OpenAI API Documentation

The official OpenAI API documentation covers API requests, text generation, natural-language processing, image analysis, agents, and related AI application development.